

NÍVEL ASSOCIATE

# SAA-C03

AWS CERTIFIED SOLUTIONS ARCHITECT - ASSOCIATE

## Guia de revisão

Computação, armazenamento, redes, segurança e decisões de arquitetura para a revisão da certificação.

Erik Nathan [eriknathan.me](https://eriknathan.me)

### DOMÍNIOS DO EXAME • PESOS OFICIAIS

**30%**

Design Secure Architectures  
Arquiteturas seguras

**26%**

Design Resilient Architectures  
Arquiteturas resilientes

**24%**

Design High-Performing  
Architectures  
Arquiteturas de alto desempenho

**20%**

Design Cost-Optimized  
Architectures  
Arquiteturas com custo  
otimizado

# Sumário

## COMO USAR ESTE GUIA

|                                     |   |
|-------------------------------------|---|
| Números e limites                   | 7 |
| Método para questões de arquitetura | 7 |

|                                            |   |
|--------------------------------------------|---|
| Domínios do exame SAA-C03 (pesos oficiais) | 8 |
|--------------------------------------------|---|

## CAPÍTULO 1 – COMPUTAÇÃO

|            |                                                               |           |
|------------|---------------------------------------------------------------|-----------|
| <b>1.1</b> | <b>EC2 — modelos de compra</b>                                | <b>9</b>  |
|            | Escolha da instância antes do modelo de compra                | 9         |
|            | Comparativo dos modelos                                       | 9         |
|            | On-Demand                                                     | 10        |
|            | Reserved Instances (RI)                                       | 10        |
|            | Savings Plans                                                 | 10        |
|            | Spot Instances                                                | 11        |
|            | Números do EC2 Spot                                           | 11        |
|            | Reservas de capacidade                                        | 11        |
|            | Tenancy: Dedicated Hosts vs. Dedicated Instances              | 12        |
|            | Padrões e pegadinhas de prova                                 | 12        |
| <b>1.2</b> | <b>Auto Scaling</b>                                           | <b>12</b> |
|            | Fundamentos do Auto Scaling group                             | 12        |
|            | Políticas de scaling                                          | 13        |
|            | Métricas de scaling                                           | 13        |
|            | Warmup, cooldown e warm pools                                 | 14        |
|            | Health checks e substituição                                  | 14        |
|            | Lifecycle hooks                                               | 14        |
|            | Scale-in: políticas de terminação e proteção                  | 14        |
|            | Atualização e manutenção do grupo                             | 15        |
|            | Padrões e pegadinhas de Auto Scaling                          | 15        |
| <b>1.3</b> | <b>Inicialização, hibernação e armazenamento local da EC2</b> | <b>15</b> |
|            | AMI (Amazon Machine Image)                                    | 16        |
|            | User data                                                     | 16        |
|            | Instance metadata (IMDS)                                      | 16        |
|            | Ciclo de vida: reboot, stop, hibernate e terminate            | 16        |
|            | Hibernate                                                     | 17        |
|            | Instance Store                                                | 17        |
|            | Padrões e pegadinhas de inicialização                         | 18        |
| <b>1.4</b> | <b>Placement Groups</b>                                       | <b>18</b> |
|            | Comparativo das estratégias                                   | 18        |
|            | Cluster                                                       | 19        |
|            | Partition                                                     | 19        |
|            | Spread                                                        | 19        |
|            | Regras operacionais                                           | 20        |
|            | Rede de alto desempenho: ENA e EFA                            | 20        |
|            | Padrões e pegadinhas de Placement Groups                      | 20        |

|             |                                                       |           |
|-------------|-------------------------------------------------------|-----------|
| <b>1.5</b>  | <b>Contêineres</b>                                    | <b>21</b> |
|             | Escolhendo a plataforma                               | 21        |
|             | Amazon ECS — conceitos                                | 21        |
|             | IAM em contêineres                                    | 21        |
|             | Rede, balanceamento e armazenamento                   | 22        |
|             | Scaling, deploy e observabilidade                     | 22        |
|             | AWS Fargate                                           | 23        |
|             | Amazon EKS                                            | 23        |
|             | Amazon ECR                                            | 23        |
|             | Outras opções de execução                             | 24        |
|             | Padrões e pegadinhas de contêineres                   | 24        |
| <b>1.6</b>  | <b>AWS Batch</b>                                      | <b>24</b> |
|             | Componentes                                           | 25        |
|             | Tipos de job                                          | 25        |
|             | Escolha da capacidade                                 | 25        |
|             | AWS Batch vs. alternativas                            | 25        |
| <b>1.7</b>  | <b>Serverless</b>                                     | <b>26</b> |
|             | AWS Lambda — fundamentos                              | 26        |
|             | Modelos de invocação                                  | 26        |
|             | Concorrência e cold start                             | 27        |
|             | Rede, dados e segurança                               | 27        |
|             | Números do Lambda                                     | 27        |
|             | Ecossistema serverless                                | 28        |
|             | Elastic Beanstalk (PaaS)                              | 28        |
|             | Padrões e pegadinhas de serverless                    | 28        |
| <b>1.8</b>  | <b>Edge e híbrido</b>                                 | <b>29</b> |
|             | Comparativo das opções                                | 29        |
|             | AWS Outposts                                          | 29        |
|             | AWS Local Zones e Wavelength                          | 29        |
|             | Computação na borda do CloudFront                     | 30        |
|             | Híbrido: VMware, contêineres e servidores on-premises | 30        |
|             | Padrões e pegadinhas de edge e híbrido                | 30        |
| <b>1.9</b>  | <b>Mobile e Frontend</b>                              | <b>31</b> |
|             | AWS Amplify                                           | 31        |
|             | Hosting de frontend sem Amplify                       | 31        |
|             | Backend para aplicações mobile e web                  | 31        |
|             | AWS Device Farm                                       | 32        |
|             | Padrões e pegadinhas de mobile e frontend             | 32        |
| <b>1.10</b> | <b>Decisão rápida — Computação</b>                    | <b>32</b> |

## CAPÍTULO 2 – ARMAZENAMENTO

|     |                                                            |    |     |                                                    |    |
|-----|------------------------------------------------------------|----|-----|----------------------------------------------------|----|
| 2.1 | Amazon S3                                                  | 33 | 2.3 | Amazon EFS                                         | 40 |
|     | Fundamentos do S3                                          | 33 |     | Fundamentos do EFS                                 | 40 |
|     | Classes de armazenamento                                   | 34 |     | Classes de armazenamento e throughput do EFS       | 41 |
|     | Lifecycle e análise de custo                               | 34 |     | Segurança e replicação do EFS                      | 41 |
|     | Proteção de dados: versionamento, Object Lock e replicação | 35 |     | Padrões e pegadinhas de EFS                        | 41 |
|     | Segurança e criptografia do S3                             | 36 | 2.4 | Amazon FSx                                         | 42 |
|     | Desempenho e transferência                                 | 36 |     | Padrões e pegadinhas de FSx                        | 42 |
|     | Eventos e operações em massa                               | 37 | 2.5 | Storage Gateway e migração de dados                | 42 |
|     | Números do S3                                              | 37 |     | AWS Storage Gateway                                | 42 |
|     | Padrões e pegadinhas de S3                                 | 38 |     | Migração e transferência de dados                  | 43 |
| 2.2 | Amazon EBS                                                 | 38 |     | Números do Snowball (legado)                       | 44 |
|     | Fundamentos do EBS                                         | 38 |     | Padrões e pegadinhas de Storage Gateway e migração | 44 |
|     | Tipos de volume                                            | 38 | 2.6 | Backup                                             | 44 |
|     | Multi-Attach                                               | 39 |     | AWS Backup                                         | 44 |
|     | Snapshots                                                  | 39 |     | Imutabilidade de backups                           | 44 |
|     | Criptografia do EBS                                        | 40 |     | Padrões e pegadinhas de backup                     | 45 |
|     | Números do EBS                                             | 40 | 2.7 | Decisão rápida – Armazenamento                     | 45 |
|     | Padrões e pegadinhas de EBS                                | 40 |     |                                                    |    |

## CAPÍTULO 3 – BANCO DE DADOS

|     |                                                 |    |     |                                       |    |
|-----|-------------------------------------------------|----|-----|---------------------------------------|----|
| 3.1 | Amazon RDS                                      | 46 |     | Chaves e índices                      | 51 |
|     | Fundamentos do RDS                              | 46 |     | Capacidade e custo do DynamoDB        | 51 |
|     | Alta disponibilidade e escala de leitura no RDS | 46 |     | Global Tables, cache e streams        | 52 |
|     | Backups e restauração do RDS                    | 47 |     | Backup e segurança do DynamoDB        | 52 |
|     | Segurança do RDS                                | 47 |     | Números do DynamoDB                   | 53 |
|     | Operação, desempenho e custo do RDS             | 48 |     | Padrões e pegadinhas de DynamoDB      | 53 |
|     | Números do RDS e do Aurora                      | 48 | 3.4 | Amazon ElastiCache (Redis/Memcached)  | 53 |
|     | Padrões e pegadinhas de RDS                     | 49 |     | Engines e modos                       | 53 |
| 3.2 | Amazon Aurora                                   | 49 |     | Estratégias de cache                  | 53 |
|     | Arquitetura do Aurora                           | 49 |     | Disponibilidade do ElastiCache        | 54 |
|     | Endpoints e réplicas do Aurora                  | 49 |     | Padrões e pegadinhas de ElastiCache   | 54 |
|     | Aurora Global Database                          | 50 | 3.5 | Outros bancos de dados                | 54 |
|     | Aurora Serverless v2                            | 50 | 3.6 | Migração e escolha econômica de banco | 55 |
|     | Recursos exclusivos do Aurora                   | 50 |     | AWS DMS                               | 55 |
|     | Padrões e pegadinhas de Aurora                  | 50 |     | Escolha econômica de banco            | 55 |
| 3.3 | Amazon DynamoDB                                 | 51 |     | Padrões e pegadinhas de migração      | 55 |
|     | Fundamentos do DynamoDB                         | 51 | 3.7 | Decisão rápida – Banco de Dados       | 56 |
|     |                                                 |    |     |                                       |    |

## CAPÍTULO 4 – REDE E ENTREGA DE CONTEÚDO

|     |                                     |    |     |                                             |    |
|-----|-------------------------------------|----|-----|---------------------------------------------|----|
| 4.1 | VPC – fundamentos                   | 57 |     | Padrões e pegadinhas de VPC                 | 59 |
|     | Estrutura da VPC                    | 57 | 4.2 | Interconexão de redes                       | 60 |
|     | Security Groups e Network ACLs      | 57 |     | Conectividade híbrida: VPN e Direct Connect | 60 |
|     | NAT                                 | 58 |     | Padrões e pegadinhas de interconexão        | 61 |
|     | VPC endpoints e PrivateLink         | 58 | 4.3 | Load Balancers                              | 62 |
|     | Acesso administrativo e remoto      | 59 |     | Recursos comuns dos load balancers          | 62 |
|     | Monitoramento e diagnóstico de rede | 59 |     | Padrões e pegadinhas de load balancers      | 63 |
|     | Números da VPC                      | 59 | 4.4 | Entrega de conteúdo e DNS                   | 63 |
|     |                                     |    |     |                                             |    |

|                                            |    |                                                 |    |
|--------------------------------------------|----|-------------------------------------------------|----|
| Amazon Route 53 — registros e hosted zones | 63 | AWS Global Accelerator                          | 65 |
| Políticas de roteamento do Route 53        | 63 | CloudFront vs. Global Accelerator vs. Route 53  | 65 |
| Números dos health checks do Route 53      | 64 | Padrões e pegadinhas de DNS e CDN               | 66 |
| Route 53 Resolver (DNS híbrido)            | 64 | 4.5 Decisão rápida — Rede e Entrega de Conteúdo | 66 |
| Amazon CloudFront                          | 64 |                                                 |    |

## CAPÍTULO 5 – SEGURANÇA, IDENTIDADE E CRIPTOGRAFIA

|                                          |    |                                                           |    |
|------------------------------------------|----|-----------------------------------------------------------|----|
| 5.1 IAM                                  | 67 | Padrões e pegadinhas de criptografia                      | 73 |
| Fundamentos do IAM                       | 67 | 5.4 Segredos                                              | 73 |
| Tipos de política                        | 68 | Padrões e pegadinhas de segredos                          | 74 |
| Avaliação de políticas                   | 68 | 5.5 Proteção de rede e aplicação                          | 74 |
| Acesso entre contas                      | 69 | Números do WAF                                            | 75 |
| Ferramentas de análise do IAM            | 69 | Padrões e pegadinhas de proteção de rede e aplicação      | 75 |
| Padrões e pegadinhas de IAM              | 69 | 5.6 Detecção, auditoria e conformidade                    | 75 |
| 5.2 AWS Organizations                    | 70 | Registro e auditoria                                      | 75 |
| Estrutura e governança multi-conta       | 70 | Detecção de ameaças e vulnerabilidades                    | 76 |
| SCPs e RCPs                              | 70 | Postura, conformidade e evidências                        | 76 |
| Compartilhamento e proteção centralizada | 70 | Padrões e pegadinhas de detecção e auditoria              | 77 |
| Padrões e pegadinhas de Organizations    | 71 | 5.7 Identidade federada e autorização                     | 77 |
| 5.3 Criptografia e certificados          | 71 | Acesso de funcionários (workforce)                        | 77 |
| AWS KMS                                  | 71 | AWS Directory Service                                     | 77 |
| Números do KMS                           | 72 | Usuários de aplicações (customer identity)                | 78 |
| Criptografia no S3 e em outros serviços  | 72 | Padrões e pegadinhas de identidade federada               | 78 |
| AWS CloudHSM                             | 72 | 5.8 Dados sensíveis em escala                             | 78 |
| AWS Certificate Manager (ACM)            | 72 | 5.9 Decisão rápida — Segurança, Identidade e Criptografia | 78 |
| Números do ACM                           | 73 |                                                           |    |

## CAPÍTULO 6 – INTEGRAÇÃO, MENSAGERIA E STREAMING

|                                         |    |                                                         |    |
|-----------------------------------------|----|---------------------------------------------------------|----|
| 6.1 Filas e notificações                | 81 | AWS Step Functions                                      | 84 |
| Amazon SQS                              | 81 | Amazon AppFlow                                          | 85 |
| Números do SQS                          | 82 | Padrões e pegadinhas de APIs e orquestração             | 85 |
| Amazon SNS                              | 82 | 6.3 Streaming                                           | 85 |
| Amazon EventBridge                      | 82 | Amazon Kinesis Data Streams                             | 85 |
| Amazon MQ                               | 83 | Amazon Data Firehose                                    | 86 |
| Padrões e pegadinhas de filas e eventos | 83 | Processamento de streams e Kafka                        | 86 |
| 6.2 APIs                                | 83 | Padrões e pegadinhas de streaming                       | 87 |
| Amazon API Gateway                      | 83 | 6.4 Decisão rápida — Integração, Mensageria e Streaming | 87 |
| AWS AppSync                             | 84 |                                                         |    |

## CAPÍTULO 7 – MIGRAÇÃO DE DADOS E BIG DATA / ANALYTICS

|                                             |    |                                      |    |
|---------------------------------------------|----|--------------------------------------|----|
| 7.1 Migração                                | 88 | Arquitetura de data lake             | 90 |
| Estratégia de migração (7 Rs)               | 88 | AWS Glue                             | 90 |
| Descoberta e planejamento                   | 88 | Amazon Athena                        | 91 |
| Servidores e máquinas virtuais              | 89 | Amazon EMR                           | 91 |
| Bancos de dados                             | 89 | Amazon Redshift                      | 91 |
| Arquivos e grandes volumes                  | 89 | Amazon OpenSearch Service            | 92 |
| Padrões e pegadinhas de migração para a AWS | 89 | Amazon Quick Sight e outros serviços | 92 |
| 7.2 ETL e análise                           | 90 | Padrões e pegadinhas de analytics    | 93 |

|     |                                                |    |
|-----|------------------------------------------------|----|
| 7.3 | Decisão rápida — Migração de Dados e Analytics | 93 |
|-----|------------------------------------------------|----|

## CAPÍTULO 8 – MACHINE LEARNING

|     |                                                   |    |                                       |    |
|-----|---------------------------------------------------|----|---------------------------------------|----|
| 8.1 | Serviços de IA pré-treinados                      | 94 | Ciclo de vida do modelo               | 96 |
|     | Documentos e texto                                | 94 | Opções de inferência                  | 96 |
|     | Fala e conversação                                | 95 | Padrões e pegadinhas de SageMaker     | 97 |
|     | Imagem e vídeo                                    | 95 | 8.3 IA generativa                     | 97 |
|     | Personalização, revisão humana e serviços legados | 95 | Amazon Bedrock                        | 97 |
|     | Padrões e pegadinhas de serviços de IA            | 96 | Padrões e pegadinhas de IA generativa | 97 |
| 8.2 | Amazon SageMaker AI                               | 96 | 8.4 Padrões de arquitetura com ML     | 98 |
|     |                                                   |    | 8.5 Decisão rápida — Machine Learning | 98 |

## CAPÍTULO 9 – GOVERNANÇA, MONITORAMENTO E CUSTOS

|     |                                            |     |                                                         |     |
|-----|--------------------------------------------|-----|---------------------------------------------------------|-----|
| 9.1 | Monitoramento e observabilidade            | 99  | Padrões e pegadinhas de operações                       | 102 |
|     | Amazon CloudWatch                          | 99  | 9.3 Governança multi-conta e conformidade               | 103 |
|     | Números do CloudWatch                      | 100 | 9.4 Gestão e otimização de custos                       | 103 |
|     | Rastreamento e observabilidade open-source | 100 | Visibilidade de custos                                  | 103 |
|     | AWS Health                                 | 100 | Controle de custos                                      | 104 |
|     | Padrões e pegadinhas de monitoramento      | 100 | Otimização de custos                                    | 104 |
| 9.2 | Operações e infraestrutura como código     | 101 | Custos de transferência de dados                        | 105 |
|     | AWS CloudFormation                         | 101 | Padrões e pegadinhas de custos                          | 105 |
|     | AWS Systems Manager                        | 101 | 9.5 Decisão rápida — Governança, Monitoramento e Custos | 105 |
|     | AWS Well-Architected                       | 102 |                                                         |     |

## CAPÍTULO 10 – ESTRATÉGIAS DE DISASTER RECOVERY

|      |                          |     |                                         |     |
|------|--------------------------|-----|-----------------------------------------|-----|
| 10.1 | Conceitos de DR          | 106 | 10.3 Blocos de construção por camada    | 108 |
| 10.2 | As quatro estratégias    | 106 | 10.4 Failover, failback e testes        | 108 |
|      | Backup & Restore         | 107 | 10.5 Ferramentas de resiliência         | 109 |
|      | Pilot Light              | 107 | 10.6 Padrões e pegadinhas de DR         | 109 |
|      | Warm Standby             | 107 | 10.7 Decisão rápida — Disaster Recovery | 110 |
|      | Multi-Site Active/Active | 107 |                                         |     |

## CAPÍTULO 11 – PADRÕES RECORRENTES CONSOLIDADOS (TOP PITFALLS DOS 6 SIMULADOS)

|      |                                                |     |                                            |     |
|------|------------------------------------------------|-----|--------------------------------------------|-----|
| 11.1 | Palavras-chave do enunciado                    | 111 | 11.6 Padrões de proteção de dados e backup | 113 |
| 11.2 | Padrões de arquitetura e esforço operacional   | 111 | 11.7 Padrões de segurança de borda         | 113 |
| 11.3 | Padrões de rede e conectividade                | 112 | 11.8 Padrões de integração e eventos       | 113 |
| 11.4 | Padrões de bancos de dados                     | 112 | 11.9 Padrões de escala e custo             | 114 |
| 11.5 | Padrões de identidade, segredos e criptografia | 112 | 11.10 Padrões de resiliência e DR          | 114 |

## CAPÍTULO 12 – MAPA DE DOMÍNIOS DO EXAME

|      |                                                  |     |                                                             |     |
|------|--------------------------------------------------|-----|-------------------------------------------------------------|-----|
| 12.1 | Domínio 1 — Design Secure Architectures (30%)    | 115 | 12.3 Domínio 3 — Design High-Performing Architectures (24%) | 116 |
| 12.2 | Domínio 2 — Design Resilient Architectures (26%) | 115 | 12.4 Domínio 4 — Design Cost-Optimized Architectures (20%)  | 116 |
|      |                                                  |     | 12.5 Observação sobre o modelo de pontuação                 | 117 |

## CAPÍTULO 13 – AUTOTESTE – FLASHCARDS DE REVISÃO RÁPIDA

|      |                         |     |       |                                                |     |
|------|-------------------------|-----|-------|------------------------------------------------|-----|
| 13.1 | Computação              | 118 | 13.7  | Migração e Analytics                           | 125 |
| 13.2 | Armazenamento           | 119 | 13.8  | Machine Learning                               | 126 |
| 13.3 | Banco de Dados          | 120 | 13.9  | Governança e Custos                            | 126 |
| 13.4 | Rede                    | 121 | 13.10 | Disaster Recovery                              | 127 |
| 13.5 | Segurança               | 122 | 13.11 | Pegadinhas Duplas (confusões mais recorrentes) | 128 |
| 13.6 | Integração e Mensageria | 124 |       |                                                |     |

## CAPÍTULO 14 – CARTÕES FLASH ADICIONAIS – CONSOLIDAÇÃO POR DOMÍNIO (SAA-C03)

|      |                                                               |     |      |                                                                                |     |
|------|---------------------------------------------------------------|-----|------|--------------------------------------------------------------------------------|-----|
| 14.1 | Domínio 1 – Projetar arquiteturas seguras                     | 130 | 14.3 | Domínio 3 – Projetar arquiteturas de alta performance                          | 142 |
|      | Armazenamento (S3 e EBS)                                      | 130 |      | Balanceamento de carga e entrega de conteúdo                                   | 142 |
|      | Contas, IAM e Organizations                                   | 130 |      | Armazenamento                                                                  | 143 |
|      | Rede (VPC)                                                    | 131 |      | Bancos de dados                                                                | 143 |
|      | Gerenciamento de segredos e certificados                      | 132 |      | Rede                                                                           | 144 |
|      | Diretório (Directory Service)                                 | 132 |      | Cartões adicionais – Volumes EBS e placement groups                            | 144 |
|      | AWS WAF                                                       | 132 |      | Cartões adicionais – Streaming, ingestão e performance de banco de dados       | 145 |
|      | Cartões adicionais – Criptografia e KMS                       | 133 |      | Cartões adicionais – Transferência de dados em larga escala                    | 145 |
|      | Cartões adicionais – Avaliação de políticas IAM               | 133 |      | Cartões adicionais – Famílias de instância e containers                        | 146 |
|      | Cartões adicionais – Detecção e monitoramento de segurança    | 133 |      | Cenários de decisão – performance                                              | 147 |
|      | Cartões adicionais – Proteção contra DDoS e acesso remoto     | 134 | 14.4 | Domínio 4 – Projetar arquiteturas econômicas                                   | 147 |
|      | Cartões adicionais – Governança e controle de acesso avançado | 134 |      | Rede e transferência de dados                                                  | 147 |
|      | Cenários de decisão – segurança                               | 136 |      | Armazenamento                                                                  | 148 |
| 14.2 | Domínio 2 – Projetar arquiteturas resilientes                 | 137 |      | Gestão e monitoramento de custos                                               | 148 |
|      | Bancos de dados                                               | 137 |      | Computação                                                                     | 148 |
|      | Conceitos de resiliência                                      | 137 |      | Cartões adicionais – Classes de armazenamento S3                               | 149 |
|      | Rede e DNS                                                    | 138 |      | Cartões adicionais – Modelos de preços de computação                           | 149 |
|      | Armazenamento                                                 | 138 |      | Cartões adicionais – Ferramentas de otimização de custo                        | 149 |
|      | Cartões adicionais – Auto Scaling                             | 138 |      | Cartões adicionais – Custo de transferência de dados e computação sem servidor | 150 |
|      | Cartões adicionais – Mensageria e desacoplamento              | 139 |      | Cartões adicionais – Estratégias de migração e otimização                      | 150 |
|      | Cartões adicionais – Roteamento com Route 53                  | 139 |      | Cenários de decisão – custos                                                   | 151 |
|      | Cartões adicionais – Cache, sessão e réplicas globais         | 139 |      |                                                                                |     |
|      | Cartões adicionais – Backup e orquestração                    | 140 |      |                                                                                |     |
|      | Cenários de decisão – resiliência                             | 141 |      |                                                                                |     |

## CAPÍTULO 15 – MAPA DE COBERTURA DO GUIA OFICIAL E QUESTÕES DE MÚLTIPLA RESPOSTA

|      |                                 |     |
|------|---------------------------------|-----|
| 15.1 | Como resolver múltipla resposta | 152 |
|------|---------------------------------|-----|

---

# Como usar este guia

- Use as seções por categoria para revisão de conteúdo.
- Use a tabela **Decisão rápida** ao final de cada capítulo (1 a 10) como cheat sheet de véspera de prova: cada uma reúne os cenários, a resposta correta e os distratores clássicos daquele tema.
- Use os **Padrões recorrentes consolidados** (seção 11) para revisar as pegadinhas que mais se repetem entre os 6 simulados (390 questões).
- Use o **Mapa de Domínios** (seção 12) para priorizar revisão conforme o peso de cada domínio na nota final.
- Use as tabelas **Números de...** dentro de cada tópico para revisar limites e números exatos que costumam confundir entre serviços — mas veja a ressalva em **Números e limites**, logo abaixo.
- Use o **Autoteste** (seção 13) para revisão ativa: os flashcards em callout do Obsidian escondem a resposta até você clicar, forçando recall em vez de releitura passiva.
- Use os **Cartões Flash Adicionais** (seção 14) como um segundo banco de revisão ativa, organizado pelos quatro domínios oficiais do exame, com cenários de decisão ao fim de cada domínio.
- Use o **Mapa de cobertura e questões de múltipla resposta** (seção 15) para conferir as 14 tarefas oficiais e praticar a seleção da quantidade de alternativas pedida no enunciado.
- Nos cartões de cenário da seção 14, identifique primeiro o **requisito decisivo**, escolha o serviço e explique por que as outras opções não atendem ao mesmo requisito.
- Para cotas, preços e disponibilidade de recursos, confira os links da documentação oficial antes da prova. As tabelas de números separam valores atuais dos números antigos ainda presentes em simulados.
- O guia oficial enumera conhecimentos e serviços **sem esgotar** todos os cenários possíveis da prova. O mapa da seção 15 cobre explicitamente as tarefas publicadas; revise também os princípios de arquitetura e as atualizações dos serviços na documentação oficial.

---

## Números e limites

Os limites e valores numéricos ficam em tabelas **Números de...** dentro do tópico de cada serviço. Essas tabelas registram números conferidos na documentação oficial da AWS (docs.aws.amazon.com) em **agosto de 2026**. São uma referência datada: confirme cotas e disponibilidade na documentação antes de depender de um valor exato.

**Vários limites clássicos de material de estudo mudaram** em 2025; quando indicado, o valor atual à data da conferência e o valor antigo aparecem juntos para explicar divergências com simulados.

*A AWS atualiza esses limites com frequência — vários desses números mudaram só em 2025 (SQS, S3, EBS gp3, RDS/Aurora storage, DynamoDB PITR, WAF). Para a prova, é mais seguro confiar no **padrão relativo** entre serviços (ex.: "io2 Block Express tem mais IOPS que io2, que por sua vez costuma superar gp3 no baseline") do que decorar o valor exato — exceto para os números clássicos que a própria AWS usa de forma consistente em material oficial e que raramente mudam, como os 15 minutos de timeout do Lambda, os 35 dias (padrão) de PITR do DynamoDB/RDS, e os 2 minutos de aviso de interrupção do EC2 Spot.*

---

## Método para questões de arquitetura

1. **Marque os requisitos obrigatórios:** protocolo, modelo de dados, RTO/RPO, latência, residência, volume, frequência e se o sistema legado pode ser alterado.
2. **Separe a restrição decisiva da preferência:** menor custo só decide entre alternativas que já atendem à segurança, à disponibilidade e ao desempenho exigidos.

3. **Compare o sistema inteiro:** uma réplica não substitui backup; um cache não resolve gravações; criptografia não concede autorização; Multi-AZ não protege de falha regional.
4. **Questões com várias respostas:** confira quantas opções o enunciado pede, avalie cada uma isoladamente e depois valide se o conjunto satisfaz todos os requisitos.

---

## Domínios do exame SAA-C03 (pesos oficiais)

| Domínio                              | Peso |
|--------------------------------------|------|
| Design Secure Architectures          | 30%  |
| Design Resilient Architectures       | 26%  |
| Design High-Performing Architectures | 24%  |
| Design Cost-Optimized Architectures  | 20%  |



# 1. Computação

## EC2 — modelos de compra

**Regra de ouro da prova:** primeiro identifique se o requisito é **desconto**, **capacidade garantida**, **isolamento físico/licenciamento** ou **tolerância a interrupção**. Cada modelo resolve só um desses problemas. Combiná-los (ex.: Capacity Reservation + Savings Plan) é comum na resposta correta.

### Escolha da instância antes do modelo de compra

- **Família e tamanho:** compute optimized (C) para CPU, memory optimized (R/X) para grandes conjuntos em RAM, storage optimized (I/D) para I/O local intenso, accelerated computing (P/G/Inf/Trn) para GPU/aceleradores e general purpose (M/T) para equilíbrio. Escolha com métricas de CPU, memória (via CloudWatch agent), EBS, rede e latência. Uma instância maior não corrige consulta ruim ou dependência externa lenta.
- **Rightsizing vem antes do compromisso:** comprometer-se com 1-3 anos de uma instância superdimensionada congela o desperdício. Use **AWS Compute Optimizer** e **Cost Explorer** (recomendações de rightsizing, RI e Savings Plans) antes de comprar.
- Scaling **vertical** aumenta a instância e pode exigir interrupção. Scaling **horizontal** acrescenta réplicas independentes atrás de um balanceador, com o estado persistido fora delas.

### Comparativo dos modelos

| Modelo                          | Desconto aproximado vs. On-Demand          | Compromisso                                          | Reserva capacidade?         | Quando usar                                                                 |
|---------------------------------|--------------------------------------------|------------------------------------------------------|-----------------------------|-----------------------------------------------------------------------------|
| On-Demand                       | —                                          | Nenhum                                               | Não                         | Cargas curtas, imprevisíveis, testes, novas aplicações sem histórico        |
| Standard Reserved Instances     | Até ~72%                                   | 1 ou 3 anos, atributos fixos                         | Só se <b>zonal</b>          | Carga base estável que não mudará de família/SO                             |
| Convertible Reserved Instances  | Até ~66%                                   | 1 ou 3 anos, permite troca                           | Só se <b>zonal</b>          | Carga base estável com expectativa de mudar família/SO/tenancy              |
| EC2 Instance Savings Plans      | Até ~72%                                   | US\$/hora por 1 ou 3 anos, uma família em uma Região | Não                         | Base estável em uma família, com flexibilidade de tamanho, SO, tenancy e AZ |
| Compute Savings Plans           | Até ~66%                                   | US\$/hora por 1 ou 3 anos, qualquer Região/família   | Não                         | Base estável que pode migrar de família/Região ou para Fargate/Lambda       |
| Spot Instances                  | Até ~90%                                   | Nenhum                                               | Não (pode ser interrompida) | Lote, CI/CD, renderização, big data, workers stateless tolerantes a falha   |
| On-Demand Capacity Reservations | Nenhum por si só                           | Nenhum (pago enquanto ativa)                         | <b>Sim</b> , em uma AZ      | Garantir capacidade para evento, DR ou lançamento, com qualquer duração     |
| Dedicated Hosts                 | Opções On-Demand/Reservation/Savings Plans | Opcional                                             | Host físico inteiro         | BYOL por socket/núcleo, conformidade com servidor dedicado                  |

| Modelo                     | Desconto aproximado vs. On-Demand     | Compromisso | Reserva capacidade? | Quando usar                                            |
|----------------------------|---------------------------------------|-------------|---------------------|--------------------------------------------------------|
| <b>Dedicated Instances</b> | Custo maior que tenancy compartilhada | Opcional    | Não                 | Isolamento de hardware por conta, sem controle do host |

Percentuais são tetos publicados pela AWS e variam por tipo, Região, prazo e forma de pagamento. Na prova importa a **ordem relativa**: Spot > RI Standard ≈ EC2 Instance SP > RI Convertible ≈ Compute SP > On-Demand.

## On-Demand

- Paga por segundo (mínimo de 60 s para Linux/Windows) ou por hora, sem compromisso, sem pagamento antecipado e sem risco de interrupção pela AWS.
- É a escolha certa para cargas **curtas, imprevisíveis ou sem histórico** e para ambientes que não podem ser interrompidos, mas ainda não justificam compromisso. **Não** é a resposta de menor custo para uma carga que roda 24/7 o ano todo.
- On-Demand **não garante capacidade**. Para garantir o lançamento em uma AZ, é preciso uma **Capacity Reservation**.

## Reserved Instances (RI)

- **Não é uma instância**: é um **desconto de cobrança** aplicado automaticamente a instâncias em execução cujos atributos correspondem à reserva (tipo, plataforma/SO, tenancy, Região ou AZ).
- **Prazo**: 1 ou 3 anos. **Pagamento**: All Upfront (maior desconto), Partial Upfront ou No Upfront (menor desconto). Três anos com All Upfront dão o maior desconto.
- **Standard vs. Convertible**: a Standard dá desconto maior e pode ter AZ, tamanho (dentro da família) e escopo alterados, mas não família/SO. Pode também ser **vendida no Reserved Instance Marketplace**. A Convertible pode ser **trocada** por outra RI Convertible de família, SO ou tenancy diferente, desde que de valor igual ou maior. Ela **não** pode ser vendida no Marketplace.
- **Regional vs. zonal**:
  - **Regional**: desconto em qualquer AZ da Região e **flexibilidade de tamanho** dentro da família (Linux/Unix, tenancy padrão). **Não reserva capacidade**.
  - **Zonal**: vinculada a uma AZ e **reserva capacidade** nela, sem flexibilidade de tamanho.
- A AWS hoje recomenda **Savings Plans** no lugar de RIs para a maioria dos casos de EC2. RIs continuam relevantes para **RDS, ElastiCache, Redshift, OpenSearch e DynamoDB** (capacidade reservada), que não são cobertos pelos Savings Plans de computação.

## Savings Plans

- Compromisso de **gasto por hora (US\$/h)** por 1 ou 3 anos, não de uma instância específica. O uso acima do compromisso é cobrado como On-Demand. As formas de pagamento são as mesmas das RIs.
- **Compute Savings Plans**: a opção mais flexível. Aplica-se a EC2 em **qualquer Região, família, tamanho, SO e tenancy**, além de **Fargate e Lambda**. É a resposta quando a empresa planeja migrar de família/Região ou modernizar para containers/serverless.
- **EC2 Instance Savings Plans**: desconto maior, mas restrito a **uma família em uma Região** (ex.: M7g em us-east-1). Continua flexível em tamanho, SO, tenancy e AZ.
- **SageMaker AI Savings Plans**: equivalente para uso de SageMaker AI.
- **Savings Plans não reservam capacidade**. Para desconto + garantia de capacidade, combine com uma **On-Demand Capacity Reservation**.

- Acompanhe com **Savings Plans budgets** (utilization e coverage) no AWS Budgets. Veja a seção 9.

## Spot Instances

- Usa capacidade EC2 ociosa com até ~90% de desconto. O preço Spot varia gradualmente com oferta e demanda de longo prazo, e o **preço máximo é opcional** (padrão = preço On-Demand). A interrupção ocorre quando a AWS precisa da capacidade de volta, não por lance.
- **Aviso de interrupção de 2 minutos** via instance metadata e evento no **EventBridge**. A **EC2 Instance Rebalance Recommendation** pode chegar antes e indica risco elevado de interrupção, permitindo substituir a instância proativamente (Capacity Rebalancing no Auto Scaling).
- **Comportamento na interrupção:** `terminate` (padrão), `stop` ou `hibernate`. Os dois últimos exigem volume root EBS e solicitação persistente ou frota mantida.
- **Spot request persistente:** depois que uma Spot Instance é interrompida, a solicitação volta a ficar aberta e tenta lançar outra instância quando houver capacidade. É adequada para um job reiniciável que deve voltar automaticamente. Uma solicitação `one-time` é encerrada após a interrupção.
- **Ideal para:** jobs em lote, renderização, CI/CD, análise de big data (EMR task nodes), workers de fila SQS, contêineres stateless e HPC com checkpoint. **Não deve ser a única capacidade** de uma aplicação crítica ou stateful (banco de dados primário, sessão em memória).
- **Diversificação:** quanto mais tipos de instância e AZs, menor a chance de interrupção simultânea. A estratégia de alocação recomendada é `price-capacity-optimized`, que escolhe os pools de menor preço entre os de maior capacidade disponível. `lowest-price` isolada aumenta o risco de interrupção.
- **Spot Fleet / EC2 Fleet:** lançam uma frota combinando Spot e On-Demand para atingir uma capacidade-alvo. Na prática, o mais comum é um **Auto Scaling group com mixed instances policy**: On-Demand base capacity + percentual On-Demand acima da base + restante em Spot, com vários tipos de instância. **AWS Batch, EKS/ECS e EMR** também usam Spot nativamente.
- *Spot Blocks (duração definida de 1-6 h) foram descontinuados. Se aparecerem em simulados antigos, desconsidere.*

## Números do EC2 Spot

| Métrica                                    | Valor     | Observação                                                          |
|--------------------------------------------|-----------|---------------------------------------------------------------------|
| Aviso de interrupção (interruption notice) | 2 minutos | Não se aplica ao modo hibernação, que não garante esse aviso prévio |

## Reservas de capacidade

- **On-Demand Capacity Reservations (ODCR):** reservam capacidade de um tipo de instância em **uma AZ específica**, por **qualquer duração**, sem compromisso de prazo. A cobrança ocorre à taxa On-Demand **estando a capacidade usada ou não**. É possível agendar reservas futuras (future-dated).
- **Sem desconto sozinha:** para reduzir o custo, aplique **Regional RIs ou Savings Plans** sobre a mesma capacidade. É a resposta clássica para "garantir capacidade em uma AZ para um evento/DR e reduzir custo".
- **Zonal RI vs. ODCR:** a Zonal RI reserva capacidade, mas exige 1-3 anos. A ODCR é flexível (cria e cancela quando quiser). Em um cenário de "evento de duas semanas", a resposta é ODCR, não RI.
- **Capacity Blocks for ML:** reservam instâncias GPU (ex.: P5) por períodos curtos e datas futuras para treinamento de ML, em EC2 UltraClusters.

## Tenancy: Dedicated Hosts vs. Dedicated Instances

- **Dedicated Hosts:** um servidor físico inteiro alocado à sua conta, com **visibilidade de sockets, núcleos e host ID e afinidade de instância ao host**. É **obrigatório para BYOL** de licenças por socket/núcleo/VM (Windows Server, SQL Server, Oracle) e para conformidade que exige servidor dedicado. Integra-se ao **AWS License Manager**. Pode ser comprado On-Demand, por Reservation ou coberto por Savings Plans.
- **Dedicated Instances:** rodam em hardware dedicado à sua conta, mas **sem visibilidade nem controle do host físico**. A cobrança é por instância, com uma taxa adicional por Região. Atendem a requisitos de isolamento físico sem licenciamento por núcleo.
- **Palavra-chave:** "licença existente vinculada a núcleos/sockets" leva a **Dedicated Hosts**. "Não compartilhar hardware com outros clientes", sem requisito de licença, leva a **Dedicated Instances**, a opção mais barata entre as duas.

## Padrões e pegadinhas de prova

- **Menor custo recorrente:** Savings Plans/Reserved Instances para a **carga base** + Spot para **picos tolerantes a interrupção** + On-Demand para o restante imprevisível. É repetido em praticamente todos os simulados.
- "Carga 24/7 por 3 anos, sem mudanças": **Standard RI ou EC2 Instance Savings Plan, 3 anos, All Upfront**.
- "Vai migrar de família/Região ou para Lambda/Fargate": **Compute Savings Plans**.
- "Precisa garantir capacidade em uma AZ": **Capacity Reservation** (ou Zonal RI). Regional RI e Savings Plans **não** garantem capacidade.
- "Job pode ser interrompido e retomado" / "ambiente não produtivo em lote": **Spot**. "Não pode ser interrompido": nunca Spot como única capacidade.
- "Carga curta e imprevisível, sem compromisso": **On-Demand**. Comprar RI para uma carga de poucos meses desperdiça o compromisso.
- "RI Standard sobrando após redução de uso": venda no **RI Marketplace**. "Precisa mudar de família mantendo o desconto": **Convertible RI**, via troca.
- Referência: [Opções de compra do EC2](#).

## Auto Scaling

**Regra de ouro da prova:** Auto Scaling resolve duas coisas diferentes. **Elasticidade** ajusta a capacidade à demanda e reduz custo. **Autorrecuperação** substitui instâncias com falha e mantém a capacidade desejada. Uma questão pode pedir só uma delas. Um grupo com `min = max = desired = 1` não escala, mas recria a instância se ela falhar.

## Fundamentos do Auto Scaling group

- **Launch template:** define AMI, tipo de instância, security groups, IAM role, user data e volumes. Use launch templates. As **launch configurations** estão descontinuadas para novos recursos e não suportam mixed instances policy nem versionamento.
- **Capacidades:** `min` (piso nunca violado), `max` (teto de custo) e `desired` (valor atual, que as políticas ajustam entre min e max). Uma ação agendada ou política nunca passa do `max`. Se o pico exigir mais, aumente o `max`.
- **Multi-AZ:** distribua o grupo em pelo menos duas AZs, com subnets de cada uma. O Auto Scaling mantém as AZs equilibradas (**AZ Rebalance**) e, se uma AZ falhar, lança a capacidade nas AZs restantes. Para suportar a perda de uma AZ **sem degradação**, dimensione o `min` para que as AZs restantes aguentem a carga sozinhas.

- **Integração com ELB:** o grupo registra e remove instâncias do target group automaticamente. O **deregistration delay** (connection draining) deixa as requisições em andamento terminarem antes do encerramento.
- **Mixed instances policy:** um grupo pode combinar vários tipos de instância e On-Demand + Spot. Veja "Spot Instances" em [EC2 — modelos de compra](#).
- **Estado fora da instância:** instâncias são descartáveis. Sessões ficam em ElastiCache/DynamoDB, arquivos em S3/EFS e dados em RDS/DynamoDB. Disco local e Instance Store se perdem no scale-in.

## Políticas de scaling

| Política                  | Como decide                                                                       | Quando usar                                                                  | Pegadinha                                                              |
|---------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Target tracking</b>    | Mantém uma métrica perto de um alvo (ex.: CPU em 50%) e cria os alarmes sozinha   | Padrão para a maioria das cargas variáveis; menor esforço operacional        | Reativa: não antecipa um pico conhecido                                |
| <b>Step scaling</b>       | Alarmes do CloudWatch com degraus (ex.: +2 se CPU > 70%, +4 se > 85%)             | Resposta proporcional ao tamanho da violação                                 | Mais configuração que target tracking                                  |
| <b>Simple scaling</b>     | Um alarme, um ajuste, depois espera o <b>cooldown</b>                             | Legado; raramente é a melhor resposta                                        | Não reage a novos alarmes durante o cooldown                           |
| <b>Scheduled scaling</b>  | Altera min/max/desired em data/hora ou cron                                       | Picos recorrentes e <b>fixos</b> (ex.: toda segunda às 8h)                   | Não se adapta se o pico mudar de horário                               |
| <b>Predictive scaling</b> | ML sobre até 14 dias de histórico; lança capacidade <b>antes</b> do pico previsto | Padrão diário/semanal recorrente, <b>com variação</b> , e bootstrap demorado | Precisa de histórico; comece no modo <i>forecast only</i> para validar |
| <b>Manual</b>             | Você altera o <code>desired</code>                                                | Testes e intervenções pontuais                                               | Não é elasticidade                                                     |

- **Combinação é permitida:** várias políticas podem coexistir. Quando conflitam, o Auto Scaling escolhe a que resulta em **maior capacidade**, tanto no scale-out quanto no scale-in. É comum usar **predictive + target tracking**: o preditivo antecipa a base e o target tracking corrige o imprevisto.
- **Scheduled vs. predictive:** se o padrão é fixo e 100% conhecido, scheduled basta e é mais simples. Se é recorrente mas varia em amplitude ou horário, predictive é a resposta.
- Ajustar apenas a **capacidade desejada** (não min/max) via ação agendada elimina a lentidão em horários de pico sem manter custo elevado o dia todo. Ajustar o `min` também funciona e impede que uma política dinâmica reduza abaixo do necessário durante o evento.

## Métricas de scaling

- **CPU média** (`CPUUtilization`): a mais comum, para cargas limitadas por CPU.
- `ALBRequestCountPerTarget`: requisições por instância no ALB. É a melhor métrica quando a carga é de requisições web cujo custo não aparece bem na CPU.
- **Memória e disco:** não são métricas padrão do EC2. Exigem o **CloudWatch agent** publicando métricas personalizadas.
- **Workers de fila SQS:** escale pela fila, não por CPU/rede. A métrica bruta `ApproximateNumberOfMessagesVisible` mede o backlog. Em target tracking, o indicador mais preciso é **backlog por instância** (mensagens visíveis ÷ instâncias em serviço), calculado com **metric math** ou métrica personalizada, com alvo = latência aceitável ÷ tempo médio de processamento por mensagem.
- **Métricas personalizadas:** qualquer métrica do CloudWatch pode guiar target tracking ou step scaling, desde que **aumente e diminua proporcionalmente** à capacidade.
- **Monitoramento detalhado** (1 minuto) reage mais rápido que o padrão de 5 minutos.



## Warmup, cooldown e warm pools

- **Instance warmup:** tempo em que uma instância nova ainda não conta nas métricas agregadas do grupo. Evita que o Auto Scaling lance instâncias demais enquanto as primeiras ainda inicializam. Configure o **default instance warmup** no grupo; ele vale para target tracking, step scaling e instance refresh.
- **Cooldown** (padrão de 300 s): aplica-se apenas a **simple scaling** e bloqueia novas ações após um ajuste. Target tracking e step scaling usam warmup, não cooldown.
- **Bootstrap lento:** se a instância leva muitos minutos para ficar pronta (instalar pacotes, carregar cache), as opções são:
  1. **AMI pré-configurada (golden AMI)**, criada com EC2 Image Builder, para reduzir o que o user data precisa fazer;
  2. **Warm pool:** instâncias pré-inicializadas mantidas `Stopped`, `Hibernated` ou `Running` fora do serviço. No scale-out, entram no grupo em segundos. Em estado `Stopped`, paga-se só o EBS;
  3. **Predictive scaling**, para lançar antes do pico.

## Health checks e substituição

- **EC2 health check** (padrão): verifica só o status da instância e do hardware. Uma aplicação travada que retorna erro 500 continua "saudável" para esse check.
- **ELB health check:** se o target group marca a instância como unhealthy, o Auto Scaling a substitui. **Precisa ser habilitado no grupo.** Ter o ALB associado não basta. É a resposta para "substituir instâncias em que a aplicação parou de responder".
- **Health checks personalizados:** a aplicação ou um script marca a instância como unhealthy via API `SetInstanceHealth`. O grupo também pode usar health checks do **VPC Lattice** e do **EBS** (volumes com I/O prejudicado).
- **Health check grace period:** tempo após o lançamento em que as falhas de health check são ignoradas. Se for curto demais, o grupo encerra instâncias que ainda estão inicializando e entra em loop de substituição.
- **Instância unhealthy é encerrada e substituída**, não reiniciada. Para reiniciar/recuperar a mesma instância (mantendo IP, EBS e ID), use **EC2 auto recovery** ou um alarme do CloudWatch com a ação `recover`, fora do Auto Scaling.

## Lifecycle hooks

- Pausam a instância em `Pending:Wait` (antes de entrar em serviço) ou `Terminating:Wait` (antes de ser encerrada) para executar uma ação personalizada.
- **Scale-out:** instalar software, baixar configuração, registrar em sistemas externos antes de receber tráfego.
- **Scale-in:** copiar logs para o S3, drenar jobs em andamento, remover o registro de DNS/licença, fazer snapshot.
- A notificação vai para **EventBridge**, SNS ou SQS, e normalmente dispara uma **Lambda** ou um documento do **Systems Manager Run Command**. A ação termina com `CompleteLifecycleAction`. O **heartbeat timeout** padrão é de 1 hora e pode ser estendido com `RecordLifecycleActionHeartbeat`, até 48 horas no total.
- É a resposta para "salvar os logs da instância antes que o Auto Scaling a encerre".

## Scale-in: políticas de terminação e proteção

- **Política de terminação padrão:** primeiro escolhe a AZ com mais instâncias (mantém o equilíbrio). Depois, segundo a estratégia de alocação (On-Demand vs. Spot). Depois, a instância com o **launch template/configuration mais antigo**. Por fim, a mais próxima da próxima hora de cobrança.
- **Outras políticas:** `OldestInstance`, `NewestInstance`, `OldestLaunchTemplate`, `ClosestToNextInstanceHour` ou uma **política personalizada em Lambda**.

- **Instance scale-in protection:** impede que o scale-in encerre instâncias específicas, por exemplo as que processam um job longo. **Não** protege contra encerramento por falha de health check nem contra encerramento manual.
- **Standby:** retira uma instância do serviço temporariamente, para troubleshooting ou atualização, sem que o grupo a substitua. Ela continua contando no `desired`.
- **Suspender processos:** `Launch`, `Terminate`, `HealthCheck`, `ReplaceUnhealthy`, `AZRebalance`, `AlarmNotification`, `ScheduledActions` e `AddToLoadBalancer` podem ser suspensos. Ex.: suspender `ReplaceUnhealthy` para investigar uma instância com falha sem que ela seja encerrada.

## Atualização e manutenção do grupo

- **Instance refresh:** substitui as instâncias de forma gradual após uma nova versão do launch template (nova AMI, novo tipo), respeitando um **percentual mínimo saudável**. Suporta checkpoints, pular instâncias já atualizadas e **rollback automático** se os alarmes do CloudWatch dispararem.
- **Maximum instance lifetime:** substitui automaticamente instâncias mais antigas que o limite configurado (mínimo de 1 dia). Útil para forçar a aplicação de patches e evitar drift de configuração.
- **Blue/green:** para trocar tudo de uma vez com retorno rápido, crie um novo grupo atrás do mesmo ALB e desloque o tráfego, ou use CodeDeploy.
- **Capacity Rebalancing (Spot):** lança uma substituição assim que chega uma rebalance recommendation, antes da interrupção de 2 minutos.
- **AWS Auto Scaling** (o serviço "guarda-chuva") gerencia planos de scaling de vários recursos ao mesmo tempo (EC2, ECS, DynamoDB, Aurora). **Application Auto Scaling** escala recursos que não são EC2 (ECS services, DynamoDB, Aurora Replicas, Lambda provisioned concurrency, SageMaker endpoints). Veja a seção 9.

## Padrões e pegadinhas de Auto Scaling

- "Instâncias com a aplicação travada continuam no grupo": habilite o **ELB health check** no Auto Scaling group.
- "Pico toda segunda às 9h": **scheduled scaling**. "Pico diário recorrente com variação e instâncias demoram a iniciar": **predictive scaling** (e/ou **warm pool**).
- "Tráfego imprevisível, menor esforço operacional": **target tracking**.
- "Escalar workers pela fila": **backlog por instância** do SQS, não CPU.
- "Salvar logs antes do encerramento" / "executar script antes de entrar em serviço": **lifecycle hook**.
- "Instâncias novas são encerradas logo após o lançamento": **health check grace period** curto demais.
- "Escala demais durante o pico": falta de **instance warmup** ou métrica que não é proporcional à carga.
- "Resistir à perda de uma AZ sem perder desempenho": grupo em várias AZs com capacidade mínima suficiente em cada uma, atrás de um ALB entre AZs.
- "Atualizar a AMI de todas as instâncias sem downtime": **instance refresh**.
- "Scheduled scaling não atingiu a capacidade esperada": o `max` do grupo está limitando.
- Referência: [Amazon EC2 Auto Scaling User Guide](#).

## Inicialização, hibernação e armazenamento local da EC2

**Regra de ouro da prova:** separe o que **automatiza** a inicialização (AMI e user data), o que **informa** a instância sobre si mesma (instance metadata) e o que **preserva** estado entre paradas (EBS e hibernate). Instance Store é rápido, mas **efêmero**: nunca é a resposta quando o dado precisa sobreviver a um stop.

## AMI (Amazon Machine Image)

- Modelo de lançamento: sistema operacional, software, configuração e o mapeamento de volumes (block device mapping). Uma AMI baseada em EBS inclui **snapshots dos volumes EBS**. O conteúdo de Instance Store não entra na AMI.
- **AMI é regional**: para lançar em outra Região (DR, expansão), **copie a AMI** para lá. A cópia pode trocar a chave KMS.
- **Golden AMI**: software já instalado e configurado reduz o tempo de boot e o trabalho do user data. **EC2 Image Builder** automatiza a criação, os testes, a aplicação de patches e a distribuição entre Regiões e contas.
- **Equilíbrio AMI vs. user data**: o que muda pouco (pacotes, agentes) fica na AMI. O que muda por ambiente ou por instância (endpoints, flags, registro em serviço) fica no user data ou no Parameter Store.
- Compartilhamento de AMI criptografada entre contas: veja a seção 5.

## User data

- Shell scripts ou diretivas `cloud-init` (Linux), ou scripts PowerShell/batch via EC2Launch (Windows), fornecidos no launch. No Linux executam como `root` e, **por padrão, somente no primeiro boot**. É possível configurar execução a cada boot, mas isso não é o padrão.
- **Limite de 16 KB** (antes da codificação base64). Scripts maiores devem baixar o restante do S3 ou usar o Systems Manager.
- Para alterar o user data, a instância precisa estar **parada**. Mudar o user data de uma instância já iniciada não reexecuta o script.
- **Não coloque segredos no user data**: ele fica legível para qualquer processo da instância via metadata e para quem tem `ec2:DescribeInstanceAttribute`. Busque segredos no **Secrets Manager** ou no **Parameter Store** usando a IAM role da instância.
- Troubleshooting no Linux: `/var/log/cloud-init-output.log`.
- Para configuração contínua após o boot (patches, comandos em frota), use **Systems Manager** (Run Command, State Manager), não user data.

## Instance metadata (IMDS)

- Informações sobre a própria instância (ID, tipo, AZ, IPs, IAM role, user data, tags opcionais, aviso de interrupção Spot), acessadas em `http://169.254.169.254/latest/meta-data/`. **Metadata não executa scripts**: é o user data que automatiza a inicialização.
- **Credenciais temporárias da IAM role** chegam à instância pelo IMDS através do **instance profile**. Por isso, **nunca guarde access keys** na instância: associe uma role.
- **IMDSv2** (orientado a sessão): exige obter um token via `PUT` antes de ler os dados. Protege contra **SSRF** e proxies mal configurados que exporiam as credenciais da role. **Exija IMDSv2** (`HttpTokens=required`) no launch template, como padrão da conta ou via SCP com a condition key `ec2:MetadataHttpTokens`.
- **Hop limit**: o padrão de 1 bloqueia o acesso de contêineres que estão atrás de uma camada extra de rede. Use 2 quando contêineres na instância precisam do IMDS.
- O IMDS pode ser **desabilitado** por completo quando a aplicação não precisa dele.

## Ciclo de vida: reboot, stop, hibernate e terminate

| Ação   | RAM     | Instance Store | Volumes EBS | IPv4 público | IP privado / Elastic IP | Host físico | Cobrança de computação |
|--------|---------|----------------|-------------|--------------|-------------------------|-------------|------------------------|
| Reboot | Perdida | Preservado     | Preservados | Mantido      | Mantidos                | Mesmo       | Continua               |



| Ação              | RAM               | Instance Store | Volumes EBS                                     | IPv4 público | IP privado / Elastic IP                   | Host físico     | Cobrança de computação                          |
|-------------------|-------------------|----------------|-------------------------------------------------|--------------|-------------------------------------------|-----------------|-------------------------------------------------|
| Stop / start      | Perdida           | Apagado        | Preservados                                     | Muda         | Mantidos                                  | Geralmente muda | Para (EBS e Elastic IP continuam cobrados)      |
| Hibernate / start | Salva no EBS root | Apagado        | Preservados                                     | Muda         | Mantidos                                  | Geralmente muda | Para após hibernar (EBS e Elastic IP continuam) |
| Terminate         | Perdida           | Apagado        | Root apagado por padrão; adicionais preservados | Liberado     | Privado liberado; Elastic IP desassociado | —               | Encerra                                         |

- **Stop/start move a instância para outro host.** Isso resolve um problema de hardware subjacente ou uma instância agendada para *retirement*. Um reboot não resolve, porque mantém o host.
- **IP público estável:** associe um **Elastic IP**. O IPv4 público automático muda a cada stop/start. Hoje todo IPv4 público é cobrado, inclusive o Elastic IP em uso.
- `DeleteOnTermination`: o padrão é `true` para o volume root e `false` para os volumes EBS adicionais. Para manter o root após o terminate, altere esse atributo.
- **Termination protection** (`DisableApiTermination`) impede o terminate pelo console/API. Ela **não** impede o encerramento por scale-in do Auto Scaling, a interrupção de Spot, nem um shutdown do sistema operacional quando o comportamento de desligamento (`InstanceInitiatedShutdownBehavior`) está como `terminate`. Existe também a **stop protection**.
- **EC2 auto recovery:** se a verificação de status do **sistema** falhar, a instância é recuperada em outro host mantendo ID, IPs, Elastic IP, EBS e metadata. É o padrão em tipos suportados e também pode ser feito por alarme do CloudWatch com a ação *recover*. Não se aplica a instâncias com Instance Store.

## Hibernate

- Salva o conteúdo da **RAM no volume root EBS** e, no próximo start, restaura memória, processos e caches. É indicado quando o bootstrap ou o aquecimento de cache é demorado e precisa ser **retomado, não reexecutado**.
- **Pré-requisitos:**
  - habilitar **no launch**, porque não é possível ativar depois;
  - volume root **EBS criptografado** e grande o bastante para a RAM;
  - RAM de até **150 GiB** (Linux) ou **16 GiB** (Windows);
  - família de instância e AMI suportadas.
- Uma instância pode ficar hibernada por no máximo **60 dias**. Funciona para On-Demand, Reserved e Spot. Não funciona com root em Instance Store nem em instâncias bare metal.
- Durante a hibernação, paga-se o armazenamento EBS (incluindo a RAM gravada) e o Elastic IP, não a computação.
- **Alternativa no Auto Scaling:** warm pool com instâncias em estado `Hibernated`. Veja [Auto Scaling](#).

## Instance Store

- Block storage temporário em discos **fisicamente ligados ao host**, muitas vezes NVMe. Oferece IOPS e throughput muito altos, com latência mínima, e o custo já está incluído no preço da instância. A quantidade e o tamanho são fixos pelo tipo de instância (ex.: famílias `i`, `d` e variantes com sufixo `d`, como `m6id`).
- **Reboot preserva** os dados. **Stop, hibernate, terminate e falha do disco ou do host apagam** os dados.
- Não pode ser destacado e anexado a outra instância, não tem snapshot, e seu conteúdo **não é incluído em uma AMI**. Para proteger os dados, replique entre instâncias ou copie para EBS/S3.

- **Casos de uso:** buffers, caches, arquivos temporários (scratch) de HPC/renderização, spill de processamento, e dados **replicados pela aplicação** entre nós (ex.: Cassandra, Kafka, Elasticsearch com réplicas), em que perder um nó não perde dados.

| Critério          | Instance Store                   | Amazon EBS                                       |
|-------------------|----------------------------------|--------------------------------------------------|
| Persistência      | Até stop/terminate/falha do host | Independente do ciclo de vida da instância       |
| Desempenho        | Máximo (disco local)             | Alto, ajustável (gp3/io2), via rede              |
| Snapshot / backup | Não                              | Snapshots incrementais no S3, AWS Backup         |
| Desanexar / mover | Não                              | Sim (na mesma AZ); Multi-Attach em io1/io2       |
| Tamanho           | Fixo pelo tipo                   | Escolhido e ampliável                            |
| Custo             | Incluído na instância            | Cobrado por GB (e IOPS/throughput provisionados) |

## Padrões e pegadinhas de inicialização

- "Instalar software automaticamente ao lançar a instância": **user data** (ou golden AMI para reduzir o tempo de boot).
- "Script deve rodar a cada reinício": user data **não** faz isso por padrão. Configure `cloud-init` para cada boot ou use o **Systems Manager State Manager**.
- "Aplicação precisa saber sua AZ/ID/IP em tempo de execução": **instance metadata**.
- "Aplicação na EC2 precisa acessar o S3 sem chaves": **IAM role via instance profile**, com credenciais entregues pelo IMDS.
- "Mitigar SSRF que roubou credenciais da role": **exigir IMDSv2**.
- "Aplicação leva 20 minutos para carregar dados em memória e precisa retomar rápido após parar": **hibernate**.
- "Maior IOPS possível para dados temporários ou replicados": **Instance Store**. "Dados precisam sobreviver a stop/start": **EBS**.
- "Instância com hardware degradado ou agendada para retirement": **stop/start** (muda o host), não reboot.
- "Precisa manter o mesmo IP público após stop/start": **Elastic IP**.
- "Lançar a mesma configuração em outra Região": **copiar a AMI**.
- Referências: [User data](#), [IMDS](#), [Hibernate](#), [Instance Store](#).

## Placement Groups

**Regra de ouro da prova:** placement group controla **onde** as instâncias ficam no hardware físico. **Cluster** aproxima as instâncias para desempenho de rede e  **aumenta** o risco de falha correlacionada. **Spread** e **Partition** afastam as instâncias para **reduzir** esse risco. Se a questão fala em "latência mínima entre nós", a resposta é Cluster. Se fala em "não podem falhar juntas", a resposta é Spread ou Partition, conforme a escala.

## Comparativo das estratégias

| Estratégia     | Onde ficam as instâncias                                     | Escopo              | Limite principal                       | Caso típico                                                                                             |
|----------------|--------------------------------------------------------------|---------------------|----------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Cluster</b> | Próximas, no mesmo segmento de rede de alta largura de banda | <b>Uma única AZ</b> | Capacidade disponível naquele segmento | HPC fortemente acoplado (MPI), treinamento de ML distribuído, bancos em memória com replicação síncrona |

| Estratégia       | Onde ficam as instâncias                                                                            | Escopo                      | Limite principal                                                    | Caso típico                                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------|-----------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Partition</b> | Divididas em partições; cada partição em racks próprios, sem compartilhar rack com outras partições | Uma ou várias AZs da Região | Até <b>7 partições por AZ</b> ; centenas de instâncias por partição | Big data e bancos distribuídos com replicação por rack: HDFS, HBase, Cassandra, Kafka                             |
| <b>Spread</b>    | Cada instância em um rack distinto, com rede e energia próprias                                     | Uma ou várias AZs da Região | Até <b>7 instâncias em execução por AZ</b> por grupo                | Poucas instâncias críticas que não podem falhar juntas: nós de quórum, primário/secundário, servidores de licença |

*Placement groups não têm custo adicional. O custo aparece indiretamente: Cluster concentra risco em uma AZ, e Spread limita a escala.*

## Cluster

- Instâncias no mesmo segmento de rede com alta bisseção de largura de banda dentro de **uma AZ**. Oferece a **menor latência** e o **maior throughput** entre instâncias. O tráfego entre instâncias do grupo pode usar até **10 Gbps por fluxo** (fora do grupo, o limite por fluxo é menor) e **jumbo frames** (MTU 9001).
- **Não é alta disponibilidade**: falha de rack ou de AZ pode derrubar todo o grupo. Para HA, replique entre grupos em AZs diferentes ou aceite o risco em cargas reiniciáveis com checkpoint.
- **Evitar** `InsufficientInstanceCapacity`:
  - lance **todas as instâncias em uma única requisição**;
  - use o **mesmo tipo de instância**;
  - se a capacidade faltar, **pare e inicie todas** as instâncias do grupo para realocá-las juntas;
  - para garantir a capacidade antecipadamente, use **On-Demand Capacity Reservations** associadas ao cluster placement group. Veja [Reservas de capacidade](#).
- Funciona melhor com instâncias com **enhanced networking** e, para HPC/ML, com **EFA**.

## Partition

- O grupo é dividido em partições lógicas, e **nenhuma partição compartilha rack** com outra. A falha de um rack afeta apenas uma partição.
- **Até 7 partições por AZ**. O grupo pode abranger várias AZs da mesma Região e comportar centenas de instâncias, o que o diferencia do Spread.
- A aplicação descobre em qual partição cada instância está pelo **instance metadata** (`placement/partition-number`). Isso permite que sistemas **topology-aware** (HDFS, HBase, Cassandra, Kafka) coloquem réplicas em partições diferentes.
- Você pode lançar instâncias em uma partição específica ou deixar a EC2 distribuí-las uniformemente.

## Spread

- Cada instância fica em um **rack distinto**, com fonte de energia e rede próprias. A falha de hardware afeta no máximo uma instância.
- **Limite de 7 instâncias em execução por AZ** por grupo. Ex.: um grupo em 3 AZs comporta até 21 instâncias. Para mais instâncias isoladas por falha, use **Partition** (isola por grupos de racks) ou vários spread groups.
- Em **AWS Outposts**, existe também o spread em nível de **host**.
- Combinado com várias AZs, protege contra falha de hardware e de AZ. É indicado para os poucos nós que sustentam uma aplicação (ex.: 3 ou 5 nós de quórum).

## Regras operacionais

- Uma instância pertence a **no máximo um** placement group. Não é possível mesclar grupos.
- Para **mover** uma instância existente para um placement group, removê-la dele ou trocar de grupo, a instância precisa estar **parada**. Depois, altere o placement pela CLI/API (`modify-instance-placement`) e inicie de novo.
- **Auto Scaling groups** e **launch templates** podem especificar o placement group. Um ASG com um Cluster placement group fica restrito a uma AZ.
- **Dedicated Hosts** não podem ser lançados em placement groups.
- O **nome do grupo** é único na conta e na Região. A estratégia não pode ser alterada depois de criado.

## Rede de alto desempenho: ENA e EFA

- **Enhanced networking com ENA (Elastic Network Adapter)**: usa SR-IOV para mais largura de banda, mais pacotes por segundo e menor latência, sem custo adicional. Vem habilitado na maioria dos tipos atuais e é o padrão para throughput alto em cargas TCP/IP comuns.
- **Elastic Fabric Adapter (EFA)**: interface de rede para **HPC e machine learning fortemente acoplados**. Inclui todos os recursos do ENA e adiciona **OS-bypass**: a aplicação comunica diretamente do user space com o hardware, via **libfabric** (usada por MPI e NCCL). É a escolha quando a questão exige máxima performance de comunicação entre nós.
- **Restrições do EFA**:
  - o tráfego OS-bypass **não é roteável**: os nós precisam estar na **mesma subnet** (e, na prática, num Cluster placement group);
  - o security group precisa permitir **todo o tráfego de entrada e saída entre os membros do próprio grupo** (regra autorreferente);
  - disponível apenas em tipos de instância específicos, e o OS-bypass depende de suporte do sistema operacional (amplo em Linux).
- **ENA Express**: aumenta a largura de banda por fluxo e reduz a latência de cauda para tráfego TCP entre instâncias na mesma AZ, sem mudar a aplicação.

## Padrões e pegadinhas de Placement Groups

- "HPC com comunicação intensa entre nós / menor latência de rede possível": **Cluster placement group + EFA**.
- "Hadoop/Cassandra/Kafka com réplicas em racks diferentes": **Partition**.
- "Poucas instâncias críticas que não podem sofrer falha simultânea de hardware": **Spread**, idealmente em várias AZs.
- "Precisa isolar 50 instâncias em hardware distinto": Spread não comporta, por causa do limite de 7 por AZ. A resposta é **Partition**.
- "Cluster placement group com erro de capacidade ao adicionar instâncias": lance tudo de uma vez com o mesmo tipo, pare e inicie o grupo, ou reserve capacidade com **ODCR**.
- "Cluster placement group resolve alta disponibilidade?" **Não**: concentra tudo em uma AZ.
- "ENA ou EFA?": ENA para throughput alto em TCP/IP comum. **EFA** para MPI/NCCL com OS-bypass.
- Referências: [Placement groups](#), [EFA](#).

## Contêineres

**Regra de ouro da prova:** separe duas decisões. **Qual orquestrador:** ECS (nativo da AWS, mais simples) ou EKS (Kubernetes, portabilidade e ecossistema). **Onde os contêineres rodam:** EC2 (você gerencia instâncias, tem controle total e GPU) ou Fargate (serverless, sem servidores). "Menor esforço operacional" costuma levar a **ECS + Fargate**. "Já usa Kubernetes / manifests existentes / multicloud" leva a **EKS**.

### Escolhendo a plataforma

| Opção                                                     | O que você gerencia                         | Quando usar                                                                                   | Quando não usar                                              |
|-----------------------------------------------------------|---------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>ECS + Fargate</b>                                      | Task definition e serviço                   | Microserviços e APIs com menor operação; sem necessidade de acesso ao host                    | GPU, contêineres privilegiados, controle do kernel/host      |
| <b>ECS + EC2</b>                                          | Instâncias (AMI, patch, capacidade) + tasks | GPU, tipos de instância específicos, Spot/RI por instância, alta densidade                    | Quando a equipe quer evitar gerenciar servidores             |
| <b>EKS</b> (node groups, Karpenter, Auto Mode ou Fargate) | Workloads Kubernetes; nós conforme a opção  | Equipe e ferramentas Kubernetes, migração de clusters sem reescrever manifests, portabilidade | App simples em que o Kubernetes é complexidade desnecessária |
| <b>App Runner</b>                                         | Código ou imagem                            | Serviço web/API HTTP simples, com build, deploy, balanceamento e scaling gerenciados          | Processamento de eventos, jobs, controle de orquestração     |
| <b>Lambda (imagem de contêiner)</b>                       | Função empacotada como imagem (até 10 GB)   | Eventos curtos (até 15 min) com dependências grandes                                          | Processos longos ou servidores sempre ativos                 |
| <b>AWS Batch</b>                                          | Definição de jobs                           | Filas de jobs em lote em contêiner, sobre EC2/Spot/Fargate                                    | Serviços de longa duração. Veja <a href="#">AWS Batch</a>    |

### Amazon ECS — conceitos

- **Cluster:** agrupamento lógico de capacidade. **Task definition:** especificação versionada (imagem, CPU, memória, portas, variáveis, roles, logs, volumes). **Task:** instância em execução de uma task definition. **Service:** mantém N tasks em execução, substitui as que falham, integra com ELB e aplica o scaling. Tasks avulsas (RunTask) servem para jobs.
- **Capacity providers:** definem onde as tasks rodam: **Fargate**, **Fargate Spot** ou um **Auto Scaling group de EC2** com *managed scaling*, em que o ECS escala as instâncias conforme a demanda das tasks. Uma estratégia pode combinar Fargate (base) e Fargate Spot (restante), como no mix On-Demand + Spot da EC2.
- **ECS Managed Instances:** opção em que a AWS provisiona e gerencia instâncias EC2 para o cluster (patching, escolha de tipo), mantendo acesso a recursos de EC2 como GPU.
- **ECS Anywhere:** estende a orquestração ECS para servidores on-premises, com gestão centralizada híbrida via IAM.

### IAM em contêineres

- **Task role** (`taskRoleArn`): permissões que **o código da aplicação** usa para chamar outros serviços (S3, DynamoDB, SQS). É o equivalente, por task, à role de uma instância EC2. Nunca use security group, usuário IAM ou chaves no código.
- **Task execution role** (`executionRoleArn`): permissões que **o agente ECS/Fargate** usa para iniciar a task: puxar a imagem do ECR, enviar logs ao CloudWatch e buscar segredos do Secrets Manager/Parameter Store.

- **Instance role** (ECS em EC2): usada pelo agente ECS da instância para se registrar no cluster. Não deve ser usada para dar permissões à aplicação, porque todas as tasks da instância herdariam as mesmas permissões.
- **EKS**: permissões por pod via **EKS Pod Identity** ou **IRSA** (IAM Roles for Service Accounts, com OIDC). Isso evita dar a todos os pods a role do nó.
- **Segredos**: referencie o Secrets Manager ou o Parameter Store no campo `secrets` da task definition (ou via Secrets Store CSI Driver no EKS), em vez de variáveis de ambiente em texto puro.

## Rede, balanceamento e armazenamento

- **Modo `awsvpc`**: cada task recebe sua própria **ENI e IP privado** na VPC, com **security group por task**. É obrigatório no Fargate e recomendado em EC2. O target group do ALB/NLB usa o tipo `ip`.
- **Modo `bridge`** (EC2): com **mapeamento dinâmico de portas**, várias tasks do mesmo serviço rodam na mesma instância, e o ALB roteia para cada porta. O target type é `instance`.
- **ALB** para HTTP/HTTPS com roteamento por path/host entre microserviços. **NLB** para TCP/UDP ou IP estático.
- **Service discovery**: **ECS Service Connect** ou **AWS Cloud Map** (DNS/API) para que serviços encontrem uns aos outros sem um balanceador para cada chamada interna.
- **Subnets privadas sem NAT**: para puxar imagens do ECR, crie **VPC endpoints de interface** `ecr.api` e `ecr.dkr`, um **gateway endpoint do S3** (as camadas da imagem ficam no S3) e um endpoint do CloudWatch Logs.
- **EKS e IPs**: o **VPC CNI** atribui um IP da VPC a cada pod, o que pode **esgotar as subnets**. As mitigações são *prefix delegation*, CIDR secundário com *custom networking* ou subnets maiores.
- **Armazenamento**:
  - o armazenamento do contêiner é **efêmero** (Fargate tem 20 GiB por padrão, ampliável até 200 GiB);
  - para dados persistentes e **compartilhados entre tasks/pods** e AZs, use **Amazon EFS** (suportado no Fargate);
  - volumes **EBS** podem ser anexados a tasks ECS e a pods EKS (via EBS CSI driver), mas ficam restritos a uma AZ;
  - estado de aplicação fica fora do contêiner (RDS, DynamoDB, ElastiCache, S3).

## Scaling, deploy e observabilidade

- **ECS Service Auto Scaling** (via Application Auto Scaling): target tracking em CPU, memória, `ALBRequestCountPerTarget` ou métrica personalizada (ex.: backlog SQS por task). Também suporta step e scheduled scaling. No ECS em EC2, o **capacity provider com managed scaling** escala as instâncias para caber as tasks.
- **EKS**: o **Horizontal Pod Autoscaler** escala pods. O **Karpenter** (recomendado) ou o Cluster Autoscaler escalam os nós. O **EKS Auto Mode** delega à AWS o gerenciamento de nós, scaling, rede e armazenamento do cluster.
- **Deploy no ECS**:
  - **rolling update** com `minimumHealthyPercent / maximumPercent`;
  - **deployment circuit breaker**, que faz rollback automático se as novas tasks não estabilizam;
  - **blue/green** nativo do ECS ou via CodeDeploy, com troca de tráfego no ALB e retorno rápido.
- **Logs e métricas**: driver `awslogs` para o CloudWatch Logs ou **FireLens** (Fluent Bit) para outros destinos. **CloudWatch Container Insights** mostra métricas de cluster, serviço, task e pod.
- **Alta disponibilidade**: distribua tasks e nós em **várias AZs** atrás de um load balancer. O serviço ECS já espalha tasks entre AZs por padrão. O control plane do EKS é gerenciado e multi-AZ. Multi-AZ na mesma Região já garante disponibilidade suficiente na maioria dos casos. Multi-região com Route 53 fica reservado para DR geográfico explícito.



## AWS Fargate

- Modo serverless de execução para ECS e EKS: sem instâncias para provisionar, corrigir ou escalar. Cada task/pod roda em um **ambiente isolado** (kernel próprio), o que é uma vantagem de segurança.
- **Cobrança por vCPU e memória por segundo** (mínimo de 1 minuto), pela configuração da task. Coberto por **Compute Savings Plans**.
- **Fargate Spot**: desconto de até ~70% para tasks tolerantes a interrupção, com aviso de 2 minutos (sinal `SIGTERM`). Disponível no ECS.
- **Limitações que decidem questões**:
  - sem **GPU**;
  - sem contêineres **privilegiados** e sem acesso ao host (SSH), mas é possível usar **ECS Exec** para acessar o contêiner;
  - no EKS, não roda **DaemonSets**;
  - há limites máximos de vCPU/memória por task. Quando qualquer um desses requisitos aparece, a resposta é **EC2** como capacidade.

## Amazon EKS

- Kubernetes gerenciado e certificado: **migra clusters on-premises sem alterar manifests**. O control plane é gerenciado pela AWS, multi-AZ e cobrado por hora por cluster.
- **Opções de nós**:
  - **Managed node groups**: a AWS gerencia o ASG e as atualizações dos nós;
  - **Self-managed nodes**: controle total;
  - **Fargate profiles**: pods serverless;
  - **Karpenter**: provisionamento just-in-time, com diversificação de tipos e Spot;
  - **EKS Auto Mode**: a AWS opera o data plane.
- **AWS Load Balancer Controller**: cria **ALB** para objetos `Ingress` e **NLB** para `Service` do tipo LoadBalancer.
- **Add-ons**: VPC CNI, CoreDNS, kube-proxy, EBS/EFS CSI drivers, gerenciados como EKS add-ons.
- **Amazon EKS Anywhere / EKS Distro**:
  - **EKS Anywhere** permite criar clusters Kubernetes em infraestrutura on-premises ou de borda. É **gerenciado pelo cliente**: a operação, a manutenção e o ciclo de vida do cluster ficam sob sua responsabilidade. Uma assinatura Enterprise oferece suporte comercial opcional.
  - **EKS Distro** é a distribuição Kubernetes open-source usada pelo EKS e pode ser executada em infraestrutura própria, sem que a AWS opere o cluster.
  - **EKS Hybrid Nodes** conecta servidores on-premises como nós de um cluster EKS com control plane na AWS.
  - [Opções de implantação](#).

## Amazon ECR

- Registry gerenciado de imagens OCI/Docker, **privado** (por conta e Região) ou **público** (ECR Public). Imagens são criptografadas em repouso (AES-256 ou **KMS**).
- **Isolamento**: repositórios separados por ambiente (dev/test/prod), com IAM roles e **repository policies** específicas por repositório. É o padrão de isolamento e controle de custo/segurança. Repository policies também permitem pull **entre contas**.
- **Image scanning**: *basic* (CVEs do sistema operacional, no push) ou *enhanced* com **Amazon Inspector** (contínuo, inclui pacotes de linguagem).

- **Lifecycle policies:** expiram imagens antigas ou sem tag automaticamente e reduzem o custo de armazenamento.
- **Tag immutability:** impede sobrescrever uma tag existente (ex.: `v1.2.0`), garantindo que o deploy use exatamente a imagem testada.
- **Replicação** cross-Region e cross-account para DR e latência de pull. **Pull through cache** espelha registries públicos (Docker Hub, ECR Public, Quay) para reduzir a dependência externa e os limites de taxa.

### Outras opções de execução

- **AWS App Runner:** executa aplicações web e APIs diretamente de código-fonte ou imagem de contêiner, com build/deploy, load balancing e scaling gerenciados. É adequado para um serviço HTTP simples com pouca operação. Não é um mecanismo geral de processamento de eventos S3 nem substitui ECS/EKS quando se precisa de controle de orquestração.
- **Elastic Beanstalk (plataforma Docker):** PaaS que orquestra EC2 + ASG + ELB para um contêiner. Veja [Elastic Beanstalk \(PaaS\)](#).
- **AWS ParallelCluster:** clusters de HPC (Slurm) sobre EC2. Não serve para aplicações web containerizadas comuns.
- **Amazon Lightsail containers:** opção simplificada com preço fixo, para projetos pequenos. Raramente é a resposta em arquitetura corporativa.

### Padrões e pegadinhas de contêineres

- "Executar contêineres sem gerenciar servidores, menor esforço operacional": **ECS com Fargate**.
- "Empresa já usa Kubernetes on-premises e quer migrar sem reescrever": **EKS**.
- "Contêiner precisa de GPU / modo privilegiado / DaemonSet": **EC2** como capacidade (ECS em EC2 ou nós EKS), não Fargate.
- "A aplicação no contêiner precisa acessar o DynamoDB": **task role** (ECS) ou **Pod Identity/IRSA** (EKS). "A task não consegue puxar a imagem nem enviar logs": **task execution role**.
- "Cada task precisa de seu próprio security group": modo de rede `awsvpc`.
- "Tasks em subnet privada sem NAT não puxam imagens": **VPC endpoints** `ecr.api`, `ecr.dkr` + **gateway endpoint do S3**.
- "Armazenamento persistente compartilhado entre tasks Fargate em várias AZs": **Amazon EFS**.
- "Reduzir custo de tasks tolerantes a interrupção": **Fargate Spot** (ou EC2 Spot via capacity provider) + **Compute Savings Plans** para a base.
- "Escalar workers de fila em ECS": Service Auto Scaling com **backlog SQS por task**.
- "Garantir que só imagens sem vulnerabilidades críticas cheguem à produção": **ECR image scanning** (enhanced/Inspector) no pipeline + **tag immutability**.
- "Deploy com rollback automático se a nova versão falhar": **deployment circuit breaker** ou **blue/green**.
- "Pods EKS esgotando IPs da subnet": **prefix delegation** ou CIDR secundário (custom networking).
- Referências: [Amazon ECS](#), [Amazon EKS](#), [AWS Fargate](#), [Amazon ECR](#).

## AWS Batch

**Regra de ouro da prova:** AWS Batch é a resposta para **filas de jobs em lote independentes** (scripts ou contêineres) que precisam de capacidade provisionada sob demanda, sem gerenciar cluster nem agendador. Se o job dura menos de 15 minutos e é disparado por evento, compare com **Lambda**. Se é processamento

distribuído com Spark/Hadoop, compare com **EMR** ou **Glue**. Se é HPC com agendador Slurm, compare com **ParallelCluster**.

## Componentes

- **Job definition:** imagem de contêiner, comando, vCPU, memória, GPU, IAM role, variáveis, volumes e estratégia de retry.
- **Job queue:** fila com **prioridade**. Várias filas podem compartilhar os mesmos compute environments, e a de maior prioridade é atendida primeiro. **Fair-share scheduling policies** dividem a capacidade entre equipes ou projetos.
- **Compute environment:** onde os jobs rodam.
  - **Managed:** a AWS escala instâncias entre `minvCpus` e `maxvCpus`. Com `minvCpus = 0`, não há custo quando a fila está vazia.
  - **Opções de capacidade:** EC2 On-Demand, **EC2 Spot**, **Fargate**, Fargate Spot ou um cluster **EKS**.
  - **Unmanaged:** você gerencia as instâncias.
- **Sem custo adicional:** paga-se apenas pelos recursos (EC2, Fargate, EBS) usados pelos jobs.

## Tipos de job

- **Job simples:** um contêiner executando até terminar.
- **Array jobs:** um único envio gera até **10.000 jobs filhos** com o mesmo definition, diferenciados por um índice (`AWS_BATCH_JOB_ARRAY_INDEX`). É o padrão para processar milhares de arquivos, imagens ou simulações em paralelo.
- **Dependências entre jobs:** um job só começa quando outros terminam (`dependsOn`), permitindo pipelines simples. Para fluxos com ramificação, tratamento de erro e etapas em outros serviços, use **Step Functions**, que tem integração nativa com o Batch.
- **Multi-node parallel jobs:** um job abrange várias instâncias EC2 que se comunicam (MPI), com suporte a **EFA** e Cluster placement group. É indicado para HPC fortemente acoplado sem manter um cluster permanente. Não roda em Fargate.

## Escolha da capacidade

- **EC2 Spot:** menor custo para jobs tolerantes a interrupção. Use a estratégia `SPOT_PRICE_CAPACITY_OPTIMIZED`, vários tipos de instância e **retry strategy** para reenviar o job interrompido. Checkpoints no S3 evitam recomeçar do zero.
- **EC2 On-Demand:** jobs que não podem ser interrompidos, **GPU**, instâncias grandes ou tipos específicos. `BEST_FIT_PROGRESSIVE` equilibra custo e disponibilidade.
- **Fargate:** inicialização mais rápida e sem instâncias para gerenciar, para jobs pequenos ou médios. Não suporta GPU nem multi-node parallel.
- **Disparo:** agendamentos do **EventBridge Scheduler**, eventos do S3 via EventBridge, ou Step Functions chamando `SubmitJob`. Os estados dos jobs geram eventos no EventBridge para notificação.

## AWS Batch vs. alternativas

| Requisito                                                               | Resposta         | Por que não as outras                                                         |
|-------------------------------------------------------------------------|------------------|-------------------------------------------------------------------------------|
| Jobs longos (> 15 min) em contêiner, em fila, com capacidade automática | <b>AWS Batch</b> | Lambda tem limite de 15 min. ECS exige que você orquestre a fila e o scaling. |

| Requisito                                                    | Resposta                                       | Por que não as outras                                              |
|--------------------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------|
| Evento curto, leve e altamente paralelo                      | <b>Lambda</b>                                  | Batch tem overhead de agendamento e provisionamento.               |
| Spark/Hadoop sobre grandes volumes                           | <b>EMR</b> ou <b>Glue</b>                      | Batch não distribui um único processamento entre nós como o Spark. |
| Cluster HPC com Slurm e ferramentas científicas tradicionais | <b>ParallelCluster</b>                         | Batch usa seu próprio agendador baseado em filas.                  |
| Processar milhões de objetos S3 com orquestração serverless  | <b>Step Functions Distributed Map</b> + Lambda | Batch é melhor quando cada item precisa de mais recursos ou tempo. |

- Referência: [AWS Batch User Guide](#).

## Serverless

**Regra de ouro da prova:** serverless significa **sem servidores para provisionar, escalar automaticamente (inclusive até zero) e pagar pelo uso**. Nas questões de "menor esforço operacional", a combinação típica é **API Gateway + Lambda + DynamoDB/S3**, com **SQS/SNS/EventBridge** para desacoplar e **Step Functions** para orquestrar. Ela perde quando há execução acima de 15 minutos, conexões de longa duração, GPU ou carga constante alta, em que contêineres ou EC2 podem custar menos.

### AWS Lambda – fundamentos

- Computação orientada a eventos. A cobrança é por **número de requisições + duração × memória (GB-segundo)**. A arquitetura **arm64 (Graviton)** costuma ter melhor preço-desempenho que x86.
- Memória de 128 MB a 10.240 MB**, e a **CPU escala junto com a memória**. Compare duração, custo, latência, concorrência, tamanho de `/tmp` e limite de execução. Teste mais memória antes de concluir que sempre aumenta o custo, pois a execução pode ficar mais rápida. **AWS Lambda Power Tuning** e o **Compute Optimizer** ajudam a achar o ponto ótimo. Para jobs longos ou dependências operacionais especiais, compare Batch, Fargate e EC2. [Boas práticas do Lambda](#).
- Timeout máximo de 15 minutos**. Os demais limites (pacote, `/tmp`, imagem de contêiner) estão em [Números do Lambda](#).
- Empacotamento:** `.zip` (com até 5 **layers** para bibliotecas compartilhadas) ou **imagem de contêiner** de até 10 GB, armazenada no ECR.
- Versions e aliases:** uma versão é imutável, e um alias (ex.: `prod`) aponta para ela. Um **alias com pesos** divide o tráfego entre duas versões (canary). O **CodeDeploy** automatiza o deslocamento gradual com rollback por alarme.

### Modelos de invocação

- Síncrona:** API Gateway, ALB, **Lambda function URL** e SDK. Quem chama espera a resposta e trata o erro e o retry.
- Assíncrona:** S3, SNS, EventBridge. O Lambda enfileira o evento e faz **até 2 novas tentativas**. Configure **destinations** (on-success/on-failure para SQS, SNS, EventBridge ou outra Lambda) ou uma **DLQ** para não perder eventos que falharam.
- Event source mapping (polling):** SQS, Kinesis Data Streams, DynamoDB Streams, MSK e Amazon MQ. O Lambda lê em **lotes**.
  - SQS:** a mensagem é **excluída só após sucesso**. Use `ReportBatchItemFailures` para reprocessar só as mensagens que falharam. Ajuste o **visibility timeout da fila para pelo menos 6× o timeout da função**. Configure DLQ/redrive na própria fila.

- **Kinesis/DynamoDB Streams:** a ordem é preservada por shard. Um registro com erro bloqueia o shard até expirar, ser dividido (*bisect batch*) ou ir para um destino on-failure.

## Concorrência e cold start

- **Concorrência** = requisições por segundo × duração média. O padrão é de **1.000 execuções concorrentes por conta e Região**, compartilhadas entre todas as funções (cota ajustável). Acima do limite, a invocação sofre **throttling** (429): a síncrona falha e a assíncrona é repetida.
- **Reserved concurrency:** **garante** capacidade para uma função e também **limita** o máximo dela. Protege um banco downstream de ser sobrecarregado. Com valor 0, a função é desativada. Não tem custo.
- **Provisioned concurrency:** mantém ambientes **pré-inicializados** e elimina o cold start para cargas sensíveis a latência. É cobrada enquanto está configurada e pode ser agendada via Application Auto Scaling.
- **SnapStart** (Java, Python, .NET): restaura um snapshot do ambiente já inicializado e reduz o cold start sem o custo contínuo da provisioned concurrency.
- Inicialize SDKs e conexões **fora do handler** para reutilizá-los entre invocações no mesmo ambiente.

## Rede, dados e segurança

- **Execution role:** o que a função **pode acessar** (S3, DynamoDB etc.). Deve ser sempre uma IAM Role, nunca credenciais estáticas.
- **Resource-based policy:** **quem pode invocar** a função (ex.: EventBridge, S3, outra conta), com o `Service` principal correto e a ação mínima `lambda:InvokeFunction`.
- **Lambda em VPC:** necessário para acessar recursos privados (RDS, ElastiCache). A função usa ENIs Hyperplane nas subnets escolhidas e **não recebe IP público**. Para acessar a internet, é preciso uma **subnet privada com NAT Gateway**. Para serviços AWS sem NAT, use **VPC endpoints**.
- **RDS Proxy:** faz pool de conexões e evita que milhares de invocações concorrentes esgotem as conexões do RDS/Aurora. Também suporta autenticação IAM e reduz o tempo de failover.
- **Armazenamento:** `/tmp` é **efêmero** (512 MB a 10 GB), mas é reutilizado enquanto o ambiente estiver quente. Para arquivos persistentes e compartilhados, monte **Amazon EFS**. Para objetos, use S3.
- **Configuração e segredos:** variáveis de ambiente são criptografadas com KMS. Segredos devem vir do **Secrets Manager** ou do **Parameter Store**, de preferência com a extensão de cache, e não de variáveis em texto puro.
- **Observabilidade:** logs no CloudWatch Logs, métricas (Invocations, Errors, Throttles, Duration, ConcurrentExecutions, IteratorAge) e tracing com **X-Ray**.

## Números do Lambda

| Métrica                                            | Valor                    | Observação                                                                                                        |
|----------------------------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------|
| Timeout máximo de execução                         | <b>15 minutos (900s)</b> | Número clássico, ainda vigente — o mais citado em provas                                                          |
| Memória                                            | 128 MB – 10.240 MB       | Incrementos de 1 MB; vCPU escala junto com a memória                                                              |
| Tamanho de <code>/tmp</code> (ephemeral storage)   | 512 MB – 10.240 MB       | Configurável; 512 MB é o padrão/mínimo                                                                            |
| Pacote de deploy — .zip direto (console/CLI/API)   | 50 MB                    |                                                                                                                   |
| Pacote de deploy — descompactado (código + layers) | 250 MB                   | Vale mesmo fazendo upload via S3: o S3 só contorna o limite de 50 MB do .zip em si, não o de 250 MB descompactado |
| Imagem de container                                | até 10 GB                |                                                                                                                   |

| Métrica                              | Valor                        | Observação                        |
|--------------------------------------|------------------------------|-----------------------------------|
| Concorrência padrão por conta/região | 1.000 execuções concorrentes | Cota ajustável via Service Quotas |

## Ecossistema serverless

- **Integração e APIs:** API Gateway, AppSync, SQS, SNS, EventBridge e Step Functions estão na seção 6. **Dados:** DynamoDB, Aurora Serverless v2 e S3 estão nas seções 2 e 3. **Contêineres sem servidor:** Fargate. Veja [Contêineres](#).
- **AWS SAM** (Serverless Application Model): extensão do CloudFormation com sintaxe simplificada para funções, APIs e tabelas, mais a CLI para build, teste local e deploy. **CDK** é a alternativa em linguagens de programação.
- **AWS Serverless Application Repository (SAR):** catálogo de aplicações serverless reutilizáveis e prontas para implantação (publicadas pela AWS, por parceiros ou pela comunidade), integrado ao AWS SAM/CloudFormation. É a resposta padrão para "reaproveitar um padrão de arquitetura serverless comum sem construir do zero".

## Elastic Beanstalk (PaaS)

- **AWS Elastic Beanstalk:** PaaS que orquestra EC2 + Auto Scaling + ELB (e opcionalmente RDS) a partir do código enviado, com suporte nativo a várias linguagens e Docker. **Não é serverless:** há instâncias EC2 cobradas continuamente, mas você não as configura manualmente.
- **Ambientes:** *web server* (atrás de ELB) e *worker* (consome uma fila SQS). Permite vários ambientes com **troca de URL (swap CNAME)** para blue/green.
- **Políticas de deploy:**
  - *all at once:* mais rápida, com downtime;
  - *rolling* e *rolling with additional batch:* mantêm a capacidade;
  - *immutable:* novas instâncias em um ASG temporário, com rollback seguro;
  - *traffic splitting:* canary.
- Costuma ser a resposta para "levar uma aplicação web tradicional para a AWS rapidamente, sem gerenciar a infraestrutura em detalhe". Costuma ser descartado quando existe alternativa serverless mais simples (S3 + Lambda) ou quando se pede controle de orquestração de contêineres.

## Padrões e pegadinhas de serverless

- "Processamento dura mais de 15 minutos": não é Lambda. Use **Batch**, **Fargate** ou **Step Functions** dividindo o trabalho em etapas.
- "Cold start inaceitável em API sensível a latência": **provisioned concurrency** (ou SnapStart).
- "Uma função consome toda a concorrência da conta e afeta as outras": **reserved concurrency**.
- "Lambda sobrecarrega o RDS com conexões": **RDS Proxy**.
- "Lambda em VPC não acessa a internet": falta **NAT Gateway** na rota da subnet privada. Atribuir IP público à função não é possível.
- "Eventos assíncronos com falha não podem se perder": **destinations on-failure** ou DLQ.
- "Mensagens SQS processadas duas vezes pela Lambda": aumente o **visibility timeout** ( $\geq 6 \times$  o timeout) e torne o processamento **idempotente**.
- "Permitir que o EventBridge/S3 invoque a função": **resource-based policy**, não a execution role.
- "Deploy gradual de nova versão com rollback automático": **alias com pesos + CodeDeploy**.
- Referência: [AWS Lambda Developer Guide](#).



## Edge e híbrido

**Regra de ouro da prova:** identifique **onde o recurso precisa estar** e **por quê**. Latência para usuários de uma metrópole: **Local Zones**. Latência para dispositivos 5G: **Wavelength**. Dados ou processamento que precisam ficar no seu datacenter (residência, latência com sistemas locais): **Outposts**. Conteúdo em cache perto de usuários globais: **CloudFront** (seção 4), não Local Zones. Workloads VMware sem refatorar: **VMware Cloud on AWS** ou **Amazon EVS**.

### Comparativo das opções

| Opção                                   | Onde fica                                                       | Quem opera o hardware                                              | Caso típico                                                                                                        | Não confundir com                                  |
|-----------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| <b>Local Zones</b>                      | Área metropolitana, extensão de uma Region                      | AWS                                                                | Latência de milissegundos de dígito único para usuários de uma cidade: jogos, mídia, estações de trabalho virtuais | CDN (não faz cache) ou Region independente         |
| <b>Wavelength</b>                       | Dentro da rede 5G da operadora                                  | AWS + operadora                                                    | Apps móveis e IoT com latência ultrabaixa (AR/VR, veículos conectados, streaming ao vivo)                          | Local Zones (sem integração com a operadora)       |
| <b>Outposts</b> (racks ou servers)      | <b>No seu datacenter</b>                                        | AWS entrega, instala e mantém; você fornece espaço, energia e rede | Residência de dados, baixa latência com sistemas on-premises, processamento local                                  | Region desconectada (precisa de link com a Region) |
| <b>Dedicated Local Zones</b>            | Infraestrutura dedicada a um cliente, em local indicado por ele | AWS                                                                | Requisitos de soberania e isolamento de governos e setores regulados                                               | Local Zones públicas                               |
| <b>VMware Cloud on AWS / Amazon EVS</b> | Region AWS, em hardware bare metal                              | Parceiro/VMware (VMC) ou você (EVS)                                | Migrar vSphere/vSAN/NSX sem converter VMs                                                                          | AWS MGN (converte para EC2 nativo)                 |

### AWS Outposts

- Estende a infraestrutura AWS (EC2, EBS, **S3 on Outposts**, ECS, EKS, RDS, ElastiCache, EMR) para datacenters on-premises, com baixa latência local e as mesmas APIs e console.
- Outposts racks** (42U, capacidade maior) e **Outposts servers** (1U/2U, para lojas, fábricas e filiais com pouco espaço).
- Precisa de conexão com a Region-mãe** (*service link*), pela internet ou pelo Direct Connect. Se o link cair, as instâncias em execução continuam, mas o **control plane** (lançar, alterar, monitorar) fica indisponível. Não serve para operação permanentemente desconectada.
- Capacidade fixa:** não há elasticidade além do hardware instalado. Planeje a capacidade e a redundância (mais de um Outpost para HA).
- Backups devem ir para o **S3 na Region AWS** (não apenas localmente no Outposts) para garantir um DR robusto. Snapshots de EBS podem ficar localmente (S3 on Outposts) quando a residência exige, mas isso não protege contra a perda do site.

### AWS Local Zones e Wavelength

- AWS Local Zones:** aproxima compute, storage e alguns serviços de usuários de uma área metropolitana, mantendo o vínculo com a Region principal. A zona precisa ser **ativada (opt-in)**. Depois, você cria uma **subnet da sua VPC** nela. Oferece só um subconjunto de serviços (EC2, EBS, ECS, EKS, ALB, FSx, entre outros). É indicada

para latência de milissegundos de dígito único quando uma Region está distante. Não equivale a CDN e não é uma Region independente. Também serve para **migração híbrida** com Direct Connect até a Local Zone.

- **AWS Wavelength:** aproxima computação e armazenamento dos usuários de redes de operadoras, reduzindo a latência para aplicações móveis. A Wavelength Zone usa uma **sub-rede de uma VPC da região principal**. A rota local da VPC permite a comunicação com recursos da Region, e um **carrier gateway** conecta a zona à rede da operadora. Direct Connect não é requisito para essa integração. [Funcionamento](#).

## Computação na borda do CloudFront

| Critério         |                                                                                          |                                                                                                     |
|------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Linguagem        | JavaScript                                                                               | Node.js e Python                                                                                    |
| Onde roda        | Edge locations (mais pontos)                                                             | Regional edge caches                                                                                |
| Eventos          | Viewer request/response                                                                  | Viewer e <b>origin</b> request/response                                                             |
| Tempo e recursos | Submilissegundo, sem rede e sem corpo da requisição                                      | Até segundos, com acesso à rede e ao corpo                                                          |
| Custo            | Muito baixo                                                                              | Maior                                                                                               |
| Casos            | Reescrever URL, headers, redirecionamentos, validar tokens simples, normalizar cache key | Autenticação com chamada externa, gerar respostas, escolher origem dinamicamente, manipular imagens |

## Híbrido: VMware, contêineres e servidores on-premises

- **VMware Cloud on AWS:** executa cargas de trabalho VMware (vSphere, vSAN, NSX) nativamente sobre a infraestrutura AWS. É a resposta padrão para "migrar workloads VMware on-premises para a nuvem sem re-arquitetar aplicações ou converter formatos de VM". É diferente do MGN, que replica servidores convertendo-os para instâncias EC2 nativas. A oferta é comercializada pela Broadcom/VMware e parceiros.
- **Amazon Elastic VMware Service (EVS):** executa o VMware Cloud Foundation em instâncias bare metal **dentro da sua VPC**, operado por você. É indicado quando se quer manter as ferramentas VMware com integração direta à VPC.
- **Contêineres on-premises:** **ECS Anywhere**, **EKS Anywhere** e **EKS Hybrid Nodes**. Veja [Contêineres](#).
- **Servidores on-premises gerenciados:** **Systems Manager** com *hybrid activations* gerencia patches, inventário e comandos em servidores fora da AWS, com o mesmo console da EC2.
- **Dispositivos de borda:** **AWS IoT Greengrass** executa Lambda/contêineres e inferência de ML **localmente** em dispositivos, funcionando mesmo desconectado e sincronizando com a nuvem depois.
- **Armazenamento híbrido:** Storage Gateway e DataSync estão na seção 2. Os dispositivos **Snowball Edge** com computação desconectada estão sendo descontinuados. Veja a ressalva em [Números do Snowball \(legado\)](#).

## Padrões e pegadinhas de edge e híbrido

- "Usuários em uma cidade distante da Region precisam de latência de dígito único para uma aplicação interativa": **Local Zones**.
- "Aplicação para dispositivos 5G com latência ultrabaixa": **Wavelength**.
- "Dados não podem sair do datacenter, mas a empresa quer as APIs da AWS": **Outposts**.
- "Outposts perdeu a conexão com a Region": as instâncias continuam rodando, mas não é possível gerenciá-las. Planeje um link redundante.
- "Servir arquivos estáticos com baixa latência globalmente": **CloudFront**, não Local Zones.
- "Reescrever URL ou adicionar headers de segurança na borda com o menor custo": **CloudFront Functions**.  
"Chamar um serviço externo ou mudar a origem na borda": **Lambda@Edge**.

- "Migrar centenas de VMs VMware rapidamente, sem conversão": **VMware Cloud on AWS** (ou EVS). "Migrar servidores para EC2 nativa": **AWS MGN**.
- "Inferência de ML em fábrica com conectividade intermitente": **IoT Greengrass**.

## Mobile e Frontend

**Regra de ouro da prova:** para frontends e apps móveis, a pergunta é **quanto do backend você quer construir**. "Acelerar o desenvolvimento full-stack com backend gerenciado" leva ao **Amplify**. "Site estático com baixo custo e alta escala" leva a **S3 + CloudFront**. "API em tempo real/GraphQL" leva ao **AppSync**. "Login de usuários finais" leva ao **Cognito**. "Testar em dispositivos reais" leva ao **Device Farm**.

### AWS Amplify

- Plataforma para desenvolver e implantar aplicações web/mobile full-stack rapidamente, com backend integrado: autenticação via **Cognito**, API via **AppSync**/API Gateway, dados no **DynamoDB**, storage via **S3** e funções **Lambda**. É a resposta padrão para "acelerar o desenvolvimento de uma aplicação web/mobile com backend AWS totalmente gerenciado".
- **Amplify Hosting:** CI/CD a partir do Git, com build e deploy automáticos a cada commit, **ambientes de preview por branch/pull request**, domínio personalizado com HTTPS e distribuição via CloudFront. Suporta sites estáticos (SPA) e frameworks com **SSR** (Next.js, Nuxt).
- **Amplify Gen 2:** o backend é definido em TypeScript (sobre o CDK), com sandbox por desenvolvedor.
- **Bibliotecas e UI:** clientes para web (JavaScript), iOS, Android e Flutter, com componentes prontos de login, storage e dados.
- É diferente do Elastic Beanstalk, que faz uma orquestração mais ampla da infraestrutura de aplicação e não é focado em frontend mobile/web nem em backend-as-a-service. Veja [Elastic Beanstalk \(PaaS\)](#).

### Hosting de frontend sem Amplify

- **S3 + CloudFront:** o padrão para sites estáticos e SPAs com baixo custo e escala global.
  - Mantenha o bucket **privado** com **Origin Access Control (OAC)**, e não com o website endpoint público;
  - use certificado do **ACM em us-east-1** para o domínio;
  - proteja com **AWS WAF** e defina cache e invalidação conforme o deploy.
- **Conteúdo dinâmico:** o CloudFront pode ter várias origens (S3 para estáticos, ALB/API Gateway para `/api/*`) sob o mesmo domínio, o que também evita problemas de CORS.

### Backend para aplicações mobile e web

- **Amazon Cognito:**
  - **User Pools** autenticam usuários finais: cadastro, login, MFA, login social e federação SAML/OIDC;
  - **Identity Pools** trocam o token por **credenciais AWS temporárias**, permitindo que o app acesse S3/DynamoDB diretamente com permissões limitadas por usuário. Veja a seção 5.
- **AWS AppSync:** API **GraphQL** gerenciada, com **subscriptions em tempo real** (WebSocket), resolvers para DynamoDB, Lambda, RDS e HTTP, cache e autorização por Cognito, IAM, API key ou OIDC. O **AppSync Events** oferece pub/sub em tempo real via WebSocket sem GraphQL. É indicado para apps que agregam várias fontes de dados em uma chamada e para dados sincronizados em tempo real.
- **Amazon API Gateway:** APIs REST/HTTP e **WebSocket** para chat e notificações. Veja a seção 6.

- **Upload direto do app: S3 presigned URLs** ou credenciais do Identity Pool, evitando que arquivos grandes passem pelo backend.
- **Notificações e mensagens:**
  - **SNS mobile push** (APNs, FCM) para notificações simples;
  - **AWS End User Messaging** para SMS, voz e push em escala;
  - o **Amazon Pinpoint** teve o fim de suporte anunciado, então não o escolha para soluções novas.
- **Amazon Location Service:** mapas, geocodificação, rotas e geofencing para apps, com dados de provedores como Esri, HERE e Open Data.

## AWS Device Farm

- Testa aplicações mobile e web em uma frota real de dispositivos físicos e emuladores na nuvem. É a resposta padrão para "testar compatibilidade de um app em múltiplos dispositivos reais sem manter um laboratório de hardware próprio".
- Suporta **testes automatizados** (Appium, Espresso, XCTest), **acesso remoto** interativo a dispositivos reais e testes em **navegadores desktop** com Selenium. Integra-se a pipelines de CI/CD e gera logs, vídeos e métricas de desempenho por dispositivo.

## Padrões e pegadinhas de mobile e frontend

- "Startup quer lançar app web/mobile com login, API e storage no menor tempo": **Amplify**.
- "Site estático com menor custo e bucket não público": **S3 privado + CloudFront com OAC**.
- "Usuários do app precisam enviar fotos direto para o S3 com permissão só da própria pasta": **Cognito Identity Pool** com política por usuário, ou **presigned URL**.
- "Dashboard com dados atualizados em tempo real para milhares de clientes": **AppSync subscriptions** (ou API Gateway WebSocket).
- "Login social (Google, Apple) e MFA para usuários finais": **Cognito User Pools**.
- "Testar o app em centenas de modelos de celular": **Device Farm**.
- "Preview automático de cada pull request do frontend": **Amplify Hosting**.

## Decisão rápida — Computação

| Cenário / requisito                                               | Resposta correta                            | Distratores clássicos                        |
|-------------------------------------------------------------------|---------------------------------------------|----------------------------------------------|
| Carga stateless tolerante a interrupção                           | EC2 Spot / Spot Fleet                       | On-Demand, Reserved                          |
| Carga base previsível + picos variáveis                           | Reserved/Savings Plan (base) + Spot (picos) | On-Demand para tudo, Spot para tudo          |
| Dimensionamento para carga recorrente e conhecida (fixa)          | Scheduled Scaling                           | Dynamic Scaling, Predictive Scaling          |
| Dimensionamento para carga com componente previsível mas variável | Predictive Scaling                          | Scheduled Scaling (não se ajusta à variação) |

## 2. Armazenamento

**Regra de ouro da prova:** primeiro identifique o **tipo de acesso** que a aplicação espera, depois o **escopo** (uma instância, várias instâncias, on-premises) e só então o custo. Bloco para uma instância leva a **EBS** (ou Instance Store, se temporário). Arquivos compartilhados via NFS/Linux levam a **EFS**. SMB/Windows/AD leva a **FSx for Windows**. HPC paralelo leva a **FSx for Lustre**. Objetos via API/HTTP, data lake e arquivamento levam a **S3**. Acesso local on-premises a armazenamento na nuvem leva a **Storage Gateway**.

| Serviço              | Tipo             | Escopo                                                  | Protocolo / acesso        | Caso típico                                                 |
|----------------------|------------------|---------------------------------------------------------|---------------------------|-------------------------------------------------------------|
| Amazon S3            | Objeto           | Regional (dados em ≥ 3 AZs, exceto classes One Zone)    | API REST/HTTPS            | Data lake, backup, conteúdo estático, arquivamento          |
| Amazon EBS           | Bloco            | Uma AZ, anexado a uma instância (Multi-Attach limitado) | Dispositivo de bloco      | Volume de boot, bancos em EC2, aplicações transacionais     |
| Instance Store       | Bloco efêmero    | Host físico                                             | Dispositivo de bloco      | Cache, buffer, scratch. Veja <a href="#">Instance Store</a> |
| Amazon EFS           | Arquivo          | Regional (ou One Zone), milhares de clientes            | NFS v4.1 (Linux)          | Conteúdo compartilhado, home dirs, contêineres e Lambda     |
| FSx for Windows      | Arquivo          | Single-AZ ou Multi-AZ                                   | SMB + Active Directory    | File server Windows, SharePoint, home folders               |
| FSx for Lustre       | Arquivo paralelo | Uma AZ                                                  | Cliente Lustre (POSIX)    | HPC, ML, renderização, com integração ao S3                 |
| FSx for NetApp ONTAP | Arquivo e bloco  | Single-AZ ou Multi-AZ                                   | NFS, SMB, iSCSI, NVMe/TCP | Migração de NetApp, multiprotocolo                          |
| FSx for OpenZFS      | Arquivo          | Single-AZ ou Multi-AZ                                   | NFS                       | Migração de ZFS/Linux com baixa latência                    |
| Storage Gateway      | Híbrido          | On-premises + AWS                                       | NFS/SMB, iSCSI, VTL       | Acesso local com dados na nuvem                             |

### Amazon S3

- Armazenamento de objetos com **11 nove de durabilidade** e escala automática. É a base de quase toda arquitetura serverless.

#### Fundamentos do S3

- Buckets são regionais**, com nome **globalmente único**. Um objeto é identificado pela *key* (o "caminho" é apenas parte do nome). O tamanho máximo de objeto e os limites de upload estão em [Números do S3](#).
- Consistência forte** de leitura após escrita (read-after-write) para PUT, DELETE e LIST, sem custo adicional.
- Desempenho por prefixo:** pelo menos **3.500 PUT/COPY/POST/DELETE** e **5.500 GET/HEAD** por segundo por prefixo. Para escalar, distribua as keys em **vários prefixos**. Não há limite de prefixos por bucket.
- Tipos de bucket:**
  - general purpose:** o padrão;
  - directory buckets:** S3 Express One Zone;
  - table buckets:** **S3 Tables**, tabelas Apache Iceberg gerenciadas para analytics, com compactação automática;
  - vector buckets:** **S3 Vectors**, armazenamento e consulta de embeddings para IA de baixo custo.

- **Mountpoint for Amazon S3:** cliente que monta um bucket como sistema de arquivos local, para cargas com **muita leitura sequencial** (ML, analytics). Não oferece semântica POSIX completa (sem renomear diretório nem editar parte de um arquivo). Para arquivos POSIX compartilhados, use EFS ou FSx.

## Classes de armazenamento

Escolha conforme a frequência de acesso, o prazo aceitável para recuperar os dados e a necessidade de resiliência entre Zonas de Disponibilidade.

| Classe                               | Quando usar                                                                                        | Características principais                                                                                                                                                                                                                                                                  |
|--------------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>S3 Standard</b>                   | Dados acessados com frequência                                                                     | Classe padrão, com acesso em milissegundos e armazenamento em múltiplas AZs.                                                                                                                                                                                                                |
| <b>S3 Intelligent-Tiering</b>        | Frequência de acesso desconhecida ou variável                                                      | Move objetos automaticamente entre níveis de acesso para otimizar o custo, <b>sem cobrança de recuperação</b> . Há cobrança de monitoramento por objeto. Objetos menores que 128 KB não são monitorados e ficam sempre no nível frequente. Níveis de arquivamento assíncrono são opcionais. |
| <b>S3 Standard-IA</b>                | Dados pouco acessados que precisam de recuperação imediata                                         | Acesso em milissegundos e armazenamento em múltiplas AZs, com cobrança pela recuperação, duração mínima de 30 dias e <b>tamanho mínimo cobrado de 128 KB</b> por objeto.                                                                                                                    |
| <b>S3 One Zone-IA</b>                | Dados pouco acessados que podem ser recriados                                                      | Acesso em milissegundos, mas os dados ficam em apenas <b>uma AZ</b> . Há cobrança pela recuperação, duração mínima de 30 dias e mínimo cobrado de 128 KB.                                                                                                                                   |
| <b>S3 Glacier Instant Retrieval</b>  | Arquivos raramente acessados (cerca de uma vez por trimestre) que precisam de recuperação imediata | Acesso em milissegundos, com cobrança pela recuperação e duração mínima de 90 dias.                                                                                                                                                                                                         |
| <b>S3 Glacier Flexible Retrieval</b> | Backups e arquivos que podem aguardar a restauração                                                | Requer solicitação de restauração: <b>Expedited</b> (1–5 min), <b>Standard</b> (3–5 h) ou <b>Bulk</b> (5–12 h, gratuito). Duração mínima de 90 dias.                                                                                                                                        |
| <b>S3 Glacier Deep Archive</b>       | Arquivamento de longo prazo com acesso muito raro                                                  | Requer solicitação de restauração: <b>Standard</b> (até 12 h) ou <b>Bulk</b> (até 48 h). Menor custo por GB. Duração mínima de 180 dias.                                                                                                                                                    |
| <b>S3 Express One Zone</b>           | Workloads frequentes e sensíveis à latência próximos da computação                                 | Usa <i>directory buckets</i> em <b>uma AZ</b> e oferece acesso em milissegundos de um dígito. Não protege contra a perda dessa AZ.                                                                                                                                                          |

- **Restauração do Glacier:** a restauração cria uma **cópia temporária** do objeto pelo número de dias pedido. O objeto original continua na classe de arquivamento. Para muitos objetos, use **S3 Batch Operations** para iniciar as restaurações em massa.
- Quando o momento de transição é conhecido, compare uma **regra de lifecycle** com Intelligent-Tiering, considerando custos de monitoramento, transição, recuperação e duração mínima. [Comparação oficial das classes](#).

## Lifecycle e análise de custo

- **S3 Lifecycle Policies:** automatizam transições entre classes (ex.: Standard → Standard-IA após 30 dias → Glacier Deep Archive). É a resposta padrão para retenção de longo prazo com custo mínimo. As regras podem filtrar por prefixo, tag ou tamanho de objeto.
- **Ações de lifecycle além da transição:**
  - **expirar** objetos;
  - expirar **versões não atuais** (essencial com versionamento, senão versões antigas acumulam custo);
  - remover *delete markers* expirados;
  - **abortar multipart uploads incompletos** (partes órfãs são cobradas e não aparecem na listagem).
- **Regras de transição:** um objeto precisa ficar ao menos **30 dias** em Standard antes de ir para Standard-IA/One Zone-IA por lifecycle. Transições para objetos pequenos (< 128 KB) geralmente não compensam.



- **S3 Storage Class Analysis:** observa padrões de acesso de um bucket/prefix/tag e recomenda transição de **S3 Standard para S3 Standard-IA**. Não recomenda diretamente One Zone-IA ou classes Glacier e não executa a transição. A análise orienta uma lifecycle policy posterior.
- **S3 Storage Lens:** métricas agregadas de uso do S3 em escala de conta/organização (ex.: uploads multipartes incompletos, buckets sem versionamento ou criptografia). Não confundir com AWS Config (compliance de configuração) ou SCP (apenas restrição).
- **S3 Inventory:** relatório diário ou semanal (CSV, ORC ou Parquet) com a lista de objetos e seus metadados (classe, criptografia, replicação). Serve de base para auditoria e para o S3 Batch Operations.
- **S3 Requester Pays:** o solicitante autorizado paga as solicitações e a transferência de dados correspondentes, e o dono do bucket continua pagando o armazenamento. Compare com distribuição via CloudFront quando houver público amplo. [Requester Pays](#).

## Proteção de dados: versionamento, Object Lock e replicação

- **S3 Versioning + MFA Delete:** protege contra exclusão **acidental**. Um DELETE sem versão cria um *delete marker*, e a versão anterior pode ser restaurada. **MFA Delete** só pode ser habilitado pelo **usuário root** via CLI/API. Não é solução para retenção regulatória (isso é Object Lock). O versionamento, uma vez habilitado, só pode ser **suspenso**, nunca desabilitado.
- **S3 Object Lock (WORM):** exige versionamento habilitado. **Governance Mode** (permissão especial `s3:BypassGovernanceRetention` pode sobrepor) vs. **Compliance Mode** (imutável mesmo para a conta raiz durante a retenção). Compliance Mode é sempre a resposta quando o requisito é imutabilidade absoluta/regulatória. **Legal Hold** é uma trava sem data de expiração definida, usada quando a liberação depende de decisão futura/indefinida (ex.: processos judiciais), diferente de um período de retenção fixo.
- **Replicação:**
  - **Cross-Region Replication (CRR)** replica objetos entre buckets de Regiões diferentes (DR, latência, conformidade) e gera custo extra de transferência;
  - **Same-Region Replication (SRR)** replica na mesma Região (agregação de logs, cópia entre contas, ambientes de teste).
- **Regras da replicação:**
  - exige **versionamento nos dois buckets** e uma IAM role;
  - **não é retroativa:** objetos já existentes exigem **S3 Batch Replication**;
  - *delete markers* não são replicados por padrão, e exclusões de versões específicas nunca são replicadas (protege contra exclusão maliciosa);
  - não há replicação encadeada ( $A \rightarrow B \rightarrow C$  não replica de A para C);
  - **S3 Replication Time Control (RTC)** oferece SLA de replicação de 99,99% dos objetos em 15 minutos, para requisitos de RPO;
  - a replicação entre contas pode trocar o dono do objeto para a conta de destino.
- **S3 Multi-Region Access Points (MRAP):** fornece um endpoint global e roteia automaticamente cada requisição para um bucket ativo próximo do cliente, usando a rede AWS Global Accelerator.
  - **Roteamento não é replicação:** configure CRR para que o objeto esteja disponível em todos os buckets necessários. Para gravações em mais de uma Região, considere replicação bidirecional.
  - Em uma interrupção regional, os **controles de failover** permitem alterar quais Regiões recebem tráfego. Não presuma que o MRAP detecta a falha e muda sozinho a configuração ativo/passivo.
  - [Roteamento e failover](#) · [Replicação](#).
- **AWS Backup para S3:** backups contínuos (restauração point-in-time de até 35 dias) e periódicos de buckets, centralizados com os demais recursos. Veja [Backup](#).

## Segurança e criptografia do S3

- **Segurança de bucket: Block Public Access** (nível de conta) + política de bucket restritiva + IAM é a combinação padrão de proteção. Novos buckets já vêm com Block Public Access ativado e **ACLs desabilitadas** (*Object Ownership: bucket owner enforced*). Use políticas, não ACLs.
- **Avaliação de acesso:** uma requisição é permitida se a política de IAM **ou** a de bucket permitir (na mesma conta) e nenhuma negar explicitamente. Entre contas, **as duas** precisam permitir.
- **Condições úteis em bucket policy:**
  - `aws:SecureTransport` para exigir HTTPS;
  - `aws:SourceVpce` / `aws:SourceVpc` para aceitar só tráfego vindo de um VPC endpoint;
  - `aws:PrincipalOrgID` para restringir à sua AWS Organization;
  - `s3:x-amz-server-side-encryption-aws-kms-key-id` para exigir uma chave KMS específica.
- **S3 Access Points:** endpoints com **política própria por aplicação ou equipe** sobre o mesmo bucket. Simplificam uma bucket policy gigante e podem ser **restritos a uma VPC**. **S3 Access Grants** mapeiam identidades corporativas (IAM Identity Center/diretório) para prefixos do S3.
- **Criptografia em repouso:**
  - **SSE-S3** (padrão em todos os buckets desde 2023): chaves gerenciadas pelo S3;
  - **SSE-KMS:** controle da chave, **auditoria no CloudTrail** e separação de permissões (quem lê precisa de `kms:Decrypt`). Habilite **S3 Bucket Keys** para reduzir chamadas e custo do KMS;
  - **DSSE-KMS:** duas camadas de criptografia, para conformidade específica;
  - **SSE-C:** o cliente fornece a chave em cada requisição, exige HTTPS, e a AWS não armazena a chave;
  - **criptografia client-side:** os dados já chegam criptografados, e a aplicação gerencia tudo. Veja a seção 5.
- **Origin Access Control (OAC):** mecanismo atual recomendado pela AWS para restringir um bucket a ser acessado só via CloudFront. Substitui o antigo **Origin Access Identity (OAI)**, que ainda aparece em provas e ambientes legados, mas não deve ser usado em novos designs.
- **Acesso a conteúdo restrito via CloudFront: signed URL do CloudFront** é ideal para conceder acesso a um arquivo específico. **Signed cookies** preservam as URLs existentes e concedem acesso a vários arquivos restritos, como toda a área de assinantes. HTTPS protege o transporte, mas não autoriza o assinante.
- **Presigned URL:** permite upload/download direto entre o cliente e o S3, sem passar pelo servidor de aplicação, o que reduz acoplamento e gargalo. A URL herda as permissões de quem a assinou e expira. Os prazos máximos estão em [Números do S3](#).
- **Acesso privado a partir da VPC: VPC Gateway Endpoint para S3** (gratuito, via route table) ou interface endpoint quando o acesso vem de on-premises/outra VPC. Veja a seção 4.
- **Deteção de dados sensíveis** em buckets: **Amazon Macie**. **Logs de acesso:** server access logging ou **CloudTrail data events**.

## Desempenho e transferência

- **Estratégia de upload e custo por requisição:** muitos objetos pequenos geram mais requisições PUT e metadados por GB armazenado. Quando o consumo permitir, agrupe registros em arquivos maiores (por exemplo, Parquet para analytics) e compare o custo das solicitações com a latência de ingestão.
- **Multipart upload:** para **objetos grandes**, permite paralelismo e retomada de partes. A AWS recomenda considerá-lo a partir de 100 MB, e ele é obrigatório acima do limite de PUT único. [Multipart upload](#) · [Preços S3](#).
- **Migração de muitos arquivos locais:** compare DataSync, Transfer Family e, quando a banda não atende ao prazo, a transferência física disponível. Veja [Storage Gateway e migração de dados](#).

- **S3 Transfer Acceleration:** acelera uploads e downloads de longa distância usando edge locations e o backbone da AWS até um bucket central. Para sites globais, o padrão é **CloudFront para downloads cacheáveis** e **Transfer Acceleration para uploads**. Global Accelerator não aceita bucket S3 como endpoint direto.
- **Byte-range fetch:** o header HTTP `Range` de `GetObject` baixa apenas os bytes necessários, por exemplo os primeiros 250 bytes de cada arquivo. Também permite paralelizar o download de um objeto grande em várias partes.
- **S3 Select:** filtra linhas/colunas de conteúdo estruturado com SQL. Seu `ScanRange` delimita a parte escaneada, mas não substitui um `Range GET` para ler bytes arbitrários do objeto. **Disponibilidade atual:** S3 Select não está disponível para novos clientes; clientes existentes podem continuar a usá-lo. Para consultas SQL sobre o S3, use **Athena**. [Documentação](#).
- **Latência mínima para dados quentes perto da computação:** **S3 Express One Zone**, na mesma AZ das instâncias.

## Eventos e operações em massa

- **S3 Event Notifications:** disparam SQS/SNS/Lambda em eventos do bucket (criação, remoção, restauração, replicação). É o padrão para pipelines assíncronos de processamento de upload.
- **EventBridge:** com a integração ativada no bucket, oferece mais flexibilidade que as notificações tradicionais: filtros avançados por metadados, **vários destinos**, arquivamento e replay de eventos.
- **Cuidado com loops:** uma Lambda que grava no **mesmo bucket/prefixo** que a dispara cria recursão. Use outro bucket ou prefixo de saída.
- **S3 Batch Operations:** executa uma ação sobre **milhões ou bilhões de objetos** a partir de um manifesto (S3 Inventory ou CSV): copiar, alterar a classe, tags, ACLs, Object Lock, restaurar do Glacier, invocar uma Lambda por objeto, ou replicar objetos existentes. Rastreia o progresso e gera um relatório de conclusão.

## Números do S3

| Métrica                                                            | Valor                               | Observação                                                                                                                                                                                |
|--------------------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tamanho máximo de objeto                                           | <b>50 TB</b>                        | ⚠ Aumentado de 5 TB em dez/2025 — mudança muito recente. 5 TB ainda é o valor "clássico" citado na maioria do material de estudo.                                                         |
| PUT single (sem multipart), via API/SDK                            | 5 GB                                |                                                                                                                                                                                           |
| PUT single via console                                             | até 160 GB                          |                                                                                                                                                                                           |
| Ponto recomendado para começar a usar multipart                    | a partir de ~100 MB                 | Recomendação da AWS, não um limite rígido                                                                                                                                                 |
| Tamanho mínimo de parte em multipart                               | 5 MiB                               | Exceto a última parte, que não tem mínimo                                                                                                                                                 |
| Número máximo de partes / tamanho máx. por parte                   | 10.000 partes / 5 GiB cada          |                                                                                                                                                                                           |
| Presigned URL — SigV4 com usuário IAM (credenciais de longo prazo) | até <b>7 dias</b> (604.800s)        | Também é o teto do <code>aws s3 presign</code> via CLI                                                                                                                                    |
| Presigned URL — credenciais temporárias (IAM Role/STS)             | limitada pela duração da sessão STS | A URL nunca excede a validade do token temporário usado para assiná-la (tipicamente até 12h, conforme <code>MaxSessionDuration</code> da role) — <b>nunca chega aos 7 dias</b> nesse caso |
| Presigned URL gerada pelo Console S3                               | até 12 horas                        |                                                                                                                                                                                           |

## Padrões e pegadinhas de S3

- "Frequência de acesso imprevisível e sem custo de recuperação": **Intelligent-Tiering**. "Acesso conhecido e cai após 30 dias": **lifecycle** para Standard-IA.
- "Arquivo acessado uma vez por trimestre, mas precisa estar disponível em milissegundos": **Glacier Instant Retrieval**.
- "Retenção de 7–10 anos, restauração em até 48 horas, menor custo": **Glacier Deep Archive**.
- "Dados reproduzíveis, menor custo com acesso imediato": **One Zone-IA**.
- "Imutabilidade regulatória, nem o root pode apagar": **Object Lock Compliance Mode** (ou Backup Vault Lock para backups).
- "Replicar objetos que já estavam no bucket antes da regra": **S3 Batch Replication**.
- "RPO de replicação de 15 minutos com SLA": **S3 RTC**.
- "Custo de armazenamento subindo em bucket versionado": lifecycle para **expirar versões não atuais** e **abortar multipart incompletos**.
- "Throttling (503 Slow Down) em alto volume": distribuir em **mais prefixos**.
- "Auditar quem acessou cada objeto com a chave": **SSE-KMS** + CloudTrail.
- "Acesso ao bucket somente a partir da VPC": gateway endpoint + bucket policy com `aws:SourceVpce`.
- "Vários times com permissões diferentes no mesmo bucket": **S3 Access Points**.

## Amazon EBS

### Fundamentos do EBS

- Armazenamento em bloco persistente, acessado pela rede e **preso a uma única AZ**. É replicado automaticamente **dentro da AZ**, o que protege contra falha de hardware, mas não contra perda da AZ. Um volume normalmente é anexado a **uma instância por vez**. A instância pode ter vários volumes.
- Mover um volume para outra AZ ou Região**: crie um **snapshot**, copie-o se for para outra Região e crie um volume a partir dele na AZ de destino.
- Elastic Volumes**: altere tamanho, tipo e IOPS/throughput **sem desanexar nem parar** a instância, por exemplo de gp2 para gp3. É preciso aguardar um intervalo entre modificações e **não é possível reduzir** o tamanho. Depois de aumentar, estenda o sistema de arquivos no sistema operacional.
- EBS-optimized**: largura de banda dedicada entre instância e EBS, padrão nos tipos atuais. **O limite de IOPS/throughput da instância** pode ser o gargalo mesmo com um volume rápido.
- RAID 0** (striping de vários volumes no sistema operacional) soma desempenho além do limite de um volume. RAID 1 não é necessário para durabilidade, porque o EBS já replica na AZ.
- `DeleteOnTermination`: o root é apagado com a instância por padrão, e os volumes adicionais não. Veja [Ciclo de vida: reboot, stop, hibernate e terminate](#).

### Tipos de volume

| Tipo | Categoria                | Desempenho                                                                                            | Boot? | Caso típico                                                                 |
|------|--------------------------|-------------------------------------------------------------------------------------------------------|-------|-----------------------------------------------------------------------------|
| gp3  | SSD uso geral            | Baseline de 3.000 IOPS e 125 MiB/s independentes do tamanho; IOPS e throughput provisionáveis à parte | Sim   | Padrão para a maioria das cargas, cerca de 20% mais barato por GB que o gp2 |
| gp2  | SSD uso geral (anterior) | IOPS <b>atrelado ao tamanho</b> (3 IOPS/GiB) com burst por créditos                                   | Sim   | Legado. Migre para gp3                                                      |

| Tipo                     | Categoria                        | Desempenho                                                                        | Boot?      | Caso típico                                    |
|--------------------------|----------------------------------|-----------------------------------------------------------------------------------|------------|------------------------------------------------|
| <b>io2 Block Express</b> | SSD IOPS provisionado            | Maior IOPS, throughput e latência abaixo de milissegundo; durabilidade de 99,999% | Sim        | Bancos críticos (Oracle, SQL Server, SAP HANA) |
| <b>io1</b>               | SSD IOPS provisionado (anterior) | IOPS provisionado, menor durabilidade que io2                                     | Sim        | Legado. Migre para io2                         |
| <b>st1</b>               | HDD throughput                   | Throughput alto para leitura <b>sequencial</b>                                    | <b>Não</b> | Big data, logs, data warehouse, ETL            |
| <b>sc1</b>               | HDD frio                         | Menor custo por GB                                                                | <b>Não</b> | Dados sequenciais raramente acessados          |

- **General Purpose SSD vs Provisioned IOPS SSD:** `gp2/gp3` equilibram preço e performance para workloads gerais. `io1/io2` são Provisioned IOPS SSD para bancos transacionais críticos que exigem IOPS sustentado, baixa latência e maior durabilidade. `io1/io2` também são os tipos associados a EBS Multi-Attach nas questões dos simulados.
- **HDD `st1` / `sc1` vs SSD:** HDD otimiza custo por GB para leitura/gravação **sequencial** de blocos grandes (`st1` para throughput frequente, `sc1` para uso frio). Para I/O aleatório, banco transacional ou volume de boot, prefira SSD. Calcule IOPS e throughput necessários, não apenas o tamanho em GiB. [Tipos de volume EBS](#).
- **gp3 vs io2:** o valor clássico de prova para `gp3` era 16.000 IOPS, mas o limite atual é maior e está detalhado em [Números do EBS](#). `io2` continua sendo escolhido por durabilidade, consistência de IOPS e latência, enquanto **io2 Block Express** atende o patamar mais alto de performance. O ganho só se concretiza se a instância suportar a largura de banda/IOPS necessários e estiver otimizada para EBS.

## Multi-Attach

- **EBS Multi-Attach** está disponível em volumes **io1/io2** e é restrito a instâncias **Nitro na mesma AZ**, com limite de **até 16 instâncias** por volume.
- Combinado com cluster placement group, é a arquitetura ideal para HPC com armazenamento de bloco compartilhado. **Mas atenção:** o EBS não gerencia a consistência de escrita entre as instâncias anexadas. A aplicação precisa implementar I/O fencing ou usar um sistema de arquivos cluster-aware (ex.: GFS2).
- Sistemas de arquivos comuns como EXT4/XFS **não** são cluster-native e não devem ser assumidos como seguros nesse cenário. Para arquivos compartilhados sem essa complexidade, use **EFS** ou **FSx**.

## Snapshots

- **Incrementais:** só os blocos alterados desde o último snapshot são armazenados (no S3, gerenciado pela AWS). Apagar um snapshot intermediário não quebra os demais.
- **Regionais:** podem ser **copiados para outra Região** (DR) e **compartilhados com outras contas**. Snapshots criptografados exigem compartilhar também a chave KMS gerenciada pelo cliente, porque a chave gerenciada pela AWS não pode ser compartilhada.
- **Automação: Amazon Data Lifecycle Manager (DLM)** cria, retém e copia snapshots e AMIs por política baseada em tags. **AWS Backup** centraliza isso junto com outros serviços.
- **Fast Snapshot Restore (FSR):** elimina a latência de primeira leitura (*lazy loading*) de volumes criados a partir de snapshot, com cobrança por AZ habilitada. Sem FSR, cada bloco é baixado do S3 no primeiro acesso. A alternativa é ler o volume inteiro antes de usar (*initialization*).
- **EBS Snapshots Archive:** camada até ~75% mais barata para snapshots retidos por longo prazo (mínimo de 90 dias), com restauração em até 72 horas.
- **Recycle Bin:** retém snapshots e AMIs excluídos por um período configurável, protegendo contra exclusão acidental. Regras com **lock** impedem que a própria regra seja desativada.



## Criptografia do EBS

- Usa **KMS** (AES-256). Criptografa os dados em repouso, **em trânsito entre a instância e o volume**, os snapshots e os volumes criados a partir deles, com impacto mínimo de latência.
- Só é garantida criando o volume já criptografado ou habilitando a **criptografia padrão da conta (por Região)**. Tags, políticas de KMS isoladas ou IAM roles não ativam criptografia automaticamente.
- **Criptografar um volume existente não criptografado**: snapshot → **copiar o snapshot com criptografia** → criar um novo volume → trocar o volume na instância. Não existe uma opção "criptografar no lugar".

## Números do EBS

| Métrica                                            | Valor              | Observação                             |
|----------------------------------------------------|--------------------|----------------------------------------|
| gp3 — IOPS baseline (incluído, sem custo extra)    | 3.000 IOPS         |                                        |
| gp3 — throughput baseline (incluído)               | 125 MiB/s          |                                        |
| gp3 — IOPS máximo provisionável                    | <b>80.000 IOPS</b> | ⚠ Aumentado de 16.000 IOPS em set/2025 |
| gp3 — throughput máximo provisionável              | <b>2.000 MiB/s</b> | ⚠ Aumentado de 1.000 MiB/s em set/2025 |
| gp3 — tamanho máximo de volume                     | <b>64 TiB</b>      | ⚠ Aumentado de 16 TiB em set/2025      |
| io2 — IOPS máximo por volume                       | 64.000 IOPS        | Não mudou                              |
| io2 Block Express — IOPS máximo                    | 256.000 IOPS       |                                        |
| io2 Block Express — throughput máximo              | 4.000 MB/s         |                                        |
| io2 / io2 Block Express — tamanho máximo de volume | 64 TiB             |                                        |

⚠ **Pegadinha clássica desatualizada**: "gp3 tem no máximo 16.000 IOPS, io2 sempre tem mais" **não é mais categoricamente verdade** — com os novos limites, gp3 pode chegar a 80.000 IOPS provisionados, superando o io2 padrão (64.000). A distinção real hoje é **durabilidade** (io2 oferece 99,999% vs. 99,8–99,9% do gp3) e **IOPS sustentado por GB provisionado**, não mais "quem tem o teto mais alto".

## Padrões e pegadinhas de EBS

- "Reduzir custo de volumes gp2 sem downtime": **Elastic Volumes para gp3**.
- "Banco precisa de IOPS consistente e durabilidade máxima": **io2 Block Express**.
- "Maior throughput sequencial pelo menor custo, sem ser boot": **st1**.
- "Levar o volume para outra AZ": **snapshot + novo volume** na AZ de destino.
- "Volume restaurado do snapshot está lento no começo": **Fast Snapshot Restore**.
- "Garantir que todo novo volume seja criptografado": **criptografia padrão do EBS** na Região (e SCP/Config para impor).
- "Volume rápido, mas a aplicação não passa de X MB/s": verifique o **limite de EBS da instância**.

## Amazon EFS

### Fundamentos do EFS

- Sistema de arquivos NFS gerenciado e elástico, compartilhável entre milhares de instâncias **Linux**, contêineres (ECS, EKS, Fargate) e **Lambda**. Cresce e encolhe sozinho, com cobrança pelo **uso**, sem provisionar capacidade.



- **EFS Regional** armazena dados em várias AZs. **EFS One Zone** reduz custo mantendo os dados em uma AZ e exige avaliar o risco dessa zona.
- Não oferece SMB/Active Directory nem protocolo Lustre, e **não suporta Windows**. É a resposta típica para armazenamento compartilhado POSIX sem refazer a aplicação.
- **Mount targets**: um por AZ, cada um com IP em uma subnet e **security group liberando NFS (TCP 2049)** a partir dos clientes. Também pode ser montado de on-premises via Direct Connect/VPN.

## Classes de armazenamento e throughput do EFS

- **EFS Standard vs EFS Infrequent Access (IA)**: Standard atende arquivos acessados frequentemente. IA reduz o custo de arquivos acessados poucas vezes por trimestre e mantém a semântica POSIX e o acesso online, cobrando pela recuperação. **EFS Lifecycle Management** move arquivos automaticamente conforme o tempo desde o último acesso, e pode trazê-los de volta ao Standard no primeiro acesso. **EFS Archive** é para acesso ainda mais raro.
- **Throughput e desempenho**:
  - **Elastic** (recomendado) acompanha picos imprevisíveis, com cobrança pelo uso;
  - **Provisioned** reserva uma vazão conhecida mesmo com poucos GiB armazenados;
  - **Bursting** depende do tamanho em Standard e de créditos. Observe IOPS, vazão, latência, tamanho dos arquivos e métricas antes de migrar para FSx for Lustre ou ampliar a computação. [Modos de throughput do EFS](#).
- **Performance mode**: *General Purpose* (menor latência, recomendado) ou *Max I/O* (legado, maior paralelismo com mais latência). O modo Elastic tornou o Max I/O raramente necessário.

## Segurança e replicação do EFS

- **Criptografia em repouso** (KMS, definida na criação) e **em trânsito** (TLS, com o *EFS mount helper*).
- **Controle de acesso**: security groups (rede), **IAM authorization** e *file system policies* (quem pode montar e gravar, exigir TLS) e permissões POSIX.
- **EFS Access Points**: impõem um **usuário/grupo POSIX** e um **diretório raiz** por aplicação. É o padrão para isolar várias aplicações ou tenants (e funções Lambda) no mesmo sistema de arquivos.
- **EFS Replication**: replica o sistema de arquivos de modo assíncrono para outra Região ou AZ.
  - Enquanto a replicação está ativa, o **destino é somente leitura**. Não há escrita ativa simultânea nos dois lados nem RPO zero garantido.
  - Para recuperação, planeje o failover e a promoção do destino, além de testar DNS, permissões e clientes.
  - [Replicação](#) · [Failover](#).
- **Backup**: AWS Backup (habilitado por padrão em novos sistemas de arquivos criados pelo console).

## Padrões e pegadinhas de EFS

- "Várias instâncias Linux em várias AZs precisam do mesmo diretório": **EFS**. "Se forem instâncias Windows": **FSx for Windows**.
- "Funções Lambda precisam de arquivos grandes compartilhados e persistentes": **EFS com access point**.
- "Reduzir custo de arquivos antigos sem mudar a aplicação": **Lifecycle Management para IA/Archive**.
- "Poucos GiB, mas precisa de alta vazão constante": **Provisioned throughput** (ou Elastic).
- "Instância não consegue montar o EFS": security group sem **TCP 2049** ou mount target ausente na AZ.

## Amazon FSx

| Opção                              | Protocolos                | Destaques                                                                                | Quando escolher                                               |
|------------------------------------|---------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>FSx for Windows File Server</b> | SMB                       | NTFS, <b>Active Directory</b> , DFS, shadow copies, deduplicação, Multi-AZ               | File server Windows, SharePoint, home folders com AD          |
| <b>FSx for Lustre</b>              | Lustre (POSIX)            | Sub-ms, centenas de GB/s, <b>integração com S3</b> ; <i>scratch</i> ou <i>persistent</i> | HPC, ML, renderização, simulações                             |
| <b>FSx for NetApp ONTAP</b>        | NFS, SMB, iSCSI, NVMe/TCP | Snapshots, clones, SnapMirror, tiering, compressão/dedup, Multi-AZ                       | Migrar NetApp; multiprotocolo (Linux + Windows no mesmo dado) |
| <b>FSx for OpenZFS</b>             | NFS                       | Snapshots e clones instantâneos, baixa latência, Multi-AZ                                | Migrar ZFS/Linux; ambientes de dev/test com clones            |

- **FSx for Windows File Server:** sistema de arquivos gerenciado com SMB nativo, permissões NTFS e integração com Active Directory (AWS Managed Microsoft AD ou AD on-premises). Substitui NAS/servidores de arquivo Windows on-premises. É a resposta padrão sempre que o requisito envolve SMB + AD + alta disponibilidade multi-AZ (ex.: migração de SharePoint, compartilhamento entre equipes Windows, arquivos confidenciais com AWS Client VPN). O **Multi-AZ** mantém um servidor standby em outra AZ com failover automático.
- **FSx for Lustre:** sistema de arquivos paralelo de altíssima performance para HPC, renderização e simulações científicas. Integra nativamente com o S3 para persistência durável: um **data repository association** apresenta os objetos do S3 como arquivos (carregados sob demanda) e exporta os resultados de volta ao bucket.
  - **Scratch:** temporário, sem replicação, menor custo, para processamento de curto prazo.
  - **Persistent:** dados replicados na AZ, para cargas longas.
- **FSx for NetApp ONTAP:** sistema de arquivos compatível com recursos ONTAP e protocolos NFS, SMB e iSCSI, incluindo snapshots, clones e tiering. É a opção natural para migrar workloads NetApp preservando funcionalidades e compatibilidade multiprotocolo. **SnapMirror** replica de um NetApp on-premises para a AWS.
- **FSx for OpenZFS:** sistema de arquivos gerenciado compatível com OpenZFS, acessado por NFS, com snapshots e clones rápidos. É indicado para migrar workloads ZFS/Linux que exigem baixa latência e semântica de sistema de arquivos, não SMB/Windows.
- **Backups do FSx:** automáticos e manuais, armazenados no S3, e integrados ao AWS Backup.

### Padrões e pegadinhas de FSx

- "Compartilhamento SMB com permissões do AD e alta disponibilidade": **FSx for Windows Multi-AZ**. EFS não atende Windows/SMB.
- "Processar dados do S3 com sistema de arquivos paralelo de alta performance": **FSx for Lustre** vinculado ao bucket.
- "Job HPC de curta duração com menor custo": Lustre **scratch**. "Dados precisam persistir": Lustre **persistent**.
- "Mesmos dados acessados por Linux (NFS) e Windows (SMB)": **FSx for NetApp ONTAP**.
- "Migrar um NetApp com snapshots e replicação": **FSx for NetApp ONTAP** + SnapMirror.

## Storage Gateway e migração de dados

### AWS Storage Gateway

- Família de gateways híbridos (VM on-premises, EC2 ou appliance) com **cache local** para acesso de baixa latência aos dados mais usados, enquanto os dados completos ficam na AWS.

- **S3 File Gateway:** expõe um bucket S3 como compartilhamento **NFS/SMB** on-premises, e cada arquivo vira um objeto no S3. Sem ACLs/AD nativos do Windows: para semântica completa de file server Windows, prefira FSx for Windows. Os objetos podem ser processados por serviços AWS e passar por lifecycle.
- **FSx File Gateway:** gateway on-premises com acesso local de baixa latência a um FSx for Windows na nuvem, preservando ACLs e AD. Verifique a disponibilidade para novos clientes antes de adotá-lo em um projeto novo.
- **Volume Gateway:** expõe volumes de **bloco iSCSI**, que não são compartilháveis entre múltiplas instâncias.
  - **Cached:** dados primários na AWS, com cache local dos dados quentes.
  - **Stored:** dados primários **locais**, com backup assíncrono para a AWS como snapshots EBS. Os snapshots podem virar volumes EBS para recuperação na AWS.
- **Tape Gateway:** **fitas virtual (VTL)** para o software de backup existente (Veeam, NetBackup), com arquivamento em S3 Glacier Flexible Retrieval/Deep Archive. Elimina a gestão de fitas físicas sem mudar o processo de backup.

## Migração e transferência de dados

| Requisito                                                                                        | Resposta                                                                          | Observação                                               |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------|
| Transferência online, grande ou <b>recorrente</b> , entre on-premises/outras nuvens e S3/EFS/FSx | <b>AWS DataSync</b>                                                               | Agente, agendamento, filtros, verificação de integridade |
| Parceiros externos enviam arquivos via <b>SFTP/FTPS/FTP/AS2</b>                                  | <b>AWS Transfer Family</b>                                                        | Mantém protocolo e credenciais dos parceiros             |
| Aplicações on-premises continuam acessando localmente, dados na nuvem                            | <b>Storage Gateway</b>                                                            | Acesso contínuo com cache, não migração pontual          |
| Banco de dados (não arquivos)                                                                    | <b>AWS DMS</b>                                                                    | Veja a seção 3                                           |
| Rede insuficiente para o prazo                                                                   | <b>Data Transfer Terminal</b> , parceiros ou Snowball (só para quem já é cliente) | Calcule volume ÷ banda efetiva antes                     |
| Enviar dados ao S3 de longe, pela internet                                                       | <b>S3 Transfer Acceleration</b>                                                   | Uploads de clientes distribuídos globalmente             |

- **AWS DataSync + agente on-premises:** transferência automatizada, otimizada e **recorrente** de grandes volumes entre on-premises e AWS (ou entre serviços AWS e outras nuvens), com compressão e verificação de integridade. Exige um agente instalado localmente com uma *location* configurada. Depende da largura de banda de rede. Pode usar **Direct Connect** ou VPC endpoints para não trafegar pela internet, limitar a banda usada e preservar metadados/permisões.
- **AWS Transfer Family:** serviço gerenciado de transferência de arquivos que expõe endpoints totalmente gerenciados para os protocolos **SFTP, FTPS, FTP e AS2**, usando S3 ou EFS como armazenamento de backend. É a resposta padrão para "migrar um workflow de transferência de arquivos legado (SFTP) para a nuvem sem mudar processos/credenciais dos parceiros externos que já usam esse protocolo". É diferente do DataSync, que faz movimentação em massa/recorrente entre sistemas sem expor um endpoint de protocolo de transferência a terceiros.
- **AWS Snowball Edge (legado):** dispositivo físico para transferência offline quando rede e prazo inviabilizam uma cópia online. Ainda pode aparecer em questões antigas. Desde novembro de 2025, só clientes existentes podem solicitar dispositivos, e o suporte comercial termina em dezembro de 2026. Para novos projetos, compare **AWS DataSync** por rede, **AWS Data Transfer Terminal** (levar a mídia a uma instalação de upload da AWS) e parceiros, considerando tempo total, disponibilidade local e custo. [Aviso Snowball](#) · [Data Transfer Terminal](#).
- **AWS Snowcone (descontinuado):** dispositivo menor citado em simulados antigos, que não pode ser solicitado desde novembro de 2024. [Atualização da família Snow](#).

## Números do Snowball (legado)

| Métrica                         | Valor                                              | Observação                                              |
|---------------------------------|----------------------------------------------------|---------------------------------------------------------|
| Snowball Edge Storage Optimized | 210 TB utilizáveis                                 |                                                         |
| Snowball Edge Compute Optimized | até 104 vCPUs, 416 GB RAM, 28 TB NVMe SSD dedicado |                                                         |
| Snowcone (histórico)            | 8 TB (HDD) ou 14 TB (SSD)                          | Descontinuado em 2024; não selecionar para novo projeto |

⚠ **Atenção — relevante para quem estuda em 2026:** a AWS está descontinuando toda a linha Snow Family. Fechada para novos clientes desde 07/nov/2025, com fim de suporte em todas as regiões comerciais previsto para 31/dez/2026. Isso ainda não está refletido na maior parte do material de curso do SAA-C03, mas é uma mudança de calendário real e recente.

## Padrões e pegadinhas de Storage Gateway e migração

- "Substituir a biblioteca de fitas sem mudar o software de backup": **Tape Gateway**.
- "Servidores on-premises gravam arquivos por NFS/SMB e os dados devem ir para o S3": **S3 File Gateway**.
- "Volumes iSCSI com os dados primários locais e backup na AWS": **Volume Gateway stored**.
- "Migrar 50 TB de um NAS para o EFS e depois sincronizar diariamente": **DataSync**.
- "Parceiros enviam arquivos via SFTP e não podem mudar": **Transfer Family**.

## Backup

### AWS Backup

- Backup centralizado com políticas de retenção, cópias entre contas/Regiões e testes de restauração para recursos compatíveis (por exemplo EC2, RDS, Aurora, EFS, EBS, FSx, S3 e DynamoDB).
- **Backup plans:** frequência, janela, retenção, transição para cold storage e cópia para outra Região ou conta. Os recursos são atribuídos **por tag**, e novos recursos com a tag entram automaticamente.
- **Com AWS Organizations:** *backup policies* aplicam planos a todas as contas, com um **cofre central em conta separada** para cópias entre contas. Isso protege contra comprometimento da conta de produção.
- **Cold storage do AWS Backup não é S3 Glacier Deep Archive:** a transição depende do tipo de recurso. Para objetos S3 arquivados, use lifecycle e classes S3 Glacier, respeitando as limitações do backup de S3.
- **Retenção de anos:** defina frequência, cópias, imutabilidade, prazo de exclusão e teste de restore. Não confunda com o PITR do DynamoDB (janela de até 35 dias).
- **Restore testing** valida automaticamente se os backups podem ser restaurados. **AWS Backup Audit Manager** gera relatórios de conformidade das políticas de backup.
- [Recursos por tipo](#) · [Plano e cold storage](#).

### Imutabilidade de backups

- **AWS Backup Vault Lock:** aplica proteção WORM (write-once-read-many) a um cofre de backup, impedindo exclusão ou alteração dos backups armazenados durante o período de retenção, mesmo por usuários administrativos.

- Em **modo compliance**, nem a conta raiz consegue burlar a trava após o período de carência. Em **modo governance**, usuários com permissão específica podem removê-la. É o equivalente do S3 Object Lock Compliance Mode, mas aplicado ao AWS Backup. É a resposta padrão para imutabilidade regulatória de backups.
- **Logically air-gapped vault**: cofre isolado e bloqueado por padrão, compartilhável com outras contas para recuperação mesmo se a conta de origem for comprometida (ex.: ransomware).

## Padrões e pegadinhas de backup

- "Centralizar e padronizar backups de EC2, RDS, EFS e DynamoDB em várias contas": **AWS Backup + Organizations backup policies**.
- "Backups não podem ser apagados nem por administradores": **Vault Lock em modo compliance**.
- "Cópia de backup em outra Região para DR": **cópia cross-Region no backup plan**.
- "Réplica substitui backup?" **Não**: replicação propaga exclusões e corrupção. Backup com retenção e imutabilidade protege contra erro humano e ransomware.
- "Provar ao auditor que os backups seguem a política": **Backup Audit Manager**.

## Decisão rápida — Armazenamento

| Cenário / requisito                               | Resposta correta                   | Distratores clássicos                       |
|---------------------------------------------------|------------------------------------|---------------------------------------------|
| Imutabilidade regulatória absoluta no S3          | S3 Object Lock — Compliance Mode   | Governance Mode, Versioning+MFA Delete, ACL |
| Retenção sem prazo definido / decisão futura      | S3 Object Lock — Legal Hold        | Retention period fixo                       |
| Compartilhamento de arquivos Windows com AD/SMB   | Amazon FSx for Windows File Server | EFS, S3, Storage Gateway                    |
| Compartilhamento POSIX/NFS entre instâncias Linux | Amazon EFS                         | EBS, S3                                     |
| HPC com protocolo Lustre                          | Amazon FSx for Lustre              | EFS, EBS                                    |

## 3. Banco de Dados

**Regra de ouro da prova:** escolha o banco pelo **modelo de dados e pelo padrão de acesso**, não pela familiaridade. Joins e transações ACID levam a **RDS/Aurora**. Acesso por chave em escala massiva com latência de milissegundos leva a **DynamoDB**. Leituras repetidas leva a **cache** (ElastiCache/DAX). Analytics sobre grandes volumes leva a **Redshift/Athena**. Depois, resolva **disponibilidade** (Multi-AZ), **escala de leitura** (réplicas), **DR** (cross-Region/Global) e **custo**, nessa ordem. Multi-AZ não escala leitura. Réplica não substitui backup.

| Modelo de dados                        | Serviço                                                     | Caso típico                                 |
|----------------------------------------|-------------------------------------------------------------|---------------------------------------------|
| Relacional (OLTP)                      | <b>RDS, Aurora, Aurora DSQL</b>                             | ERP, e-commerce, transações com joins       |
| Chave-valor / documento em escala      | <b>DynamoDB</b>                                             | Carrinho, sessões, perfis, IoT, jogos       |
| Documento (compatível com MongoDB)     | <b>DocumentDB</b>                                           | Catálogo, conteúdo, migração de MongoDB     |
| Wide-column (compatível com Cassandra) | <b>Keyspaces</b>                                            | Migração de Cassandra, alta taxa de escrita |
| Grafo                                  | <b>Neptune</b>                                              | Redes sociais, recomendação, fraude         |
| Em memória (cache)                     | <b>ElastiCache</b> (Valkey/Redis OSS/Memcached), <b>DAX</b> | Cache, sessões, leaderboards                |
| Em memória (banco primário durável)    | <b>MemoryDB</b>                                             | Dados em memória que não podem ser perdidos |
| Séries temporais                       | <b>Timestream</b>                                           | Telemetria, métricas, IoT por timestamp     |
| Busca e análise de logs                | <b>OpenSearch Service</b>                                   | Busca textual, observabilidade              |
| Data warehouse (OLAP)                  | <b>Redshift</b>                                             | BI, relatórios sobre grandes volumes        |

## Amazon RDS

### Fundamentos do RDS

- Banco relacional gerenciado: MySQL, PostgreSQL, MariaDB, SQL Server, Oracle e Db2. A AWS cuida de provisionamento, patching, backups, monitoramento e failover. **Não há acesso ao sistema operacional** (para isso, veja RDS Custom abaixo).
- Roda em subnets da sua VPC, definidas por um **DB subnet group** (subnets privadas em pelo menos duas AZs). O acesso é controlado por **security groups**, idealmente referenciando o security group da camada de aplicação, e não CIDRs.
- Armazenamento** gp3 ou io2 (EBS por baixo). **Parameter groups** configuram o engine. **Option groups** habilitam recursos específicos (ex.: TDE no Oracle/SQL Server).

### Alta disponibilidade e escala de leitura no RDS

- RDS Multi-AZ DB instance:** uma instância principal e uma réplica **síncrona em standby** em outra AZ, com failover automático, normalmente em 1–2 minutos. O **endpoint DNS não muda**: a aplicação só precisa reconectar. A standby não atende leituras. Esse modelo é voltado a disponibilidade, não a ampliar a capacidade de leitura. Backups e manutenção são feitos a partir da standby, o que reduz o impacto na principal.



- **RDS Multi-AZ DB cluster:** uma instância de gravação e **duas instâncias de leitura**, em três AZs, com replicação semissíncrona e failover mais rápido (tipicamente abaixo de 35 s). Combina disponibilidade e capacidade adicional de leitura. Não deve ser confundido com o modelo de uma única standby nem com uma Read Replica independente. Está disponível para MySQL e PostgreSQL. [Comparação dos modelos](#).
- **Read Replica:** réplica geralmente **assíncrona** para descarregar tráfego de leitura. Dimensione-a para a demanda, pois ela também pode virar gargalo.
  - Cada réplica tem **endpoint próprio**, e a aplicação precisa enviar as leituras para ele.
  - Pode ficar na mesma AZ, em outra AZ ou em **outra Região**. A cross-Region serve para leitura local e DR, com custo de transferência entre Regiões.
  - Pode ser **promovida** manualmente a banco independente, o que quebra a replicação.
  - Pode ter Multi-AZ própria.
  - No modelo RDS Multi-AZ DB instance, uma Read Replica separada atende leitura. No Multi-AZ DB cluster, as duas instâncias leitoras já podem atender esse tráfego.
- Quando o problema envolve disponibilidade e lentidão por leituras pesadas, compare **RDS Multi-AZ DB instance + Read Replica** com **RDS Multi-AZ DB cluster**. A escolha depende do mecanismo de alta disponibilidade, da capacidade de leitura e do engine exigidos no cenário.
- **RDS Proxy:** agrupa e reutiliza um pool de conexões entre a aplicação (tipicamente Lambda) e o banco.
  - É a resposta padrão quando o sintoma é "timeout de conexão apesar de CPU/memória/disco baixos", ou seja, esgotamento de conexões, não de capacidade computacional.
  - Também **reduz o tempo de failover** (mantém as conexões da aplicação e redireciona para a nova principal) e suporta **autenticação IAM** com credenciais no Secrets Manager.
  - Não resolve indisponibilidade total durante updates.

## Backups e restauração do RDS

- **Backups automáticos (PITR):** snapshot diário + logs de transação, com retenção de 1 a 35 dias. Restauram a qualquer ponto dentro do período de retenção, normalmente até cerca de 5 minutos antes do momento atual. É a resposta padrão para "restaurar a X minutos antes de uma alteração".
- **Snapshots manuais:** persistem até serem apagados, inclusive após a exclusão da instância. Podem ser **copiados entre Regiões** e **compartilhados entre contas** (se criptografados, compartilhe também a chave KMS gerenciada pelo cliente). A **replicação de backups automáticos cross-Region** permite PITR em outra Região para DR.
- **Toda restauração cria uma nova instância**, com um **novo endpoint**. A aplicação precisa apontar para ela, ou você renomeia as instâncias.
- Ao excluir a instância, os backups automáticos são removidos, a menos que você escolha retê-los. Crie um **snapshot final**.
- Para retenção acima de 35 dias, imutabilidade ou gestão centralizada, use **AWS Backup**. Veja [Backup](#).

## Segurança do RDS

- **Criptografia em repouso (KMS)** só pode ser definida **na criação**. Ela cobre armazenamento, backups, snapshots e réplicas. Para criptografar um banco existente: snapshot → **copiar com criptografia** → restaurar uma nova instância → migrar a aplicação. Uma Read Replica tem o mesmo estado de criptografia da origem.
- **Em trânsito:** TLS. Force conexões TLS com um parâmetro do engine (ex.: `rds.force_ssl` no PostgreSQL/SQL Server, `require_secure_transport` no MySQL).
- **Autenticação:**
  - **IAM database authentication** (MySQL, MariaDB, PostgreSQL) usa tokens temporários de 15 minutos, sem senha no código;

- **Kerberos/AD** para SQL Server, Oracle, MySQL e PostgreSQL;
- senha mestre **gerenciada e rotacionada pelo Secrets Manager**.
- **Auditoria:** exporte logs do engine para o CloudWatch Logs. **Database Activity Streams** envia um fluxo de atividade quase em tempo real para o Kinesis, para monitoramento de conformidade (Oracle, SQL Server e Aurora).

## Operação, desempenho e custo do RDS

- **Monitoramento:** métricas do CloudWatch (CPU, `FreeableMemory`, `ReadIOPS`, `DatabaseConnections`, `ReplicaLag`), **Enhanced Monitoring** (métricas do sistema operacional em até 1 s) e **CloudWatch Database Insights** (sucessor do Performance Insights), que mostra carga por consulta, espera e usuário.
- **RDS Storage Autoscaling:** expande automaticamente o armazenamento sem downtime quando o espaço livre cai abaixo de um limite, até o máximo configurado.
- **RDS Custom:** permite acesso ao sistema operacional subjacente para customizações específicas (Oracle e SQL Server), mantendo parte da automação gerenciada. É diferente do RDS padrão (sem acesso ao sistema operacional) e do EC2 puro (sem automação).
- **RDS Blue/Green Deployments:** cria um ambiente de staging (green) totalmente gerenciado que espelha a produção (blue) via replicação lógica, permitindo testar com segurança upgrades de versão de engine e mudanças de schema antes de promovê-los. O switchover entre blue e green é controlado pelo usuário e tipicamente leva menos de 1 minuto de downtime. É a resposta padrão para "atualizar a versão de um banco de produção com risco e downtime mínimos", mais seguro que atualizar in-place a instância diretamente.
- **Manutenção:** patches ocorrem na **janela de manutenção**. Em Multi-AZ, a standby é atualizada primeiro e depois há failover, o que reduz o downtime.
- **Parar (stop) uma instância RDS:** pode economizar em dev/test ou uso intermitente, mas a AWS reinicia automaticamente a instância após até sete dias parada, e armazenamento e backups continuam cobrados. Não planeje stop como arquivamento permanente. Para arquivar, faça snapshot e exclua a instância. [Parar instância RDS](#).
- **RDS Event Notifications:** o RDS **emite sim** eventos nativos para SNS/EventBridge, mas apenas eventos de **gerenciamento/infraestrutura** (failover, criação/exclusão de instância, conclusão de backup, mudança de parâmetro etc.). O que o RDS **não** emite nativamente são eventos de **alteração de dados** (ex.: "linha X foi atualizada na tabela Y"). Para reagir a mudanças de dados em uma tabela, é preciso um mecanismo intermediário (trigger no banco + polling, ou AWS DMS com CDC), publicando depois em SQS/SNS para os destinos.
- **Desempenho e custo de banco:** observe CPU, memória, IOPS, latência, conexões e consultas antes de ampliar instâncias. Compare índice/consulta adequada, cache, Read Replica, RDS Proxy e aumento de IOPS conforme o gargalo. No custo total, inclua instâncias ou ACUs, armazenamento/IOPS, réplicas, backup acima da franquia aplicável, transferência e retenção de snapshots. Escolha PITR curto quando atende ao negócio. Retenção longa e imutável pede plano de backup e ciclo de vida apropriados. **Reserved Instances** de RDS reduzem o custo de bancos estáveis, porque Savings Plans de computação não cobrem o RDS.

## Números do RDS e do Aurora

| Métrica                                       | Valor       | Observação                                                                               |
|-----------------------------------------------|-------------|------------------------------------------------------------------------------------------|
| Retenção de backup automático — mínimo        | 1 dia       | Default via API/CLI                                                                      |
| Retenção de backup automático — máximo        | 35 dias     | Default 7 dias via Console                                                               |
| Storage máximo — RDS MySQL/PostgreSQL/MariaDB | até 64 TiB  |                                                                                          |
| Storage máximo — RDS Oracle/SQL Server        | até 256 TiB | ⚠ Com múltiplos volumes de armazenamento adicionais — recurso de dez/2025, muito recente |

| Métrica                                                         | Valor                                                | Observação                                                                                                                                                                                        |
|-----------------------------------------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Storage máximo — Aurora                                         | 128 TiB (clássico) / até 256 TiB em engines recentes | Varia por versão do engine (desde jul/2025 para Aurora PostgreSQL 15.13+/16.9+/17.5+ e Aurora MySQL 3.10+); 128 TiB continua sendo o valor mais citado em material de estudo                      |
| Réplicas de leitura — RDS (MySQL/MariaDB/PostgreSQL/SQL Server) | até 15                                               | RDS Oracle: até 5                                                                                                                                                                                 |
| Réplicas de leitura — Aurora                                    | até 15 Aurora Replicas por cluster                   | RDS "padrão" e Aurora convergiram para o mesmo número — a distinção real hoje não é a quantidade, e sim replicação <b>síncrona via storage compartilhado</b> (Aurora) vs. <b>assíncrona</b> (RDS) |

## Padrões e pegadinhas de RDS

- "Alta disponibilidade com failover automático em outra AZ": **Multi-AZ**. "Escalar leituras": **Read Replicas** (ou Multi-AZ DB cluster).
- "A standby Multi-AZ pode atender relatórios?" No modelo DB instance, **não**.
- "DR em outra Região com RPO de minutos para o RDS": **Read Replica cross-Region** (promover no desastre) ou cópia cross-Region de backups/snapshots, conforme RPO/RTO.
- "Criptografar um RDS existente não criptografado": **snapshot** → **cópia criptografada** → **restore**.
- "Remover senhas do código": **IAM database authentication** ou **Secrets Manager** com rotação.
- "Lambda esgota as conexões do banco": **RDS Proxy**.
- "Identificar a consulta que causa lentidão": **Database Insights / Performance Insights**.
- "Upgrade de versão com downtime mínimo e rollback": **Blue/Green Deployments**.
- "Acesso ao sistema operacional do banco gerenciado": **RDS Custom**.
- "Restaurou o backup, mas a aplicação continua lendo dados antigos": a restauração criou **novo endpoint**.

## Amazon Aurora

### Arquitetura do Aurora

- Banco relacional gerenciado de alta performance, compatível com MySQL/PostgreSQL. As instâncias de computação são separadas de um **volume de cluster compartilhado**.
- **Armazenamento distribuído: 6 cópias dos dados em 3 AZs**. Tolerar a perda de 2 cópias sem afetar a escrita e de 3 sem afetar a leitura, com reparo automático. Cresce sozinho até o limite do engine ([Números do RDS e do Aurora](#)), e a cobrança é pelo uso.
- **Réplicas compartilham o mesmo armazenamento**: não há cópia de dados para criar uma réplica, e o lag normalmente fica em **milissegundos**. Até **15 Aurora Replicas** por cluster.
- **Failover**: uma Aurora Replica é promovida a writer, tipicamente em **menos de 30 segundos**. A ordem segue os **failover priority tiers**. Sem réplicas, o Aurora recria a instância, o que é mais lento.
- **Configuração de armazenamento**: **Aurora Standard** (paga por I/O) ou **Aurora I/O-Optimized** (sem cobrança por I/O e preço maior de instância/armazenamento). I/O-Optimized compensa quando o I/O passa de cerca de 25% do custo do cluster.

### Endpoints e réplicas do Aurora

- **Cluster endpoint** (writer): sempre aponta para a instância de escrita atual e acompanha o failover.
- **Reader endpoint**: balanceia conexões entre as réplicas de leitura.

- **Custom endpoints:** grupos de instâncias específicas, por exemplo réplicas maiores para relatórios analíticos, separadas das réplicas da aplicação.
- **Instance endpoints:** para diagnóstico. Evite usá-los na aplicação.
- **Aurora Auto Scaling com Aurora Replicas:** escala automaticamente réplicas de leitura conforme a demanda (CPU ou conexões), mantendo HA multi-AZ. É a resposta padrão para cargas de leitura imprevisíveis.

## Aurora Global Database

- Replicação em nível de armazenamento para **Regiões secundárias somente leitura**, com lag tipicamente **abaixo de 1 segundo** e sem impacto no desempenho da Região primária.
- **Switchover** (planejado, sem perda de dados) e **failover** gerenciado (desastre, RPO de segundos) promovem uma Região secundária em cerca de um minuto. **Write forwarding** permite que aplicações na Região secundária enviem escritas, encaminhadas à primária.
- Replica quase em tempo real entre Regiões com baixíssima latência. Combinado com estratégia *pilot light*, atinge RTO baixo com menor custo que warm standby ou ativo-ativo.
- **Global Database vs. Read Replica cross-Region (RDS):** Global Database tem lag menor, failover gerenciado e é a resposta para "RPO de segundos e RTO de minutos entre Regiões" em bancos relacionais.

## Aurora Serverless v2

- Modo sem servidor do Aurora que escala a capacidade de computação automaticamente em segundos, em incrementos finos de **ACU** (Aurora Capacity Unit), conforme a demanda. A cobrança é pela capacidade efetivamente consumida, e a escala pode ir **até 0 ACU com pausa automática** em períodos ociosos.
- É a resposta padrão para cargas de banco com tráfego intermitente/imprevisível, ambientes multi-tenant com picos, ou dev/test que não justificam capacidade provisionada fixa. É diferente do Aurora provisionado (capacidade fixa, mais previsível/econômica para cargas estáveis e constantes).
- Um mesmo cluster pode **misturar** instâncias provisionadas e Serverless v2 (ex.: writer provisionado e leitores serverless). Também suporta Multi-AZ, réplicas e Global Database. O Aurora Serverless v1 foi descontinuado.

## Recursos exclusivos do Aurora

- **Database Cloning:** cria cópias consistentes em minutos compartilhando armazenamento (copy-on-write). É ideal para staging sob demanda, sem `mysqldump`, e só cobra os blocos alterados.
- **Backtrack** (Aurora MySQL): "rebobina" o banco **no lugar** para um ponto anterior (até 72 horas) sem restaurar para uma nova instância. É a resposta para desfazer rapidamente um `DELETE` errado. PITR, ao contrário, cria um novo cluster.
- **Zero-ETL com Redshift:** replica dados do Aurora (e do RDS/DynamoDB) para o Redshift quase em tempo real, sem pipelines de ETL, para analytics sobre dados transacionais.
- **Babelfish for Aurora PostgreSQL:** entende T-SQL e o protocolo do SQL Server, reduzindo as mudanças no código ao migrar do SQL Server.
- **Aurora DSQL:** banco relacional distribuído e serverless, compatível com PostgreSQL, **ativo-ativo em várias Regiões** com consistência forte. É indicado quando se precisa de escrita em várias Regiões sem gerenciar sharding nem replicação.
- **Aurora PostgreSQL Limitless Database:** escala horizontal de escrita via sharding gerenciado, além do limite de uma única instância writer.

## Padrões e pegadinhas de Aurora

- "Failover mais rápido e réplicas com lag mínimo": **Aurora** com réplicas em várias AZs.

- "Separar leituras de relatórios pesados das leituras da aplicação": **custom endpoint** do Aurora.
- "DR entre Regiões com RPO de ~1 s e RTO de ~1 min": **Aurora Global Database**.
- "Carga intermitente com longos períodos ociosos": **Aurora Serverless v2**.
- "Desfazer em minutos uma alteração errada sem restaurar": **Backtrack** (Aurora MySQL).
- "Criar cópia do banco de produção para testes rapidamente e sem custo de cópia completa": **cloning**.
- "Custo de I/O domina a fatura": **Aurora I/O-Optimized**.
- "Analytics sobre dados transacionais sem construir ETL": **zero-ETL para Redshift**.

---

## Amazon DynamoDB

### Fundamentos do DynamoDB

- NoSQL serverless (chave-valor e documento) com escala automática, latência de milissegundos de um dígito em qualquer escala e dados replicados em **3 AZs**. Não há servidor, patch nem janela de manutenção.
- **Tabela → itens → atributos**, com item de até **400 KB**. Arquivos grandes vão para o S3, com a referência no item.
- **Chave primária**: *partition key* sozinha, ou *partition key + sort key* (chave composta). Isso permite consultar vários itens relacionados (ex.: pedidos de um cliente ordenados por data) com `Query`. `Scan` lê a tabela inteira e é caro. Evite-o em caminhos críticos.
- **Leituras**: *eventually consistent* (padrão, metade do custo) ou *strongly consistent*. Esta última não está disponível em GSIs nem em réplicas de Global Tables com consistência eventual.
- **Transações ACID** (`TransactWriteItems` / `TransactGetItems`) em até 100 itens, inclusive de tabelas diferentes, com o dobro do consumo de capacidade.

### Chaves e índices

- **Modelo de acesso**: modele a chave de partição pelas consultas e pela distribuição de tráfego. **Partições quentes** aumentam latência e throttling mesmo quando a capacidade total parece suficiente. Use chaves de **alta cardinalidade** (ex.: ID de usuário, não "status"), ou acrescente um sufixo aleatório (*write sharding*) para chaves muito acessadas.
- **LSI (Local Secondary Index)**: mesma partition key com outra sort key. **Só pode ser criado junto com a tabela**, suporta leitura fortemente consistente e tem limite de 10 GB por partition key.
- **GSI (Global Secondary Index)**: outra partition key e sort key, **criado a qualquer momento**, somente leitura eventualmente consistente e com **capacidade própria**. Se o GSI sofrer throttling de escrita, **as escritas na tabela base também sofrem**.
- Um GSI cria outro padrão de consulta, com armazenamento e consumo adicionais. Ele melhora consultas, mas não reduz custo nem tráfego. Compare custo de leitura, escrita, armazenamento, índices e backup antes de escolher entre DynamoDB, RDS/Aurora e Redshift.
- Não é indicado para armazenar arquivos binários grandes (isso é S3) nem para consultas ad hoc com joins e agregações complexas (isso é relacional ou analytics).

### Capacidade e custo do DynamoDB

- **Capacidade sob demanda**: paga por requisição, dispensa provisionamento e acompanha tráfego imprevisível. Mas **não significa escala ilimitada instantânea**: picos acima de duas vezes o pico anterior em janela curta, quotas ou partições quentes podem causar throttling. É possível definir **throughput máximo** para limitar custo. [Limites do modo sob demanda](#).



- **Provisionada + auto scaling:** você define RCU/WCU, e o auto scaling (target tracking) ajusta. Tende a ser econômica para carga previsível, mas reage ao sinal medido. **Reserved capacity** reduz ainda mais o custo de uma base estável.
- **Unidades:** 1 **RCU** = 1 leitura fortemente consistente/s de até 4 KB (ou 2 eventualmente consistentes). 1 **WCU** = 1 escrita/s de até 1 KB. Transações consomem o dobro.
- **DynamoDB Standard-IA:** classe de tabela com armazenamento cerca de 60% mais barato e leitura/escrita mais caras, para tabelas grandes com acesso pouco frequente. Combinável com capacidade provisionada para cargas estáveis.
- **TTL:** exclui automaticamente itens expirados via atributo de timestamp (epoch), sem consumir WCU. É a solução nativa e gratuita para expurgo de dados antigos. As exclusões aparecem no Streams, o que permite arquivá-las no S3.

## Global Tables, cache e streams

- **DynamoDB Global Tables:** réplicas ativas automáticas e bidirecionais entre Regiões (**multi-active:** escrita em qualquer Região). Servem para DR/replicação multi-Região e latência local para usuários globais, não para absorver picos de tráfego local.
  - Com consistência eventual (padrão), conflitos são resolvidos por **last writer wins**, e a replicação costuma levar cerca de 1 segundo.
  - A opção **multi-Region strong consistency** permite RPO zero, com maior latência de escrita.
- **DAX:** cache em memória compatível com operações do DynamoDB para leituras repetidas com latência de microssegundos.
  - Exige usar o **cliente DAX e o endpoint do cluster** na aplicação, embora as alterações costumem ser pequenas.
  - Leituras fortemente consistentes passam ao DynamoDB **sem usar o cache**. DAX não substitui modelagem de chaves e índices.
  - Compare com ElastiCache quando a aplicação não se limita ao DynamoDB.
  - [Cliente DAX](#) · [Consistência](#).
- **Streams:** captura ordenada de alterações por item (retenção de 24 horas), usada para auditoria, triggers **Lambda**, replicação e agregações. **Kinesis Data Streams for DynamoDB** é a alternativa quando se precisa de retenção maior ou de vários consumidores via Kinesis.

## Backup e segurança do DynamoDB

- **PITR:** restaura a qualquer ponto da janela configurada (até 35 dias, veja [Números do DynamoDB](#)). Como no RDS, **a restauração cria uma nova tabela**.
- **Backups on-demand:** completos, sem impacto de desempenho, retidos até serem apagados. Também via **AWS Backup**, com cópia cross-Region/conta e Vault Lock.
- **Export para o S3** (a partir do PITR, **sem consumir RCU**) em DynamoDB JSON ou Ion, para analytics com Athena. **Import do S3** cria uma nova tabela a partir de CSV/JSON/Ion.
- **Criptografia em repouso sempre ativa**, com chave própria da AWS, gerenciada pela AWS ou **KMS gerenciada pelo cliente**.
- **Controle fino:** condições IAM como `dynamodb:LeadingKeys` restringem cada usuário (ex.: via Cognito) aos itens com a própria partition key.
- **Acesso privado** a partir da VPC: gateway endpoint. **Deletion protection** impede exclusão acidental da tabela.
- **Zero-ETL** do DynamoDB para OpenSearch (busca) e Redshift (analytics).



## Números do DynamoDB

| Métrica                                          | Valor                                                                       | Observação                                                                                                                                                         |
|--------------------------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tamanho máximo de item                           | 400 KB                                                                      | O nome do atributo entra no cálculo do tamanho                                                                                                                     |
| Janela do PITR                                   | até <b>35 dias</b> (padrão)                                                 | Desde jan/2025 é configurável de 1 a 35 dias via <code>RecoveryPeriodInDays</code> — antes era fixo em 35                                                          |
| Capacidade sob demanda — comportamento de escala | escala instantaneamente; cria headroom de até 2× o pico de tráfego anterior | Não é um "burst" de 5 minutos — esse conceito (capacidade não usada acumulada) é do modo <b>provisioned</b> , não do on-demand; cuidado para não confundir os dois |
| LSI — limite de tamanho                          | 10 GB por <i>item collection</i> (mesma partition key)                      | Soma da tabela base + todas as projeções dos LSIs daquela partição; GSI <b>não</b> tem esse limite                                                                 |

## Padrões e pegadinhas de DynamoDB

- "Tráfego imprevisível, sem planejamento de capacidade": **sob demanda**. "Carga estável e previsível, menor custo": **provisionada + auto scaling + reserved capacity**.
- "Throttling com capacidade total sobrando": **partição quente**. Revise a partition key.
- "Consulta por outro atributo em tabela já existente": **GSI**. LSI só pode ser criado na criação da tabela.
- "Leituras repetidas em microssegundos sem reescrever a aplicação": **DAX**.
- "Usuários globais escrevendo em várias Regiões": **Global Tables**.
- "Remover sessões expiradas sem custo": **TTL**.
- "Disparar processamento quando um item muda": **Streams + Lambda**.
- "Analisar a tabela com SQL sem impactar a produção": **export para o S3 + Athena** (ou zero-ETL para o Redshift).
- "Cada usuário só pode acessar os próprios itens": IAM com `dynamodb:LeadingKeys`.

## Amazon ElastiCache (Redis/Memcached)

### Engines e modos

- Cache gerenciado em memória, com engines **Valkey** (fork open-source compatível com Redis, recomendado e de menor custo), **Redis OSS** e **Memcached**. Opção **ElastiCache Serverless** (sem gerenciar nós, escala automática) ou clusters baseados em nós.
- Requer integração da aplicação. DAX também exige um cliente próprio, mas é específico para DynamoDB. Uma alteração no código da aplicação é esperada: o cache não é transparente.
- Valkey/Redis**: estruturas ricas (sorted sets, hashes, streams), persistência por snapshot, replicação (até 5 réplicas por shard), **cluster mode** (sharding horizontal), pub/sub, **Global Datastore** (réplica cross-Region), criptografia e **AUTH/RBAC**.
- Memcached**: simples, multi-thread, sharding pelo cliente, **sem persistência, replicação ou backup**. É indicado apenas para cache puro, descartável e horizontalmente particionado.

### Estratégias de cache

- Lazy loading (cache-aside)**: a aplicação lê do cache e, se não encontrar (*miss*), lê do banco e grava no cache. Guarda só o que é pedido, mas o primeiro acesso é lento e o dado pode ficar desatualizado.
- Write-through**: a aplicação grava no banco e no cache. O cache fica sempre atualizado, mas guarda dados que talvez nunca sejam lidos.
- TTL**: combine qualquer estratégia com expiração para limitar a desatualização e o uso de memória.

- **Casos de uso:** resultados de consultas caras, **sessões compartilhadas**, leaderboards (sorted sets), rate limiting, filas leves e pub/sub.
- ElastiCache pode manter sessões compartilhadas entre instâncias EC2 atrás de um ALB, evitando afinidade obrigatória com uma única instância (*sticky sessions*).

## Disponibilidade do ElastiCache

- **Redis/Valkey com Multi-AZ e failover automático** melhora a disponibilidade ao promover uma réplica em outra AZ. A replicação comum é **assíncrona**: gravações recentes podem se perder no failover, então não prometa RPO zero. Memcached não oferece replicação e failover nativos equivalentes. [Failover do ElastiCache](#).
- **Amazon MemoryDB**: banco **primário** em memória, compatível com Valkey/Redis, com **log de transações Multi-AZ durável**. É a escolha quando os dados em memória **não podem ser perdidos** (ex.: estado de jogo, carrinho, contadores financeiros). O ElastiCache é cache.

## Padrões e pegadinhas de ElastiCache

- "Reduzir a carga de leitura do RDS para consultas repetidas": **ElastiCache** (lazy loading + TTL).
- "Sessões de usuário compartilhadas entre instâncias do ALB": **ElastiCache** (ou DynamoDB).
- "Cache com alta disponibilidade e failover automático": **Valkey/Redis com Multi-AZ**, não Memcached.
- "Ranking em tempo real de jogo": **sorted sets** do Valkey/Redis.
- "Banco em memória com durabilidade": **MemoryDB**.

---

## Outros bancos de dados

- **Amazon Neptune**: banco de dados de grafos totalmente gerenciado, otimizado para armazenar e consultar relacionamentos altamente conectados (redes sociais, sistemas de recomendação, detecção de fraude baseada em grafos) via Gremlin, SPARQL ou openCypher. É a resposta padrão quando o requisito menciona explicitamente "relacionamentos"/"grafo" como modelo de dados central, diferente de um banco relacional (RDS/Aurora) ou de documento (DocumentDB). Replica em 3 AZs, com até 15 réplicas de leitura. **Neptune Analytics** atende análises de grafo em memória.
- **Amazon Keyspaces (for Apache Cassandra)**: banco de dados wide-column gerenciado e compatível com a API do Apache Cassandra (CQL), serverless e com escala automática. É a resposta padrão para migrar cargas Cassandra existentes para a AWS sem gerenciar cluster, ou para novas cargas que precisam do modelo de dados wide-column com alta taxa de escrita.
- **Amazon DocumentDB**: compatível com MongoDB, com arquitetura de armazenamento semelhante à do Aurora (replicado em 3 AZs, até 15 réplicas). Para HA e DR, distribua em múltiplas AZs com backups automáticos e snapshots no S3. **Global Clusters** replicam entre Regiões. Há opção **elastic clusters** para sharding.
- **Amazon Timestream**: banco de **séries temporais** para telemetria, IoT e métricas, com camadas de memória e magnética e funções de agregação por tempo. **Timestream for InfluxDB** oferece InfluxDB gerenciado. Confira na documentação a disponibilidade de cada variante para novos clientes.
- **Amazon OpenSearch Service**: busca full-text, análise de logs e busca vetorial, também com opção serverless. Complementa o banco principal (ex.: DynamoDB → zero-ETL → OpenSearch para busca), mas não é banco transacional.
- **Amazon QLDB (histórico)**: aparece em simulados antigos como ledger verificável, mas o suporte terminou em julho de 2025. Não o escolha para uma arquitetura nova. Avalie o requisito de auditoria/imutabilidade com serviços atuais. [Aviso de fim de suporte](#).
- **Amazon Redshift**: data warehouse gerenciado para OLAP. Não serve para ingestão contínua de IoT nem para servir dados via API de baixa latência. Veja a seção 7.

---

## Migração e escolha econômica de banco

### AWS DMS

- **Migração com downtime mínimo:** **full load** dos dados existentes seguido de **CDC** (change data capture), que replica as alterações contínuas até o cutover. A origem continua operando durante a migração.
- **Execução:** replication instance (provisionada, Multi-AZ opcional) ou **DMS Serverless**, com escala automática. **Data validation** compara origem e destino.
- **Homogênea** (Oracle → RDS Oracle, MySQL → Aurora MySQL): muitas vezes as ferramentas nativas do engine, ou as *homogeneous data migrations* do DMS, bastam.
- **Heterogênea** (Oracle → Aurora PostgreSQL): **DMS move dados; DMS Schema Conversion converte esquema e código**, com relatório do que exige ajuste manual. Teste tipos, procedures, consistência e cutover. O antigo AWS SCT também aparece em material de estudo. [DMS Schema Conversion](#).
- **Destinos variados:** além de bancos, o DMS grava em S3, Kinesis, OpenSearch e Redshift, e pode alimentar pipelines com CDC.
- **Grandes volumes com banda limitada:** faça a carga inicial por outro meio (backup nativo no S3, transferência física) e use o DMS apenas para o CDC. Veja [Storage Gateway e migração de dados](#).
- **MGN** migra servidores inteiros (rehost), não é ferramenta de migração de banco para serviço gerenciado.

### Escolha econômica de banco

- **Decisão por carga:** RDS/Aurora para transações relacionais e joins; DynamoDB para acesso chave-valor/documento em grande escala com padrões definidos; Redshift para OLAP; S3 + Athena para análise eventual; ElastiCache/DAX para leituras repetidas. Uma escolha barata por GB pode sair cara em operações, réplicas, backup ou consultas. Meça padrão de acesso e retenção antes de comparar preços.
- **Formato do dado e preço total:** séries temporais (telemetria por timestamp) pedem particionamento por tempo, agregações e política de expiração. Analytics colunar favorece Redshift ou arquivos Parquet/ORC no S3 consultados pelo Athena. Compare padrão de consulta, volume ingerido, retenção quente/fria, custo de consulta e operações. Não escolha um motor apenas porque cobra menos por GiB.
- **Gerenciado vs. EC2:** banco em EC2 só se justifica quando há requisito que o RDS/RDS Custom não atende (versão, extensão, acesso total ao sistema operacional). Caso contrário, perde em esforço operacional e em HA pronta.
- **Alavancas de custo:**
  - Reserved Instances para RDS, Aurora, ElastiCache e DocumentDB;
  - reserved capacity no DynamoDB;
  - Graviton nas instâncias de banco;
  - Aurora Serverless v2 ou DynamoDB on-demand para cargas intermitentes;
  - I/O-Optimized quando o I/O domina;
  - stop em dev/test (com o limite de 7 dias);
  - retenção de snapshots adequada.

### Padrões e pegadinhas de migração

- "Migrar Oracle para Aurora PostgreSQL com o mínimo de downtime": **DMS Schema Conversion** (esquema e código) + **DMS full load + CDC** (dados).
- "Migrar SQL Server reduzindo reescrita de T-SQL": **Babelfish for Aurora PostgreSQL**.
- "Manter a origem ativa até o cutover": **CDC** do DMS.

- "Migrar MongoDB para serviço gerenciado": **DocumentDB** (via DMS). "Cassandra": **Keyspaces**.

## Decisão rápida — Banco de Dados

| Cenário / requisito                                         | Resposta correta                                                            | Distratores clássicos                                                                  |
|-------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Alta disponibilidade / failover de banco relacional         | RDS Multi-AZ DB instance, Multi-AZ DB cluster ou Aurora, conforme o cenário | Read Replica isolada e backups não substituem failover automático                      |
| Descarregar tráfego de leitura de um banco                  | Read Replica ou leitores de RDS Multi-AZ DB cluster/Aurora                  | Standby de RDS Multi-AZ DB instance não atende leitura                                 |
| Timeout de conexão com CPU/memória baixos                   | RDS Proxy                                                                   | Aumentar instância, Read Replica                                                       |
| Cache de leitura do DynamoDB com pequena mudança no cliente | DynamoDB DAX                                                                | ElastiCache exige integração própria; DAX também requer cliente e endpoint específicos |

## 4. Rede e Entrega de Conteúdo

**Regra de ouro da prova:** siga o caminho do pacote. Pergunte **de onde vem** o tráfego (internet, outra VPC, on-premises, serviço AWS), **para onde vai** e **o que precisa acontecer no caminho**: filtrar, balancear, cachear, inspecionar ou manter privado. Depois escolha o menor conjunto de componentes que resolve. Tráfego privado para S3/DynamoDB leva a **gateway endpoint**. Muitas VPCs e redes on-premises levam a **Transit Gateway**. Expor um serviço a outras VPCs sem abrir a rede leva a **PrivateLink**. Latência global com cache leva a **CloudFront**. IP estático global com failover rápido leva a **Global Accelerator**.

### VPC — fundamentos

#### Estrutura da VPC

- A **VPC é regional**, e cada **subnet pertence a uma AZ**. Os blocos CIDR IPv4 vão de **/16 a /28** ([Números da VPC](#)). É possível adicionar **CIDRs secundários** e habilitar **IPv6** (dual-stack).
- **5 IPs reservados por subnet**: os 4 primeiros (rede, roteador da VPC, DNS, uso futuro) e o último (broadcast). Uma subnet /28 tem 16 endereços, mas só 11 utilizáveis.
- **Route tables**: toda tabela tem a **rota local** (comunicação dentro da VPC), que não pode ser removida. Prevalece a **rota mais específica** (longest prefix match). Subnets sem associação explícita usam a *main route table*.
- **Plano de endereçamento e topologia**: dimensione CIDRs da VPC e das subnets para crescimento, IPs reservados pela AWS, ENIs de serviços gerenciados (Lambda, EKS, endpoints) e conectividade híbrida. Evite faixas sobrepostas antes de peering, Transit Gateway ou VPN. Distribua camadas pública/privada por AZ e associe as route tables conforme o destino. Para banco privado, permita apenas as portas e origens da camada de aplicação.
- **Internet Gateway (IGW)**: anexado à VPC. Uma subnet é pública quando sua route table tem rota direta ao IGW, e a instância precisa de endereço público (IPv4 público automático ou **Elastic IP**) para acesso direto. Uma VPC pode ter um IGW anexado por vez. O IGW é redundante e escalável, sem limite de banda próprio. [Rotas de subnet](#).
- **Egress-only Internet Gateway**: equivalente ao NAT para **IPv6**. Permite saída para a internet e bloqueia conexões iniciadas de fora. IPv6 não usa NAT, porque todos os endereços são públicos.
- **DNS da VPC**: atributos `enableDnsSupport` (Resolver da AWS em `VPC CIDR +2`) e `enableDnsHostnames` (nomes DNS públicos para instâncias com IP público). Ambos são exigidos por private hosted zones e pelo private DNS de interface endpoints.
- **Cobrança de IPv4 público**: todo IPv4 público é cobrado por hora, inclusive Elastic IPs em uso. IPv6 e NAT reduzem essa necessidade.

#### Security Groups e Network ACLs

| Critério    | Security Group                                     | Network ACL                                          |
|-------------|----------------------------------------------------|------------------------------------------------------|
| Nível       | ENI (instância, endpoint, RDS, Lambda em VPC)      | Subnet                                               |
| Estado      | <b>Stateful</b> (retorno liberado automaticamente) | <b>Stateless</b> (liberar ida e volta)               |
| Regras      | Somente <b>allow</b>                               | <b>Allow e deny</b>                                  |
| Avaliação   | Todas as regras juntas                             | Em ordem numérica; a primeira que casa decide        |
| Padrão      | Nega entrada; permite saída                        | NACL padrão permite tudo; NACL customizada nega tudo |
| Referências | Pode referenciar <b>outro security group</b>       | Apenas CIDRs                                         |

- **Security Groups:** firewall stateful por instância/interface, allow-list, sem regra de "deny" explícita. **Referenciar o security group de origem** (ex.: a camada de aplicação como origem no SG do banco) é o padrão, porque acompanha automaticamente instâncias que entram e saem do Auto Scaling.
- **Network ACLs:** firewall stateless por subnet, permite allow e deny explícitos. Como não acompanha o estado da conexão, precisa liberar o tráfego nos dois sentidos: a porta do serviço no caminho de ida e as **portas efêmeras** usadas no retorno. Em questões de prova, normalmente é o range amplo `1024-65535`, ajustado ao sistema cliente. Security Groups são stateful e liberam automaticamente o retorno de uma conexão permitida.
- **Bloquear um IP ou uma faixa específica:** só a **NACL** (deny) faz isso na VPC. Security groups não negam. Para bloqueio em camada 7 ou por reputação, use o **WAF**.

## NAT

- **NAT Gateway:** permite a saída de subnets privadas para a internet, bloqueando conexões iniciadas de fora. Fica em uma **subnet pública** com Elastic IP (no tipo zonal) e escala a banda automaticamente. É cobrado por hora e **por GB processado**.
- **NAT zonal vs. regional:** com NAT Gateway **zonal**, um por AZ evita dependência entre AZs e cobrança de tráfego cruzado. Compartilhar um único NAT zonal economiza horas de gateway, mas cria dependência daquela AZ e pode aumentar custos por GB. O tipo **regional** se expande pelas AZs com workloads e simplifica rotas. Compare preço e requisitos, pois ele não oferece NAT privado e pode levar tempo para expandir a uma AZ recém-usada. Use endpoint S3/DynamoDB quando o destino é esse serviço, para não processar esse tráfego via NAT. [Preços](#) · [NAT regional](#).
- **NAT privado:** traduz endereços para comunicação com outras redes privadas (VPCs ou on-premises com **CIDRs sobrepostos**), via Transit Gateway ou VGW, sem acesso à internet.
- **NAT instance (legado):** EC2 que você gerencia (desabilitar *source/destination check*, HA e patches por sua conta). Só é resposta quando o enunciado pede controle total ou custo mínimo em ambiente pequeno.

## VPC endpoints e PrivateLink

- **VPC Gateway Endpoint (S3 e DynamoDB):** adiciona rotas por prefix list às route tables, mantém o tráfego na rede AWS e não tem cobrança adicional do endpoint. Para workloads dentro de uma VPC acessando S3/DynamoDB, costuma ser a opção mais simples e econômica. Funciona só para tráfego originado **na própria VPC**, não de on-premises nem de VPCs pareadas.
- **VPC Interface Endpoint (via AWS PrivateLink):** cria ENIs com IPs privados e security groups, com cobrança por hora e por dados. É usado para a maioria dos serviços gerenciados (Comprehend, Translate, Step Functions, KMS, Secrets Manager, ECR, SSM etc.). Com **private DNS** habilitado, o nome padrão do serviço resolve para o IP do endpoint, sem mudar o código.
- **S3 e DynamoDB suportam hoje tanto gateway quanto interface endpoints:** o interface é necessário quando se precisa de IP privado, acesso de on-premises/outra Região ou controles por security group. Ele não é adicionado à route table como um gateway target.
- **Endpoint policies:** políticas de recurso no endpoint limitam **quais buckets, tabelas ou ações** podem ser acessados por ele (ex.: impedir a exfiltração para buckets de outras contas). Combine com `aws:SourceVpce` na bucket policy para exigir que o acesso venha do endpoint.
- **AWS PrivateLink:** acesso privado a serviços críticos entre VPCs sem exposição à internet. É recomendado tanto para comunicação entre microsserviços quanto para acesso a serviços AWS gerenciados.
  - O **provedor** publica um *endpoint service* atrás de um **NLB** (ou GWLB), e o **consumidor** cria um interface endpoint.
  - O acesso é **unidirecional** (consumidor → serviço), funciona **entre contas** e **com CIDRs sobrepostos**, e não expõe a VPC inteira.
  - É o padrão de SaaS privado.



## Acesso administrativo e remoto

- **Bastion Host:** ponto único de acesso administrativo em subnet pública. Instâncias privadas devem aceitar SSH apenas do IP **privado** do bastion (nunca do público), idealmente referenciando o security group do bastion.
- **AWS Systems Manager Session Manager:** resposta padrão para "eliminar chaves SSH compartilhadas" mantendo acesso seguro, auditável (logs de sessão no S3/CloudWatch) e escalável. Combinado com VPC Endpoints ( `ssm` , `ssmmessages` , `ec2messages` ), gerencia EC2 totalmente privado, sem bastion hosts e sem portas de entrada abertas.
- **EC2 Instance Connect Endpoint:** SSH/RDP para instâncias em subnets privadas **sem IP público e sem bastion**, com autorização IAM e registro no CloudTrail.
- **AWS Client VPN:** acesso remoto seguro (OpenVPN) de usuários a recursos em VPC, com autenticação por certificado, AD ou SAML. Habilite logs de conexão no **CloudWatch Logs**, ajuste a retenção conforme a auditoria e acompanhe as cobranças de endpoint, associações e logs. Se precisar de arquivo de longo prazo, exporte/entregue os logs ao S3 por um fluxo separado. [Logs de conexão](#).
- **AWS Verified Access:** acesso **zero trust** a aplicações internas sem VPN, avaliando identidade e postura do dispositivo a cada requisição.

## Monitoramento e diagnóstico de rede

- **VPC Flow Logs:** registram **metadados** do tráfego IP (origem, destino, portas, bytes, `ACCEPT` / `REJECT` ) no nível de VPC, subnet ou ENI, com destino no CloudWatch Logs, S3 ou Firehose. **Não capturam o conteúdo** dos pacotes. É a resposta para "descobrir por que a conexão é rejeitada" ou para auditoria de tráfego.
- **Traffic Mirroring:** copia os **pacotes completos** de uma ENI para appliances de IDS/análise forense.
- **Reachability Analyzer:** verifica, sem enviar tráfego, se há caminho entre dois recursos e **qual componente bloqueia** (SG, NACL, rota). **Network Access Analyzer:** identifica acessos de rede não intencionais em relação a requisitos (ex.: "nenhum banco acessível da internet").

## Números da VPC

| Métrica                      | Valor                  | Observação                                   |
|------------------------------|------------------------|----------------------------------------------|
| Internet Gateways por VPC    | máximo <b>1</b>        | Não é possível anexar mais de um IGW por vez |
| CIDR — menor bloco permitido | /28 (16 endereços)     |                                              |
| CIDR — maior bloco permitido | /16 (65.536 endereços) |                                              |

## Padrões e pegadinhas de VPC

- "Instância em subnet pública sem acesso à internet": falta **rota para o IGW** ou **IP público/Elastic IP**, ou SG/NACL bloqueando.
- "Instâncias privadas precisam baixar patches": **NAT Gateway** em subnet pública (um por AZ para HA).
- "Reduzir custo de NAT com tráfego alto para o S3": **gateway endpoint do S3**.
- "Bloquear um IP malicioso": **NACL deny** (ou WAF, se HTTP).
- "NACL liberou a porta 443 de entrada, mas as respostas não chegam": falta liberar as **portas efêmeras** de saída.
- "Acesso a instâncias privadas sem bastion e sem chaves SSH": **Session Manager** (ou EC2 Instance Connect Endpoint).
- "Garantir que o S3 só seja acessado pelo endpoint da VPC": **endpoint policy** + `aws:SourceVpce` na bucket policy.
- "Investigar tráfego rejeitado": **VPC Flow Logs** / **Reachability Analyzer**.
- "IPv6 com saída para a internet e sem entrada": **egress-only IGW**.

## Interconexão de redes

| Necessidade                                       | Solução                        | Observação                                                        |
|---------------------------------------------------|--------------------------------|-------------------------------------------------------------------|
| Conectar <b>duas</b> VPCs                         | <b>VPC Peering</b>             | Sem custo por hora, não transitivo, sem CIDRs sobrepostos         |
| Conectar <b> muitas</b> VPCs e on-premises        | <b>Transit Gateway</b>         | Hub regional, roteamento transitivo, segmentação por route tables |
| Rede global com política central                  | <b>AWS Cloud WAN</b>           | Backbone gerenciado entre Regiões e sites                         |
| Expor <b>um serviço</b> a outras VPCs/contas      | <b>PrivateLink</b>             | Unidirecional; funciona com CIDRs sobrepostos                     |
| Comunicação <b>service-to-service</b> em camada 7 | <b>VPC Lattice</b>             | Políticas IAM por serviço, HTTP/gRPC                              |
| Compartilhar subnets entre contas                 | <b>VPC compartilhada (RAM)</b> | Uma VPC central, recursos de várias contas                        |
| On-premises rápido e barato de ativar             | <b>Site-to-Site VPN</b>        | IPsec pela internet, 2 túneis                                     |
| On-premises com banda e latência previsíveis      | <b>Direct Connect</b>          | Link físico privado, semanas para provisionar                     |

- **VPC Peering:** conexão ponto a ponto entre duas VPCs, **não transitiva**, que não escala para muitas VPCs.
  - Funciona entre contas e entre Regiões.
  - Exige **CIDRs não sobrepostos** e **rotas nos dois lados**.
  - **Não há roteamento edge-to-edge:** uma VPC não usa o IGW, o NAT, a VPN ou o Direct Connect da outra.
  - Na mesma Região, security groups podem referenciar os da VPC pareada.
- **AWS Transit Gateway:** hub central que conecta centenas de VPCs/contas e redes on-premises via rotas (inclusive inter-regionais, por **TGW peering**). É a resposta padrão para conectar muitas VPCs de forma centralizada e escalável. VPC Peering não escala (complexidade combinatória), e múltiplas VPNs/Direct Connect dedicados são caros.
  - **Route tables do TGW** segmentam domínios (ex.: produção não fala com desenvolvimento, mas as duas falam com serviços compartilhados).
  - **ECMP** sobre várias VPNs agrega banda.
  - É **compartilhado entre contas via AWS RAM**.
  - É cobrado por attachment e por GB.
- **VPC compartilhada (AWS RAM):** a conta de rede é dona da VPC e compartilha **subnets** com outras contas da Organization. Estas lançam recursos nelas sem gerenciar a rede. Isso reduz o número de VPCs, peerings e NATs.
- **AWS Cloud WAN:** rede global gerenciada por uma **política central** que conecta VPCs, VPNs e Direct Connect em várias Regiões, com segmentação. É indicado quando o Transit Gateway por Região + peering fica difícil de operar.
- **AWS VPC Lattice:** serviço de application networking (camada 7) que conecta, monitora e protege a comunicação **entre serviços** (não entre redes inteiras) através de múltiplas VPCs e contas, sem exigir gerenciamento manual de peering/rotas. É diferente do **Transit Gateway** (camada 3, conecta VPCs/redes inteiras via rotas IP): o VPC Lattice entende HTTP/HTTPS/gRPC e permite políticas de acesso granulares por serviço (via IAM). É a resposta padrão quando o requisito é service-to-service networking e observabilidade em arquiteturas de microserviços multi-conta, não conectividade de rede em larga escala.

### Conectividade híbrida: VPN e Direct Connect

- **AWS VPN Site-to-Site:** túnel IPsec sobre a internet pública, usado como conexão principal de baixo custo ou como backup do Direct Connect.
  - Cada conexão tem **2 túneis**, em endpoints diferentes da AWS, para alta disponibilidade.

- Cada túnel tem banda limitada (em torno de 1,25 Gbps). Para mais banda, use várias conexões com **ECMP no Transit Gateway**.
- Termina em um **Virtual Private Gateway** (uma VPC) ou em um **Transit Gateway** (várias VPCs). O lado on-premises é o **Customer Gateway**.
- O roteamento pode ser estático ou dinâmico (**BGP**).
- **Accelerated VPN** usa a rede do Global Accelerator para reduzir a variação de latência.
- **AWS Direct Connect**: conexão privada dedicada on-premises ↔ AWS, com baixa latência previsível. Não passa pela internet, e a transferência de dados de saída custa menos que pela internet.
  - **Dedicated** (1, 10, 100 Gbps, porta física própria) ou **Hosted** (de 50 Mbps a 10+ Gbps via parceiro).
  - **O provisionamento leva semanas**: não é resposta para "conectar amanhã". Nesse caso, use VPN.
  - **Estratégia recomendada**: Direct Connect como principal + VPN Site-to-Site como backup barato (duplicar Direct Connect é caro). Uma única conexão compartilhada via Transit Gateway entre várias VPCs é mais barata que múltiplas conexões dedicadas.
  - **Resiliência máxima**: conexões em **dois locais de Direct Connect diferentes**, com mais de uma conexão por local (modelos do *Resiliency Toolkit*).
- **VIFs do Direct Connect**:
  - **Private VIF**: acessa uma VPC por IP privado.
  - **Public VIF**: acessa endpoints públicos de serviços AWS, como S3, usando IPs públicos, mas sem passar por um ISP.
  - **Transit VIF**: acessa Transit Gateways associados a um Direct Connect Gateway.
  - **Direct Connect Gateway**: conecta a VIF privada ou de trânsito a múltiplos gateways/VPCs compatíveis em contas e Regiões, mas não cria conectividade transitiva entre essas VPCs.
- **Criptografia no Direct Connect**: o link **não é criptografado por padrão**. Use **MACsec** (camada 2, em conexões dedicadas compatíveis) ou uma **VPN IPsec sobre o Direct Connect** (via public VIF ou VPN com IP privado sobre transit VIF) quando o requisito exige criptografia em trânsito.
- **AWS VPN CloudHub**: topologia hub-and-spoke sobre um virtual private gateway para que múltiplos sites conectados por Site-to-Site VPN (e sites ligados por Direct Connect ao mesmo gateway) comuniquem-se entre si, não apenas com a VPC.
- **Banda e limite efetivo**: estime volume diário ÷ janela de transferência, acrescente overhead e picos, e compare com o throughput contratado/real de VPN, Direct Connect, NAT, instâncias e serviços.
  - Uma VPN pode ser mais barata e rápida de contratar. O Direct Connect pode justificar-se por volume constante, previsibilidade e requisitos de conexão privada.
  - Para alta disponibilidade, planeje túneis/links redundantes e o failover.
  - Se o fluxo concorre com tráfego crítico, limite ou agende o volume de transferência (por exemplo, com o controle de banda do DataSync) e teste a vazão fim a fim.
  - Para migração offline de novos clientes, avalie o Data Transfer Terminal. [Controle de banda do DataSync](#).

## Padrões e pegadinhas de interconexão

- "Dezenas de VPCs em várias contas + on-premises, com gestão central": **Transit Gateway** (compartilhado via RAM).
- "VPC A pareada com B, e B com C: A fala com C?" **Não**. Peering não é transitivo.
- "Conectar redes com CIDRs sobrepostos": **PrivateLink** (serviço específico) ou **NAT privado**. Peering não aceita.
- "Precisa de conectividade híbrida em dias": **Site-to-Site VPN**. "Banda previsível e alta, longo prazo": **Direct Connect** (+ VPN de backup).
- "Direct Connect precisa ser criptografado": **MACsec** ou **VPN IPsec sobre o Direct Connect**.

- "Acessar o S3 pelo Direct Connect sem internet": **public VIF** (ou private/transit VIF + interface endpoint do S3).
- "Aumentar a banda da VPN além de um túnel": várias VPNs com **ECMP no Transit Gateway**.
- "Várias contas lançando recursos na mesma rede central": **VPC compartilhada via RAM**.

## Load Balancers

| Critério      |                                          |                                                                  |                                 |
|---------------|------------------------------------------|------------------------------------------------------------------|---------------------------------|
| Camada        | 7 (HTTP/HTTPS, gRPC, WebSocket)          | 4 (TCP, UDP, TLS)                                                | 3 (IP, via GENEVE)              |
| Roteamento    | Path, host, header, query string, método | Porta/protocolo                                                  | Transparente para appliances    |
| IP            | DNS (IPs mudam)                          | <b>IP estático por AZ</b> / Elastic IP                           | —                               |
| IP do cliente | Header X-Forwarded-For                   | <b>Preservado</b>                                                | Preservado                      |
| Alvos         | Instância, IP, <b>Lambda</b>             | Instância, IP, <b>ALB</b>                                        | Instância, IP (appliances)      |
| WAF           | <b>Sim</b>                               | Não                                                              | Não                             |
| Caso típico   | Web, microsserviços, APIs                | Latência ultrabaixa, milhões de req/s, UDP, IP fixo, PrivateLink | Firewalls, IDS/IPS de terceiros |

- **Application Load Balancer (ALB)**: camada 7 (HTTP/HTTPS), com health checks de conteúdo real da aplicação (detecta erros 5xx via requisição HTTP). Suporta AWS WAF diretamente e roteamento por path/host/header. É a origem recomendada no CloudFront para conteúdo dinâmico.
  - **Autenticação** integrada via Cognito ou OIDC.
  - **Redirecionamentos** (HTTP → HTTPS) e respostas fixas.
  - **Weighted target groups** para blue/green e canary.
  - Sticky sessions e *slow start*.
  - Vários certificados por listener via **SNI**.
- **Network Load Balancer (NLB)**: camada 4 (TCP/UDP), IP estático e altíssima performance. **Também suporta health checks** (TCP, HTTP ou HTTPS) e listeners TLS para descarregar a criptografia das EC2, preservando o IP de origem em cenários suportados. A diferença para o ALB é a ausência de roteamento por conteúdo HTTP (path/host/header) e de integração nativa com AWS WAF. É o load balancer usado para UDP, SSH/TCP e requisitos de camada 4.
  - É obrigatório para expor um **endpoint service do PrivateLink**.
  - Um **ALB como alvo do NLB** dá IP estático a uma aplicação que precisa de roteamento HTTP.
- **Gateway Load Balancer (GWLB)**: especializado para integrar appliances de segurança de terceiros (firewalls virtuais), combinando balanceamento com encapsulamento GENEVE (porta 6081). É a resposta padrão para "inspecionar todo o tráfego antes de chegar aos servidores". O tráfego é desviado por **GWLB endpoints** nas route tables, de forma transparente, e as appliances escalam e ficam altamente disponíveis.
- **Classic Load Balancer**: legado. Não é resposta para novos designs.

## Recursos comuns dos load balancers

- **Internet-facing vs. internal**: o internet-facing fica em subnets públicas, e os alvos podem ficar em subnets **privadas**. O internal atende tráfego dentro da VPC ou do on-premises.
- **Cross-zone load balancing**: distribui igualmente entre alvos de todas as AZs. É **ativado por padrão no ALB** (sem custo) e **desativado por padrão no NLB/GWLB**, com cobrança de tráfego entre AZs quando ativado.

- **TLS:** termine no load balancer com certificado do **ACM** (renovação automática). Para criptografia fim a fim, **re-criptografe** até os alvos ou use NLB com passthrough TCP.
- **Security groups:** o SG dos alvos deve aceitar tráfego **apenas do SG do load balancer**. ALB e NLB suportam security groups.
- **Deregistration delay** (connection draining) conclui as requisições em andamento antes de remover um alvo. **Health checks** do load balancer podem ser usados pelo Auto Scaling. Veja [Health checks e substituição](#).

## Padrões e pegadinhas de load balancers

- "Rotear `/api` e `/images` para serviços diferentes": **ALB** com regras de path.
- "Clientes exigem um IP fixo para liberar no firewall": **NLB** (Elastic IP) ou **Global Accelerator**.
- "Jogo com UDP e latência mínima": **NLB**.
- "Aplicação precisa do IP real do cliente atrás do ALB": header `X-Forwarded-For`.
- "Proteger contra SQL injection no balanceador": **ALB + WAF**.
- "Inspeccionar todo o tráfego com firewall de terceiros": **GWLB**.
- "Login com Cognito antes de chegar à aplicação": **autenticação no ALB**.
- "Alvos em uma AZ recebem mais carga que outros no NLB": **cross-zone load balancing** desativado.

## Entrega de conteúdo e DNS

### Amazon Route 53 — registros e hosted zones

- **Amazon Route 53:** DNS gerenciado com políticas de roteamento (simples, ponderado, latência, geolocalização, geoproximidade, failover, multivalue, IP-based) + health checks. Oferece SLA de 100% de disponibilidade e registro de domínios.
- **Registros:**
  - `A / AAAA` (IP);
  - `CNAME` (nome → nome, **não permitido no apex** da zona, como `exemplo.com`);
  - **Alias:** extensão da AWS que aponta para recursos como ALB, NLB, CloudFront, S3 website, API Gateway e Global Accelerator, **funciona no apex** e **não cobra consultas** para recursos AWS. Prefira Alias a CNAME para recursos AWS;
  - **TTL** alto reduz custo e latência, mas atrasa as mudanças.
- **Public hosted zone:** resolução na internet. **Private hosted zone:** resolução interna em uma ou mais VPCs associadas, inclusive de outras contas.
- **Private hosted zones:** para resolução interna em uma VPC, os atributos `enableDnsSupport` e `enableDnsHostnames` precisam estar habilitados. Namespaces sobrepostos são permitidos e a correspondência mais específica vence. Uma Route 53 Resolver rule de mesmo domínio tem precedência sobre a private hosted zone.

### Políticas de roteamento do Route 53

| Política        | Como decide                                                                            | Caso típico                         |
|-----------------|----------------------------------------------------------------------------------------|-------------------------------------|
| <b>Simple</b>   | Um registro (pode ter vários valores, retornados em ordem aleatória, sem health check) | Um único recurso                    |
| <b>Weighted</b> | Proporção por peso                                                                     | Canary, teste A/B, migração gradual |



| Política                  | Como decide                                                                  | Caso típico                                   |
|---------------------------|------------------------------------------------------------------------------|-----------------------------------------------|
| <b>Latency</b>            | Menor latência medida entre o usuário e a Região                             | Aplicação multi-Região com melhor desempenho  |
| <b>Failover</b>           | Primário enquanto saudável; secundário se falhar                             | DR ativo-passivo                              |
| <b>Geolocation</b>        | País/continente/estado do usuário                                            | Conteúdo localizado, restrição legal, idioma  |
| <b>Geoproximity</b>       | Distância, com <b>bias</b> para expandir ou encolher uma área (Traffic Flow) | Deslocar tráfego entre Regiões                |
| <b>Multivalued answer</b> | Até 8 registros <b>saudáveis</b> aleatórios                                  | Balanceamento simples no DNS com health check |
| <b>IP-based</b>           | CIDR de origem do cliente                                                    | Rotear por rede/ISP conhecido                 |

- **Geolocation ≠ latency**: geolocalização decide por **onde o usuário está** (conformidade, idioma). Latência decide por **desempenho**. Configure um registro **default** em geolocation para os locais não mapeados.
- **Health checks**: monitoram um endpoint (HTTP/HTTPS/TCP, com correspondência de texto opcional), **outros health checks** (*calculated*) ou um **alarme do CloudWatch**. O alarme é o caminho para recursos **privados**, porque os health checkers da Route 53 ficam fora da VPC. Os intervalos estão em [Números dos health checks do Route 53](#).
- **Route 53 Application Recovery Controller (ARC)**: *routing controls* para failover manual confiável entre Regiões, *readiness checks* e **zonal shift** para tirar uma AZ problemática do tráfego do ALB/NLB.

## Números dos health checks do Route 53

| Métrica                          | Valor       | Observação                                                            |
|----------------------------------|-------------|-----------------------------------------------------------------------|
| Intervalo padrão entre checagens | 30 segundos |                                                                       |
| Intervalo "fast"                 | 10 segundos | Custo adicional; não pode ser alterado após a criação do health check |

## Route 53 Resolver (DNS híbrido)

- **Inbound endpoint**: servidores DNS **on-premises** resolvem nomes da AWS (private hosted zones, endpoints) encaminhando consultas para IPs na VPC.
- **Outbound endpoint + forwarding rules**: recursos **na VPC** resolvem domínios on-premises (ex.: `corp.local`), encaminhando para o DNS corporativo. As regras podem ser **compartilhadas com outras contas via RAM**.
- **Route 53 Resolver DNS Firewall**: bloqueia consultas a domínios maliciosos ou não autorizados a partir das VPCs (prevenção de exfiltração via DNS). **Query logging** registra as consultas DNS da VPC.
- **DNSSEC**: assinatura de zonas públicas e validação no Resolver, contra falsificação de respostas.

## Amazon CloudFront

- **Amazon CloudFront**: CDN global com centenas de pontos de presença e caches regionais.
  - Suporta origem S3 (estático) e ALB/origem personalizada on-premises (dinâmico) simultaneamente em uma mesma distribuição: multi-origem, com **cache behaviors** por path (ex.: `/api/*` → ALB, `/*` → S3).
  - Absorve picos e DDoS na borda antes da origem (**Shield Standard** incluso).
  - O cache pode ser personalizado por cabeçalho (ex.: `Accept-Language`), query string ou cookie.
- **Origens**: S3 (com **OAC**, veja [Segurança e criptografia do S3](#)), ALB, EC2, API Gateway, Lambda function URL, MediaPackage ou qualquer HTTP. **VPC origins** permitem usar ALB/NLB/EC2 em **subnets privadas** como origem, sem expô-los à internet.



- **Cache:** *cache policies* definem a cache key e o TTL, e *origin request policies* definem o que é enviado à origem sem entrar na cache key. Incluir headers ou cookies demais na cache key **derruba a taxa de acerto**.
- **Invalidação** remove objetos antes do TTL, com custo após uma franquia. Para deploys frequentes, prefira **nomes de arquivo versionados** (ex.: `app.v42.js`).
- **Origin Shield:** camada de cache adicional centralizada que reduz a carga e o custo na origem.
- **Origin failover:** um **origin group** com origem primária e secundária troca automaticamente em erros ou timeouts (para GET/HEAD/OPTIONS).
- **Segurança:**
  - **viewer protocol policy** (redirecionar para HTTPS);
  - certificado do **ACM em us-east-1** para domínio personalizado;
  - **AWS WAF** na distribuição;
  - **geo restriction** (allow/deny por país);
  - **signed URLs/cookies** para conteúdo privado (seção 2);
  - **field-level encryption** para campos sensíveis de formulários;
  - header secreto da origem para impedir acesso direto ao ALB (ou VPC origins).
- **Conteúdo dinâmico e uploads:** o CloudFront também acelera requisições não cacheáveis (conexões persistentes com a origem, rede da AWS) e aceita POST/PUT e WebSocket.
- **Custo: price classes** limitam os pontos de presença usados, com custo menor e latência maior em algumas regiões. A transferência da origem AWS para o CloudFront não é cobrada.
- **Computação na borda:** CloudFront Functions e Lambda@Edge. Veja [Computação na borda do CloudFront](#).

## AWS Global Accelerator

- **AWS Global Accelerator:** usa **2 IPs anycast estáticos** e a rede backbone da AWS para rotear tráfego TCP/UDP ao endpoint mais próximo e saudável, com failover automático em segundos. Não faz cache de conteúdo (diferente do CloudFront). É ideal para aplicações multi-Região não-HTTP ou que precisam de IP estático (combinado com NLB/ALB como endpoints).
- **Endpoints:** ALB, NLB, instâncias EC2 e Elastic IPs, organizados em **endpoint groups por Região**. O **traffic dial** controla a porcentagem de tráfego por Região (ex.: para manutenção ou blue/green regional), e **pesos** distribuem entre endpoints. Há **client affinity** opcional.
- **vs. Route 53:** o failover do DNS depende do **TTL e do cache dos clientes**. O Global Accelerator muda o roteamento **sem trocar o IP**, então o failover é mais rápido e previsível.
- **Custom routing accelerator:** mapeia usuários para instâncias/portas específicas (ex.: salas de jogo multiplayer).

## CloudFront vs. Global Accelerator vs. Route 53

| Requisito                                                       | Resposta                                    |
|-----------------------------------------------------------------|---------------------------------------------|
| Cachear conteúdo estático/dinâmico HTTP perto dos usuários      | <b>CloudFront</b>                           |
| IP estático global, TCP/UDP, failover multi-Região em segundos  | <b>Global Accelerator</b>                   |
| Rotear por latência/geografia/peso no DNS, sem proxy no caminho | <b>Route 53</b>                             |
| Uploads rápidos de longe para um bucket S3                      | <b>S3 Transfer Acceleration</b>             |
| Proteger aplicação web global contra ataques de camada 7        | <b>CloudFront + WAF</b> (+ Shield Advanced) |

## Padrões e pegadinhas de DNS e CDN

- "Apontar o domínio raiz `exemplo.com` para um ALB/CloudFront": registro **Alias** (CNAME não é permitido no apex).
- "Usuários da Europa devem ser atendidos apenas pela Região da UE (conformidade)": **geolocation**. "Pelo menor tempo de resposta": **latency**.
- "Enviar 10% do tráfego para a nova versão": **weighted**.
- "DR ativo-passivo baseado em saúde": **failover** + health checks.
- "Health check de recurso privado": **alarme do CloudWatch** associado ao health check.
- "Servidores on-premises precisam resolver private hosted zones": **Resolver inbound endpoint**. "A VPC precisa resolver `corp.local`": **outbound endpoint + forwarding rule**.
- "Reduzir a carga da origem e a latência de conteúdo estático global": **CloudFront**. "Taxa de acerto do cache baixa": remova headers/cookies/query strings desnecessários da **cache key**.
- "Usuários acessam o ALB direto, contornando o CloudFront/WAF": **VPC origins** (ALB privado) ou header secreto + regra no ALB, e SG com a *prefix list* do CloudFront.
- "Certificado do CloudFront não aparece": precisa estar no **ACM em us-east-1**.
- "Failover global rápido sem depender de TTL de DNS": **Global Accelerator**.

## Decisão rápida — Rede e Entrega de Conteúdo

| Cenário / requisito                                                                         | Resposta correta                     | Distratores clássicos              |
|---------------------------------------------------------------------------------------------|--------------------------------------|------------------------------------|
| Tráfego não pode passar pela internet (S3/DynamoDB)                                         | VPC Gateway Endpoint                 | NAT Gateway, Internet Gateway, VPN |
| Tráfego privado para outros serviços gerenciados (Comprehend, Translate, Step Functions...) | VPC Interface Endpoint / PrivateLink | VPC Peering, NAT Gateway           |
| Conectar dezenas/centenas de VPCs de forma centralizada                                     | AWS Transit Gateway                  | VPC Peering manual, múltiplos VGW  |
| Inspecionar todo o tráfego com appliance de terceiros                                       | Gateway Load Balancer                | NLB, ALB, Transit Gateway          |
| Health check de conteúdo de aplicação (erro 5xx, path específico)                           | Application Load Balancer            | Network Load Balancer              |
| UDP / IP estático / baixa latência multi-região                                             | AWS Global Accelerator + NLB         | CloudFront, ALB, Route 53 sozinho  |
| CDN para conteúdo HTTP(S) cacheável                                                         | Amazon CloudFront                    | Global Accelerator                 |
| Eliminar chaves SSH compartilhadas                                                          | AWS Systems Manager Session Manager  | Bastion Host, STS                  |

## 5. Segurança, Identidade e Criptografia

**Regra de ouro da prova:** separe controles **preventivos** (impedem: IAM, SCP, permissions boundary, Block Public Access, security groups, WAF, KMS), **detectivos** (avisam: CloudTrail, Config, GuardDuty, Macie, Inspector, Security Hub) e **responsivos** (corrigem: EventBridge + Lambda/Systems Manager Automation, remediação do Config). Quando o enunciado diz "garantir que nunca aconteça", a resposta é preventiva. Quando diz "detectar e alertar", é detectiva. Prefira sempre **credenciais temporárias** (roles, federação) a chaves de longo prazo, e **serviços gerenciados** a soluções customizadas.

| Requisito                                       | Serviço                                           |
|-------------------------------------------------|---------------------------------------------------|
| Quem pode fazer o quê em uma conta              | <b>IAM</b> (políticas, roles)                     |
| Teto de permissões para contas/OUs              | <b>SCP</b> (Organizations)                        |
| Teto de permissões para recursos da organização | <b>RCP</b> (Organizations)                        |
| Acesso de funcionários a várias contas          | <b>IAM Identity Center</b>                        |
| Login de usuários finais de aplicativos         | <b>Cognito</b>                                    |
| Chaves de criptografia gerenciadas              | <b>KMS</b> (ou <b>CloudHSM</b> para HSM dedicado) |
| Certificados TLS                                | <b>ACM</b>                                        |
| Senhas e credenciais com rotação                | <b>Secrets Manager</b>                            |
| Filtrar HTTP (SQLi, XSS, bots, rate limit)      | <b>WAF</b>                                        |
| DDoS                                            | <b>Shield</b> (Standard/Advanced)                 |
| Firewall stateful/IPS na VPC                    | <b>Network Firewall</b>                           |
| Quem fez qual chamada de API                    | <b>CloudTrail</b>                                 |
| Recursos fora da configuração exigida           | <b>Config</b>                                     |
| Ameaças e comportamento malicioso               | <b>GuardDuty</b>                                  |
| Vulnerabilidades em EC2/ECR/Lambda              | <b>Inspector</b>                                  |
| PII em buckets S3                               | <b>Macie</b>                                      |
| Visão central de findings e conformidade        | <b>Security Hub</b>                               |

### IAM

#### Fundamentos do IAM

- **Raiz e MFA:** proteja o usuário raiz com MFA, evite chaves de acesso raiz e use identidades federadas ou roles para operações diárias. Reserve o root para as poucas tarefas que o exigem (ex.: alterar o plano de suporte, fechar a conta, restaurar permissões). Em organizações com **gerenciamento centralizado de acesso raiz**, credenciais raiz das contas-membro podem ser removidas. Para usuários com acesso humano, exija MFA e permissões mínimas. [Boas práticas da conta raiz](#).
- **Identidades:** *users* (identidade de longo prazo, a ser evitada para humanos: prefira o Identity Center), *groups* (agrupam usuários para anexar políticas e **não podem ser principal** em uma política) e **roles** (identidade assumida via **STS**, com credenciais temporárias).

- Usuários, grupos, roles e políticas seguem o **princípio do menor privilégio**. Evite `*` como principal, ações amplas (`lambda:*`) ou permissões concedidas a todos os recursos quando só um é necessário.
- **IAM Role > qualquer credencial estática**: sempre que EC2, Lambda ou ECS precisam acessar outro serviço AWS de forma segura, a resposta é uma IAM Role (nunca usuário IAM, grupo IAM ou credenciais copiadas/hardcoded). Role de instância EC2 (via *instance profile*), `taskRoleArn` do ECS e execution role do Lambda são os mecanismos corretos conforme o serviço.
- **Workloads fora da AWS: IAM Roles Anywhere** troca um certificado X.509 (de uma PKI própria ou do ACM Private CA) por credenciais temporárias, em vez de access keys em servidores on-premises.
- **Access keys**: quando inevitáveis, rotacione-as, nunca as coloque em código ou repositório, e monitore o uso com o *credential report* e o *last accessed*.
- **Política de senha global do IAM**: aplica requisitos de complexidade/comprimento centralizadamente. Rotação periódica obrigatória não é nativa do IAM.
- **Responsabilidade compartilhada**: a AWS protege a infraestrutura física e os serviços gerenciados que opera. O cliente configura identidade, dados, rede e controles do workload. Em EC2, o cliente também gerencia sistema operacional e patches. Em Lambda e outros serviços mais gerenciados, a AWS assume mais da camada de infraestrutura, mas a aplicação e suas permissões continuam responsabilidade do cliente. [Modelo da AWS](#).

## Tipos de política

| Tipo                        | Anexada a                                                        | Função                                                                             |
|-----------------------------|------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Identity-based</b>       | User, group, role                                                | Concede permissões à identidade                                                    |
| <b>Resource-based</b>       | Recurso (bucket S3, fila SQS, chave KMS, função Lambda, segredo) | Concede acesso ao recurso, inclusive a <b>outras contas</b> ; tem <b>Principal</b> |
| <b>Trust policy</b>         | Role                                                             | Define <b>quem pode assumir</b> a role                                             |
| <b>Permissions boundary</b> | User ou role                                                     | <b>Teto</b> do que as identity policies podem conceder                             |
| <b>SCP</b>                  | Conta, OU ou root da Organization                                | <b>Teto</b> para identidades das contas-membro                                     |
| <b>RCP</b>                  | Conta, OU ou root da Organization                                | <b>Teto</b> para recursos das contas-membro (S3, KMS, STS, SQS, Secrets Manager)   |
| <b>Session policy</b>       | Sessão de <code>AssumeRole</code> / federação                    | Restringe a sessão abaixo das permissões da role                                   |

- **Permissions boundary**: policy anexada a um IAM user ou role que define o máximo de permissões que identity policies podem conceder. Ela não concede acesso por si só e não pode ser anexada a IAM groups. É a resposta para delegar a criação e a gestão de roles sem permitir escalada de privilégio. SCP limita contas/OU's, enquanto boundary limita a identidade específica.
- **Políticas gerenciadas vs. inline**: gerenciadas (da AWS ou do cliente) são reutilizáveis e versionadas. Inline ficam presas a uma única identidade. Prefira políticas gerenciadas pelo cliente para menor privilégio.

## Avaliação de políticas

- **Ordem**: tudo começa **negado implicitamente**. Um **deny explícito** em qualquer política **sempre vence**. Um allow só vale se nenhum teto (SCP, RCP, boundary, session policy) o bloquear.
- **Mesma conta**: basta um allow na identity policy **ou** na resource-based policy. **Entre contas**: a conta de origem precisa permitir (identity policy) **e** o recurso precisa permitir (resource policy ou trust policy da role).
- **Condições em políticas**: podem combinar múltiplas condições (ex.: `aws:PrincipalOrgID`, Região, IP) e restringem a permissão apenas quando *todas* as condições se cumprem simultaneamente. Chaves frequentes:
  - `aws:SourceIp`;

- `aws:RequestedRegion` (restringir Regiões);
  - `aws:MultiFactorAuthPresent`;
  - `aws:SecureTransport`;
  - `aws:PrincipalOrgID` / `aws:ResourceOrgID`;
  - `aws:SourceVpce`;
  - `aws:PrincipalTag` / `aws:ResourceTag`.
- **ABAC (Attribute-Based Access Control)**: permissões baseadas em **tags** da identidade e do recurso (ex.: permitir quando `aws:PrincipalTag/projeto = aws:ResourceTag/projeto`). Escala melhor que criar uma política por projeto, porque novos recursos com a tag certa ganham acesso sem editar políticas.

## Acesso entre contas

- Uma role na conta de destino define **quem pode assumi-la** na trust policy. A entidade de origem precisa de autorização para `sts:AssumeRole`, e as permissões da role delimitam o que as credenciais temporárias podem fazer.
- Para S3 e outros serviços compatíveis, uma política baseada em recurso pode autorizar um principal externo diretamente. Se houver SSE-KMS, a política da chave e as permissões KMS também precisam permitir a operação. SCP, permissions boundary e deny explícito continuam limitando o acesso. [Acesso entre contas](#).
- **Terceiros e confused deputy**: quando um fornecedor externo assume uma role na sua conta, exija um **External ID** na trust policy. Para serviços AWS agindo em seu nome, restrinja com `aws:SourceArn` / `aws:SourceAccount`.

## Ferramentas de análise do IAM

- **IAM Access Analyzer**:
  - identifica recursos compartilhados com **entidades externas** à conta ou organização (buckets, roles, chaves KMS, filas);
  - aponta **acesso não utilizado** (roles, chaves e permissões ociosas);
  - **valida políticas** contra boas práticas;
  - **gera políticas de menor privilégio** a partir da atividade registrada no CloudTrail.
- **Credential report** (status de senhas, MFA e chaves de todos os usuários), **last accessed information** (quais serviços uma identidade realmente usa) e **policy simulator** (testar o efeito de políticas).

## Padrões e pegadinhas de IAM

- "Aplicação na EC2 precisa acessar o S3": **IAM role via instance profile**, nunca access keys na instância.
- "Deixar desenvolvedores criarem roles sem conseguirem se dar mais permissões": **permissions boundary**.
- "Acesso cruzado seguro para um fornecedor externo": role com trust policy + **External ID**.
- "Deny explícito vs. allow": o **deny vence** sempre.
- "Acesso a recursos por projeto sem escrever uma política por projeto": **ABAC com tags**.
- "Descobrir buckets e roles acessíveis de fora da organização": **IAM Access Analyzer**.
- "Exigir MFA para ações sensíveis": condição `aws:MultiFactorAuthPresent`.
- "Servidores on-premises precisam chamar APIs AWS sem chaves de longo prazo": **IAM Roles Anywhere**.

---

## AWS Organizations

### Estrutura e governança multi-conta

- **Management account** (pagadora, fora do alcance das SCPs), **OUs** (hierarquia por ambiente, unidade de negócio ou função) e contas-membro. Separe cargas em **várias contas** para isolar o impacto, o faturamento e as permissões: produção, desenvolvimento, segurança/log, rede compartilhada.
- **Consolidated billing**: fatura única, **descontos por volume agregados** e **compartilhamento de Reserved Instances e Savings Plans** entre as contas (pode ser desativado por conta).
- **Delegated administrator**: permite que uma conta de segurança administre GuardDuty, Security Hub, Config, Firewall Manager, IAM Access Analyzer e outros para toda a organização, sem usar a management account.
- **Outras políticas da Organization**: *tag policies* (padronizar tags), *backup policies* (planos do AWS Backup em todas as contas) e políticas de opt-out de uso de dados por serviços de IA.
- **AWS Control Tower**: configura uma **landing zone** multi-conta com boas práticas:
  - contas de log e auditoria;
  - Identity Center;
  - **Account Factory** para criar contas padronizadas;
  - **controls/guardrails** *preventivos* (SCP/RCP), *detectivos* (regras do Config) e *proativos* (hooks do CloudFormation). É a resposta para "criar rapidamente um ambiente multi-conta governado".

### SCPs e RCPs

- **Service Control Policies (SCPs)**: aplicadas em Unidades Organizacionais (OUs), definem o teto de permissões para todas as contas-filhas de forma centralizada e escalável. São a resposta padrão para restringir serviços/ações em toda a organização (ex.: impedir criação de Internet Gateway, restringir as Regiões permitidas).
- **Regras das SCPs**:
  - **não concedem** permissões, só limitam;
  - afetam **todos** os usuários e roles das contas-membro, **inclusive o root** delas;
  - **não afetam a management account** nem as service-linked roles;
  - a permissão efetiva é a interseção entre SCP e IAM.
- **Estratégias de SCP**: *deny list* (manter `FullAWSAccess` e negar o que for proibido, a mais comum) ou *allow list* (permitir só o listado).
- **Resource Control Policies (RCPs)**: teto centralizado sobre **recursos** (ex.: impedir que qualquer bucket S3 ou chave KMS da organização seja acessado por principais de fora da organização), independentemente das políticas de recurso que cada conta escreva.
- **Perímetro de dados**: combinação de SCP (identidades só acessam recursos confiáveis), RCP (recursos só aceitam identidades confiáveis) e endpoint policies (acesso só por redes esperadas), usando `aws:PrincipalOrgID`, `aws:ResourceOrgID` e `aws:SourceVpce`.

### Compartilhamento e proteção centralizada

- **AWS RAM (Resource Access Manager)**: compartilha recursos AWS (subnets de uma VPC, Transit Gateway attachments, licenças, regras do Route 53 Resolver) entre múltiplas contas de uma AWS Organization, sem duplicar o recurso nem usar peering/replicação. É a resposta padrão para "compartilhar centralmente um recurso caro ou de infraestrutura compartilhada (ex.: Transit Gateway, subnet) entre várias contas sem replicar infraestrutura", reduzindo o custo e a complexidade de gerenciamento multi-conta.



- **S3 Block Public Access (nível de conta) + SCP:** combinação preventiva para garantir que nenhum bucket fique público, à prova de alteração por usuários (a SCP impede desativar o Block Public Access). Soluções baseadas em detecção/remediação (GuardDuty, Trusted Advisor) são sempre reativas.
- **Contatos alternativos da conta + listas de distribuição de e-mail para a conta raiz:** prática recomendada para notificações críticas sem expor a caixa raiz a todos os usuários.

## Padrões e pegadinhas de Organizations

- "Impedir que qualquer conta use Regiões fora da UE": **SCP** com `aws:RequestedRegion`.
- "A SCP não bloqueou a ação na management account": SCPs **não se aplicam** a ela. Não rode cargas na management account.
- "SCP permite S3, mas o usuário não acessa": falta o **allow no IAM**. A SCP só limita.
- "Impedir que dados saiam da organização mesmo se alguém escrever uma bucket policy aberta": **RCP** (+ Block Public Access).
- "Montar um ambiente multi-conta com guardrails prontos": **Control Tower**.
- "Compartilhar RIs/Savings Plans entre contas": **consolidated billing**.

## Criptografia e certificados

### AWS KMS

- **AWS KMS:** gerenciamento central de chaves de criptografia, integrado a quase todos os serviços, com uso registrado no **CloudTrail**. **SSE-KMS com customer managed key e rotação automática** (anual ou programável, veja [Números do KMS](#)) permite controle da key policy e auditoria via CloudTrail. Não é universalmente "superior": a escolha depende de quem deve controlar a chave e de onde a criptografia precisa ocorrer.

| Tipo de chave                                                        | Quem gerencia             | Key policy editável | Compartilhável entre contas | Rotação                |
|----------------------------------------------------------------------|---------------------------|---------------------|-----------------------------|------------------------|
| <b>AWS owned</b>                                                     | AWS (invisível para você) | Não                 | —                           | AWS                    |
| <b>AWS managed</b> ( <code>aws/s3</code> , <code>aws/ebs</code> ...) | AWS, na sua conta         | Não                 | <b>Não</b>                  | Automática anual       |
| <b>Customer managed</b>                                              | Você                      | <b>Sim</b>          | <b>Sim</b>                  | Opcional, configurável |

- **Key policy é obrigatória:** toda chave KMS tem uma. As políticas de IAM só valem se a key policy delegar à conta (a declaração padrão permite o root da conta). **Grants** concedem permissões temporárias e programáticas (usados por serviços como EBS).
- **Envelope encryption:** a API `Encrypt` só aceita até **4 KB**. Para dados maiores, o serviço gera uma **data key** (`GenerateDataKey`), criptografa localmente e guarda a data key criptografada junto com os dados.
- **Escopo regional:** chaves são regionais. Copiar um snapshot ou backup criptografado para outra Região exige **re-criptografar com uma chave da Região de destino**. **Multi-Region keys** têm o mesmo material de chave em várias Regiões, para dados criptografados no cliente que se movem entre Regiões, Global Tables ou DR.
- **Exclusão:** agendada com espera de **7 a 30 dias**. Dados criptografados com uma chave excluída ficam **irrecuperáveis**. Prefira **desabilitar** a chave, e alarme o CloudWatch sobre tentativas de uso durante a espera.
- **Cotas de requisição:** muitos objetos com SSE-KMS podem gerar throttling no KMS. Use **S3 Bucket Keys** ou cache de data keys.
- **Controle adicional:** a condição `kms:ViaService` restringe o uso da chave a um serviço específico (ex.: só via S3). **Material importado** (BYOK) e **custom key stores** (CloudHSM ou external key store) atendem requisitos de controle total do material da chave.

## Números do KMS

| Métrica                                          | Valor                          | Observação                                                                                                                                                                 |
|--------------------------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rotação automática de CMK (customer managed key) | a cada <b>365 dias (1 ano)</b> | Customizável via <code>RotationPeriodInDays</code> ; só suportado em chaves simétricas de criptografia (não em assimétricas, HMAC, material importado ou custom key store) |

## Criptografia no S3 e em outros serviços

- **SSE-S3 vs SSE-KMS vs SSE-C vs client-side encryption:**
  - **SSE-S3** é a opção server-side mais simples, com chaves gerenciadas pelo S3;
  - **SSE-KMS** adiciona controle e auditoria;
  - **SSE-C** mantém a chave com o cliente, mas o S3 executa a criptografia/decriptação, e a chave precisa acompanhar cada request;
  - na **client-side encryption**, o cliente aplica inclusive algoritmo proprietário antes do upload, e o S3 recebe ciphertext, sem participar da criptografia. Detalhes em [Segurança e criptografia do S3](#).
- **Criptografia só na criação:** EBS, RDS, EFS e outros serviços definem a criptografia em repouso na criação. Para criptografar um recurso existente: snapshot/backup → cópia criptografada → novo recurso.
- **Em trânsito:** TLS em todos os endpoints. Exija-o com `aws:SecureTransport` em políticas de recurso, parâmetros do engine no RDS e viewer protocol policy no CloudFront.
- **Compartilhar AMI criptografada entre contas:** ajuste a `launchPermission` da AMI e a política da CMK do KMS para permitir `kms:Decrypt` pela conta de destino. Nunca torne a AMI pública. Chaves AWS managed não podem ser compartilhadas: re-criptografe a AMI com uma customer managed key.

## AWS CloudHSM

- **AWS CloudHSM:** HSM dedicado (**single-tenant**, validado FIPS 140-3 nível 3), em que **só o cliente** controla as chaves. A AWS não tem acesso a elas. APIs padrão (PKCS#11, JCE, CNG).
- **Casos:** requisito regulatório de HSM dedicado, **offload de SSL/TLS**, Oracle TDE, assinatura de código, CA privada, ou **custom key store do KMS** (integração do KMS com chaves no seu HSM).
- Para alta disponibilidade, implante em **múltiplas AZs** com sincronização entre os HSMs do cluster. Nunca use instância única, nem armazene chaves em EC2 Instance Store (efêmero).
- **KMS vs. CloudHSM:** o KMS é multi-tenant e gerenciado, com integração nativa a serviços, e basta na maioria dos casos. O CloudHSM é dedicado, com mais operação, para quando se exige controle exclusivo do HSM.

## AWS Certificate Manager (ACM)

- **AWS Certificate Manager (ACM):** provisiona e gerencia certificados SSL/TLS com renovação automática. Isso só funciona para certificados **emitidos pelo próprio ACM**. Certificados importados de CA externa exigem monitoramento (AWS Config + EventBridge + SNS) e rotação **manual**. **ACM Private CA** emite certificados de uma CA privada gerenciada, para serviços internos e mTLS.
- **Certificados públicos do ACM são gratuitos** para uso em serviços integrados: ELB, CloudFront, API Gateway, App Runner. Os prazos de validade e renovação estão em [Números do ACM](#).
- **Validação por DNS** (recomendada: renovação automática enquanto o registro CNAME existir) ou por e-mail (exige ação manual).
- **Escopo regional:** o certificado precisa estar **na mesma Região** do ALB/API Gateway regional. Para o **CloudFront**, precisa estar em **us-east-1**.
- **Monitorar expiração:** eventos do ACM no **EventBridge** (dias até expirar) e a regra gerenciada do Config `acm-certificate-expiration-check`. O Trusted Advisor não é a resposta.

- **Certificado em EC2/servidor próprio:** use certificados **exportáveis** do ACM (opção paga) ou ACM Private CA. Para um site comum, prefira terminar o TLS no load balancer ou no CloudFront.

## Números do ACM

| Métrica                                 | Valor                                                                               | Observação                                                                                                                                                |
|-----------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Certificado público emitido pelo ACM    | validade de <b>198 dias</b> ; tentativa de renovação a <b>45 dias</b> do vencimento | O valor antigo de 60 dias ainda aparece em simulados e pode valer para certificado público antigo emitido com validade maior. <a href="#">ACM público</a> |
| Certificado privado gerenciado pelo ACM | tentativa de renovação a <b>60 dias</b> do vencimento                               | Depende das condições de renovação e do uso com serviços integrados. <a href="#">ACM privado</a>                                                          |

## Padrões e pegadinhas de criptografia

- "Controlar a key policy, auditar o uso e rotacionar a chave": **KMS customer managed key**.
- "Compartilhar snapshot/AMI criptografado com outra conta": **customer managed key** com permissão para a outra conta. A AWS managed key não serve.
- "Copiar backup criptografado para outra Região": re-criptografar com chave da **Região de destino** (ou usar uma Multi-Region key).
- "Requisito de HSM dedicado, single-tenant, chaves sob controle exclusivo": **CloudHSM**.
- "Throttling de KMS em upload massivo para o S3": **S3 Bucket Keys**.
- "Certificado para CloudFront": **ACM em us-east-1**. "Certificado importado expirando": monitorar com **Config/EventBridge** e renovar manualmente.
- "Chave KMS agendada para exclusão por engano": **cancelar a exclusão** durante a espera de 7–30 dias.

## Segredos

| Critério                 |                                                                             |                                                        |
|--------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------|
| Finalidade               | Segredos (senhas de banco, chaves de API, tokens)                           | Configuração e segredos simples                        |
| Rotação automática       | <b>Sim</b> (nativa para RDS/Aurora/Redshift/DocumentDB; Lambda para outros) | <b>Não</b>                                             |
| Replicação entre Regiões | <b>Sim</b>                                                                  | Não                                                    |
| Acesso entre contas      | Resource policy                                                             | Limitado (parâmetros avançados compartilhados via RAM) |
| Custo                    | Por segredo/mês + chamadas                                                  | Standard gratuito; Advanced cobrado                    |
| Tamanho                  | Até 64 KB                                                                   | 4 KB (Standard) / 8 KB (Advanced)                      |

- **AWS Secrets Manager:** resposta padrão para rotação **automática** de credenciais de banco de dados (RDS/Aurora), com replicação multi-Região de segredos. Sempre vence alternativas que envolvam Lambda customizada para rotacionar segredos em S3/EFS/Parameter Store. O RDS e o Aurora também oferecem a **senha mestre gerenciada pelo Secrets Manager**, com rotação sem configurar a função.
- **AWS Systems Manager Parameter Store:** armazena parâmetros em hierarquia ( /app/prod/db-host ), em texto ou **SecureString** criptografado via KMS, mas **sem rotação automática nativa** (a diferença chave frente ao Secrets Manager). O acesso requer IAM role com permissão de leitura do parâmetro + `kms:Decrypt`. Parâmetros **Advanced** suportam políticas como expiração e notificação. O Parameter Store também pode **referenciar segredos do Secrets Manager**.

- **Onde não guardar segredos:** código, repositório, AMI, **user data**, variáveis de ambiente em texto puro ou arquivos no S3 sem controle. Injete segredos em tempo de execução (campo `secrets` da task ECS, extensão do Lambda, SDK).

## Padrões e pegadinhas de segredos

- "Rotacionar a senha do RDS automaticamente a cada 30 dias": **Secrets Manager**.
- "Guardar configurações e URLs por ambiente com menor custo": **Parameter Store** (Standard).
- "Segredo disponível em outra Região para DR": **replicação do Secrets Manager**.
- "Credencial do banco exposta no código de uma Lambda": mover para o **Secrets Manager** e ler via execution role.

## Proteção de rede e aplicação

| Camada / ameaça                                                | Controle                                                 |
|----------------------------------------------------------------|----------------------------------------------------------|
| Instância/ENI (portas, origens)                                | <b>Security group</b>                                    |
| Subnet (bloquear IP/faixa)                                     | <b>Network ACL</b>                                       |
| VPC (stateful, IPS/IDS, filtragem por domínio, TLS inspection) | <b>AWS Network Firewall</b>                              |
| Appliances de terceiros inline                                 | <b>Gateway Load Balancer</b>                             |
| HTTP/HTTPS (SQLi, XSS, bots, rate limit, geo)                  | <b>AWS WAF</b>                                           |
| DDoS de rede/transporte (L3/L4)                                | <b>Shield Standard</b> (grátis) / <b>Shield Advanced</b> |
| DNS malicioso a partir da VPC                                  | <b>Route 53 Resolver DNS Firewall</b>                    |
| Políticas centralizadas em várias contas                       | <b>Firewall Manager</b>                                  |

- **AWS WAF:** firewall de aplicação (camada 7) contra SQL injection, XSS e excesso de requisições (regras **rate-based**, contra HTTP flood, com limites em [Números do WAF](#)). Suporta regras de geolocalização. Funciona com ALB, API Gateway, CloudFront, AppSync, Cognito User Pools, App Runner e Verified Access, e **não** com NLB. Não fornece HTTPS (isso é ACM).
  - **Web ACL** com **managed rule groups** (AWS Managed Rules para OWASP, IPs maliciosos, sistemas operacionais e linguagens, ou regras do Marketplace).
  - **IP sets**, regex e correspondência de headers.
  - **Bot Control, Fraud Control** (tomada de conta e criação fraudulenta de contas) e ações de **CAPTCHA/challenge**.
  - Logs no S3, CloudWatch Logs ou Firehose.
- **AWS Shield Standard vs. Shield Advanced:**
  - **Standard** é automático e gratuito para todos os clientes, contra ataques comuns de camadas 3/4, e não protege contra SQLi/XSS;
  - **Advanced** oferece mitigação automática de DDoS volumétrico, detecção de camada 7 com regras WAF, monitoramento contínuo e suporte 24x7 do **Shield Response Team (SRT, antigo DRT)**;
  - o Advanced também dá **proteção de custo** (créditos pelo scaling causado pelo ataque) e WAF sem custo adicional nos recursos protegidos (CloudFront, Route 53, Global Accelerator, ELB, Elastic IPs).
- **WAF trata a camada de aplicação, Shield trata DDoS de rede/transporte:** um nunca substitui o outro.
- **Arquitetura resiliente a DDoS:**
  - reduza a superfície exposta: só CloudFront/ALB/Global Accelerator públicos, com os backends privados;
  - absorva na borda: **CloudFront + Route 53 + Shield**;

- filtre em camada 7: **WAF** com rate-based rules;
- escale a origem: **ALB + Auto Scaling**.
- **AWS Firewall Manager**: gerenciamento centralizado de regras WAF/Shield Advanced/security groups/Network Firewall/DNS Firewall em múltiplas contas e Regiões. **Aplica automaticamente** as políticas a novas contas e recursos. Exige AWS Organizations e AWS Config.
- **AWS Network Firewall**: firewall de rede gerenciado com filtragem stateful em nível de VPC (regras baseadas em domínio, IPS/IDS compatível com Suricata, inspeção de payload, inspeção TLS). É diferente do AWS WAF (camada 7, HTTP, aplicado a ALB/API Gateway/CloudFront) e do AWS Firewall Manager (gerencia centralmente regras de WAF/Shield/security groups em várias contas, mas não é ele mesmo um firewall de rede). É a resposta padrão para "inspecionar e filtrar tráfego de rede genérico (não só HTTP/HTTPS) dentro de uma VPC".
  - Implante em **subnets dedicadas** e direcione o tráfego pelas route tables.
  - Em multi-conta, use uma **VPC de inspeção central** ligada ao Transit Gateway.

## Números do WAF

| Métrica                            | Valor            | Observação                               |
|------------------------------------|------------------|------------------------------------------|
| Mínimo de requisições configurável | <b>10</b>        | ⚠ Reduzido do valor clássico de 100      |
| Máximo de requisições configurável | 2.000.000.000    |                                          |
| Janela de avaliação — padrão       | 5 minutos (300s) | Configurável entre 1, 2, 5 ou 10 minutos |

## Padrões e pegadinhas de proteção de rede e aplicação

- "Bloquear SQL injection e XSS na API": **WAF** no API Gateway/ALB/CloudFront.
- "Limitar requisições por IP para conter HTTP flood": **WAF rate-based rule**.
- "Proteção contra DDoS com resposta especializada 24x7 e proteção de custo": **Shield Advanced**.
- "Aplicação atrás de NLB precisa de WAF": coloque **CloudFront (ou ALB) com WAF** na frente. O WAF não se associa ao NLB.
- "Filtrar a saída da VPC para permitir apenas domínios aprovados": **Network Firewall** (ou DNS Firewall para resolução).
- "Aplicar as mesmas regras de WAF em 50 contas automaticamente": **Firewall Manager**.
- "Bloquear acesso de um país específico": **WAF geo match** ou **geo restriction do CloudFront**.

## Detecção, auditoria e conformidade

### Registro e auditoria

- **AWS CloudTrail**: registra a atividade de API para auditoria (quem, quando, o quê, de onde), mas não impede nem reverte ações. O **Event history padrão mostra 90 dias de eventos de gerenciamento**. Para chamadas em objetos S3 e itens DynamoDB, configure **eventos de dados** em uma trail ou event data store apropriada, considerando custo e retenção. [Event history](#).
  - **Retenção longa**: uma trail entrega os logs no **S3**. Use uma **trail multi-Região** e uma **organization trail** para todas as contas, com o bucket em uma **conta de log** separada, protegido por Object Lock e SSE-KMS.
  - **Log file integrity validation**: arquivos de digest provam que os logs não foram alterados nem apagados.
  - **CloudTrail Lake**: armazena e consulta eventos com SQL, sem montar Athena.
  - **CloudTrail Insights**: detecta volumes anormais de chamadas de API ou erros.



- **Resposta em tempo real:** eventos do CloudTrail chegam ao **EventBridge** (ex.: login do root → SNS/Lambda).
- **AWS Config:** registra a **configuração** e o **histórico de mudanças** de cada recurso (linha do tempo, relacionamentos) e avalia continuamente a conformidade contra regras (gerenciadas ou customizadas com Lambda/Guard). Combinado com triggers → Lambda, permite correção automática e alertas em tempo real. Combinado com CloudTrail, forma a dupla padrão de auditoria/rastreabilidade: Config diz **o que** mudou, CloudTrail diz **quem** mudou.
  - **Remediação automática** com documentos do **Systems Manager Automation**.
  - **Conformance packs:** conjuntos de regras (CIS, PCI, NIST).
  - **Aggregator:** visão multi-conta e multi-Região.
  - O Config **não impede** mudanças: detecta e corrige depois.

## Detecção de ameaças e vulnerabilidades

- **Amazon GuardDuty:** detecção de ameaças por ML e inteligência de ameaças sobre CloudTrail, VPC Flow Logs e logs de DNS. Os logs são lidos **diretamente**, sem precisar habilitá-los ou armazená-los você mesmo. Gera alertas, **não bloqueia** automaticamente.
  - **Planos de proteção:** S3, EKS, **Runtime Monitoring** (EC2/ECS/EKS), **Malware Protection** (EBS e S3), RDS (logins anômalos) e Lambda.
  - **Resposta automatizada:** findings no **EventBridge** → Lambda/Systems Manager (isolar instância, bloquear IP em NACL/WAF).
  - Multi-conta via delegated administrator.
- **Amazon Inspector:** avalia vulnerabilidades (CVEs) e exposição de rede **continuamente** em **instâncias EC2** (via agente do SSM ou sem agente), **imagens no ECR** e **funções Lambda**, com priorização de risco. Não bloqueia tráfego em tempo real. Integra-se a pipelines de CI/CD.
- **Amazon Macie:** descoberta e proteção de dados sensíveis (PII, credenciais, dados financeiros) no **S3**, com *automated sensitive data discovery* e identificadores personalizados. Também aponta buckets públicos ou sem criptografia. Combinado com EventBridge + SNS, notifica automaticamente sem Lambda intermediária.
- **Amazon Detective:** correlaciona VPC Flow Logs, CloudTrail e GuardDuty em grafos para **investigação** aprofundada de atividades suspeitas (causa raiz, escopo). Não detecta nem bloqueia por si só.

## Postura, conformidade e evidências

- **AWS Security Hub:** agrega, prioriza e correlaciona findings de segurança de múltiplos serviços (GuardDuty, Inspector, Macie, Config, IAM Access Analyzer, Firewall Manager) e de ferramentas de parceiros terceiros em um painel único, com verificações automatizadas contra padrões de conformidade (CIS, PCI DSS, AWS Foundational Security Best Practices). É a resposta padrão para "visão centralizada e priorizada de postura de segurança em toda a conta/organização", diferente do GuardDuty (só detecta ameaças) e do Config (só avalia conformidade de configuração de recursos individuais).
- **Amazon Security Lake:** coleta e normaliza automaticamente dados de segurança da AWS e de terceiros no formato OCSF em um data lake S3 multi-account/multi-Region. Security Lake centraliza os dados brutos para consulta e analytics. Security Hub agrega findings e postura de segurança já processados.
- **AWS Audit Manager:** coleta evidências continuamente para simplificar auditorias de conformidade contra frameworks pré-construídos (PCI DSS, HIPAA, GDPR). É diferente do AWS Artifact (portal estático de documentos de conformidade produzidos pela própria AWS, que não coleta evidências do ambiente do cliente) e do AWS Config (avalia configuração técnica de recursos, não organiza evidências de auditoria de processo).
- **AWS Trusted Advisor:** recomendações de custo, segurança, desempenho, tolerância a falhas e cotas de serviço. Não monitora expiração de certificados ACM nem substitui o Config.



- **AWS Artifact**: portal de documentos de conformidade e auditoria (SOC, ISO, PCI DSS) e de acordos (ex.: BAA para HIPAA). Não monitora em tempo real nem configura serviços automaticamente.

## Padrões e pegadinhas de detecção e auditoria

- "Quem apagou o bucket?": **CloudTrail**. "Como o security group estava ontem?": **Config** (histórico de configuração).
- "Logs de auditoria à prova de adulteração em todas as contas": **organization trail** → S3 em conta de log com **Object Lock + integrity validation**.
- "Detectar mineração de criptomoeda ou chamadas de API de IPs maliciosos": **GuardDuty**.
- "Corrigir automaticamente buckets que fiquem públicos": regra do **Config** + remediação via **SSM Automation** (ou EventBridge + Lambda).
- "Encontrar números de cartão e CPF em buckets": **Macie**.
- "Imagens de contêiner com CVEs críticas": **Inspector** (scanning do ECR).
- "Painel único de conformidade CIS em todas as contas": **Security Hub**.
- "Relatório SOC 2 da AWS para o auditor": **Artifact**. "Evidências do nosso ambiente para o auditor": **Audit Manager**.
- "Alerta imediato quando o root fizer login": **EventBridge** (evento do CloudTrail) → SNS.

---

## Identidade federada e autorização

### Acesso de funcionários (workforce)

- **AWS IAM Identity Center (sucessor do AWS SSO) + SAML 2.0**: federação de identidades para SSO centralizado em múltiplas contas AWS e aplicações de terceiros. É a forma recomendada de dar acesso humano à AWS.
  - **Permission sets** definem as permissões aplicadas às contas atribuídas.
  - **Fonte de identidade**: diretório próprio do Identity Center, **Active Directory** (via AWS Managed Microsoft AD ou AD Connector) ou um **IdP externo** (Okta, Microsoft Entra ID) via SAML, com provisionamento automático de usuários e grupos via **SCIM**.
  - Fornece credenciais temporárias também para a **CLI**.
  - Para manter o Active Directory local como fonte de identidade usando o AWS Managed Microsoft AD, é preciso uma confiança bidirecional com o AD on-premises.
- **Federação SAML direta no IAM** (por conta, com role para cada IdP): abordagem mais antiga. O Identity Center é preferível em multi-conta.

### AWS Directory Service

- **AWS Directory Service — as 3 opções**:
  - **AWS Managed Microsoft AD**: Active Directory real, gerenciado pela AWS em duas AZs, compatível nativamente e com suporte a confiança bidirecional com AD on-premises. É a opção que integra com o IAM Identity Center acima, e com RDS SQL Server, FSx for Windows e WorkSpaces.
  - **AD Connector**: não é um diretório em si, e sim um proxy que redireciona a autenticação para um Active Directory on-premises existente, sem replicar nenhum dado de diretório para a AWS. É a resposta padrão quando o requisito é manter o AD on-premises como única fonte de identidade. Depende da conectividade (VPN/Direct Connect) com o on-premises.
  - **Simple AD**: implementação simplificada compatível com Samba, sem recursos completos de AD como confiança de floresta/domínio ou replicação. É a opção mais barata, usada quando não há necessidade dos recursos completos do AD.

- A escolha entre as três depende de precisar ou não do conjunto completo de recursos do AD e de manter (ou não) o diretório on-premises como fonte única de identidade.

## Usuários de aplicações (customer identity)

- **Amazon Cognito:** autenticação/autorização de usuários finais.
  - **User Pools:** diretório de usuários com cadastro, login, MFA, login social, federação SAML/OIDC, hosted UI e triggers Lambda, emitindo **JWTs**. O **User Pool Authorizer** nativo do API Gateway valida tokens sem Lambda intermediária.
  - **Identity Pools:** concedem **credenciais AWS temporárias** assumindo uma IAM Role (inclusive para visitantes não autenticados). Um erro de acesso após o login geralmente é a role incorreta configurada no Identity Pool.
  - Grupos/atributos personalizados (refletidos no JWT) resolvem autorização por atributo de negócio (ex.: assinatura premium).
  - Veja [Backend para aplicações mobile e web](#).
- **Amazon Verified Permissions:** autorização fina **dentro da aplicação** com políticas na linguagem **Cedar** (RBAC/ABAC), desacoplando regras de permissão do código. Integra-se ao Cognito e ao API Gateway.

## Padrões e pegadinhas de identidade federada

- "Funcionários com login corporativo acessando 30 contas AWS": **IAM Identity Center** com o IdP corporativo (SAML + SCIM).
- "Manter o AD on-premises como única fonte, sem replicar para a AWS": **AD Connector**.
- "Aplicações AWS que precisam de AD real (FSx for Windows, RDS SQL Server) + trust com on-premises": **AWS Managed Microsoft AD**.
- "App móvel com login social e acesso direto ao S3 por usuário": **Cognito User Pool + Identity Pool**.
- "API Gateway validando o login dos usuários sem código": **Cognito User Pool Authorizer**.

## Dados sensíveis em escala

- **AWS Lake Formation:** cria um data lake governado sobre fontes existentes (S3 + RDS via Glue), com permissões granulares centralizadas, incluindo segurança em nível de **linha e coluna/célula**, sem duplicar ou mover dados fisicamente. Suporta **LF-Tags** (ABAC) e compartilhamento entre contas. Veja a seção 7.
- **Classificação orienta o desenho:** identifique dados públicos, internos e sensíveis. Mapeie requisitos de residência, retenção, exclusão, auditoria e acesso antes de escolher Região, criptografia, chaves, backup e compartilhamento. Use Macie para descobrir PII em S3, tags/metadados para aplicar políticas, e IAM/Lake Formation para limitar quem lê cada conjunto. Compliance depende dos requisitos concretos e do escopo de responsabilidade compartilhada, não apenas de ativar KMS.
- **Residência de dados:** restrinja as Regiões com **SCP** (`aws:RequestedRegion`) ou com os controles de residência do Control Tower. Replicação, backups cross-Region e CloudFront podem mover dados, então revise cada um.
- **Perímetro de dados:** garanta que só **identidades confiáveis** acessem **recursos confiáveis** a partir de **redes esperadas**, combinando SCPs, RCPs, políticas de recurso e endpoint policies. Veja [SCPs e RCPs](#).

## Decisão rápida — Segurança, Identidade e Criptografia

| Cenário / requisito | Resposta correta | Distratores clássicos |
|---------------------|------------------|-----------------------|
|---------------------|------------------|-----------------------|

| Cenário / requisito                                            | Resposta correta      | Distratores clássicos                 |
|----------------------------------------------------------------|-----------------------|---------------------------------------|
| Rotação automática de credenciais de banco                     | AWS Secrets Manager   | Parameter Store, S3, EFS              |
| Ataques de camada de aplicação (SQLi, XSS)                     | AWS WAF               | Shield, Security Groups, NACLs        |
| DDoS volumétrico em grande escala                              | AWS Shield Advanced   | Shield Standard, GuardDuty            |
| Restringir SCP/permisões em todas as contas de uma organização | AWS Organizations SCP | IAM policy por conta, Security Groups |

## 6. Integração, Mensageria e Streaming

**Regra de ouro da prova:** identifique o **padrão de comunicação**.

- **Um produtor, um consumidor, trabalho a processar** (fila com buffer): **SQS**.
- **Um evento, vários assinantes** (fan-out): **SNS** (ou EventBridge).
- **Rotear eventos por conteúdo** entre serviços AWS, SaaS e contas: **EventBridge**.
- **Fluxo contínuo com ordem, replay e vários consumidores em tempo real**: **Kinesis Data Streams** (ou MSK, se Kafka).
- **Entregar um stream em S3/Redshift/OpenSearch sem código**: **Data Firehose**.
- **Coordenar etapas com retry, ramificação e estado**: **Step Functions**.
- **Migrar um broker existente (JMS, AMQP, MQTT) sem reescrever**: **Amazon MQ**.

Desacoplar com filas **não** garante processamento exatamente uma vez: projete consumidores **idempotentes**.

**Exemplo de três camadas desacopladas:** CloudFront/ALB atende clientes. EC2 em Auto Scaling ou ECS/Fargate/Lambda executa a lógica sem guardar sessões no disco local. RDS/DynamoDB/S3 mantém os dados persistentes. Para tarefas lentas, API → SQS → workers permite escalar produtores e consumidores separadamente. Adicione DLQ, idempotência e monitoramento de backlog/idade das mensagens: filas reduzem acoplamento, mas não garantem que uma operação externa seja executada exatamente uma vez.

| Serviço                     | Modelo                       | Ordem             | Retenção / replay                         | Consumidores             | Caso típico                                 |
|-----------------------------|------------------------------|-------------------|-------------------------------------------|--------------------------|---------------------------------------------|
| <b>SQS Standard</b>         | Fila (pull)                  | Não garantida     | Até 14 dias; mensagem sai ao ser excluída | Um por mensagem          | Buffer de trabalho, desacoplamento          |
| <b>SQS FIFO</b>             | Fila (pull)                  | Por message group | Até 14 dias                               | Um por mensagem          | Pedidos, transações em ordem                |
| <b>SNS</b>                  | Pub/sub (push)               | FIFO opcional     | Não armazena                              | Muitos assinantes        | Fan-out, notificações                       |
| <b>EventBridge</b>          | Barramento de eventos (push) | Não garantida     | Archive + replay opcional                 | Várias regras e destinos | Roteamento por conteúdo, SaaS, agendamentos |
| <b>Kinesis Data Streams</b> | Stream (pull)                | Por shard         | 24 h a 365 dias, <b>replay</b>            | Vários, em paralelo      | Telemetria, clickstream, tempo real         |
| <b>Amazon MSK</b>           | Stream Kafka                 | Por partição      | Configurável, replay                      | Vários                   | Ecossistema Kafka existente                 |
| <b>Data Firehose</b>        | Entrega gerenciada           | —                 | Não (buffer curto)                        | Destino configurado      | Carregar S3/Redshift/OpenSearch             |
| <b>Amazon MQ</b>            | Broker (JMS/AMQP/MQTT/STOMP) | Conforme o broker | Conforme o broker                         | Filas e tópicos          | Migrar ActiveMQ/RabbitMQ                    |

---

## Filas e notificações

### Amazon SQS

- **Amazon SQS:** fila gerenciada para desacoplamento assíncrono, totalmente serverless, que escala sem provisionamento. Serve como buffer para processamento assíncrono de uploads e picos de pedidos, protegendo o backend de sobrecarga.
- **Standard vs. FIFO:**
  - **Standard** oferece taxa praticamente ilimitada, sem garantir ordem e podendo entregar duplicatas (*at-least-once*).
  - **FIFO** mantém a ordem **dentro de cada grupo de mensagens** (`MessageGroupId`) e deduplica envios com o mesmo ID em uma **janela de cinco minutos** (`MessageDeduplicationId` ou deduplicação baseada em conteúdo). O nome da fila termina em `.fifo`, e o throughput é limitado por API (maior com batching e com o modo *high throughput*).
  - Uma fila Standard **não pode ser convertida** em FIFO: crie uma nova.
  - Grupos de mensagens diferentes são processados em paralelo. Use vários `MessageGroupId` (ex.: um por cliente) para escalar sem perder a ordem por entidade.
- **Idempotência:** a deduplicação não garante que a lógica de negócio execute uma única vez. Uma falha do consumidor ou o vencimento do visibility timeout pode levar a reprocessamento, então projete o consumidor para ser **idempotente**. [Recuperação e duplicatas](#).
- **Visibility timeout:** ajuste-o ao tempo de processamento. Se o processamento pode demorar mais que o previsto, o consumidor estende o prazo com `ChangeMessageVisibility`. Se for curto demais, outra instância recebe a mesma mensagem.
- **Dead-Letter Queue (DLQ):** isola mensagens que falharam repetidamente, após `maxReceiveCount` recebimentos (*redrive policy*). A DLQ precisa ser do **mesmo tipo** da fila de origem (FIFO → DLQ FIFO). Com o **DLQ redrive**, as mensagens voltam à fila de origem depois da correção. Monitore o tamanho da DLQ com alarme.
- **Delay Queue vs Visibility Timeout:** a delay queue mantém **mensagens novas** invisíveis por até 15 minutos antes da primeira entrega. O visibility timeout começa **depois que uma mensagem é recebida** e impede que outros consumidores a processem enquanto o primeiro trabalha. DLQ isola mensagens que excederam tentativas, e FIFO resolve ordem/deduplicação. Nenhum desses substitui a delay queue. Para atrasar mensagens individuais, use *message timers* (`DelaySeconds` por mensagem, apenas Standard).
- **Long polling** (`ReceiveMessageWaitTimeSeconds > 0`): o consumidor espera por mensagens em vez de receber respostas vazias. Isso **reduz custo e chamadas de API** e diminui a latência. É quase sempre preferível ao short polling (padrão). Os limites estão em [Números do SQS](#).
- **Mensagens grandes:** acima do tamanho máximo, use a **Extended Client Library**, que guarda o payload no **S3** e envia apenas a referência pela fila.
- **Segurança:** criptografia **SSE-SQS** ou **SSE-KMS**. A **queue policy** (resource-based) autoriza outros serviços ou contas a enviar mensagens (ex.: S3 event notifications, SNS). Com SSE-KMS, o produtor também precisa de permissão na chave. Use um interface endpoint para acesso privado a partir da VPC.
- **Escalar consumidores:** Auto Scaling por **backlog por instância** (veja [Métricas de scaling](#)), ECS Service Auto Scaling ou Lambda com event source mapping (veja [Modelos de invocação](#)). Monitore `ApproximateNumberOfMessagesVisible` e `ApproximateAgeOfOldestMessage`.
- **SQS Temporary Queue Client:** cria filas virtuais leves, multiplexadas sobre uma única fila SQS, para padrões request-response com muitos destinos temporários de baixo tráfego. Reduz chamadas de API, tempo de desenvolvimento e o custo de criar/excluir filas físicas.

## Números do SQS

| Métrica                             | Valor                    | Observação                                                                                                                                                 |
|-------------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tamanho máximo de mensagem          | <b>1.024 KiB (1 MiB)</b> | ⚠ Aumentado de 256 KB para 1 MiB em ago/2025. Muito material de estudo (e várias questões de simulado) ainda usa 256 KB — hoje tecnicamente desatualizado. |
| Retenção de mensagem (padrão)       | 4 dias                   |                                                                                                                                                            |
| Retenção de mensagem (máxima)       | 14 dias                  |                                                                                                                                                            |
| Visibility timeout (padrão)         | 30 segundos              |                                                                                                                                                            |
| Visibility timeout (máximo)         | 12 horas                 | Range permitido: 0s a 12h                                                                                                                                  |
| Delay queue / DelaySeconds (máximo) | 15 minutos               | Afeta a primeira entrega de mensagens novas; não confundir com visibility timeout                                                                          |
| Long polling — wait time máximo     | 20 segundos              | ReceiveMessageWaitTimeSeconds                                                                                                                              |
| Long polling — padrão               | 0s (short polling)       | Precisa ser configurado explicitamente para virar long polling                                                                                             |

## Amazon SNS

- **Amazon SNS:** pub/sub para notificações e fan-out. Um tópico **empurra** a mensagem para todos os assinantes: SQS, Lambda, HTTP/S, e-mail, SMS, mobile push e Firehose. Não guarda mensagens para reentrega posterior como o SQS, então não deve ser usado sozinho como camada de comunicação garantida.
- **Fan-out SNS + SQS:** publique uma vez no tópico, e cada consumidor tem **sua própria fila SQS**, com persistência, retry e escala independentes. É o padrão para "um pedido deve disparar faturamento, estoque e e-mail" sem acoplar os três.
- **SNS + SQS com filtragem de mensagens:** um tópico único roteia para múltiplas filas separadas por tipo, sem múltiplos pipelines. As *filter policies* avaliam atributos ou o corpo da mensagem.
- **SNS com destino de fallback SQS:** a **DLQ da subscription** retém mensagens não entregues por falhas transitórias. O SNS também aplica política de retry por tipo de endpoint.
- **SNS FIFO:** ordem e deduplicação para fan-out ordenado, entregando para filas **SQS FIFO**.
- **Segurança:** criptografia com KMS e **topic policy** para permitir publicação de outros serviços ou contas (ex.: alarmes do CloudWatch, S3).

## Amazon EventBridge

- **Amazon EventBridge** (antigo CloudWatch Events): barramento de eventos serverless para regras baseadas em evento ou cron. Permite rotear um único evento para múltiplos destinos simultaneamente (é mais flexível que as notificações de evento S3 tradicionais) e captura chamadas de API via CloudTrail em tempo real.
- **Event buses:**
  - **default:** eventos dos serviços AWS;
  - **custom:** eventos das suas aplicações;
  - **partner:** eventos de SaaS (Zendesk, Datadog, Shopify etc.). Barramentos podem enviar eventos para **outras contas e Regiões**.
- **Regras:** *event patterns* filtram por qualquer campo do JSON do evento. Cada regra pode ter vários destinos (Lambda, SQS, SNS, Step Functions, Kinesis, API Gateway, ECS task, outro barramento), com **input transformer**, **política de retry** e **DLQ**.



- **EventBridge Scheduler**: agendamentos únicos ou recorrentes (cron/rate, com fuso horário) em grande escala, invocando centenas de APIs AWS diretamente. É a opção preferida às regras agendadas para novas soluções (ex.: iniciar/parar instâncias fora do horário, disparar jobs).
- **EventBridge Pipes**: integração **ponto a ponto** de uma fonte (SQS, Kinesis, DynamoDB Streams, MSK, MQ) para um destino, com **filtro e enriquecimento** opcionais, sem código de cola em Lambda.
- **Archive e replay**: guardam eventos para reprocessá-los depois. **Schema registry**: descobre e versiona o formato dos eventos. **API destinations**: enviam eventos para endpoints HTTP externos com autenticação e rate limit.
- **EventBridge vs. SNS**: o EventBridge filtra por conteúdo com mais riqueza, integra SaaS e tem arquivo/replay. O SNS tem maior fan-out, menor latência e suporta SMS, e-mail e push.

## Amazon MQ

- **Amazon MQ**: broker gerenciado compatível com **ActiveMQ** e **RabbitMQ** e com protocolos existentes como **AMQP**, **MQTT**, **STOMP**, **OpenWire** e JMS. Tem failover automático ativo/standby multi-AZ (ActiveMQ) ou cluster de três nós (RabbitMQ).
- Substitui um broker autogerenciado em EC2 quando a prioridade é **migrar sem reescrever** para as APIs do SQS/SNS. O **Lambda pode consumir** de brokers Amazon MQ via event source mapping.
- **Não escala como o SQS**: a capacidade depende do tamanho da instância do broker. Para aplicações novas nativas da nuvem, SQS/SNS/EventBridge costumam ser mais simples e baratos.

## Padrões e pegadinhas de filas e eventos

- "Desacoplar o frontend do processamento pesado e absorver picos": **SQS** entre as camadas.
- "Processar pedidos na ordem, sem duplicatas, por cliente": **SQS FIFO** com `MessageGroupId` = cliente.
- "Mensagem processada duas vezes": **visibility timeout** menor que o tempo de processamento e consumidor não idempotente.
- "Mensagens com falha repetida travam a fila": **DLQ** com `maxReceiveCount`.
- "Muitas chamadas `ReceiveMessage` vazias e custo alto": **long polling**.
- "Payload maior que o limite do SQS": **Extended Client Library + S3**.
- "Um evento deve acionar vários sistemas independentes com persistência": **SNS** → **várias filas SQS** (fan-out).
- "Rotear eventos de uma aplicação SaaS para a AWS": **EventBridge partner event bus**.
- "Executar uma tarefa todo dia às 2h": **EventBridge Scheduler**.
- "Aplicação usa JMS/AMQP com ActiveMQ on-premises": **Amazon MQ**.

## APIs

### Amazon API Gateway

- **Amazon API Gateway**: cria, publica e protege APIs REST/HTTP/WebSocket, tipicamente na frente do Lambda. Integra-se nativamente com Cognito (autorização), WAF (proteção) e Usage Plans/API Keys (controle de consumo).

| Tipo            | Destaques                                                                                                                                                | Quando escolher                                                  |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>REST API</b> | Recursos completos: <b>WAF</b> , <b>cache</b> , <b>usage plans/API keys</b> , validação de requisição, transformação de payload, endpoint <b>privado</b> | APIs públicas com controle de consumo, proteção e transformações |
| <b>HTTP API</b> | Mais <b>barata</b> e com menor latência; JWT/OIDC nativo, Lambda e HTTP                                                                                  | Proxy simples para Lambda/HTTP com menor custo                   |

| Tipo                 | Destaques                                  | Quando escolher                              |
|----------------------|--------------------------------------------|----------------------------------------------|
| <b>WebSocket API</b> | Conexões <b>bidirecionais</b> persistentes | Chat, notificações, dashboards em tempo real |

- **Tipos de endpoint:** **edge-optimized** (via rede do CloudFront, para clientes globais), **regional** (clientes na mesma Região, ou com seu próprio CloudFront) e **private** (acessível só da VPC via interface endpoint, com resource policy).
- **Integrações:**
  - Lambda (proxy);
  - HTTP;
  - **integração direta com serviços AWS:** enviar para SQS, gravar no DynamoDB, iniciar Step Functions **sem Lambda intermediária**;
  - **VPC Link** para NLB/ALB privados.
- **Autorização:**
  - **IAM** (SigV4, para chamadas entre serviços/contas);
  - **Cognito User Pools** (JWT de usuários finais);
  - **Lambda authorizer** (tokens ou regras customizadas). **API keys não são autenticação:** elas identificam o cliente para usage plans.
- **Throttling:** há um limite padrão por conta e Região, aplicado a todas as APIs e ajustável. **Usage plans** definem cota e rate por cliente. Respostas acima do limite retornam **429**.
- **Cache (REST):** reduz chamadas ao backend e a latência, com TTL configurável por stage.
- **Timeout de integração:** o padrão é de **29 segundos**. Tarefas mais longas devem virar **assíncronas**: a API grava em SQS/Step Functions, responde **202** e o cliente consulta o status depois.
- **Stages** (dev/prod) com variáveis, **canary release** por stage, CORS, **mTLS** e domínios personalizados com certificado do ACM.

## AWS AppSync

- **AWS AppSync:** API GraphQL gerenciada com sincronização em tempo real (subscriptions). Integra fontes de dados (DynamoDB, RDS/Aurora, OpenSearch, HTTP) diretamente ou via **resolvers personalizados que invocam Lambda**. O AppSync não roda dentro de uma VPC: para fontes privadas, use Lambda na VPC. APIs **privadas** podem ser expostas via interface endpoint. Veja [Backend para aplicações mobile e web](#).

## AWS Step Functions

- **AWS Step Functions:** orquestração de workflows serverless coordenando múltiplos serviços, com a lógica de **retry, catch, ramificação e paralelismo** declarada na máquina de estados (Amazon States Language), sem código de coordenação.

| Critério       |                                                    |                                                               |
|----------------|----------------------------------------------------|---------------------------------------------------------------|
| Duração máxima | <b>Até 1 ano</b>                                   | <b>Até 5 minutos</b>                                          |
| Semântica      | Exactly-once por etapa                             | At-least-once (assíncrono) ou at-most-once (síncrono)         |
| Histórico      | Completo, auditável no console                     | Via CloudWatch Logs                                           |
| Cobrança       | Por transição de estado                            | Por execução, duração e memória                               |
| Caso típico    | Processos de negócio longos, aprovação humana, ETL | Alto volume de eventos curtos (IoT, streaming, microserviços) |

- **Estados:** Task, Choice, Parallel, **Map** (itera sobre itens), Wait, Pass, Succeed/Fail. O **Distributed Map** processa milhões de objetos do S3 com milhares de execuções filhas em paralelo.
- **Integrações com mais de 200 serviços** via SDK, sem Lambda. **Padrões de integração:**
  - *request-response*;
  - `.sync` : aguarda o término de um job Batch, ECS, Glue ou EMR;
  - **callback com task token:** pausa até uma resposta externa, como **aprovação humana**.
- **Saga:** etapas de compensação no `Catch` para desfazer operações distribuídas quando uma etapa falha.
- **Recursos em VPC privada:** Step Functions não roda na sua VPC. Use VPC Endpoints (S3/RDS) + Systems Manager Session Manager (EC2), nunca Internet Gateway/NAT para esse fim. Para chamar o Step Functions a partir da VPC, use o interface endpoint.

## Amazon AppFlow

- **Amazon AppFlow:** serviço de integração totalmente gerenciado que transfere dados de forma segura entre aplicações SaaS de terceiros (Salesforce, Slack, ServiceNow, Zendesk etc.) e serviços AWS (S3, Redshift), sem escrever código de integração customizado. Os fluxos podem ser sob demanda, agendados ou por evento, com filtros, mapeamento de campos e criptografia (inclusive via PrivateLink para SaaS compatíveis). É a resposta padrão para "sincronizar dados entre uma aplicação SaaS de terceiros e um data lake/data warehouse na AWS" sem construir um pipeline ETL próprio do zero.

## Padrões e pegadinhas de APIs e orquestração

- "API pública com limite de requisições por cliente e chaves": **REST API + usage plans + API keys**.
- "Proxy barato e rápido para Lambda com JWT": **HTTP API**.
- "Proteger a API contra SQLi e bots": **WAF** na **REST API** (ou CloudFront na frente).
- "API expõe um serviço em ALB/NLB privado": **VPC Link**.
- "Gravar a requisição no SQS sem código": **integração direta do API Gateway com o SQS**.
- "Requisição leva mais de 29 s": padrão **assíncrono** (SQS/Step Functions + consulta de status).
- "Clientes de todo o mundo com menor latência": endpoint **edge-optimized** (ou regional + CloudFront).
- "Workflow com aprovação humana e duração de dias": **Step Functions Standard** com **task token**.
- "Milhões de eventos curtos por dia com menor custo de orquestração": **Step Functions Express**.
- "Processar milhões de arquivos do S3 em paralelo": **Distributed Map**.
- "Sincronizar Salesforce com o S3 sem código": **AppFlow**.

## Streaming

### Amazon Kinesis Data Streams

- **Amazon Kinesis Data Streams:** ingestão e processamento de streams em tempo real, com **ordem por shard**, **retenção configurável** (24 horas por padrão, até 365 dias) e **replay**: vários consumidores leem os mesmos dados de forma independente.
- **Modos de capacidade:** no modo **provisioned**, a equipe gerencia shards. Cada shard aceita cerca de **1 MB/s ou 1.000 registros/s de escrita e 2 MB/s de leitura** compartilhada. No modo **on-demand**, o serviço ajusta a capacidade automaticamente para tráfego imprevisível.
- **Partition key:** define o shard e a ordem. Uma chave com pouca variedade causa **hot shard** e throttling (`ProvisionedThroughputExceeded`).

- **Consumidores:** Lambda, KCL (em EC2/ECS), Managed Service for Apache Flink e Data Firehose. **Enhanced fan-out** fornece throughput de leitura dedicado (2 MB/s por shard **para cada consumidor**, via push) e reduz a contenção quando várias aplicações consomem o mesmo stream.
- **Produtores:** SDK, **KPL** (agregação e batching), Kinesis Agent, IoT Core e CloudWatch Logs.
- **Kinesis Agent:** agente instalável em servidores para enviar dados a streams. Tem mais esforço operacional que um subscription filter direto.
- Criptografia com KMS, interface endpoint e controle via IAM.

## Amazon Data Firehose

- **Amazon Data Firehose** (renomeado de "Kinesis Data Firehose"): entrega gerenciada e serverless de streams para **S3, Redshift, OpenSearch, Splunk, Snowflake, tabelas Apache Iceberg e endpoints HTTP**, com transformação via Lambda embutida. É a resposta padrão para levar logs (CloudWatch Logs subscription filter) ao OpenSearch quase em tempo real.
- **Quase tempo real:** agrupa por **tamanho ou tempo** (buffer) antes de gravar. Não é o serviço para latência de milissegundos.
- **Recursos:**
  - **conversão para Parquet/ORC** com o schema do Glue;
  - **particionamento dinâmico** no S3;
  - compressão;
  - backup dos registros originais;
  - erros enviados a um prefixo S3.
- **Sem retenção nem replay:** o Firehose entrega, não armazena. Para reprocessar ou ter vários consumidores, coloque o **Kinesis Data Streams** antes do Firehose.

## Processamento de streams e Kafka

- **Processamento de streams com estado:** para janelas, agregações contínuas e múltiplos consumidores, avalie o **Amazon Managed Service for Apache Flink**. Data Firehose é entrega, e Kinesis Data Streams/MSK fornecem ingestão e retenção do fluxo. **Kinesis Data Analytics para aplicações SQL foi descontinuado em janeiro de 2026**, e a recomendação da AWS é migrar para o Managed Service for Apache Flink. [Aviso da AWS](#).
- **Amazon MSK (Managed Streaming for Apache Kafka):** Kafka gerenciado, com streaming resiliente via clusters multi-AZ e Auto Scaling de armazenamento baseado em métricas do CloudWatch.
  - **MSK Serverless:** sem gerenciar capacidade.
  - **MSK Connect:** conectores Kafka Connect gerenciados.
  - Retenção e tamanho de mensagem configuráveis. **Tiered storage** reduz o custo de retenção longa.
  - Autenticação via IAM, TLS ou SASL.
- **Kinesis vs. MSK:** prefira **MSK** quando a equipe ou as aplicações já usam **APIs, ferramentas e conectores Kafka**, ou precisam de configurações específicas do Kafka. Prefira **Kinesis** para uma solução nativa AWS mais simples, integrada a Lambda/Firehose e com modo on-demand.
- **Amazon Kinesis Video Streams:** ingestão de vídeo em tempo real de câmeras e dispositivos. Combinado com Rekognition Video (análise) + CloudWatch (alertas) + S3 (armazenamento), forma o pipeline padrão de monitoramento de vídeo.
- **Transcodificação de mídia:** o Amazon Elastic Transcoder ainda consta da lista de serviços do guia de exame, mas **foi encerrado em novembro de 2025**. Em questões legadas, reconheça sua função histórica. Para uma arquitetura nova, consulte o AWS Elemental MediaConvert, embora esse serviço conste da lista oficial de serviços

fora de escopo. A lista do exame é não exaustiva e pode atrasar mudanças no catálogo. [Fim do Elastic Transcoder](#) · [Lista fora de escopo](#).

## Padrões e pegadinhas de streaming

- "Ingerir clickstream em tempo real para vários consumidores independentes, com replay": **Kinesis Data Streams**.
- "Levar os dados do stream ao S3 em Parquet, sem código e sem gerenciar servidores": **Data Firehose**.
- "Firehose não permite reprocessar dados": coloque o **Kinesis Data Streams** antes (retenção + replay).
- "Latência de milissegundos": Kinesis Data Streams. Firehose é **quase tempo real** (buffer).
- "Throttling em apenas alguns shards": **partition key** com baixa cardinalidade (hot shard).
- "Vários consumidores lendo o mesmo stream com latência baixa sem disputar leitura": **enhanced fan-out**.
- "Agregações por janela de tempo sobre o stream": **Managed Service for Apache Flink**.
- "Migrar um cluster Kafka on-premises sem reescrever produtores e consumidores": **Amazon MSK**.
- "Enviar logs do CloudWatch ao OpenSearch quase em tempo real": **subscription filter** → **Firehose** → **OpenSearch**.
- "Análise de vídeo de câmeras em tempo real": **Kinesis Video Streams + Rekognition Video**.

## Decisão rápida — Integração, Mensageria e Streaming

| Cenário / requisito                                                    | Resposta correta                    | Distratores clássicos                                                   |
|------------------------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------|
| Ordem por grupo de mensagens e deduplicação de envios em cinco minutos | SQS FIFO + consumidor idempotente   | SQS Standard não garante ordem; FIFO sozinho não impede reprocessamento |
| Confirmação imediata + processamento assíncrono pesado                 | SQS como buffer + worker consumidor | Lambda síncrona, Step Functions direto                                  |

## 7. Migração de Dados e Big Data / Analytics

### Regra de ouro da prova:

- **Migração:** identifique **o que** está sendo migrado. Servidores inteiros vão pelo **MGN**. Bancos de dados, pelo **DMS**. Arquivos, pelo **DataSync**/Transfer Family. VMs VMware sem conversão, pelo **VMware Cloud on AWS**. Depois veja **quanto a aplicação pode mudar**, pela estratégia dos 7 Rs.
- **Analytics:** pense no pipeline **ingestão** → **armazenamento** → **catálogo** → **processamento** → **consulta** → **visualização**. O padrão serverless de menor esforço é **S3 + Glue (Data Catalog/ETL) + Athena + Quick Sight**, com **Lake Formation** para governança. Formato **colunar comprimido e particionado** (Parquet/ORC) é a principal alavanca de custo e desempenho.

## Migração

### Estratégia de migração (7 Rs)

| Estratégia                                | O que significa                                                             | Ferramenta / exemplo                                          |
|-------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Retire</b>                             | Desligar o que não é mais necessário                                        | Resultado do inventário                                       |
| <b>Retain</b>                             | Manter on-premises por enquanto (dependência, compliance, recém-atualizado) | Revisitar depois                                              |
| <b>Rehost</b> (lift-and-shift)            | Mover sem alterar a aplicação                                               | <b>AWS MGN</b> → EC2                                          |
| <b>Relocate</b>                           | Mover o hypervisor inteiro, sem converter as VMs                            | <b>VMware Cloud on AWS</b>                                    |
| <b>Replatform</b> (lift-tinker-and-shift) | Pequenas otimizações sem mudar a arquitetura                                | Banco em EC2 → <b>RDS</b> ; app → Elastic Beanstalk/contêiner |
| <b>Repurchase</b>                         | Trocar por um produto SaaS                                                  | CRM próprio → SaaS                                            |
| <b>Refactor / Re-architect</b>            | Redesenhar como cloud-native                                                | Monólito → Lambda/ECS + DynamoDB                              |

- Os simulados antigos falam em "6 Rs". A AWS hoje usa **7 Rs**, com **Relocate** acrescentado.
- **Menor esforço e prazo curto** favorecem **Rehost**. **Reduzir operação sem reescrever** favorece **Replatform**. **Escalabilidade e agilidade no longo prazo** favorecem **Refactor**, com mais custo e tempo.

### Descoberta e planejamento

- **AWS Application Discovery Service:** coleta inventário, configuração, utilização e **dependências de rede** dos servidores on-premises, com agente (mais detalhes, inclusive processos e conexões) ou sem agente (VMware vCenter). Ajuda a agrupar aplicações em *waves* de migração.
- **AWS Migration Hub:** painel central para acompanhar o progresso das migrações entre ferramentas (MGN, DMS), com recomendações de estratégia e de tipo de instância.
- **Migration Evaluator:** cria o **business case** (TCO projetado na AWS) a partir dos dados de uso atuais.
- **AWS Transform:** agentes de IA para acelerar a modernização de VMware, mainframe e .NET. Confira o escopo atual na documentação.



## Servidores e máquinas virtuais

- **AWS Application Migration Service (MGN)**: lift-and-shift de servidores físicos/virtuais (VMware, Hyper-V, outras nuvens) para EC2, com **replicação contínua em nível de bloco** para uma área de staging de baixo custo e mínima interrupção.
  - **Fluxo cobrado**:
    1. instalar o **AWS Replication Agent** na origem;
    2. concluir a sincronização inicial;
    3. lançar *test instances* e validar;
    4. garantir que a replicação esteja atualizada;
    5. lançar a *cutover instance*.
  - O **AWS Application Discovery Service** pode ser usado antes para mapear dependências.
  - CloudEndure Disaster Recovery é orientado a DR, não a uma migração one-time moderna. Seu sucessor é o **AWS Elastic Disaster Recovery (DRS)**. Veja a seção 10.
- **VM Import/Export**: importa imagens de VM (VMDK, VHD, OVA) como AMIs ou volumes, para migrações pontuais e pequenas, sem replicação contínua.
- **Workloads VMware sem conversão**: **VMware Cloud on AWS** ou **Amazon EVS**. Veja [Híbrido: VMware, contêineres e servidores on-premises](#).

## Bancos de dados

- **AWS DMS (Database Migration Service)**: migração/replicação contínua de bancos de dados com **CDC (Change Data Capture)** para alterações incrementais quase em tempo real, mantendo o banco de origem online durante toda a migração. Para segurança, use múltiplas instâncias de replicação em subnets privadas, Multi-AZ e criptografia em trânsito e em repouso. Não serve para mover grandes volumes de arquivos (isso é DataSync).
- **AWS DMS Serverless**: cria uma serverless replication configuration e provisiona/escala automaticamente a capacidade de replicação para full load e CDC. É preferível à replication instance dimensionada manualmente quando o volume oscila e o requisito explícito é reduzir capacity planning/operação.
- Conversão de esquema, migrações heterogêneas e Babelfish: veja [AWS DMS](#).

## Arquivos e grandes volumes

- **DataSync** (online e recorrente), **Transfer Family** (SFTP/FTPS/AS2 de parceiros), **Storage Gateway** (acesso híbrido contínuo) e transferência física: veja [Migração e transferência de dados](#).
- **Cálculo de prazo**: volume ÷ banda **efetiva** (não a contratada) + overhead. Se o resultado estourar o prazo, avalie mais banda (Direct Connect), transferência física ou carga inicial offline + sincronização incremental online.

## Padrões e pegadinhas de migração para a AWS

- "Migrar centenas de servidores rapidamente, sem alterar as aplicações": **Rehost com MGN**.
- "Mapear dependências entre servidores antes de migrar": **Application Discovery Service** (com agente, para dependências detalhadas).
- "Trocar o banco em EC2 por um serviço gerenciado sem reescrever a aplicação": **Replatform** para RDS/Aurora com **DMS**.
- "Migrar VMs VMware sem converter formatos": **Relocate** com VMware Cloud on AWS.
- "Acompanhar migrações de várias ferramentas em um único lugar": **Migration Hub**.
- "Justificar financeiramente a migração": **Migration Evaluator**.
- "Banco precisa ficar online durante a migração": **DMS full load + CDC**.

## ETL e análise

### Arquitetura de data lake

- **Camadas no S3:** *raw* (dados originais, imutáveis), *processed/clean* (validados e convertidos para Parquet) e *curated* (agregados para consumo). Separe por prefixo ou bucket e aplique lifecycle às camadas antigas.
- **Catálogo e governança:** **Glue Data Catalog** (metadados de tabelas, partições e schemas, compartilhado por Athena, EMR, Redshift Spectrum e Glue) + **Lake Formation** (permissões finas centralizadas). **Tabelas Apache Iceberg** (inclusive **S3 Tables**) adicionam transações ACID, *time travel* e evolução de schema ao data lake.
- **Consumo:** Athena (SQL ad hoc), Redshift/Redshift Spectrum (data warehouse), EMR (Spark em larga escala), SageMaker (ML) e Quick Sight (BI).
- **Pipeline de ingestão de ponta a ponta:** defina volume, tamanho de evento, frequência, latência tolerada e ordem exigida.
  - **Carga periódica:** receba CSV no S3 e use o Glue para validar, transformar, compactar em Parquet e particionar pelos filtros frequentes. Catalogue com o Glue Data Catalog, governe com Lake Formation e consulte pelo Athena.
  - **Eventos contínuos:** escolha Kinesis Data Streams se múltiplos consumidores precisam processar o fluxo, ou Data Firehose se o objetivo é entrega gerenciada ao S3/Redshift/OpenSearch.
  - **Grande volume local:** compare DataSync/Direct Connect e a transferência física disponível conforme o prazo.
  - **Proteção:** transporte (TLS/endpoints privados quando aplicável), repouso (S3/KMS), roles mínimas e isolamento de dados sensíveis.
  - [Glue e S3](#) · [Athena e dados colunares](#).

| Necessidade de analytics                                               | Serviço                                      |
|------------------------------------------------------------------------|----------------------------------------------|
| SQL ad hoc sobre o S3, sem servidores, pago por consulta               | <b>Athena</b>                                |
| Data warehouse com consultas complexas, BI frequente e muitos usuários | <b>Redshift</b> (provisionado ou Serverless) |
| Spark/Hadoop/Hive em grande escala, controle do cluster                | <b>EMR</b> (EC2, EKS ou Serverless)          |
| ETL serverless e catálogo de metadados                                 | <b>Glue</b>                                  |
| Preparação visual de dados sem código                                  | <b>Glue DataBrew</b>                         |
| Busca full-text e análise de logs                                      | <b>OpenSearch Service</b>                    |
| Dashboards e BI                                                        | <b>Quick Sight</b>                           |
| Streams em tempo real                                                  | <b>Kinesis / MSK / Flink</b> (seção 6)       |
| Permissões por linha/coluna no data lake                               | <b>Lake Formation</b>                        |

### AWS Glue

- **AWS Glue:** ETL serverless gerenciado (Spark ou Python), cobrado por DPU-hora, sem cluster para administrar.
- **Crawlers:** varrem o S3 (ou bancos via JDBC), **inferem o schema** e criam/atualizam tabelas e **partições** no Data Catalog. Rode-os por agendamento ou evento para que novas partições apareçam no Athena.
- **Job Bookmarks:** rastreiam execuções anteriores para processar só dados novos (evitam reproprocessamento). `FindMatches` é ML para identificar duplicados, não controla o que já foi processado.
- **Glue Studio** (editor visual de jobs), **Glue streaming ETL** (consome Kinesis/MSK), **Glue Data Quality** (regras de qualidade), **workflows e triggers** (orquestração simples) e **Glue Schema Registry** (schemas para streams).

- **Segurança e custo:** combine VPC Endpoint Gateway (tráfego privado ao S3) com formatos colunares comprimidos (Parquet/ORC), reduzindo o volume processado.
- **AWS Glue DataBrew:** preparação visual e no-code de dados para limpeza, normalização e profiling, com mais de 250 transformações prontas. DataBrew prepara datasets, Glue ETL executa pipelines programáveis e o Glue Data Catalog mantém metadados. Não confunda essas funções.

## Amazon Athena

- **Amazon Athena:** consultas SQL serverless ad hoc direto sobre dados no S3, usando o Glue Data Catalog. É a resposta padrão para análises simples e sob demanda sem provisionar infraestrutura. Particionamento e formato colunar melhoram o desempenho e reduzem o custo por consulta.
- **Cobrança por dados escaneados (TB).** Para reduzir custo:
  - **Parquet/ORC** (lê só as colunas usadas);
  - **compressão**;
  - **particionamento** pelos filtros frequentes (ex.: `ano/mes/dia`), com *partition projection* para muitas partições sem crawler;
  - arquivos de tamanho adequado, evitando milhares de arquivos pequenos.
- **CTAS** (`CREATE TABLE AS SELECT`) e `INSERT INTO` convertem dados para Parquet particionado pelo próprio Athena.
- **Workgroups:** separam equipes e **limitam os dados escaneados** por consulta ou por período, com métricas e controle de custo. **Query result reuse** evita escanear de novo consultas repetidas.
- **Federated queries:** conectores (Lambda) consultam RDS, DynamoDB, Redshift, CloudWatch Logs e fontes on-premises na mesma SQL, sem mover dados.
- **Athena for Apache Spark:** notebooks Spark interativos sem cluster.

## Amazon EMR

- **Amazon EMR:** processamento de big data em larga escala com Spark, Hadoop, Hive, Presto/Trino, HBase e Flink. É excessivo para poucos arquivos pequenos.
- **Opções de execução:** **EMR on EC2** (controle total), **EMR on EKS** (jobs Spark em um cluster Kubernetes existente) e **EMR Serverless** (sem gerenciar cluster, para jobs Spark/Hive intermitentes).
- **Tipos de nó:**
  - **primary** (coordena, um por cluster, ou três para HA);
  - **core** (processa e guarda HDFS; perder um nó pode perder dados);
  - **task** (só processa, sem HDFS). Use **On-Demand/Reserved** para primary e core, e **Spot** para task nodes.
- **EMRFS:** dados no **S3** em vez do HDFS **desacoplam armazenamento e computação**. Isso permite **clusters transitórios** que sobem para o job e são encerrados em seguida, reduzindo muito o custo. Clusters de longa duração só quando há uso contínuo.
- **Instance fleets** diversificam tipos de instância e opções de compra, e o **managed scaling** ajusta o tamanho do cluster.

## Amazon Redshift

- **Amazon Redshift:** data warehouse gerenciado para OLAP, com armazenamento **colunar**, processamento paralelo massivo (MPP) e compressão. Veja também a seção Banco de Dados.
- **RA3 com managed storage** separa computação e armazenamento (dados no S3 com cache local). **Distribution keys** e **sort keys** orientam o desempenho de joins e filtros.

- **Carga:** **COPY a partir do S3** (paralelo, a forma recomendada). Evite **INSERT** linha a linha. Streaming ingestion lê diretamente do Kinesis/MSK. **Zero-ETL** traz dados do Aurora, RDS e DynamoDB sem pipeline. Veja [Recursos exclusivos do Aurora](#).
- **Redshift Spectrum:** consulta dados **no S3** (via Glue Data Catalog) a partir do Redshift, juntando-os com as tabelas locais, sem carregar tudo no warehouse.
- **Escala e compartilhamento:**
  - **concurrency scaling:** capacidade extra automática em picos de consultas;
  - **data sharing:** compartilha dados ao vivo entre clusters/workgroups e contas, sem copiar;
  - **materialized views;**
  - **WLM** para priorizar filas de consulta.
- **Disponibilidade e DR:** snapshots automáticos e manuais no S3, **cópia de snapshot cross-Region e Multi-AZ** (RA3).
- **Amazon Redshift Serverless:** executa analytics SQL sem provisionar cluster e ajusta a capacidade (RPU) conforme a demanda. É indicado para consultas intermitentes/imprevisíveis. **Redshift ML** permite criar e usar modelos de machine learning por SQL, com integração gerenciada ao SageMaker, sem construir o pipeline de ML manualmente.

## Amazon OpenSearch Service

- **Amazon OpenSearch Service:** busca e análise gerenciada (full-text, logs, observabilidade, busca vetorial) com **OpenSearch Dashboards**.
- **Ingestão:** o CloudWatch Logs não entrega diretamente: é preciso um subscription filter ou o Firehose no meio. Outras vias são **OpenSearch Ingestion** (pipelines gerenciados) e **zero-ETL** a partir do DynamoDB e do S3.
- **Domínios provisionados:** nós em **3 AZs** com a opção *Multi-AZ with standby*, e camadas **UltraWarm** e **cold storage** para reduzir o custo de dados antigos. **OpenSearch Serverless** dispensa o dimensionamento de nós.
- **Segurança:** acesso **dentro da VPC**, *fine-grained access control* (por índice, documento e campo), autenticação via Cognito/SAML e criptografia.

## Amazon Quick Sight e outros serviços

- **Amazon Quick Sight:** componente de BI, dashboards e visualização dentro do **Amazon Quick** (antigo QuickSight). Aparece no guia como Amazon Quick. Use Athena/Redshift como fonte analítica e Quick Sight para visualização, não como API transacional de baixa latência. [Amazon Quick](#).
  - **SPICE:** motor em memória que acelera dashboards e reduz as consultas à fonte.
  - **Row-level security** e column-level security por usuário/grupo.
  - **Dashboards incorporados** em aplicações.
  - Perguntas em linguagem natural.
- **AWS Data Exchange:** assinatura de datasets de terceiros. Combinado com S3 + IAM/políticas de bucket, dá controle de acesso granular sem necessidade de EC2 para armazenamento. Os dados podem ser entregues diretamente no S3 ou consultados via Redshift/APIs.
- **AWS Data Pipeline (legado):** aparece em materiais antigos de orquestração de ETL, mas não aceita novos clientes desde julho de 2025. Para novos pipelines, use Glue e Step Functions conforme a transformação e a orquestração. Avalie a migração de workloads existentes. [Histórico do Data Pipeline](#).
- **Orquestração de pipelines:** **Step Functions** (serverless, integrações nativas com Glue/EMR/Athena), **Glue workflows** (dentro do Glue) ou **Amazon MWAA** (Apache Airflow gerenciado, quando a equipe já usa DAGs do Airflow).

## Padrões e pegadinhas de analytics

- "Consultas SQL ocasionais em logs no S3, menor custo e sem infraestrutura": **Athena** + Parquet particionado.
- "Consulta no Athena escaneia TBs e custa caro": **Parquet/ORC + compressão + particionamento** (e workgroups com limite).
- "Novas partições não aparecem no Athena": rodar o **crawler** do Glue (ou usar partition projection).
- "ETL reprocessa arquivos antigos a cada execução": **Glue Job Bookmarks**.
- "Analistas de negócio precisam limpar dados sem código": **Glue DataBrew**.
- "Spark em larga escala com o menor custo": **EMR** com dados no S3, **cluster transitório** e **Spot nos task nodes** (ou EMR Serverless).
- "Data warehouse para BI com consultas complexas e muitos usuários": **Redshift**. "Consultas raras e imprevisíveis": **Redshift Serverless**.
- "Consultar dados do S3 junto com tabelas do Redshift sem carregá-los": **Redshift Spectrum**.
- "Carregar grandes volumes no Redshift": **COPY do S3**.
- "Compartilhar dados do Redshift com outra conta sem copiar": **data sharing**.
- "Permissões por coluna e linha para diferentes analistas no data lake": **Lake Formation**.
- "Dashboards para centenas de usuários sem sobrecarregar a fonte": **Quick Sight com SPICE**.
- "Busca textual e análise de logs de aplicação": **OpenSearch Service**.

## Decisão rápida — Migração de Dados e Analytics

| Cenário / requisito                                       | Resposta correta                                                                      | Distratores clássicos                                |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------|
| Grandes volumes, rede insuficiente para o prazo           | Data Transfer Terminal ou parceiro para novo cliente; Snowball Edge só se já elegível | DataSync depende da banda disponível                 |
| Transferência recorrente e automatizada on-premises ↔ AWS | AWS DataSync                                                                          | Mídia física é indicada para cenário pontual/offline |
| Capturar mudanças (CDC) de um banco relacional            | AWS DMS com CDC                                                                       | EventBridge direto, trigger simples                  |

## 8. Machine Learning

**Regra de ouro da prova:** suba o nível de abstração até onde o requisito permite.

1. **Serviço de IA pré-treinado** (Textract, Comprehend, Rekognition, Transcribe...) quando a tarefa é comum. Não há modelo para treinar, paga-se por uso, e é o **menor esforço operacional**.
2. **Amazon Bedrock** quando se precisa de **IA generativa** (texto, chat, resumo, RAG) com modelos de fundação via API.
3. **Amazon SageMaker AI** apenas quando é preciso **treinar ou ajustar um modelo próprio** com dados específicos do negócio.

Usar SageMaker para algo que um serviço pré-treinado já resolve é esforço desnecessário, uma pegadinha frequente.

| Entrada / tarefa                                                              | Serviço                                                   |
|-------------------------------------------------------------------------------|-----------------------------------------------------------|
| Texto e dados de <b>documentos</b> (PDF, formulários, tabelas, notas fiscais) | <b>Textract</b>                                           |
| Sentimento, entidades, PII, classificação de <b>texto</b>                     | <b>Comprehend</b> ( <b>Comprehend Medical</b> para saúde) |
| Tradução                                                                      | <b>Translate</b>                                          |
| <b>Áudio → texto</b>                                                          | <b>Transcribe</b>                                         |
| <b>Texto → fala</b>                                                           | <b>Polly</b>                                              |
| <b>Imagem/vídeo:</b> objetos, rostos, moderação, EPI                          | <b>Rekognition</b>                                        |
| <b>Chatbot</b> de voz ou texto                                                | <b>Lex</b>                                                |
| Busca inteligente em documentos corporativos                                  | <b>Kendra</b>                                             |
| Recomendações personalizadas                                                  | <b>Personalize</b>                                        |
| Revisão humana de previsões com baixa confiança                               | <b>Augmented AI (A2I)</b>                                 |
| IA generativa, RAG, agentes                                                   | <b>Bedrock</b>                                            |
| Modelo customizado: treino, ajuste, deploy                                    | <b>SageMaker AI</b>                                       |

### Serviços de IA pré-treinados

#### Documentos e texto

- **Amazon Textract:** extrai texto e dados estruturados de documentos (PDF, imagem escaneada). É OCR, não confundir com Rekognition. Vai além do OCR simples:
  - **formulários** (pares chave-valor) e **tabelas**;
  - **Queries** (perguntas em linguagem natural sobre o documento);
  - **AnalyzeExpense** (notas fiscais e recibos);
  - **AnalyzeID** (documentos de identidade). Documentos de várias páginas usam a **API assíncrona**: arquivo no S3 → job → notificação via SNS.
- **Amazon Comprehend:** NLP para texto genérico:
  - análise de sentimento, entidades, frases-chave e idioma;
  - **detecção e redação de PII**;



- modelagem de tópicos;
- **classificação e entidades personalizadas**, treinadas com seus exemplos sem precisar do SageMaker.
- **Amazon Comprehend Medical**: NLP pré-treinado para identificar entidades médicas (medicamentos, diagnósticos, dosagens) e PHI, sem treinar modelo próprio.
- **Amazon Translate**: tradução automática de idiomas, em tempo real ou em lote (documentos no S3). A **terminologia personalizada** preserva nomes de marcas e termos técnicos. O acesso seguro é via VPC Endpoint + IAM restritivo.
- **Amazon Kendra**: busca inteligente sobre documentos (ex.: em buckets S3, SharePoint, Confluence, bancos), por meio de **conectores** e com perguntas em linguagem natural. Respeita as **ACLs** dos documentos nos resultados. O controle de acesso é via IAM Role de menor privilégio + políticas de bucket restritivas. Pode servir de *retriever* para aplicações de IA generativa (RAG).

## Fala e conversação

- **Amazon Transcribe**: converte áudio em texto, em lote (arquivos no S3) ou **streaming**. Tem identificação de múltiplos falantes, **redação automática de PII**, **vocabulário personalizado** para termos do domínio, **Call Analytics** (sentimento e resumo de chamadas) e **Transcribe Medical**.
- **Amazon Polly**: converte texto em fala (text-to-speech) realista, com vozes neurais, **SSML** para controlar pronúncia, pausas e ênfase, e *lexicons* personalizados. É usado tipicamente em conjunto com o Lex (para dar resposta em voz a um chatbot) ou para acessibilidade e narração automática de conteúdo.
- **Amazon Lex**: constrói interfaces conversacionais (chatbots) usando a mesma tecnologia de NLU e reconhecimento de fala da Alexa. É a resposta padrão para "criar um chatbot de atendimento" via voz ou texto.
  - Modela **intents** (intenções) e **slots** (parâmetros).
  - Uma **Lambda** executa a ação (*fulfillment*).
  - Integra-se ao **Amazon Connect** (central de atendimento) e a canais de mensagem.

## Imagem e vídeo

- **Amazon Rekognition**: análise de imagem e vídeo. Não faz OCR de documentos: detecta texto curto em cenas (placas, legendas), e documentos são com o Textract. Recursos:
  - rótulos (objetos, cenas, atividades);
  - **comparação e busca de rostos** em *collections*;
  - **moderação de conteúdo** impróprio;
  - detecção de **EPI** (equipamento de proteção);
  - **Face Liveness** (verificação de pessoa real);
  - **Custom Labels** (seus próprios objetos, com poucas imagens).
- **Vídeo**: armazenado no S3, com análise assíncrona e resultado via SNS. Em streaming, usa **Kinesis Video Streams**. Combinado com CloudWatch (alertas) + S3 (armazenamento), forma o pipeline padrão de monitoramento de vídeo.

## Personalização, revisão humana e serviços legados

- **Amazon Personalize**: recomendações personalizadas em tempo real ("quem viu isto também viu", ranking, e-mails segmentados) a partir de dados de interação, sem expertise em ML. É a mesma tecnologia usada na Amazon.com.
- **Amazon Augmented AI (A2I)**: fluxos de **revisão humana** quando a confiança da previsão do Textract, do Rekognition ou de um modelo próprio fica abaixo de um limite (ex.: documentos financeiros que exigem conferência).

- **Amazon Fraud Detector (legado):** ainda pode aparecer em simulados antigos, mas não aceita novos clientes desde novembro de 2025. Para proteção de login/cadastro, avalie as regras gerenciadas do AWS WAF (Fraud Control). Para modelos personalizados, avalie o SageMaker AI e os dados/treinamento necessários. [Mudança de disponibilidade](#).
- **Amazon Forecast (legado):** previsões de séries temporais. Também deixou de aceitar novos clientes. Para novas previsões, avalie o **SageMaker Canvas** ou modelos do SageMaker. Confira a disponibilidade na documentação.

## Padrões e pegadinhas de serviços de IA

- "Extrair campos de formulários e tabelas de PDFs escaneados": **Textract**, não Rekognition.
- "Analisar o sentimento de avaliações de clientes": **Comprehend**.
- "Identificar medicamentos e PHI em prontuários sem treinar modelo": **Comprehend Medical**.
- "Transcrever chamadas e remover números de cartão": **Transcribe** com redação de PII.
- "Moderar imagens enviadas por usuários": **Rekognition content moderation**.
- "Chatbot de atendimento com voz": **Lex** (+ Polly, + Connect).
- "Busca em linguagem natural sobre documentos internos respeitando permissões": **Kendra**.
- "Recomendações de produtos em tempo real": **Personalize**.
- "Humano revisa extrações com baixa confiança": **Augmented AI**.

## Amazon SageMaker AI

### Ciclo de vida do modelo

- **Amazon SageMaker AI:** plataforma completa para preparar dados, treinar, ajustar, implantar e monitorar modelos customizados. Usá-la para algo já resolvido por um serviço pré-treinado (Comprehend Medical, Textract etc.) é esforço desnecessário.
- **Preparação e rotulagem:** **SageMaker Ground Truth** faz rotulagem de dados com força de trabalho humana e rotulagem automática. **Data Wrangler** faz a preparação visual. **Feature Store** guarda features reutilizáveis para treino e inferência.
- **Treinamento:** jobs gerenciados em instâncias que só existem durante o treino. O **Managed Spot Training** reduz o custo (até ~90%) com **checkpoints** no S3 para retomar após uma interrupção. O **Automatic Model Tuning** busca hiperparâmetros.
- **Sem código e modelos prontos:** **SageMaker Canvas** permite que analistas de negócio criem modelos sem programar. **JumpStart** oferece modelos pré-treinados e de fundação para implantar ou ajustar.
- **MLOps:** **SageMaker Pipelines** (CI/CD de ML), **Model Registry** (versões e aprovação), **Model Monitor** (detecta **drift** de dados e de qualidade em produção) e **Clarify** (viés e explicabilidade).

### Opções de inferência

| Opção                         | Quando usar                                 | Observação                                                  |
|-------------------------------|---------------------------------------------|-------------------------------------------------------------|
| <b>Real-time endpoint</b>     | Baixa latência, tráfego constante           | Instâncias sempre ativas, com auto scaling                  |
| <b>Serverless inference</b>   | Tráfego intermitente ou imprevisível        | Sem gerenciar instâncias; cold start possível               |
| <b>Asynchronous inference</b> | Payloads grandes, processamento longo       | Fila interna, resultado no S3 + SNS, <b>escala até zero</b> |
| <b>Batch transform</b>        | Previsões sobre um dataset inteiro, offline | Sem endpoint persistente                                    |

- **Custo:** endpoints real-time são cobrados por hora mesmo ociosos. Para uso esporádico, prefira **serverless** ou **assíncrono**. **Multi-model endpoints** hospedam vários modelos em um endpoint. **SageMaker AI Savings Plans** reduzem o custo de uso estável. Veja [Savings Plans](#).
- **Segurança:**
  - treino e endpoints **em modo VPC**, com **network isolation** opcional (o contêiner não acessa a rede);
  - **VPC interface endpoints** para as APIs e o runtime do SageMaker;
  - criptografia KMS de volumes, modelos e dados no S3;
  - **execution role** com acesso mínimo aos buckets.

## Padrões e pegadinhas de SageMaker

- "Treinar modelo próprio com dados históricos da empresa": **SageMaker AI**.
- "Reduzir o custo de treinamentos longos que podem ser retomados": **Managed Spot Training + checkpoints**.
- "Modelo com poucas requisições por dia, sem pagar instância ociosa": **serverless inference** (ou assíncrono).
- "Inferência sobre arquivos grandes que leva minutos": **asynchronous inference**.
- "Pontuar milhões de registros uma vez por dia": **batch transform**.
- "Detectar que o modelo perdeu precisão em produção": **Model Monitor**.
- "Analistas sem conhecimento de programação criam previsões": **SageMaker Canvas**.
- "Rotular milhares de imagens para treino": **Ground Truth**.

---

## IA generativa

### Amazon Bedrock

- **Amazon Bedrock:** acesso **serverless via API** a modelos de fundação de vários provedores (Anthropic Claude, Amazon Nova, Meta Llama, Mistral e outros), sem gerenciar infraestrutura. Os prompts e dados **não são usados para treinar** os modelos base, e o acesso privado é via **PrivateLink**. Confira na lista oficial do exame o escopo atual de serviços de IA generativa.
- **Knowledge Bases: RAG** gerenciado. Ingre documentos do S3 e de outras fontes, gera embeddings e os guarda em um vector store (OpenSearch Serverless, Aurora PostgreSQL com pgvector, S3 Vectors e outros). As respostas vêm fundamentadas nos seus dados, com citações. É preferível a *fine-tuning* quando o objetivo é responder sobre dados da empresa que mudam com frequência.
- **Agents:** executam tarefas em várias etapas chamando APIs e funções Lambda. **Guardrails:** filtros de conteúdo, tópicos negados, **redação de PII** e verificação de fundamentação, aplicáveis a qualquer modelo.
- **Customização:** *fine-tuning* e *continued pre-training* com seus dados, quando RAG e prompts não bastam.
- **Custo e capacidade:** on-demand (por token), **batch inference** (mais barato, para grandes volumes sem urgência) ou **provisioned throughput** (capacidade garantida).
- **Amazon Q:**
  - **Q Business:** assistente corporativo sobre dados da empresa, com conectores e respeito às permissões;
  - **Q Developer:** assistente para desenvolvimento e operação na AWS.

### Padrões e pegadinhas de IA generativa

- "Chatbot que responde sobre documentos internos atualizados com frequência": **Bedrock Knowledge Bases (RAG)**, não *fine-tuning*.
- "Impedir que o assistente revele PII ou fale de temas proibidos": **Bedrock Guardrails**.

- "Usar modelos de fundação sem gerenciar GPUs": **Bedrock**. "Hospedar um modelo open-source próprio com controle total": **SageMaker AI** (JumpStart/endpoints).
- "Gerar resumos de milhões de documentos com o menor custo, sem urgência": **batch inference** no Bedrock.
- "Assistente para funcionários sobre os sistemas da empresa, com pouco desenvolvimento": **Amazon Q Business**.

## Padrões de arquitetura com ML

- **Padrão recorrente**: para processamento de mídia/texto sob demanda com picos irregulares (imagens, sentimento, áudio), a arquitetura serverless S3 (evento) → Lambda → serviço de IA gerenciado é sempre mais econômica que EC2 dedicado/EMR batch.
- **Documentos longos**: S3 → Lambda inicia o job **assíncrono** do Textract/Transcribe → **SNS** notifica a conclusão → Lambda grava o resultado no DynamoDB/S3. Para várias etapas (extrair → classificar → revisar), use o **Step Functions**.
- **Absorver picos**: coloque **SQS** entre a ingestão e as chamadas aos serviços de IA, para respeitar as cotas de requisição e fazer retry sem perder itens.
- **Segurança e conformidade**:
  - **VPC endpoints** para os serviços de IA (Comprehend, Translate, Textract, Bedrock, SageMaker);
  - KMS nos dados de entrada e saída;
  - roles de menor privilégio;
  - Região escolhida conforme a residência de dados;
  - redação de PII (Comprehend, Transcribe, Guardrails) antes de armazenar ou exibir.
- **Custo**: serviços pré-treinados e Bedrock on-demand cobram por uso, com zero de custo ocioso. Endpoints SageMaker real-time cobram por hora. Escolha o modelo de inferência pela frequência de uso.

## Decisão rápida — Machine Learning

| Cenário / requisito                                                           | Resposta correta                               | Distratores clássicos                             |
|-------------------------------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Extrair texto, formulários e tabelas de documentos escaneados                 | Amazon Textract                                | Rekognition, Comprehend                           |
| Sentimento, entidades ou PII em texto                                         | Amazon Comprehend                              | Textract, modelo próprio no SageMaker             |
| Entidades médicas e PHI sem treinar modelo                                    | Amazon Comprehend Medical                      | SageMaker, Comprehend genérico                    |
| Áudio para texto com redação de PII                                           | Amazon Transcribe                              | Polly, Comprehend                                 |
| Análise de imagem/vídeo e moderação de conteúdo                               | Amazon Rekognition                             | Textract (documentos)                             |
| Chatbot de voz ou texto                                                       | Amazon Lex (+ Polly)                           | Comprehend, Kendra                                |
| Respostas de IA generativa sobre documentos internos que mudam com frequência | Amazon Bedrock Knowledge Bases (RAG)           | Fine-tuning, SageMaker treinado do zero           |
| Modelo próprio treinado com dados do negócio                                  | Amazon SageMaker AI                            | Serviços pré-treinados (quando não cobrem o caso) |
| Inferência esporádica sem pagar instância ociosa                              | SageMaker Serverless ou Asynchronous Inference | Real-time endpoint sempre ativo                   |

## 9. Governança, Monitoramento e Custos

### Regra de ouro da prova:

- **Monitoramento:** métricas e alarmes → **CloudWatch**; quem fez o quê → **CloudTrail**; como o recurso está configurado → **Config**; onde a requisição demorou → **X-Ray**.
- **Operação:** prefira **automação e IaC** (CloudFormation, Systems Manager) a passos manuais.
- **Custo:** primeiro **visibilidade** (tags, Cost Explorer), depois **controle** (Budgets, SCPs), por fim **otimização** (rightsizing, compromissos, Spot, lifecycle, transferência de dados). "Menor custo" só decide entre alternativas que já atendem aos requisitos de segurança, disponibilidade e desempenho.

## Monitoramento e observabilidade

### Amazon CloudWatch

- **Amazon CloudWatch:** monitoramento com métricas, logs, alarmes, dashboards e composite alarms. As granularidades e os períodos de alarme estão em [Números do CloudWatch](#).
- **Métricas:**
  - A EC2 publica CPU, rede, disco (Instance Store) e status checks. O padrão é de 5 minutos, e o **detailed monitoring** entrega a cada 1 minuto (cobrado).
  - **Memória, uso de disco EBS no sistema operacional e processos não são métricas nativas:** exigem o **CloudWatch agent**.
  - Métricas personalizadas podem ser enviadas via `PutMetricData`.
  - Métricas nativas geralmente bastam para planejamento de capacidade. Um agente customizado só se justifica quando as nativas são insuficientes.
- **Alarmes:** estados `OK`, `ALARM` e `INSUFFICIENT_DATA`. Ações possíveis:
  - notificar via SNS;
  - escalar um Auto Scaling group;
  - **parar, encerrar, reiniciar ou recuperar** uma instância EC2;
  - executar Systems Manager Automation;
  - criar itens no OpsCenter. **Composite alarms** combinam múltiplas condições (ex.: CPU e IOPS) para reduzir falsos positivos. **Anomaly detection** cria faixas esperadas por ML em vez de limites fixos.
- **CloudWatch Logs:**
  - **log groups** com **retenção configurável**. O padrão é *nunca expirar*, o que acumula custo: defina a retenção;
  - **Logs Insights** consulta logs com linguagem própria;
  - **metric filters** transformam padrões de log (ex.: `ERROR`) em métricas com alarme;
  - **exportação para o S3** permite arquivar a baixo custo;
  - criptografia com KMS.
- **Subscription filters:** encaminham logs em tempo real, sem agentes, para Lambda, Kinesis Data Streams, Data Firehose ou OpenSearch, inclusive para uma **conta central** de logs.
- **Observabilidade de aplicações:**
  - **Container Insights** e **Lambda Insights:** métricas detalhadas de contêineres e funções;

- **Application Signals:** SLOs e métricas de aplicação;
- **Synthetics:** *canaries* que testam endpoints e fluxos periodicamente, detectando falhas antes dos usuários;
- **RUM:** experiência real dos usuários no navegador;
- **Internet Monitor:** impacto de problemas da internet nos seus usuários.
- **Cross-Account Observability:** centraliza métricas, logs e traces de múltiplas contas-membro em uma conta de monitoramento, sem usuários IAM por conta.
- **EventBridge + CloudWatch:** mudanças de estado de recursos (instância parada, falha de job, evento do Health) disparam automações. Veja [Amazon EventBridge](#).

## Números do CloudWatch

| Métrica                                           | Valor                     | Observação                                                                              |
|---------------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------|
| Granularidade padrão (standard resolution)        | 60 segundos (1 min)       |                                                                                         |
| Granularidade de alta resolução (high resolution) | 1 segundo (armazenamento) | Leitura possível em períodos de 1s/5s/10s/30s ou múltiplos de 60s                       |
| Período mínimo de avaliação de alarme             | 10 segundos               | Só para alarmes de alta resolução (10s/20s/30s); alarmes padrão exigem múltiplos de 60s |

## Rastreamento e observabilidade open-source

- **AWS X-Ray:** rastreamento distribuído. Mostra o caminho de uma requisição por API Gateway, Lambda, ECS, EC2 e chamadas a serviços, com **service map**, latência por segmento e erros. É a resposta para "descobrir qual microserviço causa a lentidão". O daemon instalado localmente em cada instância EC2 é o componente nativo para coleta com mínimo impacto de desempenho. Para novas instrumentações, a AWS recomenda **OpenTelemetry** via **AWS Distro for OpenTelemetry (ADOT)** ou o CloudWatch agent.
- **Amazon Managed Grafana + Amazon Managed Service for Prometheus:** observabilidade open-source gerenciada. O Prometheus coleta e armazena métricas, especialmente úteis para cargas em contêineres/Kubernetes/EKS. O Grafana visualiza essas métricas em dashboards. É a combinação recomendada para monitorar cargas EKS/containerizadas sem operar manualmente a infraestrutura de observabilidade, como alternativa gerenciada ao stack Prometheus + Grafana autogerenciado em EC2.

## AWS Health

- **AWS Health Dashboard:** fornece visibilidade personalizada sobre eventos que afetam a conta e os recursos AWS específicos do usuário (manutenções programadas, degradações de serviço, instâncias agendadas para *retirement*). É diferente do Service Health Dashboard público (status geral da AWS para todos os clientes): o AWS Health Dashboard é personalizado por conta.
- **Eventos do AWS Health no EventBridge** permitem automatizar a resposta (ex.: parar/iniciar uma instância agendada para retirement, notificar a equipe). Com Organizations, a **visão organizacional** agrega os eventos de todas as contas.

## Padrões e pegadinhas de monitoramento

- "Monitorar o uso de memória das instâncias": **CloudWatch agent** (não é métrica nativa).
- "Alertar quando aparecer `ERROR` nos logs": **metric filter** + alarme.
- "Custo de CloudWatch Logs crescendo": definir **retenção** dos log groups e exportar o histórico para o S3.
- "Recuperar automaticamente uma instância com falha de hardware": alarme de **status check** com a ação *recover*.



- "Reduzir alarmes falsos com várias condições": **composite alarm**.
- "Identificar o serviço lento em uma cadeia de microsserviços": **X-Ray**.
- "Testar continuamente um fluxo de login do site": **CloudWatch Synthetics**.
- "Centralizar logs de todas as contas": **subscription filters** para uma conta central (ou **Cross-Account Observability**).
- "Saber se uma manutenção da AWS afetará minhas instâncias": **AWS Health Dashboard**.

---

## Operações e infraestrutura como código

### AWS CloudFormation

- **AWS CloudFormation**: Infrastructure as Code (IaC) serverless gerenciado. Define e provisiona infraestrutura AWS via templates declarativos (YAML/JSON), permitindo infraestrutura imutável e repetível entre ambientes. É a resposta padrão para "provisionar/versionar infraestrutura de forma consistente e repetível". É diferente do Elastic Beanstalk (PaaS que orquestra automaticamente EC2 + ASG + ELB para uma aplicação, sem controle granular de template) e do AWS Config (audita a conformidade de recursos já existentes, não provisiona).
- **Conceitos**:
  - **stack** (conjunto de recursos gerenciados juntos);
  - **parameters, mappings, conditions e outputs**;
  - **exports/imports** entre stacks;
  - **nested stacks** para reutilizar componentes.
- **Mudanças seguras**:
  - **change sets** mostram o que será alterado antes de aplicar;
  - **rollback automático** em falha;
  - **drift detection** identifica recursos alterados manualmente fora do template;
  - **stack policies** protegem recursos críticos contra atualizações.
- **Proteção de dados**: **DeletionPolicy: Retain** ou **Snapshot** preserva bancos, volumes e buckets quando a stack é excluída. **Termination protection** impede a exclusão acidental da stack.
- **Multi-conta e multi-Região**: **StackSets** implantam o mesmo template em várias contas e Regiões, com integração a Organizations (implantação automática em novas contas).
- **Inicialização de instâncias**: **cfn-init** e **helper scripts** configuram a instância. **CreationPolicy / WaitCondition** fazem a stack aguardar o sinal de que o software está pronto.
- **Ferramentas relacionadas**: **AWS CDK** (IaC em TypeScript, Python, Java) e **SAM** (serverless) geram CloudFormation. **Terraform** é a alternativa de terceiros multicloud.
- **Console, AWS CLI e IaC**: o console serve à exploração e a tarefas pontuais. A AWS CLI chama APIs para operações repetíveis e scripts. O CloudFormation declara o estado de múltiplos recursos, permite revisar mudanças e recriar de forma consistente. Em produção, versionar templates e validar mudanças é mais seguro do que depender de cliques manuais. [AWS CLI](#).

### AWS Systems Manager

- **AWS Systems Manager**: conjunto de ferramentas operacionais para gerenciar frotas de instâncias (EC2 e on-premises) em escala. É a resposta padrão para "gerenciar infraestrutura híbrida (on-premises + AWS) de forma centralizada e auditável, sem acesso direto às máquinas". Exige o **SSM Agent** e uma **IAM role** (instance profile) com permissões do Systems Manager. Em subnets privadas sem NAT, também exige VPC endpoints.
- **Ferramentas**:

- **Patch Manager**: aplica patches de sistema operacional automaticamente, com *patch baselines* e janelas;
  - **Maintenance Windows**: agendam tarefas;
  - **Automation**: runbooks para tarefas operacionais repetíveis, como criar AMIs, reiniciar serviços ou remediar findings do Config;
  - **Run Command**: executa comandos remotamente sem SSH;
  - **State Manager**: mantém a configuração desejada das instâncias;
  - **Inventory**: software e configuração instalados;
  - **Session Manager**: acesso sem SSH. Veja [Acesso administrativo e remoto](#);
  - **Parameter Store**: configuração e segredos. Veja [Segredos](#);
  - **OpsCenter**: centraliza itens operacionais;
  - **Fleet Manager**: gerência visual de servidores.
- **Servidores on-premises**: *hybrid activations* registram máquinas fora da AWS no Systems Manager.

## AWS Well-Architected

- **AWS Well-Architected Tool**: avaliação estruturada de workloads contra as melhores práticas, com relatório de riscos e plano de melhorias. Combinada com AWS Config + Lambda, permite avaliações automatizadas após mudanças significativas no ambiente. **Lenses** adicionam perguntas específicas (serverless, SaaS, ML).
- **AWS Well-Architected Framework — os 6 pilares**: base conceitual usada pela ferramenta acima para avaliar workloads.
  1. **Operational Excellence** (Excelência Operacional): operar e monitorar sistemas para entregar valor de negócio e melhorar processos/procedimentos continuamente (IaC, deploys pequenos e reversíveis, resposta a eventos).
  2. **Security** (Segurança): proteger informação, sistemas e ativos por meio de gestão de risco, princípio do menor privilégio e defesa em profundidade.
  3. **Reliability** (Confiabilidade): garantir que um workload funcione corretamente e de forma consistente, incluindo recuperação de falhas e escalonamento dinâmico de demanda.
  4. **Performance Efficiency** (Eficiência de Performance): usar recursos computacionais de forma eficiente para atender aos requisitos, mantendo essa eficiência conforme a demanda e a tecnologia evoluem.
  5. **Cost Optimization** (Otimização de Custos): evitar gastos desnecessários e entregar valor de negócio pelo menor custo possível.
  6. **Sustainability** (Sustentabilidade): pilar mais recente (adicionado em 2021). Minimizar os impactos ambientais de workloads em nuvem (eficiência energética, otimização de recursos).

Os 4 domínios oficiais do SAA-C03 (seção "Domínios do exame SAA-C03") mapeiam diretamente para 4 destes pilares: **Security** → **Domain 1** (Design Secure Architectures), **Reliability** → **Domain 2** (Design Resilient Architectures), **Performance Efficiency** → **Domain 3** (Design High-Performing Architectures), **Cost Optimization** → **Domain 4** (Design Cost-Optimized Architectures). Já **Operational Excellence** e **Sustainability** permeiam todos os domínios da prova, mas não têm peso isolado no exame.

## Padrões e pegadinhas de operações

- "Recriar o mesmo ambiente em várias Regiões e contas": **CloudFormation StackSets**.
- "Ver o que uma atualização de stack vai mudar antes de aplicar": **change set**.
- "Alguém alterou um security group manualmente fora do template": **drift detection**.
- "Excluir a stack sem perder o banco": `DeletionPolicy: Snapshot / Retain`.

- "Aplicar patches em centenas de instâncias Windows e Linux em janelas definidas": **Systems Manager Patch Manager** + Maintenance Windows.
- "Executar um script em toda a frota sem SSH": **Run Command**.
- "Instância não aparece no Systems Manager": falta o **SSM Agent**, a **IAM role** ou a conectividade (NAT ou VPC endpoints).
- "Avaliar a arquitetura contra boas práticas e registrar os riscos": **Well-Architected Tool**.

---

## Governança multi-conta e conformidade

- **AWS Control Tower**: orquestra a criação de uma landing zone multi-conta com boas práticas de governança pré-configuradas (estrutura de OUs, SCPs, logging centralizado) sobre o AWS Organizations. É a resposta padrão para "configurar rapidamente um ambiente multi-conta seguro e em conformidade desde o início", em vez de configurar Organizations/SCPs manualmente do zero. Detalhes dos controles em [Estrutura e governança multi-conta](#).
- **AWS Service Catalog**: permite que administradores criem catálogos de produtos de TI aprovados (tipicamente templates CloudFormation) que usuários finais lançam de forma self-service dentro de restrições de governança predefinidas. É a resposta padrão para "permitir que equipes provisionem recursos padronizados sem dar acesso total ao console/CloudFormation".
  - **Portfolios** podem ser compartilhados entre contas.
  - A **launch constraint** usa uma role própria para lançar o produto, de modo que o usuário não precisa de permissão sobre os recursos subjacentes.
- **AWS License Manager**: monitoramento do uso de licenças de software em todas as Regiões, com alertas automáticos de conformidade. **Regras de licença** (por vCPU, núcleo, socket) podem **impedir lançamentos** que violem o contrato. Integra-se a Dedicated Hosts para BYOL. Veja [Tenancy: Dedicated Hosts vs. Dedicated Instances](#).
- **Service Quotas**: consulta e **solicita aumento de cotas** (limites) por serviço e Região, com **alarmes do CloudWatch** quando o uso se aproxima do limite. Isso é essencial para DR: a Região de recuperação precisa ter cotas suficientes.
- **Tags, Resource Groups e Tag Editor**: organizam recursos por aplicação e ambiente para operação, automação (Systems Manager, Backup) e custo. **Tag policies** do Organizations padronizam chaves e valores.
- **AWS Auto Scaling** (o serviço "guarda-chuva", diferente do "EC2 Auto Scaling" da seção 1): gerencia o scaling de **múltiplos tipos de recurso simultaneamente** (EC2, ECS, DynamoDB, Aurora Replicas) por meio de planos de scaling unificados com target tracking. É a resposta padrão quando o requisito envolve escalar vários serviços diferentes de forma coordenada com uma única política, não apenas um Auto Scaling group de EC2.
- **AWS Config, CloudTrail, Security Hub e Audit Manager** (conformidade técnica e evidências): veja [Detecção, auditoria e conformidade](#).

---

## Gestão e otimização de custos

### Visibilidade de custos

- **AWS Pricing Calculator**: estima o custo **antes** de implantar uma arquitetura.
- **AWS Cost Explorer**: análise histórica detalhada de custos e uso, com filtragem granular por serviço, conta, tag e Região, **previsão** de gastos e **recomendações** de rightsizing, Reserved Instances e Savings Plans. É a resposta padrão para investigar aumentos de custo ou gerar relatórios segmentados.
- **Tags de alocação e faturamento consolidado**: padronize tags como projeto, ambiente e centro de custo, e **ative** as tags de alocação no Billing para que apareçam nos relatórios de custo (a ativação não é retroativa). Em AWS Organizations, o faturamento consolidado reúne o pagamento das contas-membro na conta de gerenciamento e

pode combinar uso elegível para descontos. Mantenha contas separadas para atribuição e governança. Compare custos por conta, tag e serviço no Cost Explorer ou no CUR/Data Exports. [Tags de custo](#) · [Faturamento consolidado](#).

- **AWS Cost and Usage Report (CUR) / Data Exports + Athena + Amazon Quick:** análise avançada de custos no nível de recurso e hora, quando o Cost Explorer ou o Budgets não têm granularidade suficiente. Exige mais configuração, então use só quando necessário.
- **AWS Cost Anomaly Detection:** usa ML para detectar **gastos fora do padrão** e alertar (SNS, e-mail), sem definir limites manualmente. É a resposta para "ser avisado de um aumento inesperado de custo em um serviço".

## Controle de custos

- **AWS Budgets:** monitora custo/uso real ou previsto e envia alertas. **Budgets Actions** pode aplicar automaticamente, ou mediante aprovação, uma IAM policy/SCP restritiva, ou atuar sobre instâncias EC2/RDS quando um limite é cruzado. Isso transforma o budget de aviso passivo em controle automatizado.
- **Savings Plans budgets:** budgets de **utilization** medem quanto do compromisso adquirido está sendo usado. Budgets de **coverage** medem qual percentual do uso elegível está coberto. Podem ser avaliados diariamente e notificar quando o percentual cai abaixo do limite. Existem budgets equivalentes para Reserved Instances.
- **Controles preventivos:** SCPs restringindo Regiões e tipos de instância caros, Service Catalog com produtos aprovados, e limites via Service Quotas.

## Otimização de custos

- **AWS Compute Optimizer:** usa machine learning para analisar métricas históricas de utilização (CloudWatch) e recomendar configurações ideais de tipo e tamanho para EC2, EBS, Lambda, ECS no Fargate, RDS e Auto Scaling groups. É a resposta padrão para "identificar recursos superdimensionados (over-provisioned)" e otimizar custo/desempenho sem análise manual. É gratuito e complementar ao Cost Explorer (que mostra gasto, não recomendações detalhadas de rightsizing).
- **Cost Optimization Hub:** consolida as recomendações de economia (rightsizing, compromissos, recursos ociosos) de todas as contas em um único lugar, com a economia estimada.
- **Trusted Advisor:** verificações de custo, como instâncias ociosas, volumes EBS não anexados, Elastic IPs sem uso e RIs subutilizadas.
- **Savings Plans para computação vs. específico de EC2:** quando há expectativa de crescimento em Lambda/Fargate além de EC2, o **Compute Savings Plan** maximiza a economia. Veja [EC2 – modelos de compra](#).
- **Principais alavancas:**
  - **rightsizing** antes de comprometer;
  - **Savings Plans/RIs** para a base;
  - **Spot** para cargas tolerantes a interrupção;
  - **Graviton**;
  - **Auto Scaling** e **desligar ambientes fora do horário** (EventBridge Scheduler / Instance Scheduler);
  - **serverless** para uso intermitente;
  - **lifecycle do S3** e Intelligent-Tiering;
  - **gp3** no lugar de gp2;
  - apagar **EBS não anexados, snapshots antigos, Elastic IPs ociosos** e load balancers sem alvos;
  - **retenção de logs**.

## Custos de transferência de dados

- **Entrada** de dados da internet para a AWS: **gratuita**. **Saída** para a internet: cobrada por GB, e a faixa gratuita é pequena.
- **Dentro da mesma AZ** via IP privado: gratuito. **Entre AZs**: cobrado nos dois sentidos, o que inclui replicação de bancos autogerenciados, tráfego para um NAT em outra AZ e cross-zone no NLB. **Entre Regiões**: cobrado.
- **Reduções típicas**:
  - **gateway endpoints** para S3/DynamoDB em vez de NAT (NAT cobra por GB processado);
  - **NAT por AZ** para evitar tráfego entre AZs;
  - **CloudFront** na frente do S3/ALB (a transferência da origem AWS para o CloudFront não é cobrada, e a saída do CloudFront costuma ser mais barata);
  - **IPs privados** em vez de públicos entre instâncias;
  - **compressão**;
  - **Direct Connect** para grandes volumes de saída para o on-premises.
- **Serviços gerenciados Multi-AZ** como RDS Multi-AZ e Aurora não cobram a replicação entre AZs do próprio serviço. Arquiteturas autogerenciadas em EC2 pagam por ela.

## Padrões e pegadinhas de custos

- "Descobrir por que a fatura aumentou no mês passado": **Cost Explorer**.
- "Ser avisado automaticamente quando um serviço gastar fora do padrão": **Cost Anomaly Detection**.
- "Impedir novos gastos ao atingir o orçamento": **Budgets Actions** (aplicar SCP/IAM policy ou parar instâncias).
- "Custo por projeto/centro de custo": **tags de alocação ativadas** + Cost Explorer.
- "Instâncias superdimensionadas": **Compute Optimizer**.
- "Conta de NAT alta por tráfego ao S3": **gateway endpoint**.
- "Ambiente de dev ligado 24/7": **agendar parada** fora do horário (EventBridge Scheduler/Instance Scheduler).
- "Estimar o custo de uma nova arquitetura antes de construir": **Pricing Calculator**.
- "Relatório de custo por recurso e por hora para análise com SQL": **CUR/Data Exports + Athena**.

## Decisão rápida — Governança, Monitoramento e Custos

| Cenário / requisito                          | Resposta correta  | Distratores clássicos   |
|----------------------------------------------|-------------------|-------------------------|
| Investigar rapidamente aumento de custo      | AWS Cost Explorer | AWS Budgets, CUR+Athena |
| Alertar quando o gasto se aproxima do limite | AWS Budgets       | Cost Explorer           |



## 10. Estratégias de Disaster Recovery

**Regra de ouro da prova:** leia o **RPO** (quanto dado pode ser perdido) e o **RTO** (quanto tempo pode ficar fora) e escolha a estratégia **mais barata que ainda atende aos dois**. Horas de RTO/RPO pedem **Backup & Restore**. Dezenas de minutos pedem **Pilot Light**. Minutos pedem **Warm Standby**. Próximo de zero pede **Multi-Site Active/Active**. Não confunda **alta disponibilidade** (Multi-AZ, dentro da Região) com **DR** (outra Região), nem **replicação** com **backup**.

### Conceitos de DR

- **RPO (Recovery Point Objective):** perda máxima de dados aceitável, medida em tempo desde o último ponto recuperável. É limitado pela frequência de backup ou pelo lag de replicação.
- **RTO (Recovery Time Objective):** tempo máximo até voltar a operar. É limitado pelo tempo de provisionar, restaurar, validar e redirecionar o tráfego.
- **Alta disponibilidade ≠ DR:** Multi-AZ (RDS, ALB, Auto Scaling em várias AZs) protege contra falha de **uma AZ** com failover automático. DR protege contra perda de **uma Região** inteira, ou contra desastres que afetam todas as cópias síncronas.
- **Replicação ≠ backup:** a replicação copia também **exclusões, corrupção e criptografia por ransomware**. Só backups com retenção, **isolamento** (outra conta) e **imutabilidade** (Vault Lock, Object Lock) permitem voltar a um ponto anterior. Uma boa estratégia combina os dois.
- **Isolamento de falhas:** a AZ isola falhas de infraestrutura, a Região isola falhas regionais, e a **conta** isola comprometimento de credenciais. Guardar backups em **outra conta e outra Região** protege contra os três.
- **Estabilidade estática:** o plano de DR não deve depender de ações no **control plane** da Região afetada (criar recursos, mudar configurações durante a crise). Quanto mais estiver pré-provisionado e pronto, mais confiável é a recuperação.

### As quatro estratégias

Da menor para a maior infraestrutura ativa e custo típico. Em geral, o RTO/RPO **diminuem** nessa ordem, mas dependem do desenho e dos testes.

| Estratégia                      | O que roda na Região de DR em operação normal                       | RPO típico         | RTO típico                        | Custo    |
|---------------------------------|---------------------------------------------------------------------|--------------------|-----------------------------------|----------|
| <b>Backup &amp; Restore</b>     | Nada além dos backups                                               | Horas              | Horas (até ~24 h)                 | \$       |
| <b>Pilot Light</b>              | Dados replicados continuamente; computação desligada ou mínima      | Segundos a minutos | Dezenas de minutos a poucas horas | \$\$     |
| <b>Warm Standby</b>             | Ambiente completo e <b>funcional</b> , em escala reduzida           | Segundos a minutos | Minutos                           | \$\$\$   |
| <b>Multi-Site Active/Active</b> | Ambiente completo <b>atendendo tráfego real</b> em todas as Regiões | Próximo de zero    | Próximo de zero                   | \$\$\$\$ |

*Os valores são ordens de grandeza para comparação. O RTO/RPO real depende da automação e de testes medidos.*



## Backup & Restore

- Backups regulares (ex.: AWS Backup com cópia entre Regiões, snapshots de banco, objetos S3 replicados quando apropriado) restaurados sob demanda na Região de DR. Tem o maior RTO/RPO e o menor custo: é a estratégia mais barata do framework de DR da AWS.
- **Na prática:**
  - **AWS Backup** com cópia **cross-Region e cross-account**;
  - cópia de **snapshots EBS e AMIs** (Data Lifecycle Manager);
  - **replicação de backups automáticos do RDS**;
  - S3 CRR para os dados de objeto.
- **Infraestrutura como código** (CloudFormation/CDK) recria rede, computação e configuração na Região de DR. Sem IaC, o RTO cresce muito e a restauração fica sujeita a erros manuais.
- Garanta que as **chaves KMS** (Multi-Region keys ou chaves na Região de destino), AMIs e segredos existam na Região de DR. Faça **restore testing** periodicamente.

## Pilot Light

- Infraestrutura mínima já pronta na Região de DR (ex.: banco replicado continuamente, como o Aurora Global Database), com o restante provisionado só na hora do failover. É um bom equilíbrio de RTO baixo com custo controlado quando a Região de DR não precisa processar carga em operação normal.
- **O núcleo de dados fica sempre ativo:** Aurora Global Database, Read Replica cross-Region do RDS, DynamoDB Global Tables e S3 CRR. **A computação fica desligada ou em zero:** AMIs prontas, Auto Scaling groups com capacidade 0 e templates prontos.
- **No failover:** promover o banco, escalar ou lançar a computação e redirecionar o DNS.
- **Diferença para o Warm Standby:** o Pilot Light **não atende tráfego** sem provisionar a computação primeiro. O Warm Standby já atende, em menor escala.

## Warm Standby

- Versão reduzida, mas funcional, do ambiente de produção sempre ativa na Região de DR. Tem RTO menor que o Pilot Light e custo maior.
- Todas as camadas estão rodando (load balancer, poucas instâncias, banco réplica) e podem ser **testadas continuamente**. No failover, **escale** os Auto Scaling groups para a capacidade de produção, promova o banco e redirecione o tráfego (Route 53 failover/weighted).
- Verifique as **cotas** da Região de DR para a escala total, porque o scale-up depende delas.

## Multi-Site Active/Active

- Ambiente completo rodando simultaneamente em múltiplas Regiões e atendendo tráfego real em todas. Tem o menor RTO/RPO possível e o maior custo. É desnecessário quando o requisito é só failover, sem balanceamento de carga multi-Região.
- **Roteamento:** Route 53 latency/geolocation/weighted com health checks, ou **Global Accelerator**.
- **Dados com escrita em várias Regiões:** **DynamoDB Global Tables**, **Aurora DSQL**, ou Aurora Global Database com *write forwarding*. Isso exige pensar em **resolução de conflitos** e consistência.
- **Variante "hot standby"** (ativo/passivo com capacidade total): ambiente completo pronto, mas sem receber tráfego até o failover.

## Blocos de construção por camada

Aurora Global Database, DynamoDB Global Tables e S3 CRR são os blocos de construção mais comuns para essas estratégias.

| Camada                        | Replicação / recuperação entre Regiões                                                                                                     |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| DNS e tráfego                 | Route 53 failover + health checks, <b>ARC routing controls</b> , <b>Global Accelerator</b> (traffic dials)                                 |
| Computação                    | Cópia de AMIs (Image Builder distribuí entre Regiões), Auto Scaling, <b>replicação do ECR</b> , Lambda implantada nas duas Regiões via IaC |
| Servidores legados            | <b>AWS Elastic Disaster Recovery (DRS)</b>                                                                                                 |
| Relacional                    | <b>Aurora Global Database</b> , Read Replica cross-Region do RDS, replicação de backups automáticos, cópia de snapshots                    |
| NoSQL                         | <b>DynamoDB Global Tables</b> , backups cross-Region via AWS Backup                                                                        |
| Cache                         | <b>ElastiCache Global Datastore</b>                                                                                                        |
| Objetos                       | <b>S3 CRR</b> (+ RTC para SLA de 15 min), Multi-Region Access Points com failover controls                                                 |
| Arquivos                      | <b>EFS Replication</b> , cópia de backups do FSx, SnapMirror (FSx for ONTAP)                                                               |
| Bloco                         | Cópia de snapshots EBS (Data Lifecycle Manager ou AWS Backup)                                                                              |
| Segredos e chaves             | <b>Replicação do Secrets Manager</b> , <b>KMS Multi-Region keys</b>                                                                        |
| Configuração e infraestrutura | CloudFormation <b>StackSets</b> , pipelines de IaC para as duas Regiões                                                                    |

- Detalhes de cada serviço:
  - [Aurora Global Database](#);
  - [Global Tables, cache e streams](#);
  - [Proteção de dados: versionamento, Object Lock e replicação](#);
  - [Segurança e replicação do EFS](#);
  - [Backup](#).

## Failover, failback e testes

- **Checklist de failover:** RPO é a perda máxima de dados aceitável, e RTO é o tempo máximo até voltar a operar.
  - Replicar dados não garante capacidade de atendimento: verifique quotas e capacidade de EC2, IPs, balanceadores, banco e serviços na **Região de contingência** para a escala necessária.
  - Prepare artefatos/IaC, segredos, chaves, DNS, rotas, dependências externas e runbooks.
  - Execute testes de failover e de retorno e meça o RTO/RPO real.
  - Em arquiteturas por camadas, mantenha frontend e workers sem estado sempre que possível, e coloque sessões, filas e dados persistentes em serviços adequados. Assim o Auto Scaling e a substituição de instâncias funcionam sem perda de sessão.
  - [Estratégias de DR da AWS](#).
- **Decisão de failover:**
  - **Automático:** health checks do Route 53, failover gerenciado do Aurora, Global Accelerator. É mais rápido, mas corre o risco de falso positivo e *flapping*.

- **Manual confiável: Route 53 ARC routing controls**, que operam no *data plane* e não dependem da Região com falha. É preferido quando a decisão de mudar de Região precisa de um humano.
  - **DNS**: use **TTL baixo** nos registros de failover. Clientes e resolvedores podem manter cache: considere o Global Accelerator quando isso for inaceitável.
  - **Failback**: planeje a **ressincronização** dos dados de volta à Região primária (inverter a replicação), a janela de retorno e os testes. Muitas vezes é mais arriscado que o failover.
  - **Testes**: *game days* e **AWS Fault Injection Service (FIS)**, que injeta falhas controladas (encerrar instâncias, latência de rede, interrupção de AZ, failover de banco) para validar a resiliência e medir o RTO.
  - **Métricas ligadas ao requisito de negócio**: saúde do target group e percentual de erros medem a disponibilidade da API. `ApproximateAgeOfOldestMessage` e o backlog indicam atraso de processamento. O lag de replicação e a idade do último backup limitam o RPO. O tempo medido do runbook valida o RTO. Alarme não substitui teste de restauração ou de failover.
- 

## Ferramentas de resiliência

- **AWS Resilience Hub**: avalia a resiliência de uma aplicação contra metas de RTO/RPO definidas pelo usuário, identifica pontos de falha na arquitetura atual (com base no AWS Well-Architected Framework) e recomenda melhorias específicas (configurações, alarmes, testes, SOPs de recuperação), com RTO/RPO e custo estimados para cada opção. É a resposta padrão para "medir/validar objetivamente se uma arquitetura atende a metas de RTO/RPO já definidas", diferente de simplesmente escolher entre as quatro estratégias de DR acima.
  - **AWS Elastic Disaster Recovery (DRS)**: replica servidores em nível de bloco, **continuamente**, para uma área de staging de baixo custo e lança instâncias de recuperação quando necessário. O RPO costuma ficar em segundos e o RTO em minutos. É útil para aplicações legadas que não podem ser redesenhadas rapidamente.
    - Funciona a partir de servidores on-premises, de outras nuvens ou de outra Região/AZ da AWS, com **pontos de recuperação no tempo** (útil contra ransomware) e **failback** para a origem.
    - Planeje VPC, quotas, ordem de inicialização e redirecionamento de tráfego. Faça recovery drills e teste o failback.
    - O DRS recupera servidores, mas a disponibilidade da aplicação inteira ainda depende de DNS, banco e serviços externos. [Guia do DRS](#).
  - **Route 53 Application Recovery Controller (ARC)**: *routing controls* e *readiness checks* para failover entre Regiões. Também oferece **zonal shift** e *zonal autoshift*, que tiram uma AZ com problema do tráfego do ALB/NLB.
  - **AWS Fault Injection Service (FIS)**: experimentos de caos gerenciados, com condições de parada por alarme do CloudWatch.
  - **AWS Backup**: cópias cross-Region/cross-account, Vault Lock e restore testing, a base da estratégia Backup & Restore. Veja [Backup](#).
- 

## Padrões e pegadinhas de DR

- "Menor custo, RTO de 24 h aceitável": **Backup & Restore** com AWS Backup cross-Region.
- "RPO de segundos e RTO de minutos para o banco, com menor custo na computação": **Pilot Light** com **Aurora Global Database**.
- "Ambiente reduzido na outra Região, capaz de assumir imediatamente e escalar": **Warm Standby**.
- "Usuários em duas Regiões, sem downtime em caso de perda de uma Região": **Multi-Site Active/Active** com DynamoDB Global Tables e Route 53 latency/Global Accelerator.
- "Multi-AZ basta para DR regional?" **Não**: protege só contra falha de AZ.
- "Réplica cross-Region protege contra exclusão acidental?" **Não**: a exclusão é replicada. Use **backups imutáveis**.

- "Recuperar servidores on-premises legados na AWS com RPO de segundos": **Elastic Disaster Recovery**.
- "Validar se a arquitetura atende RTO/RPO definidos": **Resilience Hub**.
- "Failover regional manual e confiável, sem depender da Região com falha": **Route 53 ARC routing controls**.
- "Failover funcionou, mas a Região de DR não conseguiu escalar": **cotas** não solicitadas antecipadamente (Service Quotas).
- "Backups criptografados copiados, mas não restauram na Região de DR": falta a **chave KMS** na Região de destino.

## Decisão rápida — Disaster Recovery

| Cenário / requisito                                                         | Resposta correta                                     | Distratores clássicos                              |
|-----------------------------------------------------------------------------|------------------------------------------------------|----------------------------------------------------|
| RTO/RPO de horas com o menor custo                                          | Backup & Restore (AWS Backup cross-Region + IaC)     | Warm Standby, Multi-Site                           |
| RPO de segundos e RTO de dezenas de minutos, com computação desligada na DR | Pilot Light (ex.: Aurora Global Database)            | Backup & Restore (RTO alto), Multi-Site (custo)    |
| Ambiente reduzido, já funcional, que assume e escala no failover            | Warm Standby                                         | Pilot Light (precisa provisionar antes de atender) |
| Sem downtime na perda de uma Região, usuários em várias Regiões             | Multi-Site Active/Active                             | Warm Standby, Multi-AZ                             |
| Proteção contra exclusão acidental e ransomware                             | Backups imutáveis em outra conta/Região (Vault Lock) | Réplica cross-Region, Multi-AZ                     |
| Servidores legados com RPO de segundos e RTO de minutos                     | AWS Elastic Disaster Recovery (DRS)                  | Cópia manual de AMIs, Backup & Restore             |
| Validar se a arquitetura atende metas de RTO/RPO                            | AWS Resilience Hub                                   | Well-Architected Tool, Trusted Advisor             |
| Failover regional manual e confiável                                        | Route 53 ARC routing controls                        | Editar registros DNS durante a crise               |

## 11. Padrões recorrentes consolidados (top pitfalls dos 6 simulados)

Use os padrões abaixo como **pistas de decisão**, não como gabarito automático. Confirme sempre o requisito que diferencia as alternativas: protocolo, tolerância a falhas, modelo de acesso, custo ou esforço operacional. Cada padrão aponta para o tópico em que o assunto é detalhado, e a tabela **Decisão rápida** ao fim de cada capítulo resume os cenários de cada tema.

### Palavras-chave do enunciado

| Se o enunciado diz...                                   | Normalmente favorece...                                                                    | Cuidado com...                                                            |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| "Menor esforço operacional", "sem gerenciar servidores" | Serviços gerenciados/serverless (Lambda, Fargate, DynamoDB, Aurora Serverless, S3)         | Soluções em EC2 com scripts próprios                                      |
| "Mais econômico", "menor custo"                         | Spot, Savings Plans, lifecycle, serverless para uso intermitente, gateway endpoints        | Opções mais baratas que <b>não</b> atendem a outro requisito do enunciado |
| "Sem alterar o código", "rapidamente", "lift-and-shift" | Rehost (MGN), serviços compatíveis (Amazon MQ, DocumentDB, Keyspaces, RDS do mesmo engine) | Refatorar para serverless                                                 |
| "Em tempo real", "milissegundos"                        | Kinesis Data Streams, DynamoDB, ElastiCache, WebSocket/AppSync                             | Firehose (quase tempo real), Athena, batch                                |
| "Desacoplar", "absorver picos"                          | SQS entre as camadas, SNS/EventBridge para fan-out                                         | Chamadas síncronas diretas                                                |
| "Na ordem", "sem duplicatas"                            | SQS FIFO, Kinesis por partition key, SNS FIFO                                              | SQS Standard                                                              |
| "Garantir que nunca...", "impedir"                      | Controles <b>preventivos</b> : SCP, IAM, Block Public Access, Object Lock, KMS key policy  | Controles detectivos (Config, GuardDuty, Trusted Advisor)                 |
| "Detectar", "auditar", "alertar"                        | CloudTrail, Config, GuardDuty, Macie, Security Hub, alarmes do CloudWatch                  | Controles que bloqueiam, mas não registram                                |
| "Alta disponibilidade"                                  | Multi-AZ, Auto Scaling em várias AZs, ALB                                                  | Multi-Região quando só se pede HA (custo e complexidade)                  |
| "Perda de uma Região", "DR"                             | Estratégias de DR com replicação cross-Region                                              | Multi-AZ sozinho                                                          |
| "Credenciais", "sem chaves"                             | IAM roles, credenciais temporárias, Secrets Manager                                        | Access keys em código, user data ou variáveis                             |
| "Tráfego não pode passar pela internet"                 | VPC endpoints, PrivateLink, Direct Connect                                                 | NAT Gateway, Internet Gateway                                             |

### Padrões de arquitetura e esforço operacional

- Menor esforço operacional:** quando a carga é compatível, compare serviços serverless ou gerenciados com a operação de EC2/EBS/EFS e Auto Scaling. Verifique duração, estado, controles exigidos e custo antes de escolher. → [Serverless](#), [Contêineres](#)
- Estado fora da instância:** sessões em ElastiCache/DynamoDB, arquivos em S3/EFS e dados em bancos gerenciados. Instâncias descartáveis permitem Auto Scaling, substituição e DR sem perda de sessão. → [Fundamentos do Auto Scaling group](#)
- Limites que descartam uma opção:** Lambda tem timeout de 15 minutos e o API Gateway tem timeout de integração de 29 s. Tarefas mais longas vão para Batch, Fargate ou Step Functions, com resposta assíncrona. → [Números do Lambda](#), [Amazon API Gateway](#)

4. **Serviço pré-treinado antes de modelo próprio:** Textract, Comprehend, Rekognition, Transcribe e Bedrock resolvem a maioria dos casos de IA sem SageMaker. Treinar modelo próprio para uma tarefa comum é esforço desnecessário. → [Serviços de IA pré-treinados](#)
- 

## Padrões de rede e conectividade

5. **Acesso privado a S3/DynamoDB a partir da VPC:** o gateway endpoint costuma ser a opção simples quando o tráfego não deve usar a internet. O interface endpoint também existe para esses serviços e atende requisitos diferentes, como endereços IP privados e alguns cenários híbridos. NAT Gateway não substitui um VPC endpoint para o requisito de acesso privado ao serviço. → [VPC endpoints e PrivateLink](#)
6. **Peering não é transitivo:** A-B e B-C não conectam A a C, e uma VPC não usa o IGW/NAT/VPN da outra. Para muitas VPCs, use **Transit Gateway**. → [Interconexão de redes](#)
7. **Direct Connect não é criptografado por padrão e leva semanas para ativar:** use MACsec ou VPN sobre DX para criptografia, e VPN Site-to-Site para conectividade imediata ou como backup. → [Conectividade híbrida: VPN e Direct Connect](#)
8. **Global Accelerator (rede/IP estático/UDP) ≠ CloudFront (cache de conteúdo HTTP):** nunca confunda o propósito dos dois ao otimizar performance multi-Região. → [CloudFront vs. Global Accelerator vs. Route 53](#)
9. **Security group não nega:** para bloquear um IP específico, use NACL (ou WAF para HTTP). NACL é stateless e precisa liberar as portas efêmeras de retorno. → [Security Groups e Network ACLs](#)
- 

## Padrões de bancos de dados

10. **RDS Multi-AZ DB instance, Multi-AZ DB cluster, Read Replica, RDS Proxy e backups/PITR** respondem a requisitos diferentes:
- a standby do modelo DB instance oferece failover e não atende leitura;
  - o DB cluster também tem duas leitoras;
  - a Read Replica amplia a leitura;
  - o RDS Proxy agrupa conexões;
  - o PITR restaura dados a um ponto anterior. Identifique o modelo Multi-AZ descrito antes de eliminar a opção de leitura. → [Alta disponibilidade e escala de leitura no RDS](#)
11. **Restauração cria um novo recurso:** PITR e snapshots do RDS e do DynamoDB criam uma nova instância/tabela, com **novo endpoint**. A aplicação precisa ser apontada para ela. → [Backups e restauração do RDS](#)
12. **RDS emite eventos de gerenciamento, não de dados:** Event Notifications cobrem failover, backup, mudança de parâmetro etc. Qualquer arquitetura que precise reagir a mudanças em uma **tabela/linha** precisa de um intermediário (trigger + polling, ou DMS/CDC), nunca de uma integração direta dado → EventBridge/SNS/Lambda. → [Operação, desempenho e custo do RDS](#)
13. **DynamoDB: partição quente e índices:** throttling com capacidade sobrando indica partition key ruim. LSI só pode ser criado junto com a tabela. Throttling de um GSI também trava as escritas da tabela base. → [Chaves e índices](#)
- 

## Padrões de identidade, segredos e criptografia

14. **Credenciais de workloads AWS:** prefira credenciais temporárias por IAM roles específicas (instance profile, task role, execution role) em vez de segredos estáticos no código. Uma **política baseada em recurso** é outro mecanismo de autorização, distinto de uma role, e pode ser necessária para permitir a invocação ou o acesso ao



destino. → [Fundamentos do IAM](#)

- 15. **Deny explícito sempre vence, e SCP não concede:** a permissão efetiva é a interseção de SCP, RCP, permissions boundary e identity policy. Entre contas, os dois lados precisam permitir. → [Avaliação de políticas](#)
- 16. **Rotação automática de credenciais:** o Secrets Manager oferece agendamento e mecanismos de rotação para segredos compatíveis. Verifique se o segredo usa rotação gerenciada ou uma função Lambda. O Parameter Store sozinho não fornece a mesma rotação automática de credenciais de banco. → [Segredos](#)
- 17. **Chaves KMS são regionais e a AWS managed key não é compartilhável:** copiar backups/AMIs criptografados para outra conta ou Região exige uma customer managed key com permissão e uma chave na Região de destino (ou Multi-Region key). → [AWS KMS](#)
- 18. **Certificados importados no ACM (de CA externa) nunca têm renovação automática:** só certificados emitidos pelo próprio ACM têm esse benefício. Para o CloudFront, o certificado precisa estar em **us-east-1**. → [AWS Certificate Manager \(ACM\)](#)
- 19. **Criptografia em repouso só na criação:** EBS, RDS e EFS não criptografam "no lugar". Crie um snapshot, copie-o com criptografia e restaure. → [Criptografia no S3 e em outros serviços](#)

---

## Padrões de proteção de dados e backup

- 20. **Imutabilidade de versões no S3 durante retenção regulatória:** o S3 Object Lock em modo compliance atende ao requisito de impedir a exclusão até por administradores durante o período configurado. Versionamento, ACL e lifecycle isoladamente não impõem essa retenção. Para backups, o equivalente é o **Backup Vault Lock**. → [Proteção de dados: versionamento, Object Lock e replicação](#)
- 21. **Replicação ≠ backup:** réplicas (CRR, Read Replicas, Global Tables) propagam exclusões e corrupção. Proteção contra erro humano e ransomware exige backups com retenção, em **outra conta**, e imutáveis. → [Conceitos de DR](#)
- 22. **Signed/presigned URLs para acesso controlado e temporário:** padrão recorrente tanto para upload direto do usuário ao S3 (contornando o servidor de aplicação) quanto para download controlado de conteúdo sensível via CloudFront. → [Segurança e criptografia do S3](#)
- 23. **Compartilhamento de arquivos com SMB, permissões NTFS e Active Directory** aponta para o FSx for Windows File Server. EFS usa NFS e S3 é armazenamento de objetos. Compare também requisitos de acesso híbrido e cache local antes de escolher a arquitetura completa. → [Amazon FSx](#)
- 24. **Instance Store é efêmero:** perde os dados em stop, hibernate e terminate. Dados que precisam sobreviver ficam no EBS, EFS ou S3. → [Instance Store](#)

---

## Padrões de segurança de borda

- 25. **WAF (camada de aplicação) ≠ Shield (DDoS de rede):** nunca um substitui o outro. Combinam-se em defesa por camadas junto com CloudFront/Route 53. O WAF não se associa ao NLB. → [Proteção de rede e aplicação](#)
- 26. **Preventivo vence detetativo quando o enunciado diz "garantir":** Block Public Access + SCP impedem buckets públicos. GuardDuty, Config e Trusted Advisor apenas detectam ou corrigem depois. → [SCPs e RCPs](#)

---

## Padrões de integração e eventos

- 27. **SQS como buffer de processamento assíncrono:** útil para absorver picos e desacoplar produtores de consumidores. Escolha Standard ou FIFO conforme a ordem e a deduplicação exigidas. Configure DLQ e idempotência quando houver efeitos que não podem ser repetidos. O visibility timeout precisa cobrir o tempo de processamento. → [Amazon SQS](#)

- 28. **Fan-out com persistência:** SNS → várias filas SQS, uma por consumidor. O SNS sozinho não guarda mensagens para reentrega. → [Amazon SNS](#)
  - 29. **Firehose entrega, não armazena:** para replay e vários consumidores, coloque o Kinesis Data Streams antes. O Firehose é quase tempo real (buffer). → [Amazon Data Firehose](#)
- 

## Padrões de escala e custo

- 30. **Compromisso para base previsível + Spot para trabalho interrompível** é uma combinação de custo frequente em simulados. Confira o compromisso de gasto, o risco de interrupção e a capacidade de retomada antes de aplicá-la. → [EC2 — modelos de compra](#)
  - 31. **Savings Plans e RIs regionais não reservam capacidade:** para garantir capacidade em uma AZ, use On-Demand Capacity Reservation (combinável com o desconto). → [Reservas de capacidade](#)
  - 32. **Scheduled → Dynamic → Predictive scaling:** a escolha depende do formato do padrão de carga. Fixo e 100% conhecido pede scheduled. Reativo a métricas em tempo real pede dynamic. Previsível **com variação** (não perfeitamente fixo) é o caso de uso real de predictive, não "sempre a opção mais complexa e desnecessária". → [Políticas de scaling](#)
  - 33. **ELB health check precisa ser habilitado no Auto Scaling group:** com o check padrão (EC2), instâncias com a aplicação travada continuam no grupo. → [Health checks e substituição](#)
  - 34. **Transferência de dados entra na conta:** tráfego entre AZs, NAT por GB e saída para a internet costumam explicar custos inesperados. Gateway endpoints, NAT por AZ e CloudFront reduzem esse custo. → [Custos de transferência de dados](#)
- 

## Padrões de resiliência e DR

- 35. **Multi-AZ na mesma Região** atende cenários que exigem tolerância a falha de AZ. Quando o requisito inclui perda de uma Região, residência geográfica de dados ou continuidade regional, avalie uma arquitetura multi-Região junto com RTO, RPO e custo. → [As quatro estratégias](#)
- 36. **Escolha a estratégia de DR mais barata que atende RTO e RPO:** não use Active/Active quando Pilot Light ou Warm Standby bastam. Verifique cotas, chaves e AMIs na Região de DR. → [Failover, fallback e testes](#)

## 12. Mapa de Domínios do Exame

Cada domínio oficial tem um peso diferente na nota final (30/26/24/20%). Esta seção cruza os tópicos das seções 1-9 com o domínio que eles mais provavelmente testam, para priorizar a revisão pelo que realmente pesa na prova. Muitos tópicos aparecem em mais de um domínio — o "domínio principal" indicado é o ângulo mais comum nos simulados.

### Domínio 1 — Design Secure Architectures (30%)

| Tópico                                                                                              | Onde revisar |
|-----------------------------------------------------------------------------------------------------|--------------|
| IAM (roles, políticas, condições, menor privilégio)                                                 | Seção 5      |
| IAM permissions boundaries vs SCPs                                                                  | Seção 5      |
| SCPs, AWS Organizations, Control Tower                                                              | Seções 5, 9  |
| AWS Service Catalog (self-service governado)                                                        | Seção 9      |
| KMS, CloudHSM, ACM, criptografia em repouso/trânsito                                                | Seção 5      |
| Secrets Manager vs Parameter Store                                                                  | Seção 5      |
| WAF, Shield, Firewall Manager                                                                       | Seção 5      |
| CloudTrail, Config, GuardDuty, Detective, Macie, Inspector, Security Hub, Audit Manager             | Seção 5      |
| Amazon Security Lake (dados de segurança centralizados em OCSF/S3)                                  | Seção 5      |
| Cognito, IAM Identity Center, federação SAML, Directory Service (Managed AD/AD Connector/Simple AD) | Seção 5      |
| Lake Formation (segurança de dados em escala)                                                       | Seção 5      |
| S3 Object Lock, Block Public Access, OAC/OAI                                                        | Seção 2      |
| Security Groups vs NACLs                                                                            | Seção 4      |
| Bastion Host vs Session Manager vs AWS Systems Manager (Patch Manager, Automation, Run Command)     | Seções 4, 9  |
| VPC Endpoints (Gateway vs Interface/PrivateLink)                                                    | Seção 4      |
| AWS Network Firewall (filtragem stateful de VPC)                                                    | Seção 5      |
| AWS RAM (compartilhamento de recursos entre contas)                                                 | Seção 5      |
| Compartilhamento seguro de AMI/snapshot entre contas                                                | Seção 5      |

### Domínio 2 — Design Resilient Architectures (26%)

| Tópico                                                                         | Onde revisar    |
|--------------------------------------------------------------------------------|-----------------|
| RDS/Aurora Multi-AZ, Read Replica, Global Database                             | Seção 3         |
| DynamoDB Global Tables, PITR, Streams                                          | Seção 3         |
| ElastiCache Redis Multi-AZ vs Memcached                                        | Seção 3         |
| Estratégias de DR (Backup & Restore → Pilot Light → Warm Standby → Multi-Site) | Seção 10        |
| S3 CRR, EFS Replication, Aurora Global Database (blocos de DR)                 | Seções 2, 3, 10 |
| SQS (Standard/FIFO, DLQ, visibility timeout)                                   | Seção 6         |

| Tópico                                                                     | Onde revisar |
|----------------------------------------------------------------------------|--------------|
| SNS fan-out, filtragem, fallback                                           | Seção 6      |
| ECS/EKS multi-AZ, ECS Anywhere, EKS Anywhere/EKS Distro                    | Seção 1      |
| VMware Cloud on AWS (migração de VMware sem re-arquitetar)                 | Seção 1      |
| Topologia NAT por AZ, Transit Gateway, Direct Connect + VPN backup         | Seção 4      |
| Auto Scaling (scheduled/dynamic/predictive) para resiliência de capacidade | Seção 1      |
| Route 53 health checks e failover routing                                  | Seção 4      |
| AWS Backup, snapshots, versionamento                                       | Seção 2      |
| DMS com CDC (migração com mínima interrupção)                              | Seção 7      |
| DMS Serverless; Application Migration Service (MGN), teste e cutover       | Seção 7      |
| AWS CloudFormation (IaC — infraestrutura imutável e repetível)             | Seção 9      |

## Domínio 3 — Design High-Performing Architectures (24%)

| Tópico                                                                      | Onde revisar                      |
|-----------------------------------------------------------------------------|-----------------------------------|
| EC2 (tipos, placement groups, EBS-optimized)                                | Seção 1                           |
| EC2 user data, metadata, hibernate, Instance Store e EFA                    | Seção 1                           |
| EBS gp3/io2/io2 Block Express, Multi-Attach                                 | Seção 2                           |
| EFS Provisioned Throughput, FSx for Lustre                                  | Seção 2 (e guia de Armazenamento) |
| DynamoDB DAX, capacidade sob demanda                                        | Seção 3                           |
| Escolha de tipo de banco (relacional, grafo Neptune, wide-column Keyspaces) | Seção 3                           |
| ElastiCache como cache de aplicação                                         | Seção 3                           |
| CloudFront (cache, multi-origem), Global Accelerator                        | Seção 4                           |
| ALB vs NLB vs GWLB (escolha por camada/protocolo)                           | Seção 4                           |
| Kinesis (Data Streams, Firehose, Video Streams) e processamento com Flink   | Seção 6                           |
| Kinesis on-demand e enhanced fan-out                                        | Seção 6                           |
| Transcodificação de mídia e estado legado do Elastic Transcoder             | Seção 6                           |
| AppFlow (ingestão gerenciada de dados SaaS para S3/Redshift)                | Seção 6                           |
| Lambda (concorrência, memória, timeout, /tmp)                               | Seção 1                           |
| Glue, Athena, EMR, Redshift (performance de analytics)                      | Seção 7                           |
| SageMaker vs serviços de IA pré-treinados (custo-performance)               | Seção 8                           |
| Amazon Lex, Amazon Polly (chatbot conversacional e text-to-speech)          | Seção 8                           |
| X-Ray, CloudWatch (métricas para tuning)                                    | Seção 9                           |

## Domínio 4 — Design Cost-Optimized Architectures (20%)

| Tópico                                                           | Onde revisar |
|------------------------------------------------------------------|--------------|
| Modelos de compra EC2 (On-Demand, Reserved, Spot, Savings Plans) | Seção 1      |

| Tópico                                                                | Onde revisar |
|-----------------------------------------------------------------------|--------------|
| Hosts Dedicados vs Instâncias Dedicadas (custo de licenciamento)      | Seção 1      |
| S3 Lifecycle Policies, classes de armazenamento, Intelligent-Tiering  | Seção 2      |
| S3 Storage Class Analysis, Transfer Acceleration e EFS IA             | Seção 2      |
| Parar instância RDS, Storage Autoscaling                              | Seção 3      |
| DynamoDB Standard-IA, capacidade provisionada vs sob demanda          | Seção 3      |
| DataSync vs Transfer Family vs transferência física (Snowball legado) | Seção 2      |
| Cost Explorer, Budgets, CUR + Athena + Amazon Quick                   | Seção 9      |
| Budgets Actions e budgets de utilization/coverage de Savings Plans    | Seção 9      |
| Compute Savings Plan vs EC2 Instance Savings Plan                     | Seções 1, 9  |
| Transit Gateway compartilhado vs múltiplas conexões dedicadas         | Seção 4      |
| Athena/Glue com formatos colunares (reduz custo por consulta)         | Seção 7      |
| AWS Serverless Application Repository (reuso de padrões serverless)   | Seção 1      |
| Arquitetura serverless (S3→Lambda→IA) vs EC2/EMR dedicado             | Seção 8      |

## Observação sobre o modelo de pontuação

O SAA-C03 usa pontuação **compensatória**: não é preciso atingir a nota mínima em cada domínio separadamente, só na prova como um todo. Ainda assim, como Segurança (30%) e Resiliência (26%) somam 56% da nota, dominar as seções 3, 4, 5 e 10 deste guia tem o maior retorno por hora de estudo.

## 13. Autoteste — Flashcards de Revisão Rápida

Cada card abaixo é um callout colapsável: clique para expandir e revelar a resposta só depois de tentar responder mentalmente — o objetivo é forçar recall ativo, não releitura passiva. Bom para uma rodada de revisão espaçada (ex.: 10-15 cards por dia) nos dias antes da prova, cobrindo os padrões da Seção 11 e as tabelas de decisão rápida dos capítulos 1 a 10.

### Computação

**Uma carga de trabalho tem componente de custo base previsível (roda o tempo todo) e picos de processamento em lote tolerantes a interrupção. Qual combinação de modelos de compra EC2 minimiza o custo recorrente?**

#### RESPOSTA

Reserved Instances/Savings Plans para a carga base previsível + Spot Instances para os picos tolerantes a interrupção — é o mix de menor custo mais repetido nos simulados. Nunca usar Spot para produção crítica/stateful.

**Uma equipe precisa rodar software com licenciamento BYOL que exige visibilidade total de sockets e núcleos físicos do servidor. Hosts Dedicados ou Instâncias Dedicadas?**

#### RESPOSTA

Hosts Dedicados — dão controle granular sobre o hardware físico subjacente, necessário para esse tipo de licenciamento. Instâncias Dedicadas apenas isolam fisicamente a instância, sem esse nível de visibilidade.

**Uma carga tem um padrão de demanda previsível ao longo do tempo, mas com variação mês a mês (não é fixa). Scheduled Scaling é a melhor resposta?**

#### RESPOSTA

Não necessariamente — Scheduled Scaling serve para padrões 100% fixos e conhecidos. Quando há um componente previsível **com variação**, Predictive Scaling (que usa ML para prever a demanda) é a resposta mais adequada.

**Uma fila de jobs de processamento em lote tem execuções que passam de 15 minutos e volumes variáveis de vCPU/memória. Por que AWS Batch é melhor que Lambda ou EMR aqui?**

#### RESPOSTA

Lambda tem limite de 15 minutos de execução, o que descarta jobs mais longos. EMR é voltado a processamento distribuído de big data (Hadoop/Spark), não a filas de jobs em lote independentes. AWS Batch provisiona automaticamente a capacidade ideal (EC2 On-Demand/Spot ou Fargate) conforme os requisitos dos jobs, sem gerenciamento manual de cluster.



Qual é a diferença entre EC2 user data, instance metadata e hibernate?

**RESPOSTA**

User data executa configuração/scripts — por padrão como root apenas no primeiro boot. Instance metadata apenas expõe dados sobre a instância. Hibernate salva RAM no EBS e retoma processos, evitando refazer um bootstrap demorado.

Um cluster HPC precisa de comunicação de latência mínima entre processos em EC2. ENA ou EFA?

**RESPOSTA**

EFA — inclui funcionalidades de ENA e adiciona OS-bypass para comunicação direta entre user space e hardware, otimizada para HPC/ML fortemente acoplados.

## Armazenamento

Um bucket S3 recebe dados cujo padrão de acesso é completamente imprevisível — às vezes acessados o tempo todo, às vezes ficam meses sem acesso. Qual classe de armazenamento resolve isso automaticamente?

**RESPOSTA**

S3 Intelligent-Tiering — move os objetos automaticamente entre camadas (incluindo arquivamento) conforme o padrão de acesso observado, sem precisar de uma lifecycle policy manual. Se o padrão fosse previsível ao longo do tempo, uma lifecycle policy direta seria mais barata.

Um requisito regulatório exige que objetos no S3 sejam imutáveis mesmo contra tentativas de exclusão pela conta raiz durante o período de retenção. Governance Mode do Object Lock atende isso?

**RESPOSTA**

Não. Governance Mode pode ser sobreposto por um usuário com permissão especial. Só o Compliance Mode do S3 Object Lock garante imutabilidade absoluta, nem a conta raiz consegue burlar durante a retenção. Ambos exigem versionamento habilitado.

Uma aplicação global precisa de um único endpoint S3 que roteie cada requisição para um bucket ativo próximo e permita desviar tráfego de uma região interrompida. Qual recurso e quais configurações adicionais são necessários?

**RESPOSTA**

Usar **S3 Multi-Region Access Points (MRAP)** para o endpoint global e roteamento, configurar **CRR** para disponibilizar os objetos nos buckets necessários e usar os **controles de failover** para alterar o status ativo/passivo em uma interrupção. MRAP não replica objetos por si só nem deve ser descrito como failover ativo/passivo automático. [Roteamento](#) · [Replicação](#).

**A equipe de compliance exige que os backups gerenciados pelo AWS Backup não possam ser excluídos ou alterados por ninguém, nem mesmo administradores, durante o período de retenção. Qual recurso resolve, e qual o equivalente dele no S3?**

**RESPOSTA**

AWS Backup Vault Lock em modo compliance — aplica proteção WORM ao cofre de backup. É o equivalente funcional do S3 Object Lock Compliance Mode, só que aplicado ao AWS Backup.

**Usuários globais fazem download e upload em um bucket S3 central. Qual combinação reduz a latência nos dois sentidos?**

**RESPOSTA**

CloudFront para downloads cacheáveis e S3 Transfer Acceleration para uploads pela edge/backbone AWS. Global Accelerator não aceita S3 como endpoint direto.

**É preciso ler somente os primeiros 250 bytes de milhares de objetos S3. S3 Select ou Byte-range fetch?**

**RESPOSTA**

Byte-range fetch com o header HTTP Range em GetObject. S3 Select filtra conteúdo estruturado por SQL; ScanRange divide a área escaneada pelo Select, mas não é um Range GET arbitrário. S3 Select não está disponível para novos clientes. [Documentação](#).

**Um file system POSIX é acessado apenas algumas vezes por ano, mas deve continuar online. S3 Standard-IA ou EFS IA?**

**RESPOSTA**

EFS Infrequent Access, porque preserva a interface NFS/POSIX. S3 é object storage e não substitui um file system montável.

## Banco de Dados

**Uma aplicação Lambda perde conexões com o RDS mesmo com CPU e memória do banco em níveis baixos. Qual é a causa mais provável e a solução?**

**RESPOSTA**

Esgotamento do pool de conexões do banco (não falta de capacidade computacional). Solução: RDS Proxy, que reutiliza/agrupa conexões entre a aplicação e o banco.

**Uma alteração indevida foi feita em uma tabela RDS há 10 minutos e é preciso restaurar o banco exatamente para o estado anterior a essa alteração. Qual mecanismo?**

**RESPOSTA**

Backups automáticos com Point-in-Time Recovery (PITR) — restaura o banco a qualquer ponto dentro do período de retenção configurado (não confundir com o PITR de 35 dias do DynamoDB, que é um recurso separado).

**A equipe quer atualizar a versão de engine de um banco de produção com o menor downtime e risco possíveis, testando as mudanças antes de promovê-las. Qual recurso do RDS?**

**RESPOSTA**

RDS Blue/Green Deployments — cria um ambiente "green" totalmente gerenciado, espelhado via replicação lógica, permitindo validar upgrades de engine/schema antes do switchover (controlado pelo usuário, tipicamente menos de 1 minuto de downtime). Mais seguro que atualizar a instância in-place.

**Uma arquitetura precisa reagir automaticamente sempre que uma linha específica é atualizada em uma tabela RDS. RDS Event Notifications resolve isso diretamente enviando para SNS/EventBridge?**

**RESPOSTA**

Não. RDS Event Notifications cobre apenas eventos de gerenciamento/infraestrutura (failover, criação/exclusão de instância, backup concluído, mudança de parâmetro). Para reagir a mudanças de **dados**, é preciso um intermediário: trigger no banco + polling, ou AWS DMS com CDC.

**Um cluster Aurora tem carga de leitura imprevisível e precisa escalar réplicas automaticamente mantendo alta disponibilidade multi-AZ. Qual recurso?**

**RESPOSTA**

Aurora Auto Scaling com Aurora Replicas — escala automaticamente as réplicas de leitura conforme a demanda, mantendo HA multi-AZ.

## Rede

**Uma aplicação em uma sub-rede privada precisa acessar o Amazon Comprehend (não S3, não DynamoDB) sem passar pela internet. Que tipo de VPC Endpoint é necessário?**

**RESPOSTA**

VPC Interface Endpoint (via AWS PrivateLink). S3 e DynamoDB também oferecem gateway endpoints gratuitos para acesso dentro da VPC, mas atualmente suportam interface endpoints quando private IP, acesso on-premises/cross-Region ou security groups são necessários.

**Qual VIF do Direct Connect usar para VPC por private IP, S3 por public endpoint e Transit Gateway?**

**RESPOSTA**

Private VIF para uma VPC/private IP; Public VIF para serviços AWS públicos como S3; Transit VIF para Transit Gateways associados a um Direct Connect Gateway.

**Por que liberar somente inbound numa Network ACL pode interromper uma conexão que funciona no Security Group?**

**RESPOSTA**

NACL é stateless e também precisa permitir o retorno pelas portas efêmeras. Security Group é stateful e libera automaticamente o tráfego de resposta de uma conexão permitida.

**Quais atributos da VPC devem estar ativos para uma Route 53 private hosted zone?**

**RESPOSTA**

`enableDnsSupport` e `enableDnsHostnames` devem estar definidos como `true`.

**Uma empresa precisa conectar centenas de VPCs espalhadas em várias contas e redes on-premises, de forma centralizada e escalável, incluindo rotas inter-regionais. VPC Peering resolve?**

**RESPOSTA**

Não escala — VPC Peering não é transitivo e cresce em complexidade combinatória com muitas VPCs. A resposta é AWS Transit Gateway, um hub central de rotas.

**Um requisito de segurança exige inspecionar todo o tráfego de rede com um appliance de firewall de terceiros antes que ele chegue aos servidores da aplicação. Qual tipo de load balancer usar?**

**RESPOSTA**

Gateway Load Balancer (GWLB) — combina balanceamento de carga com encapsulamento GENEVE, especializado em integrar appliances de segurança de terceiros no caminho do tráfego.

**A empresa quer eliminar o compartilhamento de chaves SSH entre a equipe, mantendo acesso administrativo seguro, auditável e escalável a instâncias EC2 totalmente privadas (sem bastion host). Qual serviço?**

**RESPOSTA**

AWS Systems Manager Session Manager, combinado com VPC Endpoints — elimina a necessidade de chaves SSH compartilhadas e de bastion hosts.

## Segurança

**Uma função Lambda precisa ler objetos de um bucket S3. Qual é a forma correta de conceder esse acesso?**

**RESPOSTA**

Uma IAM Role (execution role) anexada à função Lambda — nunca credenciais estáticas, usuário IAM, ou permissões amplas com `*`. É o mecanismo de menor privilégio recomendado sempre que um serviço AWS precisa acessar outro.

**Como permitir que um desenvolvedor gerencie IAM roles sem conseguir conceder a si mesmo `AdministratorAccess`?**

**RESPOSTA**

Aplicar uma permissions boundary ao user/role. A boundary limita o máximo concedível pelas identity policies, não concede permissões e não pode ser anexada a um IAM group.

### Quando client-side encryption vence SSE-KMS num cenário S3?

#### RESPOSTA

Quando o objeto deve ser criptografado antes de chegar à AWS ou precisa usar um algoritmo proprietário. SSE-KMS executa server-side encryption no S3 e oferece controle/auditoria da KMS key, mas não substitui esse requisito.

### Existe um requisito de rotação automática de credenciais de um banco RDS. AWS Secrets Manager ou Systems Manager Parameter Store?

#### RESPOSTA

AWS Secrets Manager — tem rotação automática nativa para credenciais de banco (RDS/Aurora). O Parameter Store armazena parâmetros criptografados via KMS, mas não tem rotação automática nativa; exigiria uma Lambda customizada.

### Uma aplicação atrás de um Network Load Balancer está sofrendo ataques de SQL Injection. O AWS WAF pode ser anexado diretamente ao NLB para mitigar isso?

#### RESPOSTA

Não. O AWS WAF opera na camada 7 e integra apenas com ALB, API Gateway e CloudFront — não com NLB (camada 4). Para proteger uma carga atrás de NLB seria necessário redesenhar a arquitetura (ex.: introduzir um ALB ou usar outra camada de proteção).

### A empresa sofre um ataque DDoS volumétrico de grande escala e precisa de mitigação automática com suporte especializado 24x7. O AWS Shield Standard (ativado por padrão) é suficiente?

#### RESPOSTA

Não. Shield Standard é básico/gratuito e não oferece mitigação automática avançada nem suporte do DRT. É preciso o AWS Shield Advanced, que adiciona mitigação automática de DDoS volumétrico, monitoramento contínuo e suporte 24x7.

### Uma organização quer garantir, de forma preventiva e à prova de alteração por qualquer usuário, que nenhum bucket S3 em nenhuma conta jamais fique público. GuardDuty e Trusted Advisor resolvem isso?

#### RESPOSTA

Não totalmente — GuardDuty e Trusted Advisor são mecanismos de detecção/remediação, portanto reativos. A resposta preventiva é combinar S3 Block Public Access (nível de conta) com uma SCP no AWS Organizations, aplicada centralizadamente a todas as contas-filhas.

## Integração e Mensageria

Um sistema de pedidos precisa preservar a ordem das mensagens de cada pedido e evitar efeitos duplicados no pagamento. SQS Standard ou FIFO? Basta escolher a fila?

### RESPOSTA

Escolher **SQS FIFO** e usar o ID do pedido como `MessageGroupId` para preservar a ordem por pedido. A deduplicação de envio vale para uma janela de cinco minutos. **Não basta escolher a fila:** o consumidor deve ser idempotente para que uma tentativa repetida não cobre duas vezes. SQS Standard não garante ordem. [Documentação](#).

Uma API precisa confirmar recebimento imediato ao usuário, mas a tarefa em si (ex.: gerar miniaturas de imagem) é pesada e deve rodar de forma assíncrona em segundo plano. Qual padrão de arquitetura?

### RESPOSTA

SQS como buffer entre a camada de API e um worker consumidor assíncrono — a API responde rápido ao enfileirar a mensagem, e o processamento pesado acontece depois, desacoplado.

Um evento único (ex.: uma chamada de API capturada via CloudTrail) precisa ser roteado para vários destinos diferentes ao mesmo tempo, de forma mais flexível do que as notificações de evento nativas do S3 permitem. Qual serviço?

### RESPOSTA

Amazon EventBridge — barramento de eventos serverless que permite rotear um único evento para múltiplos destinos simultaneamente, com mais flexibilidade que S3 Event Notifications.

Um tópico único precisa rotear mensagens para múltiplas filas SQS diferentes, cada uma recebendo apenas um subconjunto dos tipos de mensagem, sem criar múltiplos pipelines separados. Qual recurso?

### RESPOSTA

Amazon SNS com filtragem de mensagens (message filtering), fazendo fan-out para várias filas SQS conforme atributos da mensagem.

Delay queue, visibility timeout e DLQ resolvem o mesmo problema?

### RESPOSTA

Não. Delay queue adia a primeira entrega de mensagens novas; visibility timeout esconde uma mensagem já recebida durante o processamento; DLQ isola mensagens que falharam repetidamente.

Um stream Kinesis recebe picos imprevisíveis e vários consumers competem pela leitura. Quais recursos usar?

### RESPOSTA

Kinesis Data Streams em on-demand capacity mode para ajustar ingestão sem shards manuais e enhanced fan-out para throughput de leitura dedicado por consumer.



## Migração e Analytics

É preciso migrar um banco relacional on-premises para a AWS capturando as alterações incrementais em quase tempo real (CDC), mantendo o banco de origem disponível durante toda a migração. Qual serviço?

### RESPOSTA

AWS DMS (Database Migration Service) com CDC (Change Data Capture).

O volume de full load + CDC oscila e a equipe não quer dimensionar uma replication instance. Qual modalidade do DMS?

### RESPOSTA

AWS DMS Serverless, que provisiona e escala automaticamente a capacidade da replication configuration conforme o workload.

Qual é a sequência segura de lift-and-shift com AWS Application Migration Service?

### RESPOSTA

Instalar o AWS Replication Agent, concluir initial sync, lançar e validar test instances, confirmar que a replicação está atualizada e então lançar a cutover instance.

Uma empresa nova precisa transferir muitos dados, mas a rede local não cumpre o prazo. Como escolher a alternativa em 2026?

### RESPOSTA

Estime primeiro volume, banda efetiva, prazo e custo do DataSync/Direct Connect. Se a rede continuar insuficiente, avalie **AWS Data Transfer Terminal** ou parceiro para transferência física. Snowball Edge só atende clientes existentes e encerra suporte nas regiões comerciais em dezembro de 2026; respostas antigas que o tratam como opção geral estão desatualizadas. [Aviso da AWS](#).

Uma equipe de analistas precisa rodar consultas SQL ad hoc, sob demanda, direto sobre arquivos armazenados no S3, sem provisionar nenhuma infraestrutura. Qual serviço, e como reduzir o custo por consulta?

### RESPOSTA

Amazon Athena — serverless, paga por consulta. Particionar os dados e usar formato colunar comprimido (Parquet/ORC) reduz o volume escaneado e, portanto, o custo por consulta.

## Machine Learning

**É preciso extrair texto e dados estruturados de PDFs escaneados e imagens de documentos. Amazon Textract ou Amazon Rekognition?**

### RESPOSTA

Amazon Textract — especializado em OCR e extração de dados estruturados de documentos. Rekognition analisa imagem/vídeo (objetos, rostos, cenas), mas não faz OCR de documentos.

**Uma aplicação de saúde precisa identificar entidades médicas e informações de saúde protegidas (PHI) em textos clínicos, sem treinar um modelo próprio. Por que usar SageMaker aqui seria esforço desnecessário?**

### RESPOSTA

Porque o Amazon Comprehend Medical já é um serviço de NLP pré-treinado especificamente para esse caso. Usar SageMaker (que exige treinar/gerenciar um modelo customizado) para algo já resolvido por um serviço gerenciado pré-treinado é esforço desnecessário.

**Um app recebe imagens de usuários com picos de tráfego irregulares e precisa analisá-las (ex.: moderação de conteúdo) da forma mais econômica possível. Qual padrão de arquitetura é preferível a EC2 dedicado ou EMR em batch?**

### RESPOSTA

S3 (evento) → Lambda → serviço de IA gerenciado (ex.: Rekognition) — arquitetura serverless que só cobra pelo uso real, ideal para cargas de processamento de mídia/texto sob demanda com picos irregulares.

**Uma central de atendimento precisa transcrever chamadas em texto, identificando cada falante separadamente e removendo automaticamente dados pessoais sensíveis da transcrição. Qual serviço faz tudo isso nativamente?**

### RESPOSTA

Amazon Transcribe — converte áudio em texto, identifica múltiplos falantes e oferece redação automática de PII.

## Governança e Custos

**A empresa suspeita que várias instâncias EC2 e volumes EBS estão superdimensionados (over-provisioned), mas não quer fazer essa análise manualmente. Qual serviço recomenda o rightsizing automaticamente, e ele é pago?**

### RESPOSTA

AWS Compute Optimizer — usa machine learning sobre métricas históricas do CloudWatch para recomendar configurações ideais de EC2, EBS, Lambda e Auto Scaling Groups. É gratuito, e complementar ao Cost Explorer (que mostra gasto, não recomendação de rightsizing).

**Um alarme do CloudWatch baseado só em CPU alta está gerando falsos positivos, porque dispara mesmo quando outras métricas (ex.: IOPS) estão normais. Qual recurso reduz esses falsos positivos?**

**RESPOSTA**

Composite Alarms — combinam múltiplas condições (ex.: CPU alta E IOPS alto simultaneamente) em um único alarme, reduzindo disparos indevidos baseados em uma métrica isolada.

**O time de FinOps quer investigar rapidamente por que o gasto AWS subiu no mês passado, com filtragem granular por serviço/conta/tag. AWS Cost Explorer ou AWS Budgets?**

**RESPOSTA**

AWS Cost Explorer — análise histórica detalhada de custo e uso, com filtragem granular. AWS Budgets serve para alertar proativamente quando o gasto se aproxima de um limite definido, não para investigar aumentos que já ocorreram.

**AWS Budgets apenas envia alertas ou também pode impor uma resposta automática?**

**RESPOSTA**

Também pode agir: Budgets Actions aplica IAM policy/SCP ou atua sobre instâncias EC2/RDS, automática ou manualmente aprovada. Budgets de Savings Plans também monitoram utilization e coverage percentuais.

**Dos 6 pilares do AWS Well-Architected Framework, qual foi adicionado mais recentemente e não tem peso isolado em nenhum dos 4 domínios oficiais do exame SAA-C03?**

**RESPOSTA**

Sustainability (Sustentabilidade), adicionado em 2021. Junto com Operational Excellence, permeia todos os domínios da prova mas não corresponde a um domínio oficial com peso próprio (os outros 4 pilares mapeiam diretamente para os 4 domínios: Security, Reliability, Performance Efficiency, Cost Optimization).

## Disaster Recovery

**A empresa quer a estratégia de DR mais barata do framework da AWS, aceitando um RTO/RPO mais alto. Qual das quatro estratégias?**

**RESPOSTA**

Backup & Restore — backups regulares restaurados sob demanda na região de DR. É a estratégia de menor custo e maior RTO/RPO das quatro.

**Um banco (ex.: Aurora Global Database) já está replicado continuamente na região de DR, mas o restante da infraestrutura só é provisionado no momento do failover, para manter o custo controlado. Qual estratégia de DR é essa?**

**RESPOSTA**

Pilot Light — bom equilíbrio entre RTO baixo e custo controlado, adequado quando a região de DR não precisa processar carga real durante a operação normal.

Uma equipe precisa medir objetivamente se a arquitetura atual atende a metas de RTO/RPO já definidas, identificar pontos de falha e receber recomendações específicas com custo estimado. Isso é apenas escolher entre Pilot Light, Warm Standby e Multi-Site?

#### RESPOSTA

Não — isso é o papel do AWS Resilience Hub, que avalia a resiliência da aplicação contra as metas de RTO/RPO definidas pelo usuário e recomenda melhorias concretas (configurações, alarmes, testes, SOPs). Escolher entre as quatro estratégias de DR é uma decisão diferente (e anterior) a essa avaliação.

Uma aplicação precisa do menor RTO/RPO possível, com tráfego real de produção sendo atendido simultaneamente em múltiplas regiões. Quando essa estratégia (Multi-Site Active/Active) é overkill?

#### RESPOSTA

Quando o requisito real é apenas failover em caso de desastre, não balanceamento de carga multi-região — nesse caso, Multi-Site Active/Active tem custo e complexidade desnecessários frente a Warm Standby ou Pilot Light.

## Pegadinhas Duplas (confusões mais recorrentes)

Um cenário de prova pede simultaneamente: (a) failover automático de um RDS em caso de falha de AZ, (b) descarregar tráfego de leitura do banco, (c) resolver timeouts de conexão de uma Lambda com CPU/memória do banco baixas, e (d) restaurar os dados a um ponto específico no passado. Qual mecanismo resolve cada um?

#### RESPOSTA

(a) No modelo **RDS Multi-AZ DB instance**, a standby síncrona oferece failover e não atende leitura. (b) Uma **Read Replica** separada descarrega leituras desse modelo; um **RDS Multi-AZ DB cluster** já oferece duas instâncias leitoras e também deve ser considerado se o cenário permitir esse modelo. (c) **RDS Proxy** reutiliza conexões. (d) **Backups automáticos com PITR** restauram dados. [Modelos Multi-AZ](#).

Uma aplicação precisa acessar o S3 e o DynamoDB sem passar pela internet, e também precisa acessar o AWS KMS de forma privada. Os dois casos usam o mesmo tipo de VPC Endpoint?

#### RESPOSTA

Para workloads dentro da VPC, S3 e DynamoDB normalmente usam **Gateway Endpoint**, sem cobrança do endpoint e integrado à route table; KMS usa **Interface Endpoint** via PrivateLink. A pegadinha moderna é tratar isso como exclusividade: S3 e DynamoDB também suportam interface endpoints, úteis para private IP e acesso on-premises/cross-Region, porém com cobrança.

Uma aplicação sofre, ao mesmo tempo, ataques de SQL Injection na camada de aplicação e um ataque DDoS volumétrico de rede. Um único serviço da AWS resolve os dois problemas?

#### RESPOSTA

Não. AWS WAF trata ataques de camada de aplicação (SQLi, XSS, rate-based rules); AWS Shield (Advanced) trata DDoS de rede/transporte. Um nunca substitui o outro — a defesa correta combina os dois em camadas, tipicamente junto com CloudFront/Route 53.

Uma aplicação global não-HTTP (ex.: um jogo multiplayer via UDP) precisa de IP estático e baixa latência via a rede backbone da AWS; outra aplicação precisa cachear conteúdo estático HTTP na borda para reduzir carga na origem. O mesmo serviço resolve os dois casos?

**RESPOSTA**

Não. AWS Global Accelerator resolve o primeiro (TCP/UDP, IP anycast, roteamento pelo backbone AWS, sem cache de conteúdo). Amazon CloudFront resolve o segundo (CDN com cache de conteúdo HTTP(S)). Confundir os dois propósitos ao otimizar performance multi-região é um erro recorrente.

## 14. Cartões Flash Adicionais — Consolidação por Domínio (SAA-C03)

Consolidação de cartões flash adicionais para os quatro domínios do exame AWS Certified Solutions Architect - Associate (SAA-C03). Cada card abaixo é um callout colapsável: clique para expandir e revelar a resposta só depois de tentar responder mentalmente. Os **cenários de decisão** ao fim de cada domínio explicam o requisito decisivo e por que as outras opções não o atendem.

Revisão complementar: [guia oficial do exame SAA-C03](#) e documentação da AWS indicada nos cartões revisados.

### Domínio 1 — Projetar arquiteturas seguras

Conteúdo relacionado: [[04. Introdução ao Domínio 1]], [[05. Criar acesso seguro aos recursos da AWS]], [[07. Projetar cargas de trabalho e aplicativos seguros]], [[09. Determinar controles de segurança de dados apropriados]].

#### Armazenamento (S3 e EBS)

O que fazer para impedir que os objetos em buckets do Amazon S3 sejam excluídos ou sobrescritos?

##### RESPOSTA

Para impedir a exclusão permanente ou sobrescrita de **versões específicas** durante um período de retenção, usar **S3 Object Lock** (modo compliance quando nem o usuário root deve poder remover a proteção). O versionamento permite recuperar versões anteriores, mas sozinho não impede sua exclusão permanente; MFA Delete exige MFA para excluir versões permanentemente ou alterar o versionamento, mas ainda permite criar marcadores de exclusão.

[Documentação.](#)

O que fazer para proteger dados sigilosos em volumes do Amazon EBS?

##### RESPOSTA

Ativar a criptografia EBS ao criar o volume.

#### Contas, IAM e Organizations

Qual serviço da AWS você pode usar para acesso de curto prazo, atuando como credenciais temporárias de segurança para acesso aos recursos da AWS?

##### RESPOSTA

AWS Security Token Service (AWS STS).



Quando você cria usuários, grupos ou perfis (roles), com quais permissões as identidades do IAM começam?

RESPOSTA

Sem permissões — todas as permissões precisam ser concedidas explicitamente.

Quais são os dois tipos de políticas que podem ser anexadas a perfis (roles) do IAM?

RESPOSTA

Política de confiança (trust policy) e política de permissão (permissions policy).

Quais são os dois principais usos das políticas de controle de serviço (SCPs)?

RESPOSTA

Bloquear serviços por padrão e permitir apenas alguns — chamado de **lista de permissões (allow list)**. Permitir por padrão e bloquear o acesso a alguns serviços — chamado de **lista de restrições (deny list)**.

## Rede (VPC)

Quais são algumas das diferenças entre as network ACLs e os grupos de segurança?

RESPOSTA

- As NACLs são usadas com sub-redes e permitem negar ou permitir explicitamente.
- Os grupos de segurança são usados para quase todo o resto e são mais simples, pois são **stateful** — exigindo menos regras para proteger o ambiente.
- Grupos de segurança são usados com as interfaces de rede elásticas das instâncias/recursos do Amazon EC2.
- Grupos de segurança têm **negação implícita**: tudo que não for explicitamente permitido é negado.
- As NACLs são processadas na ordem do menor para o maior número de regra.
- As NACLs são **stateless**, enquanto os grupos de segurança são **stateful**.

Quais são as diferenças entre gateway endpoints e interface endpoints em uma VPC?

RESPOSTA

Dois tipos frequentes no exame são **gateway endpoints**, para S3 e DynamoDB, associados a tabelas de rotas e sem cobrança adicional, e **interface endpoints**, que criam interfaces de rede privadas nas sub-redes para serviços compatíveis com AWS PrivateLink e usam grupos de segurança e DNS privado. **Ambos podem ter endpoint policies**, quando o serviço as suporta; S3 e DynamoDB também oferecem endpoints de interface. A AWS também oferece outros tipos de VPC endpoint. [Tipos](#) · [Políticas](#).

## Gerenciamento de segredos e certificados

Qual serviço da AWS você integra para criptografar e alternar (rotate) credenciais de bancos de dados, chaves de API e outros segredos?

### RESPOSTA

AWS Secrets Manager.

Qual serviço da AWS você integra para proteger uma aplicação web e possibilitar que vários domínios veiculem tráfego SSL pelo mesmo endereço IP?

### RESPOSTA

AWS Certificate Manager (ACM).

## Diretório (Directory Service)

Qual AWS Directory Service você implementa para acessar recursos on-premises e na AWS com as mesmas credenciais locais?

### RESPOSTA

AWS Directory Service for Microsoft Active Directory (AWS Managed Microsoft AD).

## AWS WAF

Você precisa limitar o número máximo de solicitações vindas do mesmo endereço IP em uma regra do AWS WAF. O que você cria?

### RESPOSTA

Uma regra baseada em taxa (rate-based rule), definindo o limite de taxa (rate limit).

Para o AWS WAF, por que é importante ter conhecimento do modelo OSI (Open Systems Interconnection)?

### RESPOSTA

Porque o conteúdo que o firewall consegue ler depende da camada do modelo OSI em que ele opera. É preciso saber em qual camada o produto/aplicativo funciona para determinar os recursos que o firewall pode ter — quanto mais alta a camada, maior o desempenho computacional necessário e o custo.

## Cartões adicionais — Criptografia e KMS

Qual é a diferença entre chaves gerenciadas pelo cliente (customer managed keys) e chaves gerenciadas pela AWS (AWS managed keys) no AWS KMS?

### RESPOSTA

Chaves gerenciadas pelo cliente oferecem controle total sobre a política de chave, rotação e permissões de uso, além de poderem ser desativadas ou agendadas para exclusão. Chaves gerenciadas pela AWS são criadas e mantidas automaticamente pelos serviços da AWS (ex: `aws/s3`), com rotação automática anual e sem controle direto do usuário sobre a política.

O que é criptografia envelope (envelope encryption) no AWS KMS?

### RESPOSTA

É o processo de criptografar os dados com uma chave de dados (data key) e, em seguida, criptografar essa chave de dados com uma chave mestra (CMK) armazenada no KMS. Isso evita enviar grandes volumes de dados diretamente ao KMS, melhorando performance e segurança.

## Cartões adicionais — Avaliação de políticas IAM

Qual é a ordem de precedência ao avaliar políticas IAM quando há uma negação explícita, uma permissão explícita e nenhuma menção ao recurso?

### RESPOSTA

Uma **negação explícita** sempre vence, independentemente de outras políticas. Na ausência de negação, uma **permissão explícita** libera o acesso. Se não houver nenhuma declaração aplicável, o acesso é **negado implicitamente** por padrão.

Um bucket do S3 tem uma política de bucket permitindo acesso público, mas o usuário IAM não possui nenhuma política IAM concedendo acesso. O acesso é permitido?

### RESPOSTA

Sim — políticas de recursos (como a política de bucket do S3) e políticas de identidade (IAM) são avaliadas em conjunto; qualquer uma que conceda a permissão é suficiente, desde que não haja uma negação explícita em outra política e o **Block Public Access** não esteja ativado.

## Cartões adicionais — Detecção e monitoramento de segurança

Qual serviço da AWS usa machine learning para detectar atividades maliciosas e comportamento não autorizado na conta, analisando VPC Flow Logs, CloudTrail e logs de DNS?

### RESPOSTA

Amazon GuardDuty.

Qual serviço centraliza e prioriza descobertas de segurança de vários serviços da AWS (como GuardDuty, Inspector e Macie) em um único painel?

RESPOSTA

AWS Security Hub.

Qual serviço identifica automaticamente dados sensíveis (como PII) armazenados em buckets do Amazon S3?

RESPOSTA

Amazon Macie.

Qual é a diferença entre AWS CloudTrail e Amazon CloudWatch?

RESPOSTA

O **CloudTrail** registra chamadas de API e atividades para governança, conformidade e auditoria (quem fez o quê, quando). O **CloudWatch** monitora métricas de desempenho e logs de aplicações, permitindo criar alarmes com base nesses dados.

## Cartões adicionais — Proteção contra DDoS e acesso remoto

Qual é a diferença entre AWS Shield Standard e AWS Shield Advanced?

RESPOSTA

O **Shield Standard** é gratuito, ativado automaticamente para todos os clientes, e protege contra os ataques DDoS mais comuns nas camadas 3 e 4. O **Shield Advanced** é pago, oferece proteção aprimorada, acesso à equipe de resposta a DDoS (DRT), detecção mais precisa e proteção contra custos gerados por picos de escalonamento durante um ataque.

Como acessar uma instância EC2 em uma sub-rede privada sem precisar de um bastion host ou abrir portas de entrada SSH/RDP?

RESPOSTA

Usando o **AWS Systems Manager Session Manager**, que estabelece uma sessão segura via agente SSM, sem necessidade de chaves SSH, portas de entrada abertas ou um bastion host.

## Cartões adicionais — Governança e controle de acesso avançado

Como conceder acesso centralizado de funcionários a várias contas AWS usando um diretório corporativo?

RESPOSTA

Usar **AWS IAM Identity Center** integrado ao provedor de identidade e ao AWS Organizations. Atribuir usuários ou grupos a contas por meio de permission sets; eles recebem credenciais temporárias. [Documentação](#).

**No modelo de responsabilidade compartilhada, quem protege o sistema operacional de uma instância EC2 e quem protege a infraestrutura física?**

**RESPOSTA**

O **cliente** configura e atualiza o sistema operacional convidado, aplicações e dados da instância. A **AWS** protege a infraestrutura física e a virtualização subjacente. Em serviços gerenciados, a divisão muda conforme o serviço.

[Documentação.](#)

**Qual é a diferença entre permission boundaries e políticas de controle de serviço (SCPs)?**

**RESPOSTA**

Permission boundaries são políticas gerenciadas aplicadas a um usuário ou role específico do IAM, definindo o limite máximo de permissões que essa identidade pode ter (mesmo que outras políticas concedam mais acesso). SCPs são aplicadas no nível de conta/OU dentro do AWS Organizations e afetam todas as identidades daquela conta, incluindo o usuário root. Ambas atuam como "teto" de permissões, mas em escopos diferentes: permission boundary é por identidade IAM, SCP é por conta/organização.

**O que são Resource Control Policies (RCPs) no AWS Organizations e como se diferenciam das SCPs?**

**RESPOSTA**

RCPs são políticas baseadas em recursos aplicadas no nível de conta/OU que limitam o acesso máximo permitido a recursos (como buckets S3 ou chaves KMS), independentemente das permissões IAM concedidas. Enquanto as SCPs controlam quais ações as identidades de uma conta podem realizar, as RCPs controlam quem pode acessar os recursos daquela conta — inclusive de fora da organização — funcionando como uma camada adicional de guardrail sobre políticas de recurso.

**Qual serviço da AWS você usaria para avaliar continuamente a conformidade da configuração dos recursos com regras definidas (por exemplo, verificar se todos os volumes EBS estão criptografados)?**

**RESPOSTA**

AWS Config — ele monitora e registra as configurações dos recursos, avaliando-as continuamente contra regras (managed ou customizadas) e sinalizando recursos não conformes. É diferente do CloudTrail, que registra chamadas de API, não o estado/configuração dos recursos.

**O que são o Amazon S3 Access Points e qual problema eles resolvem?**

**RESPOSTA**

São endpoints de acesso nomeados, com suas próprias políticas de permissão, criados para simplificar o gerenciamento de acesso a buckets S3 compartilhados por múltiplas aplicações ou equipes. Em vez de manter uma única política de bucket complexa, cada aplicação pode ter seu próprio access point com regras específicas, simplificando a governança em buckets com muitos consumidores.

### O que é o Amazon S3 Object Lambda e quando usá-lo?

#### RESPOSTA

Permite adicionar seu próprio código (AWS Lambda) para processar e transformar dados à medida que são recuperados do S3, sem precisar criar e manter cópias adicionais dos dados. Útil para casos como redigir dados sensíveis, converter formatos ou enriquecer dados sob demanda, para diferentes aplicações que acessam o mesmo objeto.

### O que são os VPC Flow Logs e o que eles capturam?

#### RESPOSTA

Capturam informações sobre o tráfego IP que entra e sai das interfaces de rede em uma VPC (origem, destino, portas, protocolo, bytes, ação ACCEPT/REJECT). Podem ser habilitados no nível de VPC, sub-rede ou interface de rede, e enviados para CloudWatch Logs ou S3 — úteis para diagnosticar problemas de conectividade e auditoria de segurança de rede.

### Qual é a diferença entre usar uma trust policy com AssumeRole para acesso entre contas e usar uma política baseada em recurso (resource-based policy)?

#### RESPOSTA

Com AssumeRole, o usuário da conta A assume temporariamente um role na conta B (que tem uma trust policy permitindo essa conta/entidade), recebendo credenciais temporárias via STS. Com uma política baseada em recurso (como uma política de bucket S3), o recurso da conta B concede acesso diretamente a um principal da conta A, sem necessidade de assumir um role — o usuário continua usando suas credenciais originais.

## Cenários de decisão — segurança

Uma empresa deve impedir a exclusão permanente de versões de objetos S3 durante sete anos, inclusive pelo usuário root. Escolha: (A) versionamento, (B) MFA Delete ou (C) S3 Object Lock em modo compliance?

#### RESPOSTA

**C — S3 Object Lock em modo compliance.** O requisito decisivo é impedir a exclusão durante uma retenção fixa até por administradores. **A** permite recuperar versões, mas não impede sua exclusão permanente; **B** adiciona uma exigência de MFA a determinadas operações, sem impor retenção regulatória. [Documentação](#).

Instâncias em sub-redes privadas precisam acessar S3 e KMS sem usar a internet. Escolha: (A) NAT Gateway, (B) gateway endpoint para S3 e interface endpoint para KMS ou (C) Internet Gateway?

#### RESPOSTA

**B.** O requisito decisivo é acesso privado a **dois serviços com tipos de endpoint diferentes**: gateway endpoint é uma opção comum e sem cobrança do endpoint para S3; KMS usa interface endpoint. **A** e **C** não atendem ao requisito de acesso por endpoints privados. S3 também oferece interface endpoint quando o cenário exigir suas características específicas. [Tipos de endpoint](#).



## Domínio 2 — Projetar arquiteturas resilientes

Conteúdo relacionado: [[13. Introdução ao domínio 2]], [[14. Projetar arquiteturas dimensionáveis e com acoplamento fraco]], [[15. Criar arquiteturas altamente disponíveis e/ou tolerantes a falhas]].

### Bancos de dados

**Se você precisa projetar e implementar replicação de dados síncronos em Zonas de Disponibilidade diferentes para instâncias do Amazon RDS, qual é a melhor solução?**

#### RESPOSTA

Implantação **RDS Multi-AZ DB instance**, cuja standby síncrona fica em outra AZ e atende ao failover, mas não a leituras. O modelo **RDS Multi-AZ DB cluster** tem duas leitoras e usa replicação semissíncrona. [Comparação](#).

**Se você precisa projetar e implementar replicação de dados assíncronos em outra instância do Amazon RDS, em outra Região da AWS, qual é a melhor solução?**

#### RESPOSTA

Adicionar uma **réplica de leitura (read replica)**.

**Você precisa criar um plano de recuperação de desastres para um banco de dados relacional, com RTO de 60 segundos e RPO de 1 segundo. Qual banco de dados do Amazon RDS seria a melhor opção?**

#### RESPOSTA

O **Amazon Aurora Global Database**, porque permite que um único banco de dados Aurora abranja várias Regiões da AWS, usando replicação baseada em armazenamento com latência tipicamente inferior a 1 segundo. Em caso de degradação ou interrupção regional, uma das Regiões secundárias pode ser promovida para capacidade de leitura e gravação em menos de 1 minuto.

### Conceitos de resiliência

**O que é alta disponibilidade?**

#### RESPOSTA

É um modo de projetar sistemas para que continuem em execução e prestando serviço pelo máximo de tempo possível. Se ocorrer falha de um componente do sistema, ele será substituído ou corrigido assim que possível.

**O que é tolerância a falhas?**

#### RESPOSTA

Embora seja semelhante à alta disponibilidade, é a capacidade de um sistema continuar funcionando em caso de falha — ocorre falha em um ou mais componentes do sistema, mas ele continua funcionando mesmo assim. Um design tolerante a falhas precisa continuar em operação.

## O que é recuperação de desastres?

### RESPOSTA

É um pouco diferente da tolerância a falhas e da alta disponibilidade, pois estas tratam de projetar sistemas para operar **durante** um desastre. A recuperação de desastres envolve o que planejar e também o que fazer **no caso de** um desastre.

## Rede e DNS

Qual serviço da AWS pode ser implementado para criar um failover de DNS para um site estático hospedado no Amazon S3?

### RESPOSTA

**Amazon Route 53** com a política de roteamento de **failover**.

## Armazenamento

Qual classe de armazenamento do Amazon S3 você escolheria para armazenar dados frios?

### RESPOSTA

**Amazon S3 Glacier**.

Qual solução de armazenamento você escolheria se precisasse de um sistema de arquivos paralelo para dados com alta frequência de acesso?

### RESPOSTA

**Amazon FSx for Lustre**.

## Cartões adicionais — Auto Scaling

Quais são os três tipos de verificação de integridade (health check) que um grupo do Auto Scaling pode usar?

### RESPOSTA

Verificação de integridade do **EC2** (status da instância), verificação de integridade do **ELB** (se o load balancer considera a instância saudável) e verificações **personalizadas** (via API, enviando o status de integridade).

O que acontece com uma instância que falha na verificação de integridade dentro de um grupo do Auto Scaling com um Load Balancer associado?

### RESPOSTA

O Auto Scaling encerra a instância não saudável e inicia uma nova para substituí-la, mantendo a capacidade desejada do grupo.

## Cartões adicionais — Mensageria e desacoplamento

Qual é a diferença entre Amazon SQS Standard e SQS FIFO?

### RESPOSTA

A fila **Standard** oferece throughput muito alto e entrega pelo menos uma vez, sem garantia de ordem. A **FIFO** preserva a ordem dentro de cada grupo de mensagens e deduplica envios dentro de uma janela de cinco minutos; sua taxa depende da configuração. O consumidor ainda deve tolerar reproprocessamento se falhar antes de excluir a mensagem: **idempotência é responsabilidade da aplicação**. [Documentação](#).

Qual serviço da AWS você usaria para desacoplar componentes de uma aplicação, permitindo que um produtor envie mensagens para múltiplos consumidores simultaneamente?

### RESPOSTA

Amazon SNS (Simple Notification Service), com um padrão fan-out — geralmente combinado com Amazon SQS, onde múltiplas filas se inscrevem em um mesmo tópico SNS.

## Cartões adicionais — Roteamento com Route 53

Além de failover, quais são as principais políticas de roteamento do Amazon Route 53?

### RESPOSTA

**Simples**, **ponderada (weighted)**, **latência (latency-based)**, **geolocalização**, **geoproximidade** e **múltiplos valores de resposta (multivalue answer)**.

Qual política de roteamento do Route 53 você usaria para distribuir uma porcentagem específica de tráfego entre diferentes versões de uma aplicação (ex: teste A/B)?

### RESPOSTA

Política de roteamento **ponderada (weighted)**.

## Cartões adicionais — Cache, sessão e réplicas globais

Qual serviço da AWS você usaria para armazenar dados de sessão de usuários e reduzir a carga sobre um banco de dados relacional?

### RESPOSTA

Amazon ElastiCache (Redis ou Memcached).

Qual recurso do Amazon DynamoDB permite replicação multi-região ativa-ativa, com leitura e gravação em qualquer região replicada?

### RESPOSTA

DynamoDB Global Tables.

## Cartões adicionais — Backup e orquestração

### Como escolher entre backup e restauração, pilot light, warm standby e ativo-ativo para recuperação de desastres?

#### RESPOSTA

Compare **RTO, RPO, custo e complexidade**: backup e restauração mantém menos recursos ativos e costuma recuperar mais lentamente; pilot light mantém componentes essenciais; warm standby mantém uma versão reduzida em operação; ativo-ativo atende tráfego em mais de uma Região e tende a oferecer recuperação mais rápida, com maior custo operacional. [Documentação](#).

### Quando usar uma dead-letter queue (DLQ) com Amazon SQS?

#### RESPOSTA

Quando mensagens falham repetidamente, configurar uma **DLQ** para isolá-las após o número definido de tentativas. Isso evita que bloqueiem o processamento normal e permite investigar ou reprocessar a falha; ajustar também o visibility timeout ao tempo de processamento. [Documentação](#).

### Quando escolher Amazon EventBridge em vez de SNS ou SQS?

#### RESPOSTA

**EventBridge** roteia eventos de fontes AWS, aplicações próprias ou SaaS para destinos conforme regras e padrões de conteúdo. **SNS** distribui notificações a assinantes (fan-out); **SQS** guarda mensagens para consumidores processarem de forma assíncrona. Os serviços podem ser combinados. [Guia de decisão](#).

### Qual é a diferença entre o AWS Backup e a criação manual de snapshots?

#### RESPOSTA

O AWS Backup é um serviço centralizado que permite definir políticas de backup (planos) para automatizar, agendar e monitorar backups de múltiplos serviços da AWS (EBS, RDS, DynamoDB, EFS, entre outros) a partir de um único console, aplicando regras de retenção e cópia entre regiões/contas de forma consistente. Snapshots manuais exigem gerenciamento individual por serviço, sem essa visão unificada.

### O que é o cross-zone load balancing e qual é seu comportamento padrão no ALB e no NLB?

#### RESPOSTA

É o recurso que distribui o tráfego igualmente entre todas as instâncias registradas, independentemente da Zona de Disponibilidade em que estejam, em vez de distribuir apenas entre as instâncias da AZ que recebeu a requisição. No Application Load Balancer, vem habilitado por padrão (sem custo adicional). No Network Load Balancer, vem desabilitado por padrão e, quando ativado, pode gerar custo de transferência de dados entre AZs.

**O que é o AWS Elastic Beanstalk e quando você o escolheria em vez de gerenciar a infraestrutura manualmente?**

**RESPOSTA**

É um serviço de orquestração que provisiona e gerencia automaticamente a infraestrutura (EC2, Auto Scaling, ELB, monitoramento) necessária para executar uma aplicação, a partir do código-fonte enviado pelo desenvolvedor. É indicado quando você quer fazer deploy rápido de uma aplicação sem gerenciar manualmente cada componente de infraestrutura, mantendo acesso aos recursos subjacentes caso precise de ajustes finos — diferente de uma solução totalmente serverless.

**O que é o AWS Step Functions e para que tipo de arquitetura ele é indicado?**

**RESPOSTA**

É um serviço de orquestração que permite coordenar múltiplos serviços da AWS (como Lambda, ECS, SNS) em fluxos de trabalho visuais definidos como máquinas de estado, com lógica de retry, tratamento de erro e paralelismo integrados. É indicado para workflows distribuídos de múltiplas etapas, como pipelines de processamento de pedidos ou orquestração de microserviços, sem precisar codificar manualmente a lógica de coordenação e tratamento de falhas.

**Qual é a diferença conceitual entre RTO (Recovery Time Objective) e RPO (Recovery Point Objective)?**

**RESPOSTA**

RTO é o tempo máximo aceitável para restaurar um sistema após uma interrupção (quanto tempo até voltar a funcionar). RPO é a quantidade máxima aceitável de perda de dados, medida em tempo (até quantos minutos/segundos de dados antes da falha podem ser perdidos). Por exemplo, um RTO de 1 hora e RPO de 5 minutos significam que o sistema deve voltar a funcionar em até 1 hora, com no máximo 5 minutos de dados perdidos desde o último ponto de recuperação.

## Cenários de decisão — resiliência

**Um banco RDS precisa de failover entre AZs e de duas instâncias que atendam relatórios de leitura, usando uma implantação integrada. Escolha: (A) RDS Multi-AZ DB instance, (B) RDS Multi-AZ DB cluster ou (C) apenas backups automáticos?**

**RESPOSTA**

**B — RDS Multi-AZ DB cluster**, se o engine e os requisitos do cenário forem compatíveis. Ele reúne uma instância de gravação e duas leitoras em três AZs. **A** tem uma standby que não atende leituras; seria necessário acrescentar Read Replica separada. **C** permite restauração, mas não oferece instâncias leitoras nem substitui failover automático.

[Modelos Multi-AZ.](#)

Um consumidor processa pagamentos a partir de SQS FIFO e pode falhar depois de cobrar, mas antes de excluir a mensagem. Escolha: (A) confiar apenas em FIFO, (B) tornar a cobrança idempotente e ajustar o visibility timeout ou (C) trocar por SQS Standard?

**RESPOSTA**

**B.** O requisito decisivo é impedir **efeitos duplicados** quando uma mensagem é reprocessada. O consumidor deve reconhecer a mesma operação por uma chave de idempotência; o visibility timeout precisa ser compatível com o processamento. **A** confunde deduplicação de envio com execução única da lógica de negócio. **C** também não impede duplicatas e não preserva a ordem. [Recuperação do SQS](#).

## Domínio 3 — Projetar arquiteturas de alta performance

Conteúdo relacionado: [[23. Determinar soluções de armazenamento dimensionáveis e/ou de alta performance]], [[24. Projetar soluções de computação elásticas e de alto desempenho]], [[25. Determinar soluções de banco de dados de alta performance]], [[26. Determinar arquiteturas de rede dimensionáveis e de alto desempenho]], [[27. Determinar soluções de ingestão e transformação de dados de alta performance]].

### Balanceamento de carga e entrega de conteúdo

Qual balanceador de carga está na camada 7 do modelo de interconexão de sistemas abertos (OSI)?

**RESPOSTA**

☐ Application Load Balancer.

Qual balanceador de carga você integraria para dimensionar e processar milhões de solicitações por segundo?

**RESPOSTA**

☐ Network Load Balancer.

Qual é a diferença entre o Amazon CloudFront e o AWS Global Accelerator?

**RESPOSTA**

O **CloudFront** melhora a entrega de conteúdo armazenado em cache, estático ou dinâmico, veiculado por locais de borda, na maior parte do tempo.

O **Global Accelerator** melhora o desempenho de aplicativos em TCP e UDP, pois os pacotes são transferidos dos locais de borda para os aplicativos em execução em uma ou mais Regiões. Todas as solicitações chegam à borda, mas não há armazenamento em cache disponível. É útil para casos de uso que precisam de IP estático ou de failover Regional rápido.



## Armazenamento

Quais são os dois tipos de replicação do Amazon S3?

### RESPOSTA

Replicação **entre Regiões** e replicação **na mesma Região**.

Quais são as três opções de armazenamento do AWS Storage Gateway?

### RESPOSTA

**File Gateway**, **Volume Gateway** e **Tape Gateway**.

Quais são as opções de armazenamento para instâncias do Amazon EC2?

### RESPOSTA

**Armazenamento de instância** (temporário) e **volumes EBS** (permanentes).

Qual é a resiliência do Amazon EBS?

### RESPOSTA

O volume EBS pertence a **uma Zona de Disponibilidade** e é replicado dentro dela para tolerar falhas de componentes. **Snapshots são backups**, armazenados redundantemente em várias AZs; permitem restaurar um novo volume, mas não tornam o volume original altamente disponível entre AZs automaticamente. [Documentação](#).

Qual tipo de instâncias você pode usar com o Amazon Elastic File System (Amazon EFS)?

### RESPOSTA

Instâncias do **Linux**.

## Bancos de dados

Qual é a diferença entre uma consulta (query) e uma varredura (scan) em uma tabela do Amazon DynamoDB?

### RESPOSTA

Uma **consulta** é um modo de recuperar dados usando uma chave-valor de partição para iniciar a busca. Um único valor é especificado para a chave de partição, retornando um ou mais itens. É mais eficiente recuperar o máximo de dados possível em uma consulta, para não precisar executar várias leituras e desperdiçar RCUs. Também é possível filtrar adicionando uma chave-valor de classificação ou um intervalo de chaves-valores.

Uma **varredura** é mais flexível, mas menos eficiente: é possível escolher o atributo ou filtro aplicado, porém a varredura passa por toda a tabela e consome a capacidade de cada item acessado — mesmo que o item não corresponda ao filtro selecionado, sua capacidade de leitura é consumida.

O que você pode integrar a uma tabela do Amazon DynamoDB para fazer ações com base em alterações nos dados?

RESPOSTA

**DynamoDB Streams** e o **AWS Lambda**, para criar gatilhos e agir com base nas alterações capturadas no fluxo.

Qual banco de dados da AWS pode ser usado como data warehouse para transações de Analytics e de resumo?

RESPOSTA

○ **Amazon Redshift**.

## Rede

Onde fica o gateway NAT? Em uma sub-rede pública ou privada?

RESPOSTA

Em uma sub-rede **pública**.

Se você usa peering de VPC para permitir acesso entre duas VPCs da Amazon, o que você pode usar para controlar esse acesso?

RESPOSTA

**ACLs de rede** e **grupos de segurança**.

## Cartões adicionais — Volumes EBS e placement groups

Quais são os principais tipos de volume do Amazon EBS e seus casos de uso?

RESPOSTA

**gp3/gp2** (SSD de uso geral) — cargas de trabalho balanceadas; **io2/io1** (SSD provisionado) — bancos de dados críticos com IOPS altas e consistentes; **st1** (HDD otimizado para throughput) — big data e logs; **sc1** (HDD frio) — dados acessados raramente, com menor custo.

Quais são os três tipos de grupos de posicionamento (placement groups) do Amazon EC2?

RESPOSTA

**Cluster** (instâncias próximas, baixa latência e alto throughput de rede), **Spread** (instâncias distribuídas em hardware distinto para reduzir falhas correlacionadas) e **Partition** (instâncias divididas em partições lógicas com hardware separado, útil para cargas distribuídas como HDFS/Kafka).

## Cartões adicionais — Streaming, ingestão e performance de banco de dados

### Quando usar AWS DataSync e quando usar AWS DMS em uma migração?

#### RESPOSTA

**DataSync** transfere arquivos e objetos entre armazenamento local, serviços AWS e outras nuvens; **AWS DMS** migra ou replica dados de bancos de dados, inclusive com captura contínua de mudanças (CDC) quando suportada. Escolha pelo tipo de dado e pela necessidade de manter a origem sincronizada. [DataSync](#) · [DMS](#).

### Qual é a diferença entre VPC peering e AWS Transit Gateway para conectar várias VPCs?

#### RESPOSTA

**VPC peering** liga diretamente duas VPCs e não permite roteamento transitivo. **Transit Gateway** atua como hub de roteamento para várias VPCs e conexões híbridas, reduzindo a quantidade de relações ponto a ponto em redes maiores. [Documentação](#).

### Qual é a diferença entre Amazon Kinesis Data Streams e Amazon Kinesis Data Firehose?

#### RESPOSTA

O **Kinesis Data Streams** permite processamento customizado em tempo real, com retenção configurável e múltiplos consumidores. O **Kinesis Data Firehose** carrega automaticamente dados de streaming para destinos como S3, Redshift ou OpenSearch, sem necessidade de gerenciar infraestrutura, mas com menos controle sobre o processamento em tempo real.

### O que é o Amazon DynamoDB Accelerator (DAX)?

#### RESPOSTA

Um cache em memória para leituras repetidas do DynamoDB, capaz de reduzir a latência a microssegundos. A aplicação precisa usar o **cliente DAX** e o **endpoint do cluster**; não é ativado sem mudança na aplicação. [Cliente DAX](#).

### O que é o Amazon RDS Proxy e quando usá-lo?

#### RESPOSTA

Um proxy de banco de dados totalmente gerenciado que agrupa (pools) e compartilha conexões de banco de dados, melhorando a escalabilidade de aplicações (como as sem servidor com Lambda) que abrem muitas conexões simultâneas, e reduzindo a sobrecarga no banco de dados.

## Cartões adicionais — Transferência de dados em larga escala

### Qual serviço você usaria para transferir petabytes de dados para a AWS quando a conexão de rede é lenta ou não é viável?

#### RESPOSTA

Em questões históricas, Snowball Edge era a resposta para transferência física offline de grandes volumes. Para **novos clientes em 2026**, compare DataSync/Direct Connect e AWS Data Transfer Terminal; Snowball Edge está restrito a clientes existentes e perto do fim de suporte. Não use Snowmobile como resposta de arquitetura nova. [Aviso Snowball](#).

## Cartões adicionais — Famílias de instância e containers

### Quais são as principais famílias de instância do Amazon EC2 e seus casos de uso?

#### RESPOSTA

**Uso geral (M, T)** — equilíbrio entre computação, memória e rede, para a maioria das cargas de trabalho. **Otimizadas para computação (C)** — alta relação de CPU por custo, para processamento intensivo (ex: modelagem, jogos, HPC). **Otimizadas para memória (R, X, z1d)** — para bancos de dados e cargas com grandes conjuntos de dados em memória. **Otimizadas para armazenamento (I, D, H)** — alta taxa de IOPS/throughput local, para bancos de dados NoSQL e data warehousing. **Aceleradas (P, G, Inf)** — com GPUs ou aceleradores, para machine learning e processamento gráfico.

### Qual é a diferença entre Amazon ECS, Amazon EKS e AWS Fargate?

#### RESPOSTA

**ECS** é o orquestrador de containers nativo da AWS, mais simples de configurar e totalmente integrado ao ecossistema AWS. **EKS** é o serviço gerenciado de Kubernetes, indicado para equipes que já usam ou precisam de compatibilidade com Kubernetes padrão (portabilidade entre nuvens). **Fargate** não é um orquestrador, mas um modo de computação serverless que pode ser usado com ECS ou EKS, eliminando a necessidade de provisionar e gerenciar instâncias EC2 subjacentes para rodar os containers.

### Qual é a diferença entre os modos de capacidade "on-demand" e "provisioned" do Amazon DynamoDB?

#### RESPOSTA

No modo **on-demand**, a tabela escala automaticamente a capacidade de leitura/gravação conforme a demanda, cobrando por solicitação — indicado para cargas imprevisíveis ou com picos. No modo **provisioned**, você define capacidade de leitura (RCUs) e gravação (WCUs) fixas (podendo usar Auto Scaling para ajustar dentro de limites), sendo mais econômico para cargas previsíveis e estáveis.

### O que é concorrência provisionada (provisioned concurrency) no AWS Lambda e por que ela reduz cold starts?

#### RESPOSTA

É um recurso que mantém um número definido de ambientes de execução do Lambda inicializados e prontos para responder imediatamente, evitando o atraso (cold start) causado pela inicialização de um novo ambiente sob demanda. É útil para aplicações sensíveis à latência, como APIs síncronas com requisitos de tempo de resposta consistentes.

### O que são instâncias Graviton e qual sua principal vantagem?

#### RESPOSTA

São instâncias EC2 baseadas em processadores ARM projetados pela própria AWS, que oferecem melhor relação custo-desempenho (até ~40% mais eficientes) e menor consumo de energia em comparação com instâncias equivalentes baseadas em x86, para cargas de trabalho compatíveis com arquitetura ARM.

## Cenários de decisão — performance

Usuários em vários países baixam repetidamente arquivos estáticos por HTTP(S). Escolha: (A) CloudFront, (B) Global Accelerator ou (C) S3 Transfer Acceleration?

### RESPOSTA

**A — CloudFront.** O requisito decisivo é **cache de conteúdo HTTP(S)** perto dos usuários. **B** otimiza o caminho de rede e oferece IPs estáticos, mas não armazena arquivos em cache. **C** acelera a transferência de longa distância para um bucket S3, especialmente uploads, sem substituir uma CDN de downloads repetidos. [CloudFront](#) · [Global Accelerator](#).

Um processamento independente de vídeos pode durar várias horas e chega em lotes irregulares. Escolha: (A) uma única invocação Lambda, (B) AWS Batch ou (C) CloudFront?

### RESPOSTA

**B — AWS Batch.** O requisito decisivo é executar jobs em lote que podem ultrapassar o limite de duração de uma invocação Lambda. **A** está sujeita ao timeout máximo de 15 minutos; **C** distribui conteúdo e não executa jobs em lote. [AWS Batch](#) · [Limites do Lambda](#).

## Domínio 4 — Projetar arquiteturas econômicas

Conteúdo relacionado: [[36. Projetar soluções de armazenamento econômicas]], [[37. Projetar soluções de computação econômicas]], [[38. Projetar soluções de banco de dados econômicas]], [[39. Criar arquiteturas de rede com custo otimizado]].

## Rede e transferência de dados

Qual é uma opção para minimizar o custo da transferência de dados entre duas instâncias do Amazon EC2?

### RESPOSTA

Quando a arquitetura permitir, implantá-las na mesma **Zona de Disponibilidade** e usar endereços IP privados. Estar apenas na mesma Região não elimina a cobrança de tráfego entre AZs. Equilibre essa economia com a necessidade de distribuir a aplicação entre AZs para resiliência. [Preços do EC2](#).

Recentemente, você configurou uma conexão de rede dedicada com o AWS Direct Connect para conectar o data center on-premises à AWS, proporcionando o acesso consistente e dedicado que era necessário. Agora é preciso que as novas contas da AWS tenham o mesmo acesso consistente e dedicado. O que você pode adicionar ao design e implementar para cumprir essa exigência de forma econômica, com o menor gasto administrativo possível?

### RESPOSTA

Criar um **gateway do DX** para integrar à conexão recém-criada do Direct Connect. Depois, configurar um **gateway de trânsito (Transit Gateway)** entre as novas contas da AWS e associá-lo ao gateway do DX.

Ao anexar um gateway de trânsito a um gateway do DX (usando uma interface virtual de trânsito), é possível gerenciar uma única conexão para várias VPCs ou VPNs da Amazon que estão na mesma Região da AWS. Essa solução simplifica o gerenciamento de conexões entre uma Amazon VPC e as redes do cliente por uma conexão privada, minimiza o custo da rede, torna a conexão mais confiável e melhora o throughput da largura de banda.

## Armazenamento

O que você pode implementar como uma solução econômica para armazenar objetos do Amazon S3 que não são acessados com frequência?

### RESPOSTA

Criar uma **política de ciclo de vida do Amazon S3** para mover os objetos que não são acessados com frequência para uma classe de armazenamento mais barata.

Como reduzir a despesa da execução de instâncias do Amazon EC2 não utilizadas?

### RESPOSTA

Criar um **snapshot do volume EBS** e excluir as instâncias do Amazon EC2 não utilizadas.

## Gestão e monitoramento de custos

O que você deve ativar para usar o painel do console AWS Billing and Cost Management para acompanhar despesas e uso?

### RESPOSTA

☐ AWS Cost Explorer.

## Computação

Você precisa criar uma solução econômica para um aplicativo web em um grupo do Auto Scaling de instâncias do Amazon EC2, evitando o excesso de provisionamento ou o alto custo operacional, sem degradar o desempenho do aplicativo. Qual política de scaling dinâmico atende a esse requisito?

### RESPOSTA

Uma política de **scaling com monitoramento de alvo (target tracking)** — ela aumenta ou reduz a capacidade atual do grupo de instâncias com base no valor almejado de uma métrica específica, adicionando ou removendo capacidade quando necessário para manter a métrica próxima ao valor definido, evitando o excesso de provisionamento.

Você precisa escolher um modelo de preços para instâncias do Amazon EC2 que processam cargas em batch, em um ambiente que não é de produção e pode sofrer interrupções. Qual opção você escolhe?

### RESPOSTA

**Spot Instances.**

O que são as reservas de capacidade sob demanda (On-Demand Capacity Reservations)?

### RESPOSTA

Reservam capacidade computacional para instâncias do Amazon EC2 em uma determinada Zona de Disponibilidade, por qualquer período.



## O que são as instâncias reservadas (Reserved Instances)?

### RESPOSTA

Oferecem um desconto significativo (até **72%**) em relação ao preço das instâncias sob demanda. Quando atribuídas a uma Zona de Disponibilidade específica, também podem reservar capacidade.

## Cartões adicionais — Classes de armazenamento S3

### Como escolher entre S3 Glacier Instant Retrieval, Flexible Retrieval e Deep Archive?

#### RESPOSTA

**Instant Retrieval** para arquivo que precisa de acesso imediato; **Flexible Retrieval** quando é aceitável restaurar em minutos ou horas; **Deep Archive** para retenção longa com restauração em horas. Considere também frequência de acesso, cobranças de recuperação e duração mínima de armazenamento. [Documentação](#).

### Qual classe de armazenamento do Amazon S3 move automaticamente objetos entre camadas de acesso frequente e infrequente, com base no padrão de acesso, sem penalidade de recuperação?

#### RESPOSTA

Amazon S3 Intelligent-Tiering.

### Qual é a diferença entre S3 Standard-IA e S3 One Zone-IA?

#### RESPOSTA

O **Standard-IA** replica dados em pelo menos 3 Zonas de Disponibilidade, com alta durabilidade e disponibilidade. O **One Zone-IA** armazena dados em uma única Zona de Disponibilidade, custando cerca de 20% menos, mas com risco de perda de dados se aquela zona falhar — indicado para dados recriáveis ou cópias secundárias.

## Cartões adicionais — Modelos de preços de computação

### Qual é a diferença entre Reserved Instances e Savings Plans?

#### RESPOSTA

As **Reserved Instances** oferecem desconto para uma família e tipo específico de instância em uma Região (ou AZ), com compromisso de 1 ou 3 anos. Os **Savings Plans** (Compute Savings Plans) oferecem descontos semelhantes, mas com mais flexibilidade — aplicam-se automaticamente a qualquer instância EC2, Fargate ou Lambda, independentemente da família, tamanho, sistema operacional ou Região, desde que o compromisso de gasto por hora seja atingido.

## Cartões adicionais — Ferramentas de otimização de custo

### Quando usar AWS Cost Explorer e quando usar AWS Cost and Usage Report?

#### RESPOSTA

**Cost Explorer** permite visualizar tendências e analisar gastos interativamente. **Cost and Usage Report (CUR)** entrega dados detalhados de custo e uso para consultas e análises próprias, inclusive por conta, serviço e recurso quando habilitado. [Documentação](#).

Qual serviço da AWS oferece recomendações automatizadas em tempo real sobre custo, desempenho, segurança e limites de serviço, com base em boas práticas?

RESPOSTA

AWS Trusted Advisor.

Qual serviço você usaria para definir limites de gastos personalizados e receber alertas quando os custos ou o uso ultrapassarem (ou estiverem previstos a ultrapassar) esse limite?

RESPOSTA

AWS Budgets.

### Cartões adicionais — Custo de transferência de dados e computação sem servidor

Como o custo de transferência de dados varia entre instâncias na mesma Zona de Disponibilidade, entre Zonas de Disponibilidade diferentes e entre Regiões diferentes?

RESPOSTA

Transferência de dados dentro da **mesma AZ** (usando IP privado) geralmente não tem custo. Transferência **entre AZs** na mesma Região tem custo por GB transferido (em ambas as direções). Transferência **entre Regiões** tem o custo mais alto por GB. Minimizar a distância reduz o custo.

Por que o AWS Lambda é considerado uma opção econômica para cargas de trabalho esporádicas ou imprevisíveis?

RESPOSTA

Porque você paga apenas pelo tempo de computação consumido (por milissegundo) e pelo número de solicitações, sem custo quando o código não está em execução — eliminando a necessidade de provisionar e pagar por capacidade ociosa.

### Cartões adicionais — Estratégias de migração e otimização

Quais são os "6 Rs" das estratégias de migração para a nuvem?

RESPOSTA

**Rehost** (lift-and-shift, migrar sem alterações), **Replatform** (pequenas otimizações durante a migração, "lift-tinker-and-shift"), **Repurchase** (trocar por um produto SaaS), **Refactor/Re-architect** (redesenhar a aplicação para aproveitar recursos nativos da nuvem), **Retire** (desativar aplicações que não são mais necessárias) e **Retain** (manter on-premises por enquanto, adiando a migração).

O que é o AWS Compute Optimizer e como ele ajuda a reduzir custos?

RESPOSTA

É um serviço que analisa métricas de utilização (CPU, memória, rede) de recursos como instâncias EC2, Auto Scaling Groups, volumes EBS e funções Lambda, usando machine learning para recomendar o tipo e tamanho ideais (rightsizing), evitando superprovisionamento e reduzindo custos sem comprometer desempenho.

### O que é o Amazon S3 Storage Lens?

#### RESPOSTA

É uma ferramenta de análise que fornece visibilidade organizacional sobre o uso e a atividade de armazenamento em todas as contas e buckets S3, com métricas e recomendações para otimizar custos (como identificar buckets sem política de ciclo de vida ou com muitas versões não removidas) e melhorar a proteção de dados.

### Do ponto de vista de custo, qual é a diferença entre usar um NAT Gateway, uma NAT Instance e um VPC Endpoint para acessar serviços da AWS a partir de uma sub-rede privada?

#### RESPOSTA

O **NAT Gateway** é totalmente gerenciado, cobra por hora de uso mais por GB processado, e é necessário para acessar a internet em geral. A **NAT Instance** é uma instância EC2 configurada manualmente como NAT, com custo apenas da instância (mais barata em baixo volume, mas exige gerenciamento e não escala automaticamente). O **VPC Endpoint** (gateway ou interface) permite acesso privado direto a serviços específicos da AWS (como S3 ou DynamoDB) sem passar pela internet nem por um NAT, geralmente com custo menor e melhor desempenho quando o destino é um serviço AWS suportado.

### Cenários de decisão — custos

Uma aplicação EC2 tem carga base estável durante todo o ano e executa tarefas adicionais que podem ser interrompidas e retomadas. Escolha: (A) On-Demand para tudo, (B) compromisso de gasto para a base e Spot para as tarefas tolerantes a interrupção ou (C) Spot como única capacidade de produção?

#### RESPOSTA

**B.** A carga base previsível pode aproveitar **Savings Plans ou Reserved Instances**; as tarefas reiniciáveis podem aproveitar **Spot**. **A** deixa de aproveitar descontos da demanda estável. **C** expõe toda a aplicação a interrupções de capacidade. Compare preços e requisitos reais antes de assumir qual compromisso é mais econômico. [Opções de compra do EC2](#).

Objetos S3 seguem uma regra conhecida: acesso frequente por 90 dias e depois arquivamento de longo prazo. Escolha: (A) uma regra de lifecycle, (B) Intelligent-Tiering apenas porque existe ou (C) manter tudo em Standard indefinidamente?

#### RESPOSTA

**A — regra de lifecycle**, após comparar duração mínima, custos de transição e recuperação da classe de destino. O requisito decisivo é que **o momento da mudança já é conhecido**. **B** é mais útil quando o padrão de acesso é incerto e precisa de monitoramento automático; **C** pode manter dados frios numa classe mais cara. [Lifecycle](#) · [Intelligent-Tiering](#).

## 15. Mapa de cobertura do guia oficial e questões de múltipla resposta

O [guia oficial SAA-C03](#) divide a prova em **14 tarefas**. A tabela localiza a revisão de cada tarefa neste material. Cobrir as tarefas publicadas **não garante** conhecer todas as questões: exemplos de serviços e habilidades não são exaustivos, e cenários novos exigem aplicar os mesmos princípios. Para priorizar estudo, use as tarefas oficiais; para escolher um serviço em projeto novo, confira a disponibilidade atual na documentação do produto. A [lista oficial de serviços](#) também declara que é incompleta e sujeita a alterações; nesta revisão ela ainda cita Elastic Transcoder, cujo suporte terminou em 2025.

| Tarefa oficial                                        | Decisões e conceitos a dominar                                                                  | Onde revisar                                     |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <b>1.1 Acesso seguro</b>                              | Raiz/MFA, federação, roles temporárias, menor privilégio, SCP, boundaries, políticas de recurso | Seção 5: IAM, Organizations, identidade federada |
| <b>1.2 Cargas seguras</b>                             | Segmentação, SG/NACL, endpoints, inspeção, proteção de aplicação, logs e detecção               | Seções 4–5: VPC, WAF/Shield, Config/CloudTrail   |
| <b>1.3 Dados seguros</b>                              | Classificação, residência, criptografia, KMS, segredos, retenção, acesso e auditoria            | Seções 2 e 5: S3/backup, dados sensíveis, KMS    |
| <b>2.1 Escala e baixo acoplamento</b>                 | Camadas sem estado, ASG, ELB, filas, eventos, cache e persistência externa                      | Seções 1, 3, 4, 6 e 10                           |
| <b>2.2 Alta disponibilidade e tolerância a falhas</b> | Multi-AZ, replicação, backup, RTO/RPO, failover, quotas e testes de DR                          | Seções 2–4, 6 e 10                               |
| <b>3.1 Armazenamento</b>                              | Objeto/bloco/arquivo, classes, IOPS, throughput, paralelismo e escala                           | Seção 2 (Números do S3 e do EBS)                 |
| <b>3.2 Computação</b>                                 | Tipos EC2, Auto Scaling, Lambda, contêineres e gargalos medidos                                 | Seções 1 e 9                                     |
| <b>3.3 Banco de dados</b>                             | Modelo de dados, consultas, índices, cache, réplicas, conexões e capacidade                     | Seção 3                                          |
| <b>3.4 Rede</b>                                       | CIDR, AZs, DNS, ELB, CloudFront, conectividade híbrida e banda                                  | Seção 4                                          |
| <b>3.5 Ingestão e transformação</b>                   | Batch/stream, frequência, volume, Glue, Kinesis/Firehose, CSV → Parquet, partições              | Seções 6–7                                       |
| <b>4.1 Armazenamento econômico</b>                    | Lifecycle, acesso, requisições, agrupamento de arquivos, recuperação e retenção                 | Seção 2                                          |
| <b>4.2 Computação econômica</b>                       | Rightsizing, compromissos, Spot, serverless e utilização                                        | Seções 1 e 9                                     |
| <b>4.3 Banco de dados econômico</b>                   | Engine, capacidade, índices, réplicas, backups, retenção e conversão de esquema                 | Seção 3 e migração na seção 7                    |
| <b>4.4 Rede econômica</b>                             | NAT por AZ ou compartilhado, endpoints, transferência entre AZ/Region, banda e topologia        | Seções 4 e 9                                     |

### Como resolver múltipla resposta

O exame também inclui questões de **múltiplas respostas**, com duas ou mais opções corretas entre cinco ou mais alternativas. Nas questões abaixo, marque a **quantidade solicitada no enunciado** antes de abrir a solução. Leia cada requisito separadamente: uma opção pode resolver segurança e falhar no custo, ou resolver disponibilidade sem atender ao RPO. [Formato oficial](#).

**1. Uma equipe precisa acessar uma conta AWS com identidade corporativa e evitar credenciais permanentes nas instâncias EC2. Escolha duas ações: (A) usar IAM Identity Center com federação; (B) guardar access keys de um usuário IAM no AMI; (C) usar IAM role no perfil da instância; (D) compartilhar a senha da conta raiz; (E) liberar todas as ações e recursos em todas as políticas.**

**RESPOSTA**

**A e C.** Federação atende ao acesso humano, e instance profile fornece credenciais temporárias à EC2. B, D e E ampliam exposição e privilégio. Revise a seção 5.

**2. Um bucket com PII exige acesso restrito e retenção imutável pelo prazo legal. Escolha duas ações: (A) habilitar S3 Block Public Access e política mínima; (B) usar apenas SSE-S3 e considerar a retenção satisfeita; (C) configurar Object Lock em Compliance Mode com versionamento e período adequado; (D) usar URL pré-assinada pública sem expiração; (E) confiar só em CloudTrail para impedir exclusões.**

**RESPOSTA**

**A e C.** O primeiro conjunto restringe acesso; Object Lock com retenção protege contra exclusão durante o período. Criptografia e auditoria não substituem imutabilidade. Revise as seções 2 e 5.

**3. Uma API recebe picos e processa tarefas longas sem bloquear a resposta HTTP. Escolha duas decisões: (A) colocar trabalhos em SQS para workers escaláveis; (B) armazenar estado apenas no disco local de cada instância; (C) usar Auto Scaling orientado ao backlog por worker; (D) aumentar apenas o timeout do ALB; (E) manter um único servidor de processamento.**

**RESPOSTA**

**A e C.** SQS desacopla as camadas; escalar pelo backlog mantém vazão quando a fila cresce. B e E criam dependência de instância; D não resolve a capacidade. Revise as seções 1, 6 e 10.

**4. Um ambiente pilot light precisa atender à produção após falha regional dentro do RTO contratado. Escolha duas preparações: (A) verificar quotas e capacidade de expansão na região de DR; (B) presumir que replicação de dados garante RTO; (C) testar runbook, DNS, segredos e failover completo; (D) manter apenas o nome do bucket anotado; (E) ignorar dependências externas.**

**RESPOSTA**

**A e C.** Capacidade e testes são necessários para transformar dados replicados em serviço operacional. Revise a seção 10.

**5. Uma empresa recebe CSV diariamente no S3 e consulta meses de dados pelo Athena. Escolha duas melhorias: (A) transformar e compactar em Parquet particionado pelos filtros usuais; (B) consultar sempre todos os CSV sem partição; (C) catalogar dados com Glue Data Catalog; (D) copiar tudo para EBS por consulta; (E) aumentar o tamanho de uma instância EC2 inexistente.**

**RESPOSTA**

**A e C.** Arquivos colunares/partições reduzem leitura; catálogo fornece esquema descoberto e consultável. Revise a seção 7.

6. Uma VPC crescerá e conectará redes locais com múltiplas AZs. Escolha duas ações de projeto: (A) planejar CIDRs sem sobreposição e espaço para sub-redes/ENIs; (B) usar a mesma faixa CIDR da rede local para simplificar roteamento; (C) estimar throughput fim a fim e limites do link; (D) assumir que a velocidade nominal elimina todos os gargalos; (E) usar um único IP para todos os serviços privados.

**RESPOSTA**

**A e C.** Endereçamento não sobreposto viabiliza roteamento; capacidade real depende do elo mais restritivo e de overhead. Revise a seção 4.

7. Em várias contas, a equipe quer atribuir gastos por produto e evitar surpresas no orçamento. Escolha duas ações: (A) padronizar e ativar tags de alocação de custos; (B) analisar custos por conta/tag e configurar AWS Budgets; (C) usar CloudTrail como única ferramenta de orçamento; (D) compartilhar conta raiz para todas as equipes; (E) desligar relatórios de uso.

**RESPOSTA**

**A e B.** Tags precisam ser ativadas para relatórios; análise e alertas tornam o gasto visível. Revise a seção 9.

8. Instâncias em duas AZs enviam grande volume ao S3 e precisam de acesso geral à internet. Escolha duas medidas para avaliar custo e disponibilidade: (A) usar gateway endpoint para o tráfego de S3; (B) comparar NAT zonal por AZ com NAT compartilhado, incluindo horas, GB e transferência entre AZs; (C) forçar todo tráfego S3 por NAT; (D) remover a segunda AZ para eliminar o risco de failover; (E) assumir que todo endpoint de interface é grátis.

**RESPOSTA**

**A e B.** Endpoint gateway evita cobrança do NAT para S3; a topologia NAT depende do volume e da disponibilidade desejada. Revise a seção 4.

9. Uma API em EC2 deve continuar funcionando após perder uma AZ, absorver picos e processar pedidos sem perder tarefas. Escolha três decisões: (A) distribuir ALB e Auto Scaling group em duas AZs; (B) guardar estado e pedidos apenas no disco de uma instância; (C) colocar o banco em configuração Multi-AZ compatível; (D) usar apenas uma Read Replica na mesma AZ como mecanismo de failover; (E) desacoplar o processamento com SQS e consumidores idempotentes; (F) excluir backups porque há Multi-AZ.

**RESPOSTA**

**A, C e E.** ALB/ASG mantêm a camada de aplicação disponível e elástica; Multi-AZ protege o banco da falha zonal; SQS desacopla os pedidos e consumidores idempotentes lidam com entregas repetidas. Estado local, Read Replica isolada e ausência de backup não atendem a todos os requisitos. Revise as seções 1, 3, 4 e 6.



## Síntese de revisão

Este guia organiza a revisão pelos quatro domínios do exame e reúne formatos de consulta rápida e prática ativa.

- Os pesos oficiais são 30% para segurança, 26% para resiliência, 24% para desempenho e 20% para custos.
- As tabelas de decisão rápida ao fim de cada capítulo e os padrões recorrentes (seção 11) concentram comparações e pegadinhas dos simulados.
- O mapa de domínios (seção 12) orienta prioridades; as tabelas de números em cada tópico reúnem valores sujeitos a atualização.
- O autoteste, os cartões adicionais e as questões de múltipla resposta (seções 13 a 15) oferecem 182 perguntas para revisão ativa.
- O mapa da seção 15 relaciona as 14 tarefas publicadas no guia oficial às seções correspondentes.